Bachelor of Science in Information Technology

E-COMMERCE APPLICATION DEVELOPMENT

Building, Running and Securing an Online Shop

Unified Lecture Handout

Complete Course in One Volume • Fall 2026

Anyone can install a storefront in an afternoon. This course is everything after that.

A shop is not a website. It takes money, holds other people's data, and is answerable in law — and each of those changes what "finished" means.

What this volume covers. Every distinct clause of the HEC outline for *E-Commerce*, in the order the outline gives them, traced chapter by chapter in the Course Specification overleaf.

That outline is published once only, in the **2017** curriculum (p. 94), where the course is an *elective in BS Software Engineering* rather than a BSIT course at all. The 2023 curriculum drops it completely; the 2025 curriculum renames it *Digital Marketing & E-Commerce* and publishes no contents. The department teaches it as **INFT-4041**, BSIT, 5th semester, 3 credit hours. All of that is recorded in the Course Specification rather than smoothed over.

The course is taught on a **running shop**. One storefront is installed in Chapter 5 and every chapter afterwards adds a layer to it; by Chapter 23 it is the semester project. Appendix A sets up the stack in one sitting.

Six Parts • Twenty-Three Chapters • Seven Appendices

How to Use This Handout

There are two ways to teach this subject badly.

The first is to teach it as business studies: models, channels, trends, the history of the dot-com boom, and a graduate who can describe an online shop but has never configured a tax rule. The second is to teach it as programming exercises: write a cart class, write a checkout function, and a graduate who has built a toy that no one would take money through.

This handout takes the third path, which is also the one the published outline describes if you read it closely. You install a real, open-source storefront in Chapter 5, and you spend the rest of the course making it into a shop that could actually trade: a catalogue that models real products, a checkout that handles shipping and tax, a payment path that survives a failed transaction, an admin who can process a refund, a server that does not leak its database, and a listing a search engine can find. Where the platform cannot do what you need — and in Chapter 19 it cannot — you write the code that extends it.

The six parts

Part I — E-Commerce and Its Models (Chapters 1–4) is the business grounding, and it is short on purpose. What electronic commerce actually changed; the models and where the revenue comes from; auctions, portals and communities; and the build-buy-or-extend decision that justifies everything the rest of the book does.

Part II — Standing the Shop Up (Chapters 5–8) gets a storefront running and fills it. The stack and what each layer does; products and categories, with the tables underneath them; variants, options and uploads; then the theme.

Part III — From Basket to Order (Chapters 9–12) is the pipeline a customer walks through. Where basket state lives; the checkout and the order state machine; shipping, tax and zones; discounts, vouchers and referrals.

Part IV — Money and Identity (Chapters 13–16) is the part where mistakes cost money. Payment systems and who carries the risk; integrating a gateway and testing it in a sandbox; accounts, sessions and password storage; then securing the whole thing.

Part V — Running and Extending the Shop (Chapters 17–20) is the half of the job nobody demonstrates: the dashboard and what to watch, the back office, writing your first module, and deployment with everything that follows it.

Part VI — Reach, Law and Practice (Chapters 21–23) closes with being found, being lawful, and your project.



Why Chapter 19 is the hinge of the course. Everything before it is *configuration*: you are choosing among the options a platform already offers, and a careful non-programmer could do most of it. Chapter 19 is the first time the answer to "can the shop do this?" is no, and you have to write the code that makes it yes. That is the difference between the outline's bare title, *E-Commerce*, and the one this department teaches, *E-Commerce Application Development*.

The furniture of every chapter

Box	What it is for
Outcomes	What you will be able to *do*. Bloom verbs, not topics.
Before you start	Which earlier chapters this one stands on, and what state your shop should be in.
Key terms	The vocabulary introduced; all of it is in the glossary of Appendix D.
Definition	The canonical statement. In this subject several ordinary English words — order, basket, customer, product — are technical terms with narrower meanings, and confusing the two causes real bugs.
Concept	The judgement call: what an experienced practitioner would decide, and why.
Warning	Something that will bite you, usually in production rather than in the examination.
Worked example	Real arithmetic — a tax calculation, a gateway fee, a discount applied in the wrong order — with every step shown.
Measured	A number produced by running something, not by asserting it. Twelve chapters carry one; see below.
Common pitfalls	The mistakes that are actually made.
Try it yourself	The laboratory. Every one of them changes the same shop, and they are cumulative: skip Chapter 9's and Chapter 10's will have nothing to check out.
Checkpoint	Three questions. Attempt them before reading Appendix C.
Chapter summary	What to retain.
Review questions	Examination practice: multiple choice, short answer, applied.

Two typographic conventions carry a lot of weight in a course taught through a graphical interface, and they are worth learning on this page:

- **Catalog › Products › Add New** is a **click path** through the administration panel. Read the chevrons as "then".
- **Save** on its own is a single control — a button, a field, a tab or a menu item. Anything in `typewriter` is text you type or a file you edit, which is a different thing entirely.

⚠ The measured strand, and why it is only in twelve chapters

Twelve of the twenty-three chapters end with a number that was produced rather than asserted: a page time, a query count, a request rate, a hash cost, a list of security headers present and absent.

They are the twelve where measuring genuinely teaches something. "Add an index and the category page gets faster" is a claim a student can be told and will forget; watching the same page go from a table scan at fifty thousand products to an index seek, and reading both numbers off your own machine, is not. The other eleven chapters argue in prose, because dressing up a legal obligation or a business model with a stopwatch would be theatre.

Every figure in a **Measured** box was produced by the file named beside it, in `code/`, against the stack in Appendix A. Rerun them. Your numbers will differ from the book's — different machine, different disk, different day — and the *shape* should not. Where a result depends on the particular machine, the box says so.

Course Specification

This page is the course file. It states what HEC requires of a course called *E-Commerce* and shows where each requirement is met.

That is harder here than in most courses, because the requirement has been published exactly once and then withdrawn. The three editions are traced below so that an accreditation review against any of them has something to check.

Course particulars

Item	Value
Course title	*E-Commerce Application Development*, as taught. HEC 2017 calls it simply *E-Commerce*; HEC 2025 calls it *Digital Marketing & E-Commerce*. The department's title is the accurate one, and the Course Specification justifies the two added words
Course code	INFT-4041
Programme	BS Information Technology, 5th semester. Under HEC 2017 the course is an **elective in BS Software Engineering** (2017, p. 38, "select any FIVE") and does not appear in the BSIT scheme at all
Credit hours	3: three credit hours of lectures, no separate laboratory credit. The laboratory work in this handout is therefore done in lecture time and as homework, which is why every lab is small and cumulative
Prerequisite	*Web Technologies* (CS-207 / INFT-4005, 4th semester). HEC 2017 names the prerequisite *Web Engineering*; the department's course of that content is called Web Technologies. The name differs, the dependency is satisfied — see Appendix F
Duration	Sixteen teaching weeks, midterm in Week 8, project due Week 16 (Appendix G)
Teaching methodology	As published (HEC 2017): lecturing, written assignments, **project**, report writing
Assessment	As published: sessional examination, home assignments, quizzes, project, presentations, final examination
Set texts	As published: Laudon and Traver, *E-Commerce*; Peacock, *PHP 5 E-commerce Development*; Rayport, *Introduction to E-Commerce*; Schneider, *Electronic Commerce*. See defect 5 below, and Appendix E

The three editions

Ed.	Title	Status	Contents
2017	E-Commerce	Elective in BS Software Engineering (p. 38); absent from BSIT. 3(3,0); prerequisite *Web Engineering*	**Yes, p. 94 — the only published outline in any edition**
2023	—	**Absent entirely.** The course appears nowhere in the computing-disciplines curriculum except as an incidental application domain (p. 23)	No
2025	Digital Marketing & E-Commerce	Interdisciplinary / Allied course, item 11 (p. 16). Not a computing core; universities may add others	No — title and credit hours only

⚠ Five defects in the published document

Recorded here so that a reader comparing this handout with the HEC booklet is not confused, and so that the department's own course file does not inherit them.

1. Two learning outcomes, both too broad to assess. HEC publishes exactly two: "Understand the concepts and standards related to the discipline of E-Commerce" and "Analyze complex real world problems found in E-Commerce". Sibling courses publish eight. Two cannot carry sixteen weeks or a question paper, so this department adds six, each traced back to one of HEC's two and each marked as departmental in the table below.

2. The Bloom's Taxonomy column is blank for both outcomes — only the Domain column ("C") is filled. The levels below are therefore *assigned by this department*, and are marked as such.

3. The "Course Content" paragraph is the table of contents of its own reference book. Clause for clause, in order, it is the chapter list of Michael Peacock's *PHP 5 E-commerce Development* (Packt, 2010), which is reference 2 on the same page. HEC did not write an outline for this course; it pasted a contents page.

4. Two clauses are duplicated, which is what pasting a contents page does. "Checkout" appears twice — once as "The Checkout and Order Process" — and "Managing Products and Categories" appears twice, once on its own and once under Administration. Twenty-four clauses; twenty-two distinct.

5. The prescribed technology is end-of-life and the editions are stale. PHP 5 lost security support on 31 December 2018: the prescribed text targets a runtime that has been unsafe to deploy for longer than most of this class has been studying. Laudon and Traver is cited at the 13th edition (2017) and Rayport at 2007. Appendix E cites current editions and explains each gap.

Coverage of the HEC 2017 course outline

The published outline is a single run-on paragraph. Each of its clauses is traced below, in the order HEC writes them. Nothing is omitted, and the two duplicates are shown where they fall rather than quietly merged.

HEC 2017 clause	Chapter	Where exactly
"An overview of E-Commerce & its business models and concepts"	1, 2	What changed and why; then the models, with the revenue model treated as the question that decides the rest
"Planning an E-Commerce Framework"	4	Requirements, and the build-buy-or-extend decision this course's answer rests on
"Managing Products and Categories"	6	The admin screens, and the tables beneath them
"Product Variations and User Uploads"	7	Options and variants, the combinatorics they generate, and why an upload field is a security question
"Enhancing the User Experience"	8	Themes and template overrides, measured by page weight rather than taste
"The Shopping Basket"	9	Session, cookie or database, and what survives a restart
"The Checkout and Order Process"	10	The order state machine, step by step
"Shipping and Tax"	11	Geo zones, tax classes, and sales tax on a Pakistani online sale
"Discounts, Vouchers, and Referrals"	12	All three, and the order in which they must be applied
"Checkout" *(duplicate of clause 7)*	10	Same chapter; see defect 4
"Taking Payment for Orders"	14	The gateway handshake, the sandbox, and what happens when it fails halfway
"User Account Management"	15	Registration, sessions, password storage, reset flows
"Administration: Dashboard"	17	The dashboard, and which of its numbers are worth acting on
"Managing Products and Categories" *(duplicate of clause 3)*	6, 17	Same material from the back office; see defect 4
"Managing Orders"	18	Order states, fulfilment, partial shipment
"Customers"	18	Customer records, groups and approval
"Refunds"	18	Returns, credit and what the order state machine does next
"Voucher Codes"	12, 18	Created in 12, administered in 18
"Shipping" *(administration)*	11, 18	Configured in 11, operated in 18
"Deploying, Security, and Maintenance"	16, 20	Security is given its own chapter — see the note below — and deployment, backup and maintenance share 20
"Web Payment Systems"	13	Cards, wallets, bank transfer and cash on delivery; cost and risk for each

HEC 2017 clause	Chapter	Where exactly
"Social, Legal, and Ethical Issues of E-Commerce"	22	Consumer rights, privacy, dark patterns, accessibility, and the Pakistani statute
"Auctions, Portals, and Communities"	3	Moved forward to Part I, where it belongs: these are business models
"SEO"	21	Crawlability, URL structure, metadata, sitemaps

Course learning outcomes

The two outcomes HEC publishes, verbatim, followed by the six this department adds. Bloom levels are assigned by this department throughout, because the published table leaves the column blank.

CLO	By the end of the course the student will be able to	Bloom	Chapters
CLO-1	*(HEC, verbatim)* Understand the concepts and standards related to the discipline of E-Commerce	C2 Understand	1–4, 13, 22
CLO-2	*(HEC, verbatim)* Analyze complex real world problems found in E-Commerce	C4 Analyse	6, 10, 16, 20, 23
CLO-3	*(dept.)* Explain e-commerce business and revenue models, and classify a given venture against them	C2 Understand	2, 3
CLO-4	*(dept.)* Install, configure and secure an open-source storefront on a reproducible stack	C3 Apply	5, 16, 20
CLO-5	*(dept.)* Model a catalogue — products, categories, options and variants — and justify the schema that results	C3 Apply	6, 7
CLO-6	*(dept.)* Configure the order pipeline end to end: basket, checkout, shipping, tax, discounts and payment	C3 Apply	9–12, 14
CLO-7	*(dept.)* Analyse a storefront's performance and security posture from measurement rather than assertion	C4 Analyse	6, 8, 15, 16, 20, 21
CLO-8	*(dept.)* Extend the platform with a module meeting a stated requirement, then deploy, present and defend the result	C5 Evaluate	19, 20, 23

Three topics taught that the outline does not name

Each is justified here rather than smuggled in, because a reviewer will ask.

Chapter 6's data model. The outline names the screens — "Managing Products and Categories" — but never the tables. A graduate who can fill in an admin form but cannot say why a product's name lives in a different table from its price has not understood the catalogue, and *Database Systems* is a prerequisite that ought to be paid off rather than left on the shelf.

Chapter 16, Security, as a chapter of its own. HEC buries it inside "Deploying, Security, and Maintenance", one clause among three. A shop holds addresses, order histories and password hashes, and CLO-7 asks for a security posture to be *analysed*. One third of one clause will not do it.

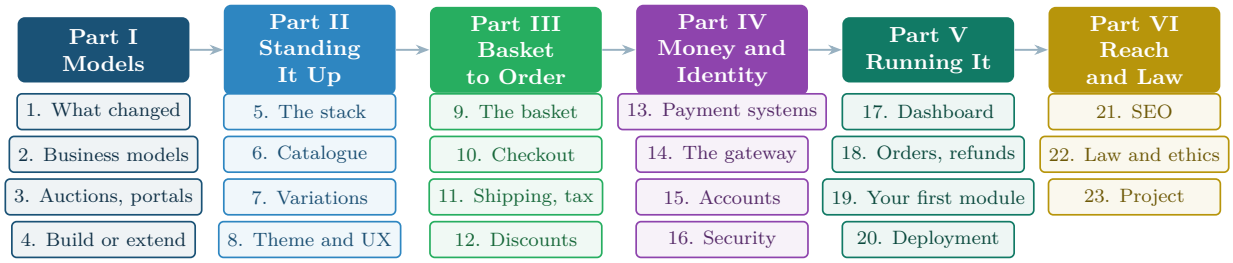
Chapter 19, module development. The outline is configuration from end to end; nothing in it requires a student to write a line of code. The department's title says *Application Development*, the HEC teaching methodology requires a project, and a project that is entirely configuration is not a computing project.

⚠ Where this course sits, and the one awkwardness

The outline this handout traces was written for *software engineering* students as an elective, and it shows: it assumes the reader will build a cart from scratch in PHP. This department teaches it to *information technology* students as a 5th-semester course, where the useful skill is standing up, configuring, securing and extending a real platform rather than reimplementing one badly.

This handout therefore follows the outline's *clauses* faithfully and its *method* not at all. Every topic HEC lists is taught; none of it is taught by writing a shopping cart in PHP 5. Chapter 4 makes that argument explicitly, so that the student can defend the choice rather than merely inherit it.

The Course at a Glance



One shop, installed in Chapter 5 and added to every week until it is the project in Chapter 23.

Figure F.1: The six parts and the chapters in each. Chapter 19 is the hinge: it is the first chapter in which configuring the platform is not enough and you have to write code.

This course stands on Web Technologies *and* Database Systems*, and shares its deployment ground with* Virtual Systems and Services *in the same semester. Appendix F states what belongs to each.*

The Shop We Build

Every lab in this book changes the same shop. Nothing is thrown away, and nothing is built twice. This page is the map of what gets added when, so that you can tell at any point in the semester whether you are behind.

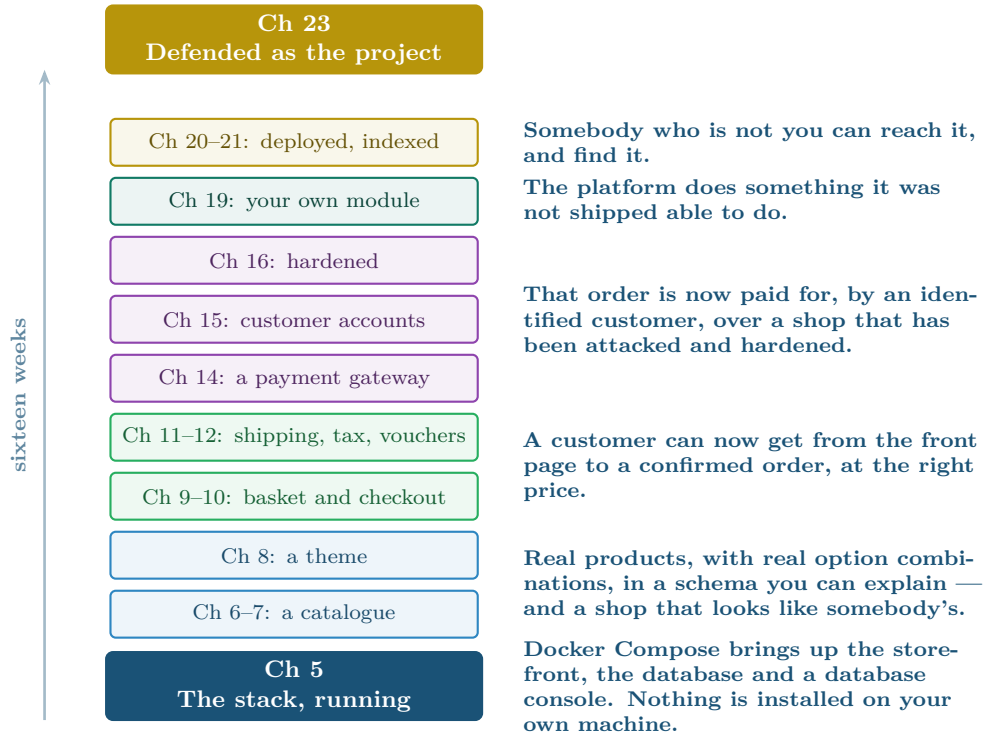


Figure F.2: What each part adds to the one shop. A lab skipped is a lab the next chapter cannot build on — Chapter 10 has nothing to check out if Chapter 9's basket was never configured.

Before Week 2

The stack runs in containers, so nothing in this course asks you to install a web server, PHP or a database on your own machine. It does ask you to install **one** thing: a container runtime. Appendix A does it in one sitting and lists what to do when it will not start, which on a university laptop it sometimes will not.

Do it before Week 2. From Chapter 5 onward every chapter assumes a shop you can reach in a browser, and a student without one spends the semester reading about a course the rest of the class is taking.

Prerequisite Self-Check

Twelve questions. If you can answer ten, begin at Chapter 1. If you cannot, Appendix B covers the PHP, SQL and template syntax this book assumes, your *Web Technologies* and *Database Systems* notes cover the rest — and the gap is worth closing in Week 1 rather than Week 9.

1. What is the difference between an HTTP `GET` and an HTTP `POST`, and which one must a form that charges a card use? Why? (Web Technologies)
2. What do the status codes 200, 302, 404 and 500 mean, and which one does a browser follow without telling the user? (Web Technologies)
3. HTTP is stateless. How, then, does a server know that two requests came from the same visitor? (Web Technologies, Chapter 9)
4. Write a `SELECT` that joins two tables and returns one row per match. (Database Systems, Appendix B)
5. What is a database index for, and what does adding one cost? (Database Systems, Chapter 6)
6. Distinguish a primary key from a foreign key, and say what the database guarantees about each. (Database Systems)
7. One product has many photographs. Which table holds the photographs, and why not the product table? (Database Systems, Chapter 6)
8. In an HTML form, what does the `name` attribute do, and where does its value arrive on the server? (Web Technologies)
9. A theme sets a heading to 18 px and you want 24 px. Give two ways to override it in CSS, and say which one you would not use. (Web Technologies, Chapter 8)
10. In PHP, what does `$_POST['qty']` contain, and why may you never use it in a query without doing something to it first? (Appendix B, Chapter 16)
11. What does HTTPS protect, and name one thing it does *not* protect. (Information Security, Chapter 16)
12. Open a terminal in a named directory, run a command, and read its output. If that sentence is not routine, start with Appendix A. (Appendix A)

Notice what is *not* on that list. You do not need to be a strong PHP programmer: until Chapter 19 you will read far more code than you write, and Appendix B gives you enough syntax to read it. You do not need any business background. You do not need a powerful machine.

What you do need is the habit of *checking* — opening the database to see what the form actually wrote, reading the error log rather than guessing, rerunning a measurement after changing one thing. Most of what goes wrong in this course is invisible from the front page of the shop, and every chapter tries to teach you where to look.

Contents

How to Use This Handout ii

Course Specification v

The Course at a Glance x

The Shop We Build xi

Prerequisite Self-Check xii

I E-Commerce and Its Models 1

1 What E-Commerce Is, and What It Changed 3
- 1.1 A Shop With No Closing Time . . . 3
- 1.2 The Four Functions . . . 4
- 1.3 What Actually Changed . . . 4
- 1.4 Why This Is a Software Problem . . . 5
- 1.5 Reading a Shop You Have Not Seen Before . . . 6

2 Business Models and Revenue Models 9
- 2.1 Two Questions That Get Confused . . . 9
- 2.2 Classifying by Who Is at Each End . . . 10
- 2.3 The Revenue Models . . . 10
- 2.4 The Arithmetic of One Order . . . 11

3 Auctions, Portals and Communities 15
- 3.1 Why This Chapter Is Here Rather Than at the End . . . 15
- 3.2 Letting the Buyer Set the Price . . . 15
- 3.3 What an Auction Costs You in Engineering . . . 16
- 3.4 Holding Other People's Money . . . 17
- 3.5 Portals, Marketplaces and Communities . . . 17

4 Planning an E-Commerce Framework 21
- 4.1 The Question This Chapter Answers . . . 21
- 4.2 The Four Options, Honestly . . . 22
- 4.3 Requirements That Actually Discriminate . . . 23
- 4.4 The Cost That Is Not on the Price List . . . 23
- 4.5 Making the Decision Defensible . . . 24

II Standing the Shop Up **28**

5 The Stack, and Your Shop in One Command **30**
5.1 Four Layers, and What Each One Does . . . 30
5.2 Containers, and What They Are Not . . . 31
5.3 The Three Services in This Course's Stack . . . 32
5.4 What Happens the First Time . . . 33
5.5 Looking At What You Were Given . . . 34

6 Products, Categories and Their Tables **37**
6.1 One Product Is Not One Row . . . 37
6.2 The Screen, and What It Writes . . . 38
6.3 Categories, and the Table That Is Not Normalised . . . 39
6.4 What the Category Page Actually Asks . . . 40
6.5 What It Costs, and What an Index Buys . . . 41

7 Product Variations, Options and Uploads **46**
7.1 Two Ways to Say "Comes in Three Colours" . . . 46
7.2 What Each Choice Costs . . . 48
7.3 Uploads, and What Your Shop Will Accept . . . 48

8 Themes, Layout and the Customer's Experience **53**
8.1 What a Theme Actually Is . . . 53
8.2 Layouts, or How to Change a Page Without Editing It . . . 54
8.3 What a Page Weighs . . . 54
8.4 Judging a Change by the Customer . . . 56

III From Basket to Order **59**

9 The Shopping Basket, and Where State Lives **61**
9.1 The Problem: A Protocol With No Memory . . . 61
9.2 Three Places State Can Live . . . 62
9.3 Where This Shop Actually Puts It . . . 62
9.4 What a Session Costs . . . 63
9.5 Two Things the Source Does That the Screens Do Not Show . . . 64
9.6 What Survives . . . 65
9.7 The Settings That Govern Any of This . . . 66

10 The Checkout and the Order State Machine **69**
10.1 A Checkout Is a Sequence, Not a Page . . . 69
10.2 The Fault That Stops Every Order . . . 70
10.3 What Each Step Costs . . . 71
10.4 An Order Is a State Machine . . . 72
10.5 Where a Checkout Can Be Shortened, and Where It Cannot . . . 73

11 Shipping, Tax and Zones **76**
11.1 Everything Here Depends on Where the Customer Is . . . 76
11.2 Tax: Three Things, Not One . . . 77
11.3 An Order Total, Computed by Hand . . . 78
11.4 Choosing How to Charge for Delivery . . . 79
11.5 Sales Tax on an Online Sale in Pakistan . . . 79

12 Discounts, Vouchers and Referrals **83**
12.1 Four Devices, and What Each One Is For . . . 83
12.2 The Clause the Platform Does Not Implement . . . 84
12.3 Totals Are a Pipeline, and Position Decides the Bill . . . 84
12.4 What a Discount Actually Costs . . . 86
12.5 Referrals: Paying Someone Else to Send Customers . . . 86

IV Money and Identity **90**

13 Web Payment Systems **92**
13.1 What Happens When Somebody Pays . . . 92
13.2 The Instruments a Pakistani Shop Can Actually Offer . . . 93
13.3 Cash on Delivery Is Not Free . . . 94
13.4 The Local Rails . . . 95

14 Taking Payment for Orders **98**
14.1 Three Ways to Take a Card, and Why One of Them Wins . . . 98
14.2 The Course Ships Its Own Gateway . . . 99
14.3 Choosing a Timeout, Which Is Not a Performance Setting . . . 99
14.4 Idempotency, Demonstrated . . . 101

15 User Account Management **105**
15.1 Why a Password Is Stored Slowly on Purpose . . . 105
15.2 What the Login Form Tells an Attacker Who Has No Password . . . 106
15.3 Lockout: What It Prevents, and What It Enables . . . 107

16 Securing the Shop **112**
16.1 Where You May Point This Chapter . . . 112
16.2 Measure First, and Measure the Right Thing . . . 113
16.3 What Is Actually Wrong, and It Is All Deployment . . . 113
16.4 Three Mistakes Made While Writing This Chapter . . . 114
16.5 What a Header Cannot Fix . . . 115

V Running and Extending the Shop **119**

17 Administration: The Dashboard, and What to Watch **121**
17.1 Eight Widgets, and What Each One Counts . . . 121

17.2 The Numbers Disagree, and Nothing Says So . . . 122
17.3 What the Dashboard Does Not Show . . . 123

18 Managing Orders, Customers and Refunds 126
18.1 The Admin Panel Does Not Edit Orders . . . 126
18.2 Stock Moves on Transitions, Not on Statuses . . . 127
18.3 Two Different Things Are Called a Refund . . . 128
18.4 Customers, and What You Are Keeping . . . 129

19 Extending the Platform: Your First Module 132
19.1 Why Not Just Edit the File . . . 132
19.2 What an Extension Is Made Of . . . 133
19.3 The Storefront Half, in Full . . . 133
19.4 Installing It, in Three Steps You Can Read . . . 134
19.5 What a Module Costs . . . 135

20 Deployment, Maintenance and Performance 139
20.1 Two Numbers, and Why One of Them Lies . . . 139
20.2 The Cheapest Optimisation Is Already a Setting . . . 140
20.3 Backups, and the Number Nobody Measures . . . 141
20.4 Going Live . . . 141

VI Reach, Law and Practice 145

21 SEO and Digital Marketing 147
21.1 What Is Checkable, and What Is Not . . . 147
21.2 Readable URLs Are Two Halves . . . 148
21.3 Can a Crawler Tell the Pages Apart? . . . 149
21.4 The Order to Work In . . . 150

22 Social, Legal and Ethical Issues 153
22.1 What Makes an Online Order a Contract . . . 153
22.2 The Instruments, and What Each Governs . . . 154
22.3 What Your Shop Already Knows About People . . . 154
22.4 Dark Patterns, Including the Ones We Built . . . 155
22.5 Accessibility, Which Is Not Optional and Is Usually Skipped . . . 156
22.6 Being Asked to Do Something You Should Not . . . 156

23 The Semester Project 160
23.1 What Is Being Assessed . . . 160
23.2 The Deliverable . . . 160
23.3 The Measurement Report . . . 161
23.4 The Defence . . . 162
23.5 A Realistic Schedule . . . 162

Appendices **166**

A The Stack: Setting It Up and Keeping It **167**
A.1 What You Install . . . 167
A.2 Starting the Shop . . . 167
A.3 The Four Commands Worth Memorising . . . 168
A.4 When It Will Not Start . . . 168
A.5 Backing Up, and Why You Will Wish You Had . . . 168
A.6 The Machine Every Measurement Was Taken On . . . 169

B Enough PHP, SQL and Twig to Read the Shop **170**
B.1 PHP, in Two Pages . . . 170
B.2 SQL, as This Course Uses It . . . 172
B.3 Twig, Which Is Just the Page . . . 173
B.4 YAML, in Ten Lines . . . 173
B.5 Where To Look Next . . . 174

C Answers to the Checkpoints **175**

D Glossary **190**

E Where to Read Further **192**
E.1 What HEC Prescribed . . . 192
E.2 What This Course Used Instead . . . 192
E.3 A Note on Using Tutorials . . . 194

F The Boundary With Other Courses **195**
F.1 The Edition History, Which Is Unusual . . . 195
F.2 Web Technologies — the Prerequisite . . . 195
F.3 Database Systems . . . 196
F.4 Information Security . . . 196
F.5 Software Engineering . . . 197
F.6 Virtual Systems and Services — the Same Semester . . . 197
F.7 What This Course Does Not Teach . . . 197

G The Sixteen-Week Schedule **198**
G.1 The Schedule . . . 198
G.2 How the Assessment Adds Up . . . 199

List of Figures

F.1 The six parts and the chapters in each. Chapter 19 is the hinge: it is the first chapter in which configuring the platform is not enough and you have to write code. x

F.2 What each part adds to the one shop. A lab skipped is a lab the next chapter cannot build on — Chapter 10 has nothing to check out if Chapter 9's basket was never configured. xi

P1.1 Reading path through Part I. Chapters 1–3 are the business grounding; Chapter 4 turns it into an engineering decision, and is the one to argue with. 2

1.1 The four functions of any shop, and the parts of this book that build them. The dashed line is the one that matters: everything to its left can be undone by closing the browser tab, and nothing to its right can. 4

2.1 The five revenue models a shop can use, and the single fact about each that most shapes the software. Most real businesses run two of them at once. 10

4.1 The four ways to obtain a shop. They are a spectrum, not a menu: each step to the right buys control and costs effort, and the right answer depends on which of the two you are shorter of. 21

P2.1 Reading path through Part II. Chapter 6 is the one to slow down on: it is the only chapter that looks at the same thing from the admin panel and from the database at once. 29

5.1 The four layers. The traditional name for this arrangement is LAMP — Linux, Apache, MySQL, PHP — and although every letter has alternatives now, the shape has not changed in twenty-five years. 31

P3.1 Reading path through Part III. Chapter 10 is the hinge of the whole book — an order is the first object in the course that cannot simply be deleted and redone. . . 60

9.1 The three stores, and what each one buys. Nothing forces a shop to pick one: most use all three at once, for different things. The question worth asking of any shop is not which one it uses, but which one holds the basket. 62

10.1 The guest checkout as the shop actually runs it. The first five steps collect; the last three commit. The gap between the order row appearing and the payment confirming it is where orders go missing. 70

11.1 Tax in three layers. A rate is a number in a place; a class is a kind of product; a rule says which rates a class attracts, in what order, and which of the three addresses on the order decides. 77

12.1 The totals pipeline as the shop ships it, with each extension's sort order above it. The numbers are editable fields in the admin panel, and nothing warns you what they mean. 85

P4.1 Reading path through Part IV. Chapter 16's laboratory runs against your own container and nothing else; the chapter says so in its first paragraph, and means it. 91

13.1 The parties to a card payment, and the direction a chargeback travels. The shop sits at one end of a chain it cannot see into, which is the central fact about accepting cards. 93

14.1 The three outcomes of a payment request. Most integration tutorials cover the first two. 101

16.1 Where a stock shop's security problems actually live. The layer students expect to fix is the one that was already correct. 115

P5.1 Reading path through Part V. Chapter 19 is where the course earns the words *Application Development* in its title; everything before it could have been done without writing code. 120

17.1 A test for every number you are thinking of putting on a screen. All eight shipped widgets sit at the left-hand end. 123

18.1 What a refund actually requires, and the one part of it the software performs. Four of the five are somebody's responsibility, and in most small shops nobody has been told which. 128

19.1 The twelve files of this chapter's extension. The namespace of every class is its path, which is how the loader finds it and the first thing to get wrong. 133

P6.1 Reading path through Part VI. Chapter 22 is the only chapter whose content is set by statute rather than by the platform, and the statute it uses is Pakistan's. . . . 146

Part I

E-Commerce and Its Models

Before you install anything, four questions have to have answers: what electronic commerce actually changed, how a shop makes money, what shapes other than a shop exist, and — given all that — whether you should build a storefront, buy one, or extend one.

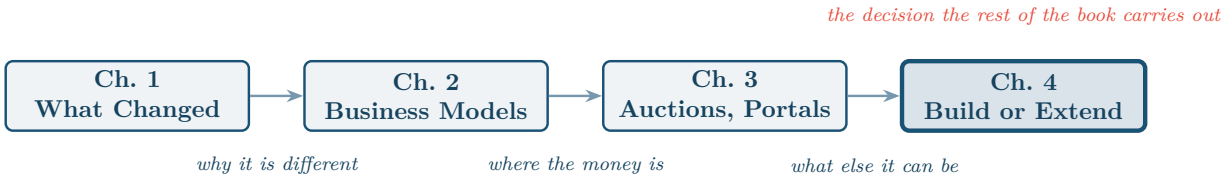


Figure P1.1: Reading path through Part I. Chapters 1–3 are the business grounding; Chapter 4 turns it into an engineering decision, and is the one to argue with.

Chapter 1

What E-Commerce Is, and What It Changed

Learning Outcomes

By the end of this chapter you will be able to:

- **Define** electronic commerce, and **distinguish** it from the wider category of electronic business.
- **Name** the four functions every shop must perform, and **locate** each of them in a storefront you have not seen before.
- **Explain** why selling online is a software problem rather than a marketing one, giving two consequences that follow.
- **Distinguish** the parts of a transaction that electronic commerce actually changed from the parts it only moved.
- **Classify** a given online business by the four functions, and **identify** which of them it has chosen not to perform itself.

Before You Start

Nothing in this book. You need to have used an online shop as a customer, which is the only prerequisite this chapter has. Appendix A sets up the software you will need from Chapter 5 onward; start it this week, because it sometimes needs a second attempt.

Key Terms in This Chapter

electronic commerce • electronic business • storefront • catalogue • basket • checkout • fulfilment • transaction • disintermediation • long tail • friction • conversion • abandonment • trust

1.1 A Shop With No Closing Time

Consider a bookshop on a street in Bahawalpur. It opens at nine and closes at nine. It holds perhaps four thousand titles, because that is what the shelves take. Its customers are the people who can reach the street. If you want a book it does not stock, the shopkeeper can order it, and you will come back next week.

Now consider the same bookshop online. It is open at three in the morning. It can list four hundred thousand titles, because a listing is a database row and rows are cheap. Its customers are anyone with a connection. If you want a book it does not stock, the website can know that instantly, and can offer to tell you when it arrives.

Every one of those differences is real, and none of them is the interesting one.

The interesting difference is this: in the street shop, a human being stands between the customer and every mistake. If the price label is wrong, the shopkeeper notices. If you try to buy two of something when one is left, the shopkeeper tells you. If you hand over a torn note, the shopkeeper decides. The shop runs on a continuous supply of human judgement, and that judgement is invisible precisely because it never fails conspicuously.

Online, there is no shopkeeper. Every one of those judgements has to have been written down, in advance, as a rule — and anything that was not written down will happen.

Electronic Commerce

Electronic commerce is the buying and selling of goods and services over a computer network, together with the transfer of money and data that completes those transactions.

It is a subset of **electronic business**, which is the use of networks for *any* business process — a supplier portal, an internal stock system, a staff payroll. The distinguishing feature of e-commerce is the **transaction**: value changes hands, and both parties are entitled to expect that it did so exactly once.

That last clause is not pedantry. It is the reason Chapter 10 is the longest chapter in this book.

1.2 The Four Functions

Strip away the branding and every shop on the internet does the same four things, in the same order. This is worth fixing now, because it is the skeleton of the whole course: each function is a part of this book, and each is a cluster of screens in the software you will install in Chapter 5.

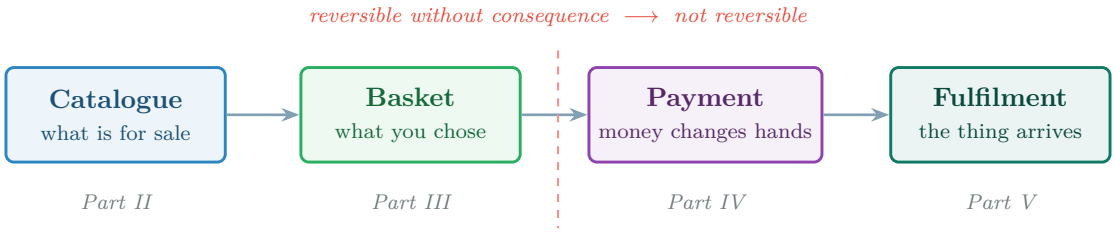


Figure 1.1: The four functions of any shop, and the parts of this book that build them. The dashed line is the one that matters: everything to its left can be undone by closing the browser tab, and nothing to its right can.

The dashed line is the whole subject

A customer browsing a catalogue and filling a basket has committed to nothing. They can close the tab and the shop is exactly as it was. Mistakes on that side of the line cost a sale at worst.

The moment payment is attempted, that stops being true. Money may have moved while the network dropped. An order may exist that the customer does not know about, or the customer may believe in an order that does not exist. Stock may have been reserved for someone who never paid.

Most of what distinguishes a real shop from a student exercise is the handling of that line, and most student projects put all their effort on the left of it because that is the half you can see.

1.3 What Actually Changed

It is easy to overstate what e-commerce changed. A great deal of it is the same commerce, conducted through a different window. Three things, though, genuinely changed, and they explain most of what the rest of this book worries about.

1.3.1 The cost of listing fell to nearly nothing

Shelf space is expensive and finite; a database row is neither. A street shop must stock what sells often, because an item that sells twice a year occupies the same shelf as one that sells twice a week. An online shop has no such pressure, and can profitably carry items that sell twice a year — the **long tail**.

That is a business observation with an engineering consequence that this course will measure: a catalogue of four hundred thousand items does not behave like a catalogue of four thousand. Chapter 6 puts fifty thousand products into your shop and times the result, and the answer is not comfortable.

1.3.2 Some middlemen became unnecessary, and others appeared

Disintermediation is the removal of intermediaries: a manufacturer can sell directly to you, and the distributor and the retailer are not needed.

The prediction made in the 1990s was that this would flatten commerce. What happened instead was that old intermediaries were replaced by new and larger ones — marketplaces, payment processors, delivery networks, search engines. A shop today typically pays a marketplace for access to customers, a gateway for the ability to take money, and a courier to deliver, and each of those is a dependency with a price and a failure mode. Chapter 13 is about one of them in detail.

1.3.3 Trust had to be manufactured

In a street shop, trust is cheap. The shop is physically there; you can see the goods; if it cheats you, you know where it is.

Online, none of that holds, and every substitute has to be built deliberately: a visible address and a returns policy, a padlock in the address bar, reviews, an order confirmation that arrives within seconds, a refund that works. These are not decoration — they are the mechanism that replaces a shopfront, and each of them is a feature somebody has to implement and maintain.

</> Three signals, and what each costs to fake

A customer deciding whether to buy from an unfamiliar shop is reading signals. It is worth knowing which ones are hard to fake and which are not.

- **A professional-looking design.** Costs almost nothing to fake; a template is free. Carries little information.
- **A valid certificate and HTTPS.** Costs nothing now that certificates are free — which is why its absence is damning and its presence proves very little. Chapter 16 returns to exactly what it does and does not prove.
- **A confirmation email, with an order number, that arrives in four seconds.** Hard to fake, because it requires the shop to actually have the machinery. This is one reason Chapter 10 treats the confirmation as part of the transaction rather than as an afterthought.

1.4 Why This Is a Software Problem

The common way to teach this subject is as business studies with screenshots. That gets the emphasis wrong, and the published outline this course follows — see the Course Specification — gets it right: eighteen of its twenty-two clauses are about building, configuring and running the software.

Here is the argument in one paragraph. A shop is a set of rules that must hold without supervision: this price applies to this customer in this quantity in this region on this date; this voucher may be used once; this stock is reserved for eleven minutes; this payment either completed or it did not. Every one of those is a statement about state, every one of them has an edge, and nobody is watching. That is not a marketing problem. It is the same problem as any other concurrent, stateful, partially-trusted system, and it is solved with the same tools.

The two failures that are specific to this subject

Almost everything that goes badly wrong in an online shop is one of two things, and both are invisible from the front page.

The state got out of step. Money taken, no order recorded. Order recorded, stock never reduced. Refund issued twice. These are not rare, and they do not announce themselves — the shop looks fine, and the accounts do not balance at the end of the month. Chapter 10 and Chapter 18 are about not causing them.

Something was trusted that should not have been. A price taken from a form field rather than from the database. A quantity that was allowed to be negative. An administrative page that was never actually checked for a login. Chapter 16 is a whole chapter about this, and its laboratory finds several of them in the shop you will have built by then.

1.5 Reading a Shop You Have Not Seen Before

The skill this chapter is really teaching is a diagnostic one, and it is worth practising before you build anything. Given any online business, you should be able to say within a few minutes which of the four functions it performs itself, which it has handed to somebody else, and what that choice costs it.

Worked Example: Classifying three businesses by the four functions

A clothing brand selling from its own site. Catalogue: its own. Basket: its own. Payment: a gateway — it does not hold a banking licence, so this is handed over. Fulfilment: usually a courier. *Performs two of four; the two that define it.*

The same brand selling only through a large marketplace. Catalogue: the marketplace's, in the marketplace's format. Basket: the marketplace's. Payment: the marketplace's. Fulfilment: possibly the marketplace's warehouse. *Performs none of the four.* It has bought access to customers and sold its relationship with them; it does not know who they are, cannot email them, and can be delisted. The commission is the visible price and is not the largest one.

A food delivery platform. Catalogue: aggregated from restaurants. Basket: its own. Payment: its own. Fulfilment: its own riders — which is the expensive one, and the one that is hard to copy. *Performs three and a half of four*, and the half it does not perform (the cooking) is the one with the thinnest margin.

The general rule. The function a business insists on performing itself is usually the one it believes is its advantage. Ask which function a business refuses to outsource, and you have found what it thinks it is.

Common Pitfalls

- **Confusing e-commerce with having a website.** A site that shows a catalogue and a phone number is a brochure. The transaction is the thing; without it, none of the hard problems in this book arise.
- **Assuming the hard part is the shopping cart.** It is not, and it has not been for twenty years — you will configure one in an afternoon in Chapter 9. The hard parts are everything after the dashed line in Figure 1.1.
- **Treating trust signals as design.** A returns policy is not a page; it is a promise that somebody has to keep, and a refund path somebody has to build (Chapter 18).
- **Believing disintermediation removed the middlemen.** It replaced them. Count your dependencies before claiming you have none.

- **Saying "we will add payments later".** Everything to the right of the dashed line constrains everything to its left. Deciding how you take money late is how projects discover their data model is wrong.

Try It Yourself: Reading four real shops

No software is needed for this one. Chapter 5 installs the stack; this week you are learning to look.

Choose **four** online businesses that you can actually reach, of deliberately different shapes — for example a single-brand store, a large general marketplace, a services business that takes bookings rather than shipping goods, and a local business selling through social media only.

For each one, as a customer and without buying anything, record:

1. **The four functions.** Who performs each — the business, or somebody else? How can you tell? For payment, the giveaway is usually whether the address bar changes during checkout.
2. **Where the catalogue ends.** How many categories deep does it go? Can a product appear in two categories? Does it have options — size, colour — and are they separate products or one product with choices? You are looking at a data model through a window; Chapter 6 will show you the model itself.
3. **The furthest point you can reach without an account.** Can you fill a basket? Reach the checkout? See the shipping cost before committing? Write down the exact step at which it stopped.
4. **Three trust signals**, and for each, whether it is cheap or expensive to fake.
5. **One thing that is broken or confusing.** There is always one. Describe it precisely enough that somebody could fix it.

Write up one page comparing the four, ending with a single sentence for each: what does this business think it is? Keep it — Chapter 4 asks you to do the same exercise for the shop you are going to build, and it is much easier the second time.

What to notice. The four shops will differ far more in what they *refuse* to do than in what they do. Everyone has a catalogue. Not everyone will tell you the delivery charge before you have handed over your address, and the ones that will not have usually decided that on purpose.

Checkpoint — Test Yourself

1. A university lets students view results and download transcripts online, but fee payment happens at a bank counter. Is this electronic commerce, electronic business, both or neither? Justify your answer from the definitions.
2. Give one judgement a human shopkeeper makes without noticing, and describe precisely what has to exist in software to replace it.
3. A marketplace charges a 15% commission. A seller calculates that running its own site would cost less than 15% of revenue and concludes it should leave. Name two costs the seller has not counted, and one thing it would gain that is not money.

Chapter Summary

- Electronic commerce is the buying and selling of goods and services over a network, *including* the transfer of money. The transaction is what distinguishes it from electronic business generally.

- Every shop performs four functions in order: catalogue, basket, payment, fulfilment. They are Parts II to V of this book.
- The line between basket and payment is the one that matters. Everything before it is reversible by closing a tab; nothing after it is.
- Three things genuinely changed: the cost of listing collapsed, old intermediaries were replaced by new and larger ones, and trust stopped being free and had to be manufactured.
- The absent shopkeeper is the engineering problem. Every judgement a human would have made has to have been written down in advance, and whatever was not written down will eventually happen.
- Almost all serious failures are one of two kinds: state that got out of step, or something trusted that should not have been.
- To read an unfamiliar business, ask which of the four functions it performs itself. The one it refuses to outsource is what it believes it is.

Review Questions

1. **(MCQ)** Which of these is electronic business but *not* electronic commerce? (a) A customer buying a phone from a web shop; (b) a supplier checking a manufacturer's stock levels through a portal; (c) a customer paying a utility bill through an app; (d) a marketplace taking commission on a sale.

2. **(MCQ)** In Figure 1.1, the dashed line separates: (a) front end from back end; (b) free actions from paid actions; (c) what a customer can undo by closing the tab from what they cannot; (d) catalogue data from order data.

3. **(Short)** Explain the long tail, and give one engineering consequence of it that a shop owner would not anticipate.

4. **(Short)** "Disintermediation removed the middleman." Argue against this with a concrete example.

5. **(Short)** A friend says the hard part of building an online shop is the shopping cart. Give the strongest version of why they are wrong, and name the two failure modes you would be more worried about.

6. **(Applied)** A bookshop with four thousand titles in a shop wants to list eighty thousand online, most of which it does not hold in stock. Identify three decisions this forces that would not arise at four thousand titles, and say for each which of the four functions it affects.

7. **(Applied)** Take any one of the four businesses from this chapter's laboratory. Describe, in enough detail that it could be checked, an experiment a customer could run from outside that would reveal whether the shop reserves stock during checkout. State what result would mean "yes", what would mean "no", and what would be inconclusive.

Chapter 2

Business Models and Revenue Models

Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** a business model from a revenue model, and **explain** why a venture can have a sound one and a hopeless other.
- **Classify** an online business by participant — B2C, B2B, C2C, C2B — and say what each classification predicts about the software.
- **Name** the principal revenue models and **identify** which one a given shop is using.
- **Explain** the unit economics of a single order, and **compute** the contribution margin from a set of figures.
- **Argue** which revenue model a given venture should adopt, and **state** the software consequences of that choice.

Before You Start

Chapter 1: the four functions, and the habit of asking which of them a business performs itself.

Key Terms in This Chapter

business model • revenue model • value proposition • B2C • B2B • C2C • C2B • marketplace • commission • subscription • freemium • advertising • affiliate • unit economics • contribution margin • customer acquisition cost • lifetime value • network effect

2.1 Two Questions That Get Confused

Almost every failed online venture answered one of these two questions well and the other not at all.

Business Model and Revenue Model

A **business model** says *what value you create and for whom*: who the customer is, what problem is solved, and why they would choose you rather than the alternative. It is the answer to "why does this deserve to exist?"

A **revenue model** says *how the money reaches you*: who pays, for what, how often, and how much. It is the answer to "who writes the cheque?"

They are independent. The same business model — say, connecting people who want a ride with people who will drive — can be monetised by commission per ride, by a driver subscription, by advertising, or by charging nothing and selling the data. Those four are different companies with different software.

The confusion matters because the two fail differently and the failures look alike from outside. A venture with no business model and a clear revenue model builds something nobody wants and charges correctly for it. A venture with a strong business model and no revenue model builds something people love and cannot pay for it — and it is usually the second that students build.

2.2 Classifying by Who Is at Each End

The oldest classification asks only who is transacting with whom. It is coarse, and it is still the first question worth asking, because it predicts a surprising amount about the software.

Type	Who to whom	What it implies for the software
B2C	A business sells to a consumer	Many small orders, card and wallet payment, a catalogue tuned for browsing, and heavy traffic at unpredictable moments. The default assumption of almost every storefront platform, including the one this course uses
B2B	A business sells to another business	Few large orders, negotiated and customer-specific prices, credit terms rather than immediate payment, purchase orders, approval chains, and quantity breaks. Almost everything in Chapter 10 changes, and most of Chapter 12
C2C	A consumer sells to another consumer	The operator holds no stock and never owns the goods. The hard parts move to trust, dispute resolution and holding money in escrow — Chapter 3 is about this
C2B	A consumer sells to a business	Freelance marketplaces, stock photography, influencer deals. The "product" is a person's time or licence, which breaks the stock model entirely

Why the B2B/B2C split is the one that bites

A platform built for B2C assumes one price per product. B2B does not have one price per product — it has a price per product *per customer*, often negotiated, often with quantity breaks, sometimes expiring.

This is the most common way a student project, or a real small business, picks the wrong tool. Everything looks fine until the first wholesale customer asks for their agreed rate, and the answer turns out to require a different data model rather than a setting. Chapter 4 makes you ask this question before you choose, which is the only time asking it is cheap.

2.3 The Revenue Models

There are not very many, and most businesses combine two or three.



Figure 2.1: The five revenue models a shop can use, and the single fact about each that most shapes the software. Most real businesses run two of them at once.

2.3.1 Sales margin

You buy at one price and sell at another. The oldest model, and the one the storefront software in this course is built around by default.

Its distinguishing feature, from a software point of view, is that *you hold stock*. Stock means a quantity that can be wrong, can be reserved, can go negative under concurrency, and must be reconciled against physical reality. Chapter 6 and Chapter 18 spend real effort on this.

2.3.2 Commission

Somebody else sells; you take a percentage for having made it possible. This is the marketplace model, and it is attractive because you hold no stock and your costs barely rise with volume.

What you take on instead is everything in Chapter 3: two parties who do not trust each other, money that must be held until both are satisfied, and disputes that someone has to adjudicate. You have also acquired a problem the sales model does not have — you must attract *both* sides, and neither arrives before the other.

2.3.3 Subscription

The customer pays regularly for continued access or delivery. Revenue becomes predictable, which is why investors like it and why so much of the web has moved to it.

The software consequence is recurring billing, which is substantially harder than one-off payment: cards expire, payments fail silently, customers pause and resume, and the price may change mid-contract. Chapter 14 explains why a stored card is a different kind of integration from a single charge.

2.3.4 Advertising and affiliate

Advertising sells the attention of your visitors to somebody else. It needs large traffic before it pays anything at all, which makes it a poor first revenue model and a common fantasy.

Affiliate revenue is the mirror image: you send a visitor to somebody else's shop and are paid if they buy. Cheap to start, and it leaves you without a customer — you never learn who bought, cannot serve them again, and the relationship belongs to the merchant. Chapter 21 is where this becomes a strategy rather than an accident.

2.4 The Arithmetic of One Order

A revenue model is a claim that can be checked, and the check is simpler than students expect. Take one order and follow the money out of it.

Worked Example: Does this order make money?

A shop sells a pair of shoes at **Rs. 4,500**, delivered anywhere in Pakistan, with free returns. Work out what is left.

Step 1 — what came in.
Revenue per order: **Rs. 4,500**.

Step 2 — the cost of the thing itself.
The shoes cost the shop Rs. 2,800 to buy.
Remaining: $4{,}500 - 2{,}800 =$ **Rs. 1,700** (a gross margin of 38%).

Step 3 — the cost of taking the money.
The gateway charges 2.5% plus Rs. 15 per transaction: $0.025 \times 4{,}500 + 15 = 112.5 + 15 =$ Rs. 127.50.
Remaining: $1{,}700 - 127.50 =$ **Rs. 1,572.50**.

Step 4 — the cost of delivery.
Courier, Rs. 250, absorbed by the shop because delivery was advertised free.
Remaining: $1{,}572.50 - 250 =$ **Rs. 1,322.50**.

Step 5 — returns, which are not optional.
20% of shoe orders come back. A return costs the courier fee again (Rs. 250) and the item cannot always be resold; assume a further Rs. 200 of loss on average. Expected cost per *order placed*: $0.20 \times (250 + 200) =$ Rs. 90.
Remaining: $1{,}322.50 - 90 =$ **Rs. 1,232.50**.

Step 6 — the cost of finding the customer.
The shop spends Rs. 60,000 a month on advertising and gets 50 orders from it: a **customer acquisition cost** of $60{,}000/50 =$ Rs. 1,200.
Remaining: $1{,}232.50 - 1{,}200 =$ **Rs. 32.50**.

The answer. The order that looked like a 38% margin earns **Rs. 32.50**, which is **0.7%** of the sale price — and that is before rent, salaries, hosting or tax. One extra return, or a 3% gateway instead of 2.5%, and it is a loss.

What this is really telling you. The two numbers that decided it were *returns* and *acquisition cost*, and neither appears anywhere in the product's price. A shop that measures only gross margin believes it is making 38%. This is why Chapter 17 is careful about which numbers on a dashboard are worth acting on.

Contribution Margin, CAC and LTV

Contribution margin is what one order contributes after all the costs that vary with that order — goods, payment fees, delivery, expected returns. It is what is left to pay the costs that do not vary.

Customer acquisition cost (CAC) is the marketing spend divided by the customers it produced.

Lifetime value (LTV) is the total contribution margin a customer produces before they stop buying.

The test a business must pass is LTV > CAC, by a comfortable multiple. The worked example above passes only if the customer orders again — its CAC is nearly its whole first-order margin, which is survivable for a business with repeat customers and fatal for one without.

Where the network effect comes from, and where it does not

A **network effect** exists when the service gets better for each user as more users join. A marketplace has one: more sellers make it more useful to buyers, which attracts more sellers.

A shop selling its own goods has *no* network effect. Your thousandth customer makes the shop no better for your first. Volume may buy you better purchase prices, which is a real advantage, but it is not a network effect and it does not protect you the same way.

This matters because the two justify completely different strategies. Running at a loss to grow is rational when a network effect will eventually make you defensible, and is just losing money when it will not. Plenty of ventures have adopted the first strategy while having the second business.

Common Pitfalls

- **Naming a business model and thinking you have named a revenue model.** "We are a marketplace for tutors" says nothing about who pays.

- **Quoting gross margin as if it were profit.** The worked example went from 38% to 0.7% without anything unusual happening.
- **Forgetting returns.** In clothing and footwear they run at 20–40% and they are the difference between a viable shop and a dead one.
- **Advertising as a first revenue model.** It pays at scale and nothing before it, so a new venture choosing it has chosen to earn nothing for a long time.
- **Assuming the platform is model-agnostic.** It is not. A B2C storefront has one price per product, and discovering that during implementation is expensive — see Chapter 4.
- **Treating "free delivery" as marketing.** It is a cost of Rs. 250 per order that appears in no price list and is paid out of the margin.

Try It Yourself: The model behind the shop you will build

Still no software — Chapter 5 starts that. This laboratory produces the one-page brief that Chapter 4 turns into a decision and Chapter 23 is eventually marked against, so it is worth real effort.

Choose a shop to build. It must be plausible, small, and local enough that you can check your assumptions: a bookshop, a bakery, a tailor, a spare-parts dealer, a plant nursery. Do not choose "an Amazon competitor".

Then produce, on one page:

1. **The business model in two sentences.** Who is the customer, what problem is solved, and why you rather than the shop down the road or the marketplace they already use.
2. **The classification**: B2C, B2B, C2C or C2B — and if you are tempted to say "both", say which one the *software* must assume, because it can only assume one.
3. **The revenue model**, from Figure 2.1, with the reason for choosing it over the next most plausible one.
4. **The arithmetic of one order**, following the six steps of the worked example with your own figures. Find real numbers where you can: gateway fees and courier rates are published, and a shopkeeper will usually tell you their return rate if you ask politely. Mark every number you guessed.
5. **The two numbers that decide it.** From your own arithmetic, name the two inputs the result is most sensitive to, and say what would have to be true for the shop to lose money.

What to notice. Almost everyone finds that the contribution margin is far thinner than they expected, and that the deciding numbers are not the ones they thought they were building a business around. That discovery is the point of the exercise, and it is much cheaper to make here than in Chapter 23.

Checkpoint — Test Yourself

1. A food delivery app charges restaurants 20% commission, charges customers a delivery fee, and sells promoted placement in search results. Name the business model and all the revenue models, and say which single one you would expect to be most profitable per unit.
2. A shop has a contribution margin of Rs. 400 per order and a customer acquisition cost of Rs. 1,500. Under what condition is this business viable? State it as an inequality and say what would need measuring.
3. Explain why a B2B shop cannot usually be built by configuring a B2C platform, naming the specific assumption that breaks.

Chapter Summary

- A business model says what value is created and for whom; a revenue model says who pays. They are independent, and a venture can have a good one and a fatal other.
- B2C, B2B, C2C and C2B classify by who transacts. The split that most affects the software is B2B against B2C: B2B has a price per customer, not a price per product.
- The five revenue models are sales margin, commission, subscription, advertising and affiliate. Holding stock or not is the fact about each that most shapes the system you have to build.
- Follow one order through six deductions — goods, payment fee, delivery, returns, acquisition — and the margin usually collapses. In the worked example, 38% became 0.7%.
- The viability test is LTV > CAC by a comfortable multiple, which for a thin first-order margin means repeat custom is not optional.
- Marketplaces have network effects; shops selling their own goods do not, and strategies that assume one are expensive when it is absent.

Review Questions

1. (MCQ) "We connect home cooks with office workers who want lunch." This is: (a) a revenue model; (b) a business model; (c) both; (d) neither.

2. (MCQ) Which revenue model most changes the *payment* integration a shop needs? (a) Sales margin; (b) affiliate; (c) subscription; (d) advertising.

3. (Short) Distinguish gross margin from contribution margin, and give two costs that appear in one and not the other.

4. (Short) Explain why a marketplace has a harder launch than a shop, despite holding no stock.

5. (Short) A clothing shop reports 45% gross margin and is losing money. Give the two most likely explanations, and say what you would measure first.

6. (Applied) A shop sells a product at Rs. 2,000 that costs it Rs. 1,100. Gateway: 2.9% + Rs. 20. Delivery: Rs. 200, charged to the customer at Rs. 150. Returns: 12%, each costing Rs. 200 in courier and Rs. 150 in unsellable stock. Compute the contribution margin per order, showing each step. Then find the return rate at which the shop breaks even.

7. (Applied) Take the venture from this chapter's laboratory and argue for a *different* revenue model than the one you chose. Give the strongest case you can, then say which software decisions in Parts III and IV would have to change if you adopted it.

Chapter 3

Auctions, Portals and Communities

Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the four common auction formats and **explain** what each one does to a bidder's incentive to bid honestly.
- **Identify** the engineering problems an auction creates that a fixed-price shop does not have.
- **Explain** the role of escrow, and **describe** what a marketplace operator must do when two users disagree.
- **Distinguish** a portal, a marketplace and a community, and **state** where each one's revenue comes from.
- **Explain** the two-sided launch problem and **evaluate** strategies for escaping it.

Before You Start

Chapter 2: the revenue models, especially commission, and the note on network effects at the end.

Key Terms in This Chapter

auction • English auction • Dutch auction • sealed-bid auction • Vickrey auction • reserve price • proxy bidding • sniping • shill bidding • escrow • dispute resolution • reputation system • portal • marketplace • two-sided market • chicken-and-egg problem

3.1 Why This Chapter Is Here Rather Than at the End

The published outline lists "Auctions, Portals, and Communities" second from last, after security and deployment. That is where it appears in the book whose contents page the outline was copied from, and it is the wrong place: these are not an afterthought to building a shop, they are *alternative shapes* a business can take, and the decision between them belongs in Part I alongside the other models.

So this chapter sits where it belongs. It is also the chapter that explains why the rest of the book assumes a fixed-price shop: the alternatives are genuinely harder, and it is worth knowing exactly how.

3.2 Letting the Buyer Set the Price

A fixed-price shop answers "what is this worth?" before the customer arrives. An auction refuses to answer and lets the buyers decide.

That is attractive when the seller genuinely does not know the price — a second-hand item, a one-off, something whose value depends on who wants it. It is unnecessary when the seller does know, which is why most commerce is not conducted by auction.

Four Auction Formats

English auction — ascending, open. The price rises until one bidder remains. What nearly everyone means by "auction".

Dutch auction — descending, open. The price starts high and falls until somebody accepts. Fast, and used for perishable goods where speed matters more than the last rupee.

Sealed-bid first-price — everyone submits one hidden bid; the highest wins and pays what they bid. Standard for tenders and contracts.

Vickrey auction (sealed-bid second-price) — the highest bid wins but pays the *second*-highest price. Its remarkable property is that bidding your true valuation is the best strategy whatever anyone else does, so there is nothing to be gained from strategising.

Why the second-price rule removes the guessing

Suppose an item is worth Rs. 10,000 to you.

In a *first-price* auction, bidding Rs. 10,000 guarantees you gain nothing even if you win — so you must shade your bid downward, and how far depends on guessing what others will do. Everyone does this, and everyone guesses.

In a *second-price* auction, bid Rs. 10,000. If the next highest bid is Rs. 8,000, you win and pay Rs. 8,000, gaining Rs. 2,000. If it is Rs. 11,000, you lose — and you are glad, because winning would have cost more than the item was worth to you. Raising your bid above your valuation can only make you win auctions you did not want; lowering it can only make you lose auctions you did want. Telling the truth is simply best.

Where you have met this already. The "proxy bidding" on consumer auction sites — you enter a maximum and the site bids on your behalf up to it — is a Vickrey auction wearing an English auction's clothes. The winner pays one increment above the second-highest maximum, not their own.

3.3 What an Auction Costs You in Engineering

A fixed-price product is a row that changes rarely. An auction is a small real-time system, and it brings four problems the rest of this book does not have to solve.

Problem	Why a fixed-price shop does not have it
Time is part of the state	A listing must close at an exact moment, unattended, and settle correctly whether or not anyone is looking. A product page has no deadline. This needs a scheduler, and a scheduler that did not run is a silent failure
Concurrent bids	Two bids arriving in the same instant must produce one consistent outcome. This is the classic lost-update problem and it needs a transaction and a lock, not a quick read-then-write
Sniping	A bid placed one second before closing wins without giving anyone a chance to respond. Fixes exist — extend the closing time on a late bid — and each changes the rules of the auction, which is a design decision rather than a bug fix
Shill bidding	The seller, or a friend, bids to push the price up with no intention of buying. Detecting it means analysing bidding patterns across accounts, and it is fraud rather than a feature gap

The lost update, concretely

Two bidders submit Rs. 5,000 at the same instant on an item currently at Rs. 4,500.

A naive implementation reads the current high bid, checks that the new bid beats it, and writes. Interleaved, both read Rs. 4,500, both pass the check, and both write Rs. 5,000 — the second overwriting the first. One bidder's money is now owed on an item they did not win, and the audit trail shows a single bid.

The fix is the one Chapter 10 uses for stock: do the check and the write in one atomic statement, inside a transaction, with the row locked. "Read, decide, write" is three steps, and anything can happen between them.

3.4 Holding Other People's Money

An auction site that merely introduces a buyer to a seller has introduced two strangers and left. Whoever moves first — sends the money, or sends the goods — carries all the risk.

Escrow is the operator's answer: it holds the buyer's money, tells the seller to ship, and releases the money when the buyer confirms receipt or a time limit expires.

Escrow and Dispute Resolution

Escrow is the holding of a payment by a trusted third party until the conditions of the transaction are met.

It makes the operator a custodian of money that is not theirs, which in most jurisdictions — Pakistan included — carries regulatory obligations (Chapter 22). It also makes the operator the **dispute resolution** authority, because somebody has to decide who is telling the truth when the buyer says the item never arrived and the seller says it did.

That adjudication cannot be automated away, and it is why marketplace commissions are higher than payment processing fees: the commission buys arbitration, not just plumbing.

Reputation as the cheap substitute, and what it cannot do

Escrow is expensive. The cheaper substitute is a **reputation system**: record how every past transaction went and let future counterparties read it.

It works, within limits, and the limits are worth knowing:

- **A new seller has no reputation**, so the system gives no protection in exactly the case where it is most needed.
- **Reputation can be bought** — a hundred small honest sales, then one large dishonest one. The score is an average; the fraud is a maximum.
- **Retaliation distorts it.** If each side rates the other, a buyer who reports a problem risks a bad rating in return, so both sides learn to stay quiet and the scores drift upward until they mean nothing.
- **It cannot be transferred.** Years of standing on one platform are worth nothing on another, which is exactly why the platform likes it.

3.5 Portals, Marketplaces and Communities

The three are often lumped together and should not be.

A **portal** aggregates *information* and sells attention — a price comparison site, a news aggregator with shopping links. It carries no stock, handles no payment, and earns from advertising or affiliate referral. Its engineering problem is ingesting other people's data and keeping it fresh, and its

commercial weakness is that it owns no transaction and can be cut out the moment a supplier decides to stop feeding it.

A **marketplace** aggregates *sellers* and hosts the transaction. It earns commission, and it is a shop in which the catalogue is written by strangers — which turns catalogue quality, duplicate detection and fraud into daily operational work rather than a one-time setup.

A **community** aggregates *people* who are there for each other rather than to transact. Commerce, if any, is secondary, and the usual mistake is assuming an engaged community will convert into a profitable marketplace. Many do not: the thing people valued was the absence of selling.

The two-sided launch problem

A marketplace is worthless to buyers with no sellers, and worthless to sellers with no buyers. On day one it has neither, and every new one faces this.

The strategies that actually work are all variations on *refusing to be two-sided at first*:

- **Be one of the sellers yourself** until there are enough others. You take on stock you did not want, and you compete with your own sellers, and it works.
- **Pick one narrow side and own it completely** — a single city, a single category — so that the market is small enough to fill.
- **Be useful to one side alone.** Offer sellers a tool worth having even with no buyers (stock management, a storefront), then introduce buyers once the sellers are already there.
- **Fake the thin side** by listing supply gathered from elsewhere. Common, legally risky, and the point at which several well-known companies acquired their reputations.

Notice that none of these is "build a better marketplace". The problem is not a product problem, and a student project that proposes a marketplace without addressing it has not yet met its hardest part.

Common Pitfalls

- **Implementing bidding with read-then-write.** Two simultaneous bids will eventually produce one lost bid, and it will be unreproducible when reported.
- **Closing auctions with a page request.** If closing happens when somebody loads the page, an auction nobody looks at never closes. It needs a scheduled job, and the job needs monitoring.
- **Treating escrow as a feature.** It is custody of other people's money and it is regulated. Chapter 22 is not optional reading if you intend to hold funds.
- **Believing a reputation score protects the first transaction.** It protects the hundredth.
- **Proposing a marketplace without a plan for the empty side.** This is the single most common fatal flaw in student project proposals for this course.
- **Assuming a community will monetise.** The engagement you measured may exist precisely because nothing was being sold.

Try It Yourself: Three shapes, and what each one must get right

No software yet. This is the last of the three analysis laboratories; Chapter 5 starts building.

Find one live example of each: an **auction**, a **marketplace** and a **portal**. They may be Pakistani or international, but you must be able to browse all three.

For each, record:

1. **Who takes the risk** if the goods never arrive — buyer, seller or operator? Find the actual policy page, do not guess. Quote it.

2. **Where the money is** between payment and delivery. Is there escrow? How long is it held, and what releases it?
3. **What the operator earns**, and from which side.
4. **One rule that exists because of a specific abuse.** Every mature platform has several — a bidding increment, a minimum feedback score, a holding period for new sellers, a limit on listing edits. Name one and work out what it is defending against.

Then, for the **auction** only: find the closing rules. Does a late bid extend the auction? By how much? Say which auction format that makes it, and whether it rewards bidding your true valuation.

Write up half a page ending with one sentence: of the three, which would be hardest to build, and which single problem makes it so?

What to notice. The rules that look arbitrary are nearly always scar tissue. Each one is a record of something that went wrong, and reading a mature platform's rules is the cheapest way to learn what will go wrong with yours.

Checkpoint — Test Yourself

1. In a Vickrey auction an item is worth Rs. 15,000 to you. Bids of Rs. 12,000 and Rs. 13,500 have been placed by others. What should you bid, what do you pay if you win, and why would bidding Rs. 18,000 be a mistake?
2. An auction site closes listings when a visitor loads the listing page. Describe precisely what happens to an auction for an unpopular item, and what the seller sees.
3. A marketplace has plenty of buyers and too few sellers. Give two strategies from this chapter that would address it, and name a cost of each.

Chapter Summary

- Auctions, portals, marketplaces and communities are alternative shapes a business can take, which is why this chapter sits in Part I rather than where the published outline puts it.
- The four formats are English, Dutch, sealed-bid first-price and Vickrey. Under the Vickrey rule, bidding your true valuation is optimal whatever others do — and consumer proxy bidding is Vickrey in disguise.
- An auction adds four problems a fixed-price shop does not have: time as state, concurrent bids, sniping and shill bidding. The first two are engineering; the second two are rule design.
- "Read the high bid, check it, write it" loses bids under concurrency. The check and the write must be one atomic operation.
- Escrow makes the operator a custodian of other people's money and the arbiter of their disputes. Reputation is the cheap substitute and fails exactly on the first transaction and the large one.
- Portals aggregate information, marketplaces aggregate sellers, communities aggregate people. Only the marketplace owns the transaction.
- Every two-sided market launches empty, and every strategy that works amounts to refusing to be two-sided at the start.

Review Questions

1. **(MCQ)** In a second-price sealed-bid auction, the optimal strategy is to bid: (a) slightly above your valuation; (b) exactly your valuation; (c) slightly below it; (d) it depends on the other bidders.
2. **(MCQ)** Which problem does a fixed-price shop share with an auction site? (a) Sniping; (b) shill bidding; (c) concurrent updates to one row; (d) scheduled closing.
3. **(Short)** Explain sniping, and give one rule change that discourages it together with the cost of that change.
4. **(Short)** Distinguish a portal from a marketplace, and say which is more easily cut out of its own business.
5. **(Short)** Give three distinct ways a reputation system can be gamed or rendered meaningless.
6. **(Applied)** Design the database handling for a single bid. Give the table columns you would need, the exact check that must happen atomically, and state what goes wrong if the check and the write are separate statements. You may write it as SQL or as precise prose.
7. **(Applied)** A student proposes a marketplace connecting Bahawalpur tailors with customers. Identify the two-sided launch problem in this specific case, choose one strategy from this chapter, and describe concretely what the first eight weeks would consist of.

Chapter 4

Planning an E-Commerce Framework

Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** the four ways a shop can be obtained, and **state** what each costs in money, time and control.
- **Write** requirements for a shop in a form that discriminates between those four rather than describing all of them equally.
- **Identify** the small number of requirements that actually decide the choice, and **explain** why most requirements do not.
- **Compute** a total cost of ownership over three years, including the costs that do not appear on a price list.
- **Defend** a build-buy-or-extend decision against the obvious objection to it.

Before You Start

Chapter 2 for the business and revenue model, and its laboratory — this chapter turns that one-page brief into a decision. Chapter 3 if your venture is a marketplace rather than a shop.

Key Terms in This Chapter

requirement • functional requirement • non-functional requirement • SaaS • hosted platform • self-hosted • open source • bespoke development • total cost of ownership • vendor lock-in • extensibility • data portability • decision matrix • minimum viable product

4.1 The Question This Chapter Answers

You have a business model, a revenue model and some idea of what you are selling. There is now exactly one decision to make before any of the rest of this book applies, and it is this: *where does the software come from?*

There are four answers, and the whole of Parts II to V is a consequence of which one you pick.

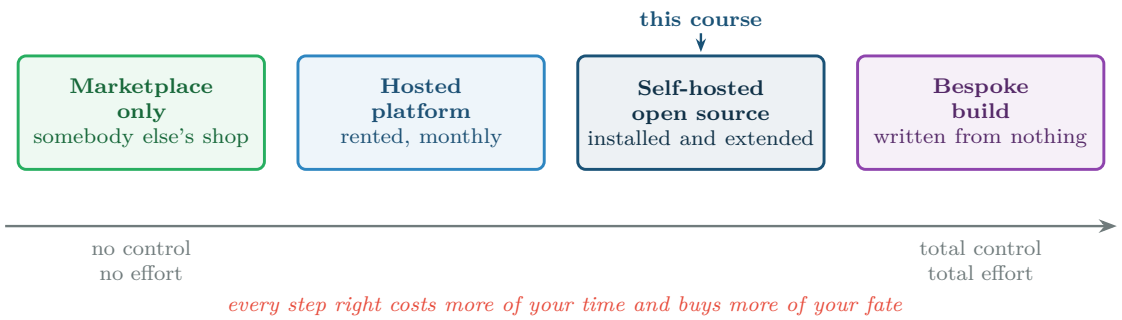


Figure 4.1: The four ways to obtain a shop. They are a spectrum, not a menu: each step to the right buys control and costs effort, and the right answer depends on which of the two you are shorter of.

4.2 The Four Options, Honestly

Option	What it is good at	What it costs you
Marketplace only	Customers on day one. No software, no hosting, no payment integration. Genuinely the right answer for many small sellers	You perform none of the four functions (Chapter 1). You do not own the customer relationship, cannot email them, and can be delisted without appeal. The commission is the visible cost and the smallest one
Hosted platform	Running in an afternoon, someone else patches it, payment and shipping already integrated	A monthly fee that scales with your success, a transaction cut on top, and a hard ceiling: when the platform cannot do something, it cannot do it. Getting your data out ranges from awkward to impossible
Self-hosted open source	You can change anything, including the things the vendor never anticipated. No per-sale cut. The data is in your database	You are now responsible for hosting, backups, updates and *security* — an unpatched shop is a liability, not an inconvenience. Needs someone who can do Chapter 20
Bespoke build	Fits the business exactly, because it was built for it. The only option when the business is genuinely unusual	Twelve to eighteen months before you sell anything, and you will reimplement baskets, tax tables and payment retries that have been solved for twenty years — badly, because you will meet each edge case for the first time in production

⚠ The outline this course follows assumes the fourth option

The HEC outline for this course is, clause for clause, the contents page of a 2010 book about writing a shopping cart from scratch in PHP 5 — see the Course Specification. Taken literally, it asks a 5th-semester class to choose *bespoke build*.

That was defensible in 2010 and is not now, for a reason this chapter can state precisely: the work has moved. In 2010, writing a basket and a checkout was most of the job. Today a mature, free, auditable implementation of both is one command away, and the job that remains — modelling the catalogue, configuring tax and shipping, integrating payment, securing the result, extending what the platform cannot do — is both larger and more representative of what an IT graduate is actually hired to do.

So this course teaches every *topic* the outline lists, and reaches them by the third route rather than the fourth. You should be able to defend that choice, not merely inherit it, which is what this chapter is for — and Chapter 19 is where the "extend" in "self-hosted and extend" stops being a word and becomes code you write.

4.3 Requirements That Actually Discriminate

Students write requirements like "the system shall allow customers to add products to a basket". Every one of the four options does that. A requirement that all four satisfy carries no information and cannot help you choose.

Functional and Non-Functional Requirements

A **functional requirement** says what the system must *do*: accept a voucher code, calculate tax by province, email an invoice.

A **non-functional requirement** says how well, under what constraints: how fast, how many at once, how available, how secure, how maintainable, by whom.

For the build-buy-extend decision the useful requirements are nearly all non-functional or unusual-functional. The ordinary functional ones are table stakes — everything does them — and listing them at length is the most common way a requirements document ends up being thirty pages that decide nothing.

The five questions that decide it

In practice the choice turns on a handful of questions. Ask these first, and most of the rest is detail.

1. **Is there anything genuinely unusual about how you price, sell or deliver?** Per-customer negotiated pricing, made-to-measure items, rental with a return date, services booked by time slot. If yes, every option except the last two gets harder fast.
2. **Who will maintain it in two years?** Not who will build it. A self-hosted shop with nobody to patch it becomes a breach (Chapter 16). If the honest answer is nobody, choose a hosted platform and accept the ceiling.
3. **What volume, and how uneven?** Thirty orders a day and three thousand are different systems; so are steady traffic and traffic that multiplies tenfold for one week a year.
4. **How much does leaving cost?** Can you export your products, customers and order history in a usable form? Ask it on day one, because the answer never improves.
5. **What is the budget, over three years, including somebody's time?** Not the setup cost. See below.

4.4 The Cost That Is Not on the Price List

The commonest error here is comparing a monthly fee against "free" and concluding that open source is cheaper. Both numbers are wrong.

Worked Example: Three years of a small shop, two ways

A shop expecting **Rs. 800,000 of sales a month**, growing slowly. Compare a hosted platform against self-hosted open source over three years.

Hosted platform.

- Subscription: \$79/month ≈ Rs. 22,000. Over 36 months: **Rs. 792,000**.
- Platform transaction fee, 0.5% of sales — charged on top of the gateway's own fee: $0.005 \times 800{,}000 \times 36 =$ **Rs. 144,000**.
- Two paid add-ons at \$15/month each ≈ Rs. 8,400/month: **Rs. 302,400**.
- Theme and setup, one-off: **Rs. 60,000**.
- Maintenance labour: near zero; the vendor patches.
- **Total: Rs. 1,298,400.**

Self-hosted open source.

- Licence: **Rs. 0**. This is the number people stop at.
- Hosting, a modest virtual server plus backups, Rs. 9,000/month: **Rs. 324,000**.
- Setup and theming, 70 hours of somebody at Rs. 2,500/hour: **Rs. 175,000**.
- Two custom modules the platform does not ship, 40 hours: **Rs. 100,000**.
- **Maintenance: 4 hours a month** — updates, patches, backups checked, the occasional breakage. $4 \times 36 \times 2{,}500 =$ **Rs. 360,000**.
- **Total: Rs. 959,000.**

The result. Self-hosting wins by Rs. 339,400 over three years — about 26%. But look at what decided it: **maintenance labour is Rs. 360,000**, the single largest line in the self-hosted column and the one most often entered as zero. Set it to zero and self-hosting appears to win by 87%. Double it, to eight hours a month because nobody on staff really knows the platform, and the two options are level.

The lesson. This calculation is not really about software prices. It is a question about whether you have, and will keep, somebody who can maintain a server — and if the answer is no, the hosted option is cheaper at any subscription price, because the alternative is not a cheaper shop but an unpatched one.

⚠ Two costs that appear in neither column

Vendor lock-in is not a line item until the day you want to leave, at which point it is the whole bill. Before committing, actually export your catalogue and customers from a trial account and look at what comes out. "We support CSV export" and "you can reconstruct your shop from this" are different claims.

The ceiling has no price at all. It is the feature you cannot have, discovered in month fourteen, when the business depends on it. The hosted option's real cost is the probability that this happens multiplied by what it does to you — which is why question 1 above is first.

4.5 Making the Decision Defensible

Write it down as a matrix: criteria in rows, weights, options scored. Not because the arithmetic is wise — it is easy to fit the weights to the answer you wanted — but because it forces the weights into the open where somebody can disagree with them.

</> A decision matrix for the shop in this course

Criteria weighted out of 100, each option scored 1–5.

Criterion	Weight	Hosted	Self-host	Bespoke
Students must see and change real code	30	1	5	5
Working shop within three weeks	25	5	4	1
Zero licence or subscription cost	15	2	5	5
Runs identically on every laptop	15	3	5	4
Covers the HEC clause list directly	15	3	5	4

Weighted total	**100**	**2.75**	**4.75**	**3.70**

Read the weights, not the totals. The first row is worth 30 and it is where hosted loses the contest — a course whose students cannot read the code is not this course. Change that weight to 5, as a small shopkeeper would, and hosted wins outright. The matrix did not make the decision; it made the decision *visible*, which is all a matrix is for.

Common Pitfalls

- **Requirements that every option satisfies.** "Customers can add to a basket" discriminates between nothing.
- **Entering maintenance as zero.** It was the largest single line in the worked example, and leaving it out reverses the answer.
- **Comparing setup cost instead of three-year cost.** Subscriptions and labour both accumulate; neither appears in a setup quote.
- **Choosing bespoke because the platform is "bloated".** You will reimplement the parts you called bloat, in production, one incident at a time.
- **Choosing self-hosted with nobody to maintain it.** This is not a cheaper shop. It is a shop that will be compromised, and Chapter 16 shows how quickly.
- **Not testing the export before committing.** Lock-in costs nothing until it costs everything.
- **Weighting the matrix after seeing the totals.** Everyone is tempted; write the weights down first and date them.

Try It Yourself: Your requirements, and your decision

The last analysis laboratory. Chapter 5 installs software.

Using the venture from Chapter 2's laboratory, produce a two-page document:

1. **Twelve requirements**, each labelled functional or non-functional, each numbered so you can refer to it later. At least four must be non-functional, with a number attached: not "must be fast" but "a category page must render in under one second with 5,000 products". You will test some of these in Chapter 6 and Chapter 20, so make them checkable.
2. **Mark the discriminating ones.** Go through your twelve and strike out every requirement that all four options satisfy. Most students strike out eight or nine. What is left is your real requirements document.
3. **Answer the five questions** from this chapter, in writing, including the honest answer to "who maintains this in two years?"
4. **A three-year total cost of ownership**, two columns, following the worked example. Mark every figure you guessed, and state the one number the answer is most sensitive to.
5. **A decision matrix**, weights written before scores, and a two-sentence conclusion.
6. **The strongest objection to your decision**, and your answer to it. If you cannot state a real objection, you have not understood the option you rejected.

What to notice. Your decision does not have to be the one this course makes. A student whose venture is three products sold through Instagram should conclude "marketplace only" and say so — and will still build the shop in Parts II–V, because the course has to teach the general case. Being right about your own venture and building something else deliberately is a perfectly coherent position, and a more useful one than pretending.

Checkpoint — Test Yourself

1. Classify each as functional or non-functional, and say which of them discriminates between the four options: (a) customers can reset a forgotten password; (b) prices are agreed per customer and may differ for the same product; (c) the shop stays up during a sale that brings twenty times normal traffic.
2. A shop compares Rs. 22,000/month hosted against "free" open source and concludes open source saves Rs. 792,000 over three years. Name the three largest costs they have omitted.
3. Give a specific business for which this course's choice — self-hosted open source — would be the wrong answer, and say which option you would choose instead and why.

Chapter Summary

- A shop can be obtained four ways: marketplace only, hosted platform, self-hosted open source, or bespoke build. They form a spectrum trading control against effort.
- Requirements that every option satisfies cannot help you choose. Strike them out; what remains is the document that matters.
- Five questions decide most cases: anything unusual about pricing or delivery, who maintains it in two years, volume and its unevenness, the cost of leaving, and three-year budget including labour.
- Compare total cost of ownership over three years, not setup cost. In the worked example maintenance labour was the largest self-hosted line, and entering it as zero reverses the conclusion.
- Vendor lock-in and the feature ceiling appear in no price list and are often the real cost of the hosted option.
- A decision matrix does not make the decision; it makes the weights visible so somebody can argue with them. Write the weights first.
- This course chooses self-hosted open source and teaches every clause of the outline through it, rather than writing a cart from scratch as the 2010 book behind the outline assumed. You should be able to defend that, including knowing when it is the wrong answer.

Review Questions

1. (MCQ) Which is a non-functional requirement? (a) The shop emails an invoice after each order; (b) the checkout completes in under two seconds at 200 concurrent users; (c) customers can apply a voucher code; (d) administrators can issue refunds.

2. (MCQ) In the worked example, the largest single cost in the self-hosted column was: (a) hosting; (b) setup and theming; (c) custom modules; (d) maintenance labour.

3. (Short) Explain why a requirement satisfied by all four options is useless for this decision, and give an example of one.

4. (Short) What is vendor lock-in, and what concrete test would you run during a free trial to measure it?

5. (Short) Give the strongest argument *against* the choice this course makes, and then answer it.

6. (Applied) A tailoring business takes measurements in the shop, makes garments to order over two weeks, and prices each job individually after seeing the cloth. Work through the five questions and recommend one of the four options, justifying it. Identify the one requirement that makes this case different from a normal shop.

7. **(Applied)** Build a three-year total cost of ownership for a shop doing Rs. 3,000,000 a month — nearly four times the worked example — on both options, keeping all other assumptions. State which option wins, and identify the single assumption that, if changed, would reverse it.

Part II

Standing the Shop Up

From here on there is a shop. Part II brings it up, fills it with a catalogue that survives contact with real products, and makes it look like somebody's rather than nobody's — and it does all of that without installing a single thing on your own machine.

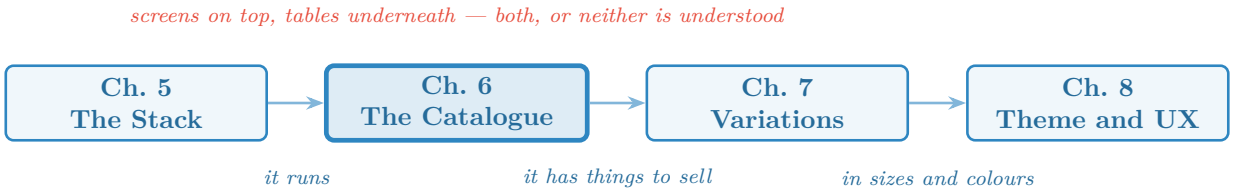


Figure P2.1: Reading path through Part II. Chapter 6 is the one to slow down on: it is the only chapter that looks at the same thing from the admin panel and from the database at once.

Chapter 5

The Stack, and Your Shop in One Command

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the four layers a web shop runs on and **state** what each one is responsible for.
- **Explain** what a container is, and **distinguish** it from a virtual machine and from an installer.
- **Read** a Compose file and **say** what each service, port and volume in it does.
- **Bring up** a working storefront, **verify** it from outside, and **reset** it to a known state.
- **Measure** what the stack costs to start and to store, and **distinguish** the cost paid once from the cost paid daily.

Before You Start

Chapter 4: the decision to use a self-hosted open-source platform, and why. Appendix A must be finished before this chapter — you need a working container runtime, and if it is going to fight you, it will fight you now rather than in Week 9.

Key Terms in This Chapter

stack • LAMP • web server • application server • database server • container • image • volume • port mapping • Docker Compose • service • environment variable • reproducibility • idempotent • pinning • checksum

5.1 Four Layers, and What Each One Does

When you type a shop's address into a browser, four distinct pieces of software answer in turn. Knowing which is which is the difference between fixing a problem and restarting things until it goes away.

Which layer is this problem in?

A diagnostic habit worth building on day one, because it turns "the site is broken" into a question with an answer.

- **Nothing answers at all**, the browser cannot connect: the web server is down, or the port is wrong. Nothing above it has run.
- **A page appears but says "500"**: the web server is fine; the application failed. The reason is in the PHP error log, never on the screen — and Chapter 16 explains why you want it that way.
- **A page appears, looks right, but data is missing or stale**: the application ran and the database answered something unexpected.
- **Everything works and is slow**: usually the database, and Chapter 6 measures exactly that.

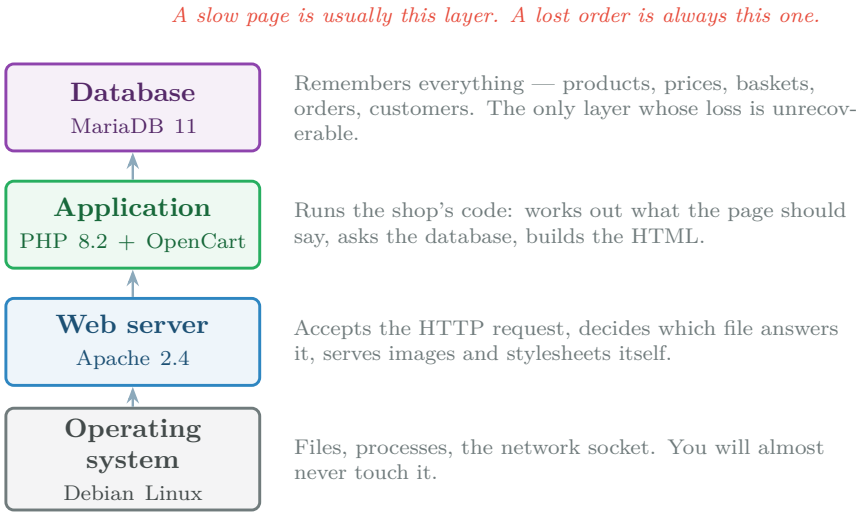


Figure 5.1: The four layers. The traditional name for this arrangement is LAMP — Linux, Apache, MySQL, PHP — and although every letter has alternatives now, the shape has not changed in twenty-five years.

5.2 Containers, and What They Are Not

The traditional way to get those four layers is to install them: a web server, a database, a PHP runtime, each with its own installer, its own configuration file in its own place, and its own way of being slightly different on your machine from on everybody else's.

A **container** replaces that with a package that already contains the software *and* everything it depends on, down to the system libraries, arranged exactly as its author arranged it.

Image, Container, Volume

An **image** is a read-only, layered filesystem containing an application and its dependencies. It is built once and is identical everywhere.

A **container** is a running instance of an image. It has its own filesystem view and its own network identity, but it shares the host's kernel — which is what makes it start in a second rather than a minute.

A **volume** is storage that lives *outside* the container and survives it. This is the one that matters: everything inside a container is thrown away when the container is removed, so anything you want to keep — your database, your shop — must be in a volume.

Three things a container is not

It is not a virtual machine. A VM carries a whole operating system including its own kernel, takes a minute to boot and gigabytes of memory. A container shares the host kernel and starts in about a second. The isolation is correspondingly weaker, which matters in production and not in this course.

It is not a backup. "It is in the container" means it will be gone when the container is replaced, which happens every time you rebuild. Your shop survives because it is in a *volume*, and Chapter 20 is about what happens when somebody has not understood that distinction.

It is not automatically secure. A container with a weak password is a weak password that starts quickly. Chapter 16 attacks this very stack and finds things.

5.3 The Three Services in This Course's Stack

The whole stack is described by one file, `code/docker-compose.yml`, and it declares three **services**.

```
services:
  db:                               # MariaDB 11: remembers everything
    image: mariadb:11
    volumes:
      - dbdata:/var/lib/mysql       # the shop's memory, kept outside
    healthcheck: ...                # "accepts a login", not "started"

  shop:                             # Apache + PHP 8.2 + OpenCart 4.1.0.4
    build: ./shop
    depends_on:
      db: { condition: service_healthy }
    ports:
      - "8090:80"                   # your 8090 -> the container's 80
    volumes:
      - shopdata:/var/www/html

  adminer:                          # a database console in the browser
    image: adminer:latest
    ports:
      - "8091:8080"
```

Three decisions in that file are worth understanding, because each one is a mistake people make.

The healthcheck, not just `depends_on`. Ordering container *starts* is not enough. MariaDB accepts TCP connections several seconds before it will accept a login, so a shop that waits only for the database to have started will try to install itself against a database that refuses it — intermittently, and more often on a fast machine. The condition `service_healthy` waits for the database to say it is actually ready.

The port mapping is a translation, not a setting. `8090:80` means "traffic arriving at port 8090 on my machine goes to port 80 inside the container". The shop always believes it is on port 80. If 8090 is busy on your machine — and on a university laptop it often is — change the left-hand number only.

Named volumes, so that a reset is deliberate. `docker compose down` stops everything and keeps your shop. `docker compose down -v` removes the volumes too, and the next start reinstalls from nothing. That second command is how you recover from a shop you have broken beyond understanding, and it is also how you destroy a semester's work in week 14. It is worth knowing which is which before you need either.

Why the image is built here rather than downloaded

Most teaching material says "pull the OpenCart image". There is no longer one to pull: there has never been an official OpenCart image, and the widely-used third-party one was withdrawn. Every such instruction on the web is now dead.

So `code/shop/Dockerfile` builds it, from the official PHP image plus a *pinned* OpenCart release — pinned by version and by SHA-256 checksum, so that the shop you build this year is the shop a classmate builds next year. Two lines in that file do the real work:

- `ARG OC_VERSION=4.1.0.4` — the exact release.
- `echo "${OC_SHA256} /tmp/oc.zip | sha256sum -c -` — the download really is the file we tested against. A version number alone is not reproducible, because a release asset can be replaced.

There is a second reason, and it is the pedagogical one: Figure 5.1 is a diagram of something, and pulling one opaque image would have left you unable to point at any of it. Here the layers are in the file, in order.

OpenCart ships its own Dockerfile, and it does not work

The release archive contains a `Dockerfile` and a `docker-compose.yml` at the top level. This course uses neither, for two reasons you can verify yourself in five minutes.

The Dockerfile contains a shell syntax error — `docker-php-ext-install zip && && docker-php-ext-enable zip` — so it has never built. The compose file pins `mysql:5.7`, which reached end of life in October 2023 and receives no further security fixes, and starts Redis, Memcached and PostgreSQL that a stock install does not use.

The lesson is not that OpenCart is bad software; it is a mature project that runs a large share of the world's small shops. The lesson is that an artefact being official is evidence and not authority. You are allowed to check, and in this course you are expected to.

5.4 What Happens the First Time

One command brings the whole thing up:

```
docker compose up -d --build
```

On a first run this does rather a lot: downloads about 1.5 GB of images, builds the shop image, starts three containers, waits for the database, and then runs OpenCart's command-line installer, which creates 159 tables and populates them with a demonstration catalogue. On every later run it does almost none of it.

That difference is worth measuring rather than asserting.

Measured: What the Stack Costs to Start

Produced by `code/m05_coldstart.py -scratch`, which removes every image and volume first, on the machine in Appendix A — 4 cores, 7.7 GiB available to the container runtime.

Start	To first 200	What it had to do
From scratch (no images at all)	**338.4 s**	Download 1.5 GB, compile three PHP extensions, fetch and verify OpenCart, install 159 tables. Of this, 214.6 s was the build alone
Cold (images built, no shop)	**34.1 s**	Start three containers, wait for the database, install OpenCart
Warm (the shop already exists)	**16.4 s**	Start three containers. Nothing else

Footprint. Images: shop 882 MB, MariaDB 462 MB, Adminer 164 MB — **1,508 MB** in all. Volumes after installation: database 181 MB, shop files 40 MB. About 1.7 GB of disk for a complete shop.

The ratio is the point. From scratch to warm is **20.6×**. You pay the 338 seconds once, in Week 3, and the 16 seconds perhaps sixty times over the semester. Quoting a single "startup time" for this stack would describe an experience nobody has.

On variance. An earlier run of the same measurement on the same machine gave 17.6 s cold and 7.7 s warm — roughly half — because less was running at the time. Your numbers will differ from these by more than you expect. The *ratio* between cold and warm should not.

5.5 Looking At What You Were Given

The installer leaves you a working shop with a demonstration catalogue. It is worth knowing its shape, because every chapter in Part II changes part of it.

</> The shop, counted

Asked of the database directly, through Adminer or the command line:

```
SELECT (SELECT COUNT(*) FROM oc_product)    AS products,
       (SELECT COUNT(*) FROM oc_category)   AS categories,
       (SELECT COUNT(*) FROM oc_order)      AS orders,
       (SELECT COUNT(*) FROM oc_customer)   AS customers;
```

- **159 tables** in total.
- **19 products** and **38 categories** of demonstration data.
- **0 orders** and **0 customers** — nobody has shopped here yet.
- **18 tables whose name begins `oc_product`.** One product is not one row, and Chapter 6 is about why.

That last number is the one to sit with. A beginner expects a products table. There are eighteen, because a product has descriptions in several languages, images, options, attributes, discounts, related items and category memberships — and each of those is a different relationship.

Common Pitfalls

- **Running `down -v` when you meant `down`.** The `-v` removes the volumes, which is your shop. There is no undo.
- **Editing files inside a running container.** They are gone at the next rebuild. Changes that must survive belong in the image or in a volume.
- **Assuming `up` means ready.** It means *started*. On a first run the shop answers about twenty seconds later; every measurement in this book calls `waitfor.py` first for exactly this reason.
- **Changing the right-hand side of a port mapping.** In `8090:80`, the 80 is the port inside the container and the shop expects it. Change the left number.
- **Treating a container as a backup.** It is the opposite of one.
- **Giving up when it does not start.** Appendix A lists the five failures that account for nearly all of them, and four are a busy port or a runtime that is not actually running.

Try It Yourself: Your shop, up, verified, broken and restored

From here to Chapter 23 this is the same shop. Treat it accordingly.

```
# --- 1. Bring the whole stack up ----------------------------------
# First run: expect about five minutes and 1.5 GB of download.
cd code
cp .env.example .env
docker compose up -d --build

# --- 2. Watch it install ------------------------------------------
# Not optional. You should see "SUCCESS! OpenCart successfully
# installed" go past; if you never see it, nothing below will work.
docker compose logs -f shop

# --- 3. Verify from outside, not from the logs --------------------
# Three URLs must answer 200. waitfor.py checks all three and is the
# answer to "is my shop up?" for the rest of the semester.
python waitfor.py

# --- 4. Look at the shop, as a customer and as the owner ----------
# Storefront   http://localhost:8090/
# Admin        http://localhost:8090/admin/   (admin / admin123)
# Database     http://localhost:8091/         (shop / shoppw / opencart)

# --- 5. Count what you were given ---------------------------------
# In Adminer, or here. 159 tables; 19 products; 0 orders.
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT COUNT(*) FROM information_schema.tables \
      WHERE table_schema='opencart';"

# --- 6. Prove the volume is doing what you think ------------------
# Change something in the admin panel first -- edit a product name.
# Then stop WITHOUT -v and start again. Your change is still there.
docker compose down
docker compose up -d
python waitfor.py

# --- 7. Now reset it deliberately ---------------------------------
# -v throws the volumes away. The shop reinstalls from nothing and
# your edit is gone. Do this once, now, while it costs you nothing.
docker compose down -v
docker compose up -d
python waitfor.py

# --- 8. Measure it ------------------------------------------------
# Produces the table quoted above, on YOUR machine.
python m05_coldstart.py --warm
```

What to notice. Step 6 and step 7 differ by two characters and by everything else. Having done both deliberately, in a week when the shop contains nothing you care about, is the cheapest possible way to learn the difference.

Also notice that step 3 does not trust step 2. The log said the install succeeded; step 3 asks the shop. Throughout this course, the thing that reports success and the thing that is true are kept apart on purpose.

Checkpoint — Test Yourself

1. A classmate's shop returns "500 Internal Server Error" on every page. Which of the four layers in Figure 5.1 are known to be working, which is known to have failed, and where would you look first?

2. Explain why `depends_on` alone is not enough to order the database before the shop, and describe the failure it produces.
3. You run `docker compose down -v` by mistake in Week 12. State exactly what is lost, what is not, and what would have had to exist beforehand for this to be a minor inconvenience.

Chapter Summary

- A shop runs on four layers — operating system, web server, application, database. Knowing which layer a symptom belongs to is most of diagnosis; only the database's loss is unrecoverable.
- An image is a built filesystem, a container is a running instance of one, and a volume is the only part that survives the container. A container is not a virtual machine, not a backup, and not secure by itself.
- The whole stack is three services in one Compose file: database, shop, database console.
- `depends_on` orders starts, not readiness. A healthcheck is what makes the shop wait until the database will actually accept a login.
- `down` keeps your shop; `down -v` destroys it. Learn the difference in Week 3.
- The image is built from a pinned release verified by checksum, because no maintained OpenCart image exists to pull and because a version number alone is not reproducible.
- Measured on this machine: 338.4 s from scratch, 34.1 s cold, 16.4 s warm — a ratio of 20.6× between the cost paid once and the cost paid daily. The stack occupies about 1.7 GB.
- The installed shop has 159 tables and 19 products, and eighteen of those tables are about products. One product is not one row.

Review Questions

1. (MCQ) In the port mapping `8090:80`, the 80 is: (a) the port on your machine; (b) the port inside the container; (c) the database port; (d) an arbitrary label.

2. (MCQ) Which survives `docker compose down`? (a) Nothing; (b) the containers; (c) data written to a named volume; (d) files edited inside a running container.

3. (Short) Distinguish a container from a virtual machine, and give one consequence of the difference that matters in this course.

4. (Short) Why does this course build the shop image from a pinned release rather than pulling a ready-made one? Give both reasons.

5. (Short) A page loads but shows yesterday's price. Which layer is at fault, and what are the two most likely explanations?

6. (Applied) Your stack will not start: port 8090 is already in use. Describe exactly what you would change, in which file, and what you would then have to change in the shop's own configuration for links to keep working. (Look at `OC_HTTP_SERVER` before answering.)

7. (Applied) Design a measurement that would show whether the *database* or the *application* layer is responsible for a slow category page, using only tools in this chapter. State what you would record, at what catalogue sizes, and what result would point at each layer. Chapter 6 runs your experiment; write it first.

Chapter 6

Products, Categories and Their Tables

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why one product occupies eighteen tables, and **say** what decides which table a given fact belongs in.
- **Create** products and categories through the admin panel, and **find** in the database exactly what each screen wrote.
- **Describe** how a category hierarchy is stored, and **say** what the path table buys and what it costs.
- **Read** the query a category page runs, and **use** `EXPLAIN` to say what the database does with it.
- **Measure** how the catalogue's size affects the page, and **decide**, from evidence, whether an index would help.

Before You Start

Chapter 5: a running shop. *Database Systems* for joins, keys and indexes; Appendix B if those need refreshing. You will spend as much of this chapter in the database console as in the admin panel.

Key Terms in This Chapter

catalogue • product • category • SKU • model • normalisation • junction table • many-to-many • localisation • materialised path • denormalisation • index • full table scan • filesort • correlated subquery • pagination

6.1 One Product Is Not One Row

Ask a student to design a products table and you get something like this:

```
CREATE TABLE product (
  product_id  INT PRIMARY KEY,
  name        VARCHAR(255),
  description TEXT,
  price       DECIMAL(10,2),
  image       VARCHAR(255),
  category    VARCHAR(100)
);
```

It is a reasonable first attempt and every part of it is wrong in a way that matters. Your shop has **eighteen** tables whose names begin `oc_product`, and the reasons for each are worth knowing, because the same reasons will apply to whatever you build after this course.

```
SELECT table_name FROM information_schema.tables
 WHERE table_schema = 'opencart'
   AND table_name LIKE 'oc_product%'
 ORDER BY table_name;
```

Four questions that split a table

Every one of those eighteen tables exists because of one of four questions. Learn the questions and the table list stops being arbitrary.

1. **"Does this fact depend on the language?"** A price does not; a name does. So `price` lives in `oc_product` and `name` lives in `oc_product_description`, keyed on *both* the product and the language.
2. **"Can there be more than one?"** A product has one price and many photographs, so photographs are rows in `oc_product_image` rather than columns.
3. **"Does this relate two things that both exist already?"** A product belongs to several categories and a category holds several products. Neither can hold the other, so a junction table — `oc_product_to_category` — holds the relationship.
4. **"Is this about the product, or about something that happened to it?"** `oc_product_viewed` and `oc_product_report` record events, not properties, and grow without bound.

6.1.1 The two tables you will use constantly

```
DESCRIBE oc_product;              -- the language-independent facts
DESCRIBE oc_product_description;  -- the words, per language
```

`oc_product` holds what is true regardless of who is reading: `price`, `quantity`, `weight`, `status`, `date_available`, `sort_order`. Its primary key is `product_id`.

`oc_product_description` holds `name`, `description`, `meta_title` and the rest — and its primary key is the *pair* `(product_id, language_id)`. One product, one row per language.

Why the split is not over-engineering

It looks like extra work for a shop that will only ever be in English. It is not, and the reason is worth more than the example.

Suppose name and price shared a row, and the shop later added Urdu. You now need a second row for the Urdu name — which duplicates the price. Change the price and you must remember to change it in both rows, and the day somebody forgets, the shop charges two different prices depending on the language the customer is reading. That is not a bug that gets noticed quickly; it is a bug that gets noticed by an accountant, months later.

The split makes that mistake *impossible to express*. There is exactly one place the price lives, so there is exactly one price. This is what normalisation is for, and it is the clearest small example of it you will meet.

6.2 The Screen, and What It Writes

Create a product: **Catalog › Products › Add New**. The form has tabs — **General**, **Data**, **Links**, **SEO** — and the tabs are not a presentational choice. They are very nearly the table list.

Tab	What you type	Where it lands
General	Product name, description, meta title and tags	One row in `oc_product_description`, for the language you were editing
Data	Model, price, quantity, weight, dimensions, date available, status	The single row in `oc_product`

Tab	What you type	Where it lands
Links	Manufacturer, categories, stores, related products	Rows in `oc_product_to_category`, `oc_product_to_store` and `oc_product_related` — one per tick
Image	Main image and additional images	`image` on the product row; the extras become rows in `oc_product_image`
SEO	The URL keyword	`oc_seo_url` — not a product table at all, which Chapter 21 explains

⚠ Do this once, deliberately, and you will never be confused again

Create one product. Then, in the database console at `http://localhost:8091/`, run:

```
SELECT MAX(product_id) FROM oc_product;   -- yours, the newest
```

and look at that id in `oc_product`, then in `oc_product_description`, then in `oc_product_to_category`. Three tables, one product, and you typed it all on one screen.

Every chapter after this one assumes you can do this. When something in the shop is not what you expected, the question is always "what did it actually write?", and the admin panel is the wrong place to look for the answer.

6.3 Categories, and the Table That Is Not Normalised

Categories form a tree: **Components › Monitors** sits under **Components**, which sits at the top. The obvious storage is a `parent_id` on each category, and that is indeed what `oc_category` has.

It is not enough, and the reason is the one interesting thing about category storage.

⚠ The question a parent pointer cannot answer cheaply

A customer opens **Components**. The page must show every product in *Components or anything beneath it*, however deep.

With only `parent_id`, answering that means: find the children of Components; find their children; find theirs; stop when there are no more. That is a loop with a query inside it, and the number of queries depends on how deep the tree happens to be — so the page gets slower as the shop's categories get better organised, which is precisely the wrong incentive.

So OpenCart keeps a second table that stores the answer directly.

```
DESCRIBE oc_category_path;
-- category_id, path_id, level

SELECT * FROM oc_category_path WHERE category_id = 27 ORDER BY level;
-- category_id  path_id  level
--          27       20      0      <- Components, the ancestor
--          27       27      1      <- Monitors, itself
```

Materialised Path

A **materialised path** stores, for every node, the complete list of its ancestors — one row per ancestor, with the depth. The tree is still in `parent_id`; this table is a *precomputed answer* to "who is above me?", and therefore also to "who is below me?"

It is a deliberate **denormalisation**: the same fact is now stored twice, in `oc_category.parent_id` and in `oc_category_path`. That buys a single-query answer to the

descendants question, and costs the obligation to keep the two in step — so moving one category in the admin panel rewrites every path row beneath it.

This is the trade-off in miniature: **denormalisation buys read speed with write complexity and the risk of disagreement.** Make it deliberately, and only where the reads vastly outnumber the writes — which for a category tree they do, by perhaps a million to one.

6.4 What the Category Page Actually Asks

Here is where this chapter stops being about design and starts being about evidence. The shop's own source is in your container, and you can read it:

```
docker compose exec shop \
  cat /var/www/html/catalog/model/catalog/product.php
```

The listing query is built up in `getProducts()`, and reduced to its skeleton it looks like this:

```
SELECT DISTINCT *, pd.name, p.image,
       (SELECT ... FROM oc_product_discount pd2
         WHERE pd2.product_id = p.product_id ... LIMIT 1) AS discount,
       (SELECT ... FROM oc_product_discount ps
         WHERE ps.product_id = p.product_id ... LIMIT 1) AS special,
       (SELECT pr.points FROM oc_product_reward pr
         WHERE pr.product_id = p.product_id ...)         AS reward,
       (SELECT COUNT(*) FROM oc_review r
         WHERE r.product_id = p.product_id ...)          AS reviews
  FROM oc_category_to_store c2s
  LEFT JOIN oc_category_path cp  ON (cp.category_id = c2s.category_id)
  LEFT JOIN oc_product_to_category p2c ON (p2c.category_id = cp.category_id)
 INNER JOIN oc_product_to_store p2s ON (p2s.product_id = p2c.product_id)
  LEFT JOIN oc_product p  ON (p.product_id = p2s.product_id
                              AND p.status = 1
                              AND p.date_available <= NOW())
  LEFT JOIN oc_product_description pd ON (p.product_id = pd.product_id)
 WHERE ...
 ORDER BY p.sort_order
 LIMIT 0, 20;
```

Four things in that query are worth naming, because each is a decision somebody made and each has a consequence you can measure.

1. **`SELECT DISTINCT *`** — every column of every joined table, then deduplicated. The `DISTINCT` is there because the path join can produce a product twice; it forces a temporary table.
2. **Four correlated subqueries.** Discount, special, reward and review count are each evaluated *per row returned*. With `LIMIT 20` that is eighty extra queries' worth of work per page, and it is the SQL form of the N+1 problem.
3. **`ORDER BY p.sort_order`** is the default sort, and `oc_product` has **no index on `sort_order`** — or on `price`, `status` or `date_available`. It has a primary key and nothing else.
4. **The whole join runs twice.** `getTotalProducts()` repeats it with `COUNT(DISTINCT p.product_id)`, because the page needs to know how many pages there are. Every category page you have ever loaded ran that join twice.

</> Reading EXPLAIN on the real query

Put `EXPLAIN` in front of it and the database says what it intends:

```
p:ALL [Using where; Using temporary; Using filesort]
```

Three findings in one line. `ALL` is a full table scan of `oc_product` — every row read, at any catalogue size. `Using temporary` is the `DISTINCT` building a temporary table. `Using filesort` is the `ORDER BY` sorting the result afterwards, because no index offers it in that order.

Now ask for the same page sorted by name, which is `ORDER BY LCASE(pd.name)`:

```
pd:index [Using where; Using index; Using temporary; Using filesort]
```

Still a filesort — and `oc_product_description` *does* have an index on `name`. Chapter 6's key lesson is in that contradiction, and the next section measures it.

A function around a column hides the column

`ORDER BY LCASE(pd.name)` cannot use an index on `name`.

An index on `name` stores the *names*, in order. The query does not ask for names in order — it asks for `LCASE(name)` in order, and the database has no index on that. It cannot assume the two orderings agree, because for some collations they do not. So it reads every row, computes `LCASE` on each, and sorts the results.

The rule, which is worth more than this example: the moment you wrap a column in a function in `WHERE` or `ORDER BY`, any index on that column stops being usable. `WHERE YEAR(date_added) = 2026` and `WHERE UPPER(email) = ?` are the same mistake, and they are extremely common.

The fixes are to store the value you actually search by, to use a case-insensitive collation so `LCASE` is unnecessary, or — in modern MariaDB and MySQL — to build an index on the expression itself.

6.5 What It Costs, and What an Index Buys

The argument so far is that this query ought to be slow and that an index ought to help. Both halves are worth checking, and only one of them survives.

Measured: The Category Page Against Catalogue Size

Produced by `code/m06_catalogue.py` on the machine in Appendix A. Products are seeded into one category; the page is requested seven times at each size and the median taken; then the same page is requested once more with MariaDB's query log recording *every* statement it ran and how long each took.
The whole run was done **twice**, because one of its results was surprising enough that a single observation would not have been worth reporting. Both runs are shown.

Products	Page	Queries	DB time	DB as % of page
100	112 / 111 ms	**144**	60 / 35 ms	54% / 32%
1,000	203 / 101 ms	**144**	60 / 60 ms	30% / 60%
10,000	171 / 302 ms	**144**	114 / 128 ms	67% / 43%
50,000	541 / 476 ms	**146 / 144**	402 / 477 ms	74% / 100%

One page, one hundred and forty-four queries — and that is the most stable number in the whole experiment. It is the same at a hundred products as at fifty thousand, in both runs. Twenty products are displayed, each needing its images, its discount, its special price and its review count, and around forty more queries fetch settings, languages, currencies, layout modules and the category tree. The $N + 1$ problem here is per *displayed* product, which is why growing the catalogue does not grow it.

The catalogue's size does cost, and it lands on the database. Five hundred times the products cost roughly 4× the page time and 8–13× the database time, and the database's share rises from about a third to between 74% and 100%. At fifty thousand products this page is database-bound; at a hundred it is not.

Now the surprise. The indexes help in the middle and hurt at the top.

Products	DB before	DB after	Change (run 1 / run 2)
100	60 / 35 ms	43 / 42 ms	−28% / +19% — noise
1,000	60 / 60 ms	41 / 48 ms	**−31%** / **−20%**
10,000	114 / 128 ms	107 / 106 ms	−6% / **−18%**
50,000	402 / 477 ms	616 / 574 ms	**+53%** / **+20%**

At a thousand and ten thousand products the indexes make the database faster, in both runs. At fifty thousand they make it **slower**, in both runs. The direction reproduces even though the magnitudes do not, which is what makes it worth believing.

Why an index can lose. `EXPLAIN` shows what changed: `p:ALL [... Using filesort]` became `p:index [...]`. Given an index on `sort_order`, the optimiser walks the index in order and avoids the sort — but it must then fetch each row by primary key, which is random access. Without the index it reads the table straight through, which is sequential, and sorts afterwards. The `DISTINCT` means it has to touch nearly every row either way, so the question is only whether those reads are sequential or scattered. At fifty thousand rows, **scattered reads plus no sort lose to sequential reads plus a sort.**

The chapter's conclusion, which is not the one the chapter expected. "Add an index" is not a diagnosis, and removing a filesort is not automatically a win. The index is right for a mid-sized catalogue and wrong for a large one, and nothing in `EXPLAIN` told us that — only the clock did. Meanwhile the page still runs 144 queries at every size, which no index touches at all. That is a structural problem, and Chapter 20 meets it with the only tool that works at that level: caching.

⚠ Three earlier versions of this measurement were wrong

All three are recorded because all three mistakes are ones you will make. The third is the worst kind.

Third, and the one that should frighten you: the products were not there. The seeder wrote `oc_product_to_store` with `store_id = 1`. The default store is **0**. Every storefront query filters on `p2s.store_id = config_store_id`, so fifty thousand products sat in the database, appeared in the admin panel, and were invisible to every customer — and to the measurement, which was timing a category page showing the nineteen demonstration products.

Nothing failed. No error, no warning, no empty page: the page rendered perfectly, returned 200, and meant something else. The numbers it produced were real numbers about a different

question, and they were only caught because a *later* chapter tried to open one of the seeded products and got a 404.

That is the characteristic shape of a measurement bug. It does not crash; it quietly answers a question you did not ask. The defence is to check that your setup did what you think before you trust what it tells you — here, one `curl` for a seeded product's page would have caught it in week one.

First: the harness timed itself. It ran the listing query twenty-five times through `docker compose exec` and divided the wall clock by twenty-five. It reported **707 ms of query time inside a 277 ms page** — impossible, and the impossibility was the only reason it was questioned. What it was really measuring was the cost of spawning a process and opening a connection, which dwarfs the query. Timing a trivial statement first and subtracting it fixed the small sizes.

Second, and worse: it was timing the wrong query. The harness used a hand-written query that *resembled* the one OpenCart runs. At 50,000 products it still reported more database time than the whole page took, because the hand-copied version lacked the real one's restrictions and did far more work. Two queries that look alike are not alike.

The fix was to stop guessing. MariaDB will record every statement it executes, with its duration, if you set `long_query_time=0` and turn the slow query log on. That is what produced the table above — the real queries, timed by the server, including the 146 of them.

Two rules, both expensive to learn any other way. **If a measurement gives you a number that cannot be true, believe the impossibility.** And **never hand-copy the query you mean to measure** — ask the server what it ran.

Common Pitfalls

- **Putting the name in `oc_product`.** It is the one column that depends on the language, and the one students most want to move.
- **Editing `oc_category_path` by hand.** It is derived. Move categories in the admin panel and let it rewrite the paths, or the tree and the paths disagree and the storefront shows products in categories they are not in.
- **Wrapping a column in a function and expecting the index to work.** `LCASE(name)`, `YEAR(date_added)`, `UPPER(email)` — all the same mistake.
- **Adding indexes because a page is slow.** Measure first. This chapter added three, removed a full scan and a filesort, and gained nothing.
- **Forgetting that the join runs twice.** Optimising the listing query and ignoring the `COUNT` halves your improvement at best.
- **Testing a catalogue with nineteen products.** Every interesting behaviour in this chapter is invisible below about a thousand.

Try It Yourself: Fifty thousand products, and where the time goes

Your shop has nineteen products. By the end of this laboratory it will have had fifty thousand, and you will know what that costs.

```
# --- 1. Look at what one product is, before changing anything -------
# Create a product in the admin panel: Catalog > Products > Add New.
# Fill in General (name), Data (model, price) and Links (a category).
# Then find it three times, in three tables.
cd code
docker compose exec db mariadb -ushop -pshoppw opencart

# --- 2. Count the tables one product touches ------------------------
```

```
# 18 of them begin oc_product. Read the list and, for each, say which
# of the four questions in this chapter put it there.

# --- 3. Fill the catalogue ---------------------------------------
# Seeded from a fixed seed, so your classmate's shop matches yours.
python seed/catalogue.py 50000

# --- 4. Load the category page and feel it -----------------------
# http://localhost:8090/index.php?route=product/category&path=20
# Then sort by name, and by price. One of those is slower; predict
# which before you try it.

# --- 5. Ask the database what it is doing ------------------------
# EXPLAIN the listing query. Find ALL, Using temporary, Using filesort.
# Then EXPLAIN it with ORDER BY LCASE(pd.name) and explain why the
# existing index on name does not save you.

# --- 6. Run the measurement --------------------------------------
# Four catalogue sizes, with and without the missing indexes. Watch
# the query COUNT as the catalogue grows -- it is the finding.
python m06_catalogue.py

# --- 7. Disagree with the book -----------------------------------
# The book found 146 queries per page, and that the indexes halved
# the database time without reliably moving the page time. Does
# yours? Then answer the question the book leaves open: of those 146
# queries, how many are for the 20 products shown, and how many would
# still run on an EMPTY category? Find out by requesting a category
# with no products in it, with the query log on.

# --- 8. Put your shop back ---------------------------------------
python seed/catalogue.py --restore
```

What to notice. Step 7 is the real exercise. The measurement in this book is a claim about one machine on one afternoon, and your job is not to reproduce the number but to find out whether the *shape* holds on yours — and, if it does, to say where the time is actually going. A student who can answer that can optimise a shop; one who can only add indexes cannot.

Checkpoint — Test Yourself

1. A shop adds Urdu. Which of these move to `oc_product_description` and which stay in `oc_product`, and why: price, product name, weight, meta description, stock quantity?
2. `oc_category_path` stores facts already derivable from `oc_category.parent_id`. Name what this buys, what it costs, and the one operation that makes the cost visible.
3. A colleague reports that a category page is slow, runs `EXPLAIN`, sees `Using filesort`, adds an index, and the page is no faster — although the database's own time halved. Give two distinct explanations for why the page did not improve, and say what you would measure to tell them apart.

Chapter Summary

- One product occupies eighteen tables. Four questions decide which: does the fact depend on language, can there be more than one, does it relate two existing things, and is it a property or an event?
- `oc_product` holds the language-independent facts; `oc_product_description` holds the words, keyed on `(product_id, language_id)`. The split makes a price that differs by language impossible to express.

- The admin form's tabs are very nearly the table list. Create one product and find it in three tables; every later chapter assumes you can.
- A parent pointer cannot answer "everything beneath this category" in one query, so `oc_category_path` stores the answer — a deliberate denormalisation buying read speed with write complexity.
- The category listing query does `SELECT DISTINCT *` with four correlated subqueries, sorts on an unindexed column, and the whole join runs twice because the page needs a count.
- `ORDER BY LCASE(name)` cannot use the index on `name`. A function around a column hides the column from the optimiser.
- Measured: **one category page runs 146 queries**, and that number barely moves between a hundred products and fifty thousand. The $N + 1$ is per displayed product, not per catalogue product.
- Five hundred times the catalogue cost about 2.5× the page time, because `LIMIT 20` bounds the work.
- The three missing indexes cut database time by 45–62% at every size, and the page time did not reliably follow — the gain is real and smaller than the run-to-run noise. Measure the database to see it.
- An index makes one query faster; it cannot make 145 others disappear. The page's real problem is structural, and Chapter 20 meets it with caching.

Review Questions

1. **(MCQ)** The primary key of `oc_product_description` is: (a) `product_id`; (b) `(product_id, language_id)`; (c) `name`; (d) an auto-increment id.

2. **(MCQ)** `Using filesort` in `EXPLAIN` means: (a) the query wrote a temporary file to disk; (b) the result had to be sorted after fetching, because no index gave that order; (c) the table has no primary key; (d) the query failed.

3. **(Short)** Give the four questions that decide which table a product fact belongs in, with one example each.

4. **(Short)** Explain why `WHERE YEAR(date_added) = 2026` cannot use an index on `date_added`, and rewrite it so it can.

5. **(Short)** What is a materialised path, and what has to happen when a category is moved in the admin panel?

6. **(Applied)** A shop wants products to carry a "minimum order quantity" that differs per customer group. Say which existing table this belongs in or what new table is needed, give the columns and the primary key, and justify it against the four questions.

7. **(Applied)** The measurement in this chapter found that removing a full table scan and a filesort did not speed the page up. Design an experiment that would locate where the time actually goes, using only what is in your container. State what you would vary, what you would hold fixed, what you would record, and what result would point at each of the four layers in Figure 5.1.

Chapter 7

Product Variations, Options and Uploads

Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** an option from a variant, and **say** which of the two a given requirement needs.
- **Explain** why options do not store the combinations they imply, and **state** what that makes impossible.
- **Configure** option sets on a product and **predict** the number of rows each one creates.
- **Describe** what a shop must check before accepting an uploaded file, and **identify** which of those checks a client can defeat.
- **Test** your own shop's upload handling and **report** what it accepts.

Before You Start

Chapter 6: the eighteen product tables, and the habit of looking at what a screen wrote. Your shop should still have its demonstration catalogue; `seed/catalogue.py -restore` puts it back if not.

Key Terms in This Chapter

option • option value • variant • SKU • modifier • price prefix • combinatorial explosion • stock keeping • required option • file upload • allow-list • MIME type • content sniffing • stored cross-site scripting

7.1 Two Ways to Say "Comes in Three Colours"

A shirt in three colours and four sizes. Twelve things a customer might buy. How many rows is that?

The honest answer is that it depends on a decision nobody will prompt you to make, and that the two answers have different consequences for the rest of the shop's life.

Option and Variant

An **option** is a choice attached to one product, whose values *modify* it. The shirt is one product; "Large" adds Rs. 200 and "Red" adds nothing. The twelve combinations are **not stored anywhere** — they are computed when a customer picks.

A **variant** is a product in its own right, linked to a master. "Red Large Shirt" is a row in `oc_product`, with `master_id` pointing at the parent, its own model number, its own stock and its own price. Twelve combinations are twelve rows.

OpenCart 4 supports both — `oc_product` carries `master_id`, `variant` and `override` columns alongside the older option tables. The admin panel shows options first, so options are what most shops end up with, including shops that needed variants.

7.1.1 What options actually store

Two tables do the work, and a third names the choices globally.

Table	What a row means
`oc_option`	A choice that exists in the shop at all — "Size", "Colour" — with its `type`: `select`, `radio`, `checkbox`, `text`, `textarea`, `date`, `time`, `datetime`, `file`
`oc_option_value`	One possible value of it: "Large". Shared across every product that uses the option
`oc_product_option`	*This* product offers *that* option, and whether it is `required`
`oc_product_option_value`	This product offers that value, and what choosing it does: `price` with a `price_prefix` of `+` or `-`, plus `quantity`, `weight` and `points` adjustments

⚠ What options make impossible

Look at where `quantity` lives: on `oc_product_option_value`, which is *per option value*, not per combination.

So the shop can know it has 40 Large shirts and 30 Red shirts. It **cannot** know how many Red Large shirts it has, because there is no row for that — and no amount of configuration will add one. Three consequences follow, and all three are discovered late:

- **No stock per combination.** You will oversell the popular combination and sit on the unpopular one.
- **No SKU or barcode per combination**, so the shop cannot speak to a warehouse system that works in SKUs — which is all of them.
- **No price history or reporting per combination.** "How many Red Large did we sell?" has no answer.

If any of those three matter, you need variants, and you need to know that *before* the catalogue is built. Migrating two thousand option-based products to variants is not a setting; it is a project.

Which one does this requirement need?

A test that settles most cases in one question: **does the shop need to count, price or identify the combination on its own?**

Options are right when the choice does not change what you hold in stock: an engraving message, a delivery date, a gift-wrap tick, a cable length that you cut to order. Also when the combination genuinely shares stock — which is rarer than people assume.

Variants are right whenever the combination is a thing in a box on a shelf. Clothing, shoes, anything with a barcode. If a warehouse person could pick up exactly one combination and scan it, it is a variant.

The common mistake is to choose options because the admin panel offers them first and they are quicker to set up, and to discover in month nine — during the first real stock-take — that the shop has never known what it holds.

7.2 What Each Choice Costs

Options are cheap to store and variants are not. That much is obvious. What is worth measuring is *how fast* the difference grows, because it decides whether a catalogue is manageable.

Measured: Options Against Variants

Produced by `code/m07_variations.py` on the machine in Appendix A. Option sets are attached to one product and the product page is requested seven times, median taken; the variant column is the number of `oc_product` rows the same catalogue would need.

Options	Combin.	Value rows	Page	KB	As variants
3	3	3	87 ms	30.0	4 rows
3×4	12	7	104 ms	30.7	13 rows
3×4×5	60	12	101 ms	31.3	61 rows
6×8×5	**240**	**19**	103 ms	31.3	**241 rows**

Options grow by addition; variants grow by multiplication. Three option sets of 6, 8 and 5 values are **19 rows** — $6+8+5$ — and express 240 combinations. The same catalogue as variants is **241 product rows**, each with its own description row, its own store row and its own category rows. Add a sixth colour and options gain one row while variants gain forty.

And the product page does not care. From 3 combinations to 240, the page stayed at about 100 ms and grew by **1.3 kilobytes** — because the page renders the *choices*, not the combinations. Nineteen values in three dropdowns, whatever the 240 products those values imply.

So the cost of variants is not page speed; it is catalogue management. Two hundred and forty-one rows that must each be priced, stocked, described, photographed and kept in step. That is the real bill, it is paid by a person rather than a server, and it is the reason shops that should have chosen variants did not.

7.3 Uploads, and What Your Shop Will Accept

One of the nine option types is `file`. Switch it on and a customer can send you a file — a logo to print, a document to quote against, a photograph to frame.

You have now accepted arbitrary bytes from a stranger, and everything that follows is about not regretting it.

What OpenCart checks, in order

From `catalog/controller/tool/upload.php` in your own container. Read it; it is forty lines.

1. **The filename is sanitised** — everything outside `[a-zA-Z0-9.␣+]` is stripped, then `basename()` removes any path. This defeats `../../etc/passwd` and it is solid.
2. **The length is checked**, 3 to 64 characters.
3. **The extension is checked against an allow-list** (`config_file_ext_allowed`). An allow-list is the right shape: anything not named is refused.
4. **The MIME type is checked against a second allow-list** (`config_file_mime_allowed`).

5. **The stored name is randomised** — the file is saved as the name plus a 32-character token, so it cannot be guessed, and the real name is kept in the database.

Check 4 is worth almost nothing, and it is not a bug in OpenCart

The MIME type it compares is `$_FILES['file']['type']` — and that value is **sent by the browser**. It is a claim made by the client about its own file. Anyone using anything other than a browser sets it to whatever passes.

The default allow-list makes this worse by including `application/octet-stream`, which means "unidentified binary" — so the honest answer for almost any file is already on the list.

This is not a criticism of OpenCart so much as a lesson about where a check must run. A check performed on data the attacker controls is not a check. The extension allow-list is real because the server derives it from the name it has already sanitised. The MIME check is advisory, and the only server-side equivalent is to inspect the file's actual contents — `finfo_file()` in PHP, which reads the magic bytes.

Two defaults on your shop, which you can verify in five minutes

Both of these are true of the stock install in Chapter 5, and both are measured in Chapter 16.

The allow-list includes `svg`. An SVG is not a picture; it is an XML document, and it may contain `<script>`. A browser asked to display one from your origin will run it. The default list also carries `eps` and `ps`, which are PostScript — a programming language — and `msi`, `cab` and `rar`.

Uploads are served over HTTP. They land in `system/storage/upload/`, which is *inside* the web root. Requesting a file there returns it: a text file comes back as `text/plain` and an SVG as `image/svg+xml`. The directory cannot be listed — there is an empty `index.html` — and names carry a 32-character token, so this is not trivially exploitable. But the protection is secrecy of the name, not inaccessibility of the directory, and Apache is configured with `AllowOverride None`, so the `.htaccess` OpenCart ships cannot change it.

What to do about it is Chapter 16's subject. The short version: remove `svg` from the list unless you need it, serve uploads through a script that sets `Content-Disposition: attachment` rather than from the filesystem, and keep the storage directory outside the web root.

Two size limits, and the smaller one wins

`config_file_max_size` is a setting in the admin panel. PHP has its own, `upload_max_filesize`, in `code/shop/php.ini`, and this course sets it to 8 MB.

If the admin setting is larger, it has no effect: PHP refuses the upload before any OpenCart code runs, and the error the customer sees is unhelpful. Whenever a limit appears not to work, look for the second limit — there is nearly always a second limit, in the web server, the runtime or the proxy, and the smallest one is the one in force.

Common Pitfalls

- **Choosing options when the requirement needs variants.** The symptom arrives at the first stock-take, and the fix is a migration.
- **Believing option `quantity` tracks combinations.** It tracks one value. Red Large and Red Small share Red's stock.
- **Trusting the uploaded MIME type.** The client supplies it.
- **Allowing `svg` without thinking about it.** It is script that renders as a picture.

- **Setting a size limit in one place only.** The smaller of the admin setting and `upload_max_filesize` wins.
- **Making every option `required`.** The customer cannot buy without choosing, and a required text option with no sensible default costs sales.
- **Forgetting that option values are global.** Editing "Large" edits it for every product that uses it.

Try It Yourself: Options, variants, and what your shop will swallow

```
# --- 1. Look at a product that already has every option type ---------
# Product 42 in the demo catalogue carries all nine. Open it in the
# admin panel: Catalog > Products > edit > Option.
cd code
docker compose exec db mariadb -ushop -pshoppw opencart

# --- 2. Count what one option set costs -----------------------------
# Add a Size option with three values, through the admin panel. Then
# count the rows it created. Predict the number before you look.
#   SELECT COUNT(*) FROM oc_product_option       WHERE product_id = 42;
#   SELECT COUNT(*) FROM oc_product_option_value WHERE product_id = 42;

# --- 3. Find the row that does not exist ----------------------------
# Set stock on two option values. Then try to write a query that
# answers "how many Red Large do we hold?". You cannot. Write down,
# in one sentence, why -- that sentence is the whole of section 7.1.

# --- 4. Measure options against variants ----------------------------
python m07_variations.py

# --- 5. Read the upload controller ----------------------------------
# Forty lines. Find the five checks, and find the one the client
# controls.
docker compose exec shop \
  cat /var/www/html/catalog/controller/tool/upload.php

# --- 6. Find out what your shop allows ------------------------------
docker compose exec db mariadb -ushop -pshoppw opencart -e \
  "SELECT \`key\`,\`value\` FROM oc_setting \
    WHERE \`key\` LIKE 'config_file%';"

# --- 7. Prove the upload directory is served ------------------------
# Put a file there yourself and fetch it. This is YOUR container; see
# the scope statement in Appendix A before you try this anywhere else.
docker compose exec shop sh -c \
  'echo hello > /var/www/html/system/storage/upload/probe.txt'
curl -i http://localhost:8090/system/storage/upload/probe.txt
docker compose exec shop rm /var/www/html/system/storage/upload/probe.txt

# --- 8. Decide, and write it down -----------------------------------
# For the shop you described in Chapter 4: options or variants? One
# paragraph, with the stock question answered explicitly. Chapter 23
# marks you on whether the catalogue matches this decision.
```

What to notice. Step 3 is the one that teaches. Everybody believes option stock tracks combinations until they try to write the query, and the minute spent failing to write it is worth more than this chapter's prose.

Checkpoint — Test Yourself

1. A shop sells mugs with a customer-supplied photograph printed on them, in two sizes. Which choices are options and which are variants, and why? Say what the shop must hold in stock.
2. A product has option sets of 4, 3 and 2 values. How many rows in `oc_product_option_value`, and how many `oc_product` rows would the same catalogue need as variants?
3. An attacker uploads a file named `logo.svg` containing `<script>`, sending `Content-Type: image/png`. Which of OpenCart's five checks does it pass, which does it fail, and what would have to be true for this to harm a visitor?

Chapter Summary

- An option modifies one product; a variant is a product of its own with `master_id` pointing at a master. OpenCart 4 supports both and shows options first.
- Options store choices, not combinations. Stock, price and weight sit on the option *value*, so there is no row for "Red Large" and no query can produce one.
- That makes three things impossible: stock per combination, a SKU per combination, and reporting per combination. If any matters, use variants, and decide before the catalogue is built.
- The test: could a warehouse person pick up exactly this combination and scan it? Then it is a variant.
- Measured: option sets of 6, 8 and 5 values are 19 rows and express 240 combinations; the same catalogue as variants is 241 product rows. Options grow by addition, variants by multiplication — and the page time is unaffected either way, because the page renders choices.
- The real cost of variants is catalogue management by a person, not server time.
- A `file` option accepts arbitrary bytes from a stranger. OpenCart sanitises the name, checks the extension against an allow-list and randomises the stored name — all sound. Its MIME check is not, because the client supplies the MIME type.
- On the stock install `svg` is allowed and the upload directory is served over HTTP. The protection is an unguessable name, not an unreachable directory. Chapter 16 fixes both.

Review Questions

1. (MCQ) Stock quantity for an option lives on: (a) `oc_product`; (b) `oc_product_option`; (c) `oc_product_option_value`; (d) a combination table.

2. (MCQ) Three option sets of 5, 4 and 3 values imply how many combinations, and cost how many `oc_product_option_value` rows? (a) 12 and 12; (b) 60 and 12; (c) 60 and 60; (d) 12 and 60.

3. (Short) Give the one-question test for options against variants, and apply it to a shop selling curtains cut to a customer-specified width.

4. (Short) Why is checking `$_FILES['file']['type']` not a security control, and what is the server-side equivalent?

5. (Short) An administrator raises the upload size limit in the admin panel and large files are still refused. Explain, and say where else to look.

6. (Applied) A clothing shop has 300 styles, each in 5 sizes and 4 colours. Compute the row counts for options and for variants across `oc_product` and `oc_product_description`. Then make a recommendation, naming the single requirement that decides it.

7. **(Applied)** Design the upload handling you would write for a shop that must accept customer artwork. State every check, say which side it runs on, say what each one defends against, and say what you would do with the file after accepting it so that serving it later is safe.

Chapter 8

Themes, Layout and the Customer's Experience

Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** how a theme is built from templates, and **override** one without editing the original.
- **Explain** what a layout is, and **place** a module on a chosen page without touching a template at all.
- **Measure** a page's weight and **say** which part of it is the weight.
- **Distinguish** the optimisations that are worth doing from the ones that are merely popular, using evidence.
- **Judge** a storefront change by its effect on the customer rather than by taste.

Before You Start

Chapter 6 and Chapter 7: a catalogue worth looking at. Appendix B's Twig section — this chapter edits templates. HTML and CSS from *Web Technologies*.

Key Terms in This Chapter

theme • template • Twig • template override • layout • module • position • page weight • request count • render-blocking • minification • responsive design • above the fold • conversion

8.1 What a Theme Actually Is

"Change the theme" sounds like changing a colour. It is not. A theme in OpenCart is a directory of Twig templates, and each one is responsible for one piece of the page.

```
docker compose exec shop \
  ls /var/www/html/catalog/view/template/product/
# category.twig  product.twig  search.twig  special.twig ...
```

The default theme lives in `catalog/view/template/`. The important rule about it is one you should adopt before you change a single line:

Never edit the default theme

Edit `catalog/view/template/product/category.twig` directly and three things happen. Your change is lost at the next upgrade. Nobody can tell what you changed, because there is no original to compare against. And a second developer, or you in March, will assume the file is stock and read it as documentation of how OpenCart works.

Make a **child theme** instead: a directory of your own that contains *only* the templates you changed. OpenCart looks there first and falls back to the default for everything else, so your directory *is* the list of your changes. A child theme with four files in it is a complete and honest statement that you changed four things.

8.2 Layouts, or How to Change a Page Without Editing It

A surprising amount of what people reach for a template to do can be done without touching one.

Layout, Module, Position

A **module** is a self-contained block — a category list, a banner, a carousel, "featured products".

A **layout** says which modules appear on which *kind* of page, and where. **Design › Layouts** lists them: Home, Category, Product, Checkout, and so on.

A **position** is a slot in the page: **Column Left**, **Column Right**, **Content Top**, **Content Bottom**. A module is placed in a position of a layout, with a sort order.

So "put a banner above the products on every category page" is three clicks in **Design › Layouts › Category** and no code at all.

Ask this before editing a template

"Is this a layout change or a template change?"

If you want something *moved, added or removed* and it already exists as a module, it is a layout change — do it in the admin panel, where it is visible to whoever comes next and survives upgrades.

Edit a template only when the *markup itself* must differ: a field shown that the theme does not show, a different arrangement within a block, a tag the theme gets wrong. Then override exactly that file in your child theme and nothing else.

The commonest waste of effort in this chapter's laboratory is a student editing `category.twig` to do something **Design › Layouts** would have done.

8.3 What a Page Weighs

Here the chapter stops being about arrangement. A customer's experience of your shop is mostly its speed, and speed is mostly *weight* — how many bytes and how many separate requests the browser must complete before the page is usable.

Note that this is a different question from Chapter 6's. That chapter measured how long the *server* took to produce the HTML. This one measures everything that happens afterwards, and it is usually larger.

Measured: What the Storefront Weighs

Produced by `code/m08_pageweight.py` on the machine in Appendix A. The harness fetches a page, finds every stylesheet, script and image the HTML names, requests each one once, and totals them.

Page	Requests	HTML	CSS	JS	Images	Total
Home	25	32 KB	348 KB	223 KB	293 KB	**895 KB**
Category	20	46 KB	348 KB	223 KB	122 KB	**738 KB**
Product	18	54 KB	354 KB	243 KB	61 KB	**713 KB**

The framework is the page. CSS and JavaScript come to about **570 KB on every page**, and they do not vary with the content — the same bytes on the home page, a category of 50,000 products and a single product. On the product page they are **84% of the weight**. The HTML that the whole of Chapter 6 was spent optimising is 6–7%.

Images are not the problem here, although everyone optimises them first. They are 33% of the home page, 16% of a category page and 9% of a product page. The largest single image in the demonstration catalogue is 13.9 KB. You could halve every image on the category page and save 61 KB — about 8%.

The single largest file is 265 KB of unminified CSS. `bootstrap.css` ships at 271,637 bytes, and **no minified version ships with it** — there is no `bootstrap.min.css` anywhere in the tree. A minified build of the same framework is around a tenth of that. That one file is 36% of the category page.

This measurement undercounts, and you should know how. It follows only assets the *HTML* names. Fonts are requested by the CSS, so they are invisible to it: the FontAwesome directory alone holds 248 KB of `.woff2` files, of which a browser fetches whichever subsets it needs. The totals above are therefore a floor, not a figure. A tool that drove a real browser would see more; this one is honest about seeing less.

The order to optimise in, which is not the popular order

From the numbers above, for this shop:

1. **Minify the CSS.** One file, 265 KB to roughly 27 KB. The largest single saving available, and it changes nothing a customer can see.
2. **Drop what you do not use.** Bootstrap and FontAwesome are general-purpose libraries; a shop uses a fraction of either. Removing the unused parts is the second-largest saving and the fiddliest.
3. **Cut the request count.** Twenty requests is twenty round trips. On a fast link this hardly matters; on a phone on a slow connection in Bahawalpur it is most of the wait.
4. **Then images** — resize to the size actually displayed, and serve modern formats.

Most advice reverses this list, because images are the thing people can see and a 2 MB photograph is a vivid story. On *this* shop images are 16% and the framework is 77%. On a shop with photographic product shots the balance moves. That is the point: the order depends on the measurement, and the measurement takes two minutes.

Render-blocking, and why position matters as much as size

A stylesheet in the `<head>` is **render-blocking**: the browser will not paint *anything* until it has fetched and parsed it. So 348 KB of CSS is not merely 348 KB of transfer — it is 348 KB standing between the customer and the first pixel.

Scripts block too, unless marked `defer` or `async`. A `<script src>` in the head stops the parser dead while it downloads.

This is why page weight and page time are not the same measurement, and why "it is only 200 KB" can still be the reason a shop feels slow.

8.4 Judging a Change by the Customer

The hardest discipline in this chapter is not technical. It is resisting the urge to change the storefront because you would prefer it differently.

</> Three changes, and how you would know if they helped

"Make the Add to Cart button green." You cannot know without an experiment: show half your visitors one colour and half the other, and compare the proportion who buy. With thirty visitors a day you will wait months for an answer, which is itself the answer — a shop this size should spend its effort elsewhere.

"Show the delivery charge on the product page." This you can reason about. Chapter 1's laboratory found that shops which hide the delivery charge until checkout lose customers there; it is the most-cited reason for abandonment. No experiment needed.

"Minify the CSS." Measurable directly, in bytes, with no customers involved at all. 265 KB to 27 KB is a fact, and it cannot make anything worse.

The pattern. Prefer changes you can verify without an experiment you cannot afford to run. Most small shops cannot generate enough traffic to measure a colour, and should spend their effort on the things that are measurable in bytes or known from everybody else's evidence.

Common Pitfalls

- **Editing the default theme.** Lost at upgrade, invisible to the next person. Override in a child theme.
- **Editing a template to do a layout's job.** If the block exists as a module, move it in **Design › Layouts**.
- **Optimising images first because they are visible.** On this shop that addresses 16% of the weight and ignores 77%.
- **Assuming a framework file is minified.** `bootstrap.css` here is 265 KB and there is no minified copy in the tree.
- **Measuring page weight by counting every `<img>` tag.** The same thumbnail on twenty cards is one request; counting it twenty times overstates the weight several-fold.
- **Forgetting fonts.** They are requested by CSS, so they are invisible to any harness that reads only the HTML — including this chapter's.
- **Redesigning on taste with no way to tell whether it helped.**

Try It Yourself: Weigh your shop, then make it lighter

```
# --- 1. Weigh three pages ---------------------------------------
cd code
python m08_pageweight.py --compare

# --- 2. Find the biggest single file ----------------------------
# Predict it before you look. Most people guess an image.
docker compose exec shop sh -c \
  'ls -laS /var/www/html/catalog/view/stylesheet/ | head -5'

# --- 3. Make a child theme --------------------------------------
# Admin: Design > Theme Editor. Override ONE template -- the category
# product card -- and add the model number under each product name.
# Your override directory is now the list of your changes.

# --- 4. Do a layout change with no code -------------------------
# Admin: Design > Layouts > Category. Add a module to Content Top.
# Reload a category page. No template was edited.
```

```
# --- 5. Weigh it again ------------------------------------------------
# Did your two changes move the number? By how much, and which one?
python m08_pageweight.py

# --- 6. Make a real saving --------------------------------------------
# Minify bootstrap.css and serve the minified copy instead. Any
# minifier will do. Record the before and after byte counts.

# --- 7. Weigh it a third time, and account for the difference -------
# The saving should be most of 240 KB. If it is not, find out what
# else changed -- do not just record the number.
python m08_pageweight.py

# --- 8. Say what you would do next, and why ---------------------------
# One paragraph, ordered by measured saving rather than by what is
# easiest to see. Chapter 20 returns to this with caching.
```

What to notice. Step 2 against step 1: almost everyone predicts an image and the answer is a stylesheet. That gap between where people look for weight and where weight is, is the whole chapter.

Checkpoint — Test Yourself

1. A product page is 713 KB, of which 597 KB is CSS and JavaScript. A colleague proposes compressing all the product photographs. What is the best case for that change, and what would you propose instead?
2. Distinguish a layout change from a template change, and give one example of each for the request "show a trust badge under the Add to Cart button on every product page".
3. This chapter's harness reports 738 KB for a category page, and a browser's developer tools report more. Give two reasons the harness is lower, and say which of them it could fix.

Chapter Summary

- A theme is a directory of Twig templates. Never edit the default: override only the files you change in a child theme, so that the directory is the list of your changes and survives upgrades.
- A layout places modules in positions on a kind of page. Moving, adding or removing an existing block is a layout change made in the admin panel, not a template edit.
- Measured: the storefront weighs 713–895 KB per page across 18–25 requests. CSS and JavaScript are about 570 KB of that *on every page* and do not vary with content — 84% of the product page.
- Images are 9–33% depending on the page, although they are what people optimise first.
- The largest single file is `bootstrap.css` at 265 KB, unminified, with no minified copy anywhere in the tree.
- The harness undercounts: it follows only assets named in the HTML, so fonts requested by CSS (248 KB in one directory) are invisible to it. Know what your instrument cannot see.
- Weight and time are not the same: a stylesheet in the head is render-blocking and stands between the customer and the first pixel.
- Prefer changes you can verify without an experiment you cannot afford to run. A small shop cannot measure a button colour and can measure 240 KB.

Review Questions

1. **(MCQ)** On the measured product page, CSS and JavaScript are what share of the weight? (a) 9%; (b) about a third; (c) 84%; (d) it depends on the catalogue size.
2. **(MCQ)** "Show the manufacturer's name under the product title" where the theme does not show it at all is: (a) a layout change; (b) a template override; (c) a module setting; (d) a database change.
3. **(Short)** Why is a child theme better than editing the default, giving two reasons that are not about upgrades.
4. **(Short)** Explain render-blocking, and why page weight and page time are different measurements.
5. **(Short)** This chapter's harness counts each unique asset once rather than once per mention. Explain what would go wrong otherwise, with a number.
6. **(Applied)** You are given a shop whose category page is 2.1 MB: HTML 40 KB, CSS 90 KB, JS 150 KB, images 1.82 MB across 40 requests. Give your optimisation order with a predicted saving for each step, and say how this differs from the order you would use on the shop in this chapter and why.
7. **(Applied)** Design a measurement that would tell you whether showing the delivery charge on the product page increases or decreases completed orders. State what you would record, how long you would need to run it, and — honestly — whether a shop taking thirty orders a day could reach a conclusion at all.

Part III

From Basket to Order

A catalogue is a brochure until somebody can buy from it. Part III is the path from a product page to a confirmed order: where the basket lives, what the checkout is really doing, and the two things that change the total after the customer thought they knew it — shipping with tax, and discounts.

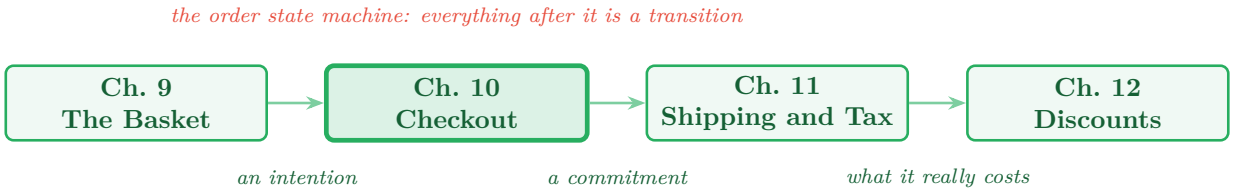


Figure P3.1: Reading path through Part III. Chapter 10 is the hinge of the whole book — an order is the first object in the course that cannot simply be deleted and redone.

Chapter 9

The Shopping Basket, and Where State Lives

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why HTTP has no memory, and **name** the three places a shop can keep what it remembers about a visitor.
- **Locate** a basket in a running shop, and **state** which store holds it, with evidence rather than with belief.
- **Measure** what a session costs in rows, bytes and queries, and **distinguish** the cost of a visitor from the cost of a customer.
- **Predict** what survives a restart, a redeployment and a volume deletion, and **verify** each prediction.
- **Judge** a session configuration — engine, expiry, cookie flags — against what the shop actually does.

Before You Start

Chapter 5: containers, volumes, and the difference between restarting a container and deleting one. Chapter 6: the tables behind the screens, and how to read a query the server actually ran rather than one you assumed it ran. From Web Technologies: HTTP requests and responses, and what a cookie is.

Key Terms in This Chapter

state • stateless protocol • cookie • session • session identifier • session engine • server-side state • client-side state • persistent cart • abandoned basket • session expiry • garbage collection • volume • cache miss

9.1 The Problem: A Protocol With No Memory

A customer puts a laptop in the basket, browses to another category, comes back, and the laptop is still there. Nothing about HTTP makes that happen.

Every HTTP request is independent. The server receives a request, answers it, and keeps nothing. The next request from the same person arrives looking exactly like a request from a stranger. If the shop is to know that these two requests belong to the same visitor, *something carried between them* must say so, and that something can only travel in the request itself — because the request is all there is.

State, Cookie, Session

State is anything the shop remembers between one request and the next: what is in the basket, who is logged in, which currency is being shown.

A **cookie** is a small named value the server asks the browser to store and send back on every subsequent request to that site. It is the only standard way to make a second request

identifiably related to a first. It lives on the customer's machine, which means the customer can read it, change it and delete it.

A **session** is state the *server* keeps, filed under a key. The browser carries only the key — the **session identifier** — in a cookie; the data stays on the server, where the customer cannot touch it.

The distinction is the whole of this chapter. A cookie is a place. A session is an arrangement that *uses* a cookie to point at a place somewhere else.

9.2 Three Places State Can Live

Given that the browser must carry something, a shop has three choices about *how much* it carries, and they trade off against each other in ways worth seeing before measuring anything.

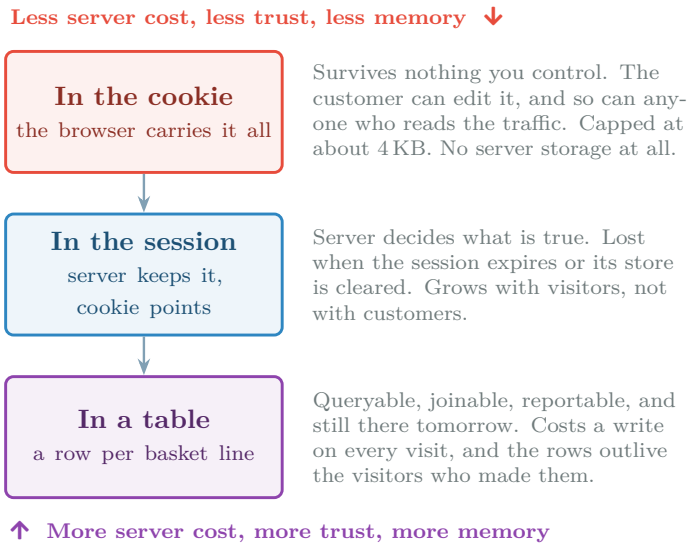


Figure 9.1: The three stores, and what each one buys. Nothing forces a shop to pick one: most use all three at once, for different things. The question worth asking of any shop is not which one it uses, but which one holds the basket.

A rule of thumb, before any measurement

Put in the **cookie** only what you would not mind the customer editing — a chosen language, a dismissed banner. Put in the **session** what must be true for this visit and need not outlive it. Put in a **table** anything you will later want to count, report on, or still have tomorrow.

Applying that rule to a basket gives an answer immediately: a shop wants to know how many baskets were abandoned last week, so the basket belongs in a table. That is an argument. The rest of this chapter is evidence.

9.3 Where This Shop Actually Puts It

Almost every tutorial on this subject says the cart is kept in the session. Before believing that of the shop in front of you, look.

The method is the one Chapter 6 arrived at the hard way: do not reason about what the code probably does, make the shop do it and read what changed. Add three products, and after each one measure all three stores at once.

Measured: Add three products; watch all three stores

Produced by `code/m09_basket.py -where` against the stack in Appendix A. "Cookie" is what the browser sends up on every request; "session" is the length of the stored session record; "cart" is rows in `oc_cart` and the bytes in them.

After adding	Cookie	Session	Cart rows	Cart bytes
(nothing)	49 B	18 B	0	0
2 × MacBook	49 B	18 B	1	53
1 × iPhone	49 B	18 B	2	106
3 × HTC Touch HD	49 B	18 B	3	159

The cookie does not move, and the session does not move. Both are constant across an empty basket and a three-line one. The entire session record is eighteen bytes:

```
{"currency":"USD"}
```

The basket is a table. It is rows in `oc_cart`, one per line, about 53 bytes each, keyed by the session identifier. The cookie carries a 26-character identifier and a currency, 49 bytes in all, and that is the whole of what the browser knows.

So the common claim is false here. It is worth knowing *why* it is so widely believed: in a simpler shop it is usually true, and the arrangement OpenCart uses — session for identity, table for the basket — is the one a shop adopts when it wants to report on baskets that were never checked out.

"The cart is in the session" — check before you repeat it

This matters beyond trivia. If you believe the basket is in the session, you will also believe:

- that lengthening the session expiry lengthens the life of a basket (here it does both, but for two different reasons, in two places);
- that switching the session engine moves the basket (it does not — see the next measurement);
- that clearing sessions clears baskets (it does not; it orphans them);
- that you cannot report on abandoned baskets without extra work (you can: they are rows, and Chapter 17 reads them).

Each of those is a decision someone will make on a Friday afternoon. The evidence took one command.

9.4 What a Session Costs

The basket is in a table, but the shop still keeps sessions, and sessions are not free. Two questions: how many are created, and what does each one cost per request?

Measured: A session is written before the visitor asks for anything

From `code/m09_basket.py -sessions`. Twenty requests made as twenty first-time visitors, then twenty made by one visitor who keeps the cookie.

20 first-time visitors	**20** new session rows
1 visitor, 20 page views	**1** new session row

The session table on this machine held **645 rows, about 35.6 KB**, accumulated entirely by the measurements in this handout — no human has ever shopped here.

Every request writes a session, including the first. A visitor who loads one page and leaves has cost a row. A crawler that ignores cookies costs one row per URL it touches, and the rows are removed on a clock rather than on departure: `config_session_expire` is **86,400 seconds**, a full day.

This is the first number in the course that scales with *traffic* rather than with *business*. The catalogue in Chapter 6 grew because the shop sold more things. This grows because more robots visited.

Measured: Database sessions against file sessions

From `code/m09_basket.py -engine`. OpenCart ships three session engines — `db`, `file` and `redis` — selected in `system/config/catalog.php`. The storefront ships configured for `db`. Both rows below are the same home page, one engine apart.

Engine	Page	Queries	On session	Cart rows	Files
`db`	89 ms	117	2	1	1
`file`	71 ms	115	0	1	14

Exactly two queries per request are the session's — one `SELECT` to read it and one `REPLACE` to write it back. Moving to files removes both and takes about 20% off the page on this machine, where the database and the web server are two containers on one host.

The basket did not move. The `Cart rows` column is 1 under both engines. The engine decides where eighteen bytes live; it has nothing to do with the basket.

Files are not free either, they are just billed elsewhere. Each session file holds those same eighteen bytes and occupies a 4 KB filesystem block — an amplification of more than two hundred. And a file session is tied to one machine's disk, so the moment the shop runs on two web servers, a customer served by the second one is a stranger. That, not speed, is why the shipped default is `db`.

Reading that 20% honestly

It was measured on one host where the database is a container a virtual network hop away. On a deployment where the database is a separate machine, removing two round trips per request would save more. On a deployment behind a load balancer, the file engine would not work at all.

A measurement tells you what happened on the machine it ran on. Appendix A names that machine for exactly this reason. Quoting 20% as a property of OpenCart would be a misuse of your own result.

9.5 Two Things the Source Does That the Screens Do Not Show

Reading `system/library/cart/cart.php` and `system/library/session.php` turns up two behaviours that no admin screen mentions and both measurements confirm.

Measured: An empty basket costs more than a full one

From `code/m09_basket.py -cache`. The same home page, with the basket holding 0, 1, 2 and 3 lines.

Basket lines	Queries	SELECT on cart	DELETE on cart
0	117	**6**	1
1	119	1	1
2	126	1	1
3	129	1	1

An empty basket is read six times; a full one is read once. The cache in `Cart::getProducts()` begins `if (!$this->data)`, which is true both when the basket has not been loaded yet *and* when it is empty. PHP cannot tell those apart that way, so an empty basket misses its own cache on every call — and a shop's commonest visitor has an empty basket.

Every storefront page deletes before it reads. The `DELETE` column is 1 on every row, including the empty basket. The `Cart` constructor removes expired guest baskets each time it is built, so a page view is a write even when the visitor only looked.

Measured: One request in five rebuilds the session table

From `code/m09_basket.py -gc`, over two runs.

25 requests	`OPTIMIZE` ran 7 (28%)	555 ms in total
60 requests	`OPTIMIZE` ran 15 (25%)	889 ms in total

`Session::__construct` registers `gc()` as a shutdown function, so garbage collection is *considered* on every request; `session_probability` and `session_divisor` make it *run* on about one in five. When it runs it deletes expired rows and then issues `OPTIMIZE TABLE`, which on InnoDB is a full table rebuild — around 60 ms here, on a 645-row table.

Both runs are printed because neither is the answer. The configured rate is one in five; 7 of 25 and 15 of 60 are what one in five looks like in small samples, and reporting only the first would have claimed 28%.

9.6 What Survives

Chapter 5 introduced volumes with the claim that they are what makes a container's data outlive the container. A basket is the smallest honest test of that claim, because you can put one somewhere and then try to destroy it three different ways.

Measured: Three ways to stop a shop, and what is left afterwards

From `code/m09_basket.py -restart -destroy`. A three-line basket is created, then each operation is applied in turn and the basket looked for again — in the tables, and in the page the same browser is shown.

Operation	Took	Cart rows	Session rows	Basket shown
— (before)	—	3	1	yes
`restart shop`	3 s	3	1	yes
`down, up -d`	10 s	3	1	yes
`down -v, up -d`	17 s	**0**	**0**	**no**

Destroying the containers destroyed nothing. `down` removes the containers; the named volumes stay, so the database files stay, so the basket stays. The browser's cookie still pointed at a session that still existed.

`-v` is the whole difference. It deletes the volumes. The shop reinstalled itself from scratch in 17 seconds — which is a good property of the stack and a bad surprise for anyone who typed it to free disk space.

Notice what the third row does *not* say. It does not say the shop broke. It came back, fast, looking perfectly healthy, with every customer's basket gone. Chapter 20 returns to this with backups, where the lesson is the same one Chapter 6 taught about the store identifier: a shop that answers 200 is not thereby a shop that is right.

9.7 The Settings That Govern Any of This

Three of the behaviours above are configurable, and two of the three are in the admin panel rather than in a file.

</> Where each decision is actually made

- **How long an abandoned basket lives.** System › Settings › Edit › Option — the session expiry, `config_session_expire`, 86,400 seconds by default. It governs both the session record and the `DELETE` in the `Cart` constructor, so shortening it discards baskets sooner.
- **Whether a basket needs an account at all.** System › Settings › Edit › Option — Guest Checkout. With it on, the basket is keyed by session alone and vanishes with the cookie. With it off, every basket belongs to a customer and follows them to another machine.
- **Where the session itself is kept.** Not in the admin panel at all, but in a file: the `session_engine` line of `system/config/catalog.php`. A setting that changes both a performance characteristic and a deployment constraint, and that can be reached only by editing a file, is worth noticing as a design choice — it is not meant to be changed by the shopkeeper.

Common Pitfalls

- **Assuming where the basket is instead of looking.** Every wrong decision in this chapter follows from that one.
- **Putting a price in the cookie.** Anything the browser carries, the customer can edit. `oc_cart` has a `price` column and the shop recomputes rather than trusting it; your own extensions must do the same.
- **Lengthening session expiry to "keep baskets"** without noticing that it also keeps every robot's session row for the same period.
- **Clearing `oc_session` to tidy up.** It does not delete baskets; it strips them of the identifier that reaches them, leaving rows nothing will ever read and nothing will ever remove.
- **Typing `docker compose down -v` to free space.** It is the one command in this chapter that cannot be undone.
- **Reading a timing from one machine as a property of the software.** The 20% above is a fact about this host.

Try It Yourself: Find the basket, then try to destroy it

```
# --- 1. Predict, in writing, before running anything -----------------
# Where is the basket: cookie, session, or a table? Commit to one.
```

```
# --- 2. Look ------------------------------------------------------
cd code
python m09_basket.py --where

# --- 3. Read the session record yourself --------------------------
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT session_id, LENGTH(data), data FROM oc_session LIMIT 3;"

# --- 4. Find your own basket in the table -------------------------
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT * FROM oc_cart;"

# --- 5. Move the session to files, and check the basket -----------
python m09_basket.py --engine
# Did the basket move? Write down why not, in one sentence.

# --- 6. Count what an empty basket costs --------------------------
python m09_basket.py --cache
# Explain the first row to someone who has not read the source.

# --- 7. Try to destroy it, in increasing order of violence --------
python m09_basket.py --restart
# Predict each row before you read it.

# --- 8. Now the one that works ------------------------------------
# Only when you are ready to reinstall the shop:
python m09_basket.py --restart --destroy

# --- 9. Change the expiry and show that it did something ----------
# Admin: System > Settings > Edit > Option. Set session expiry to 60
# seconds. Add a product, wait, reload. Record what you see and which
# of the two mechanisms in this chapter produced it.
```

What to notice. Step 1 against step 2 is the point of the lab. Keep your written prediction: the gap between what you expected and what the shop does is the only part of this exercise you will still remember in a year.

Checkpoint — Test Yourself

1. A colleague says "move the sessions to Redis and the basket will be faster". Using this chapter's measurements, say what would and would not change, and give the number that settles it.
2. The session table holds 645 rows on a shop that has never had a customer. Explain where they came from, why they are not removed when the visitor leaves, and what you would measure before deciding to shorten the expiry.
3. `docker compose down` left a three-line basket intact and `docker compose down -v` did not. Explain the difference in terms of what each command removes, and name one other thing in your shop that the second command destroys.

Chapter Summary

- HTTP keeps nothing between requests. A cookie is the only standard way to make the second request identifiably related to the first.
- A cookie is a place on the customer's machine; a session is server-side state that *uses* a cookie to point at itself. Confusing the two confuses who can change what.
- Measured: adding three products left the cookie at 49 bytes and the session at 18 bytes, and added three rows to `oc_cart`. The basket in this shop is a table, not a session.

- Measured: 20 first-time visitors created 20 session rows; one visitor making 20 page views created one. Sessions scale with traffic, not with trade.
- Measured: the `db` engine costs exactly two queries per request; the `file` engine costs none and took about 20% off the page on this host — while moving no part of the basket, and while making a second web server impossible.
- Measured: an empty basket is read six times per page and a full one once, because the cache test cannot distinguish "empty" from "not loaded". Every storefront page also deletes from `oc_cart` before reading it.
- Measured: about one request in five rebuilds the session table with `OPTIMIZE TABLE`, at roughly 60 ms.
- Measured: `restart` and `down`/`up` left the basket untouched; `down -v` destroyed it and the shop reinstalled itself in 17 seconds looking perfectly healthy.
- Where state lives decides what survives, what can be reported on, and who is able to tamper with it. It is a design decision, not a detail.

Review Questions

1. **(MCQ)** In the measured shop, adding a product to the basket changes: (a) the cookie; (b) the session record; (c) rows in a table; (d) all three.

2. **(MCQ)** Switching `session_engine` from `db` to `file` moved: (a) the basket; (b) the session record only; (c) both; (d) neither.

3. **(Short)** Explain why a session identifier must be unguessable, and what an attacker who obtained one could do in this shop.

4. **(Short)** Give two things that belong in a cookie and two that must never go in one, with your reason in each case.

5. **(Short)** An empty basket costs six cart queries and a full one costs one. Explain the cause, and propose a one-line change to the condition that would fix it.

6. **(Applied)** A shop reports 400,000 session rows and 12 orders a day. Work out what is producing the rows, state what you would measure to confirm it, and give two changes you would make — one to configuration and one to the shop's robots policy.

7. **(Applied)** You are asked to make baskets survive for thirty days so that customers can come back to them. Say exactly what you would change, what each change costs, what it would break for a guest with no account, and how you would measure whether it increased completed orders at all.

Chapter 10

The Checkout and the Order State Machine

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the steps of a checkout in order, and **state** what each one validates and what it saves.
- **Explain** why an order is a state machine rather than a record, and **read** a shop's order statuses as the states of one.
- **Measure** the cost of each checkout step in time, queries and writes, and **identify** where an order is actually created.
- **Diagnose** a checkout that fails silently, using the shop's own responses rather than its screens.
- **Judge** where a checkout can safely be shortened, and where shortening it would lose something the shop needs.

Before You Start

Chapter 9: where the basket lives, and that it is rows keyed by a session identifier. Chapter 6: reading the queries a page actually ran. Chapter 5's container, with a shop you can place a test order in.

Key Terms in This Chapter

checkout • checkout step • guest checkout • order • order status • state machine • order history • confirmation • idempotency • abandonment • conversion funnel • write amplification • silent failure

10.1 A Checkout Is a Sequence, Not a Page

The basket in Chapter 9 is a list of rows. An **order** is something else entirely: a commitment, by a named person, to pay a stated amount for stated goods delivered to a stated address. Turning the first into the second is what a checkout does, and it cannot be done in one step because each piece of information constrains the next — the delivery address decides which shipping methods exist, the shipping method decides the total, and the total decides what the customer is agreeing to.

Order, Order Status, Order History

An **order** is a row recording an intention to buy, together with rows for the products, the totals and the addresses as they were *at the moment of ordering*. It does not point at the catalogue; it copies from it, because the price of a product next month must not change what a customer agreed to pay today.

An **order status** is the state the order is in: pending, processing, shipped, complete, refunded. The set of statuses is the shop's, not the standard's.

The **order history** is the list of status changes, each with a date and an optional comment. The status column says where the order is; the history says how it got there, and it is the only part a dispute can be settled from.

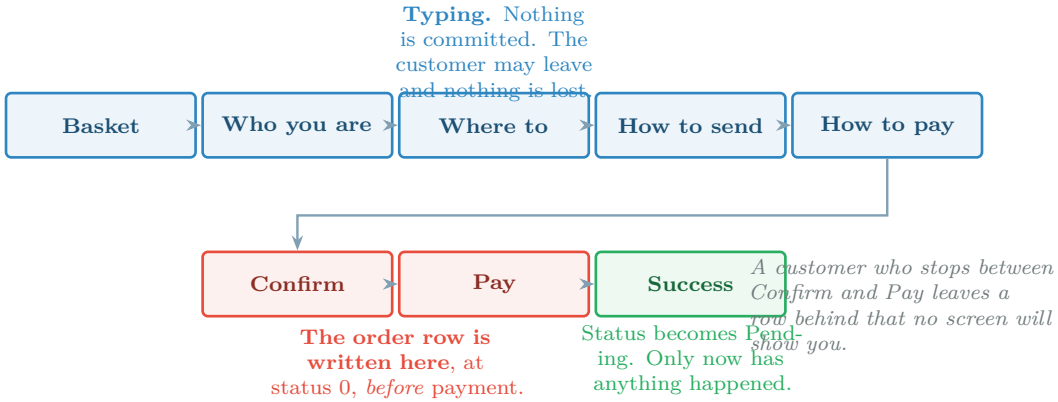


Figure 10.1: The guest checkout as the shop actually runs it. The first five steps collect; the last three commit. The gap between the order row appearing and the payment confirming it is where orders go missing.

10.2 The Fault That Stops Every Order

Before any of that could be measured, the checkout had to be made to work, and on a freshly installed shop it does not. This is worth the space because the way the failure presents itself is the lesson, not the fault itself.

A stock install cannot complete a guest checkout

Driving the sequence by hand against the shop built in Chapter 5, every step succeeded until the shipping quote, which answered:

```
{"error":"Shipping address required!"}
```

— although a shipping address had been supplied and accepted one step earlier, with `{"success":"... has been saved!"}`.

The cause. The guest registration step copies the billing address to the shipping address, but only inside a branch guarded by the setting `config_checkout_payment_address`. That setting is read by **ten** controllers in `catalog/`, and appears as a checkbox in the admin settings form — but **the installer never writes it**. An absent setting reads as false, the branch never runs, no shipping address is ever stored, and the quote has nothing to work from.

The fix is to open **System › Settings › Edit › Option**, tick **Payment Address required** and save, which writes every field on that form, including the missing one. One checkbox.

What makes this worth a page. No page returned an error code. No log recorded a failure. Every screen rendered. A shopkeeper would see customers reaching the shipping step and leaving, and would reasonably conclude that the shipping charges were too high. This is the same shape of fault as the store identifier in Chapter 6: **the system did not break, it answered a different question**, and only driving it end to end and reading the replies found it.

How to find a fault of this shape

1. **Read the reply, not the page.** Each step answers with JSON. The error was in the reply all along; the browser merely rendered it in a place nobody looks.
2. **Trust the sequence, not the screens.** Drive the steps in order from a script. The step that first disagrees with the previous one is the step to read.
3. **Then read the source for that step only.** Not before — the measurement tells you which of four thousand files to open.
4. **Change one thing and repeat.** Setting the one flag made the whole sequence complete, which is what turns a hypothesis into a cause.

10.3 What Each Step Costs

With the checkout working, the sequence can be measured. The question put to it is a specific one: *is the last step the expensive one?* It is the common belief, and it is worth settling with numbers.

Measured: A complete guest checkout, step by step

Produced by `code/m10_checkout.py -repeat 5` against the stack in Appendix A: medians of five complete checkouts, each buying two products. Queries and writes are counted from MariaDB's slow query log, not estimated.

Step	Time	Queries	Writes	DB time	Share
fill basket	55 ms	57	6	23 ms	11%
register (guest)	30 ms	41	3	9 ms	6%
shipping address*	21 ms	32	2	6 ms	4%
shipping method	56 ms	67	4	22 ms	11%
payment method	54 ms	74	4	17 ms	10%
confirm	**149 ms**	**125**	**25**	93 ms	**28%**
pay	64 ms	102	8	25 ms	12%
success	94 ms	109	3	37 ms	18%
Total	**523 ms**	**607**	**55**	—	

* *A guest never uses this step.* It redirects to the login page, because its guard requires a logged-in customer. The round trip costs 21 ms and 32 queries and achieves nothing; a guest's addresses were both set by the registration step.

The common belief is wrong as stated. No single step costs more than the rest together. `confirm` is the most expensive at 28%, not 50%.

But split the sequence where the customer stops typing:

	Time	Queries	Writes
Typing (5 steps)	217 ms	271	19
Ordering (3 steps)	307 ms	336	**36**

The three steps after the last keystroke cost more than the five before it — 40% more time, and nearly twice the writes. That is the defensible version of the claim.

The sharpest number in the table is the writes column. `confirm` performs **25 of the 55 writes** in the entire checkout, against two to six for every other step. It is not slow because it is complicated; it is slow because it is the step that *writes the order*: the order row, the product rows, the totals, the addresses, all copied out of the basket and the session so that they can never change again.

For scale: one checkout is **607 queries**. A category page in Chapter 6 was 144. A single customer buying two items costs the database more than four people browsing.

10.4 An Order Is a State Machine

The reason `confirm` and `pay` are separate steps — and the reason the gap between them matters — is that an order has a life after it is created, and that life is a sequence of states.

</> The fourteen states this shop defines

System › Localisation › Order Statuses lists them. They are not a standard; they are this shop's vocabulary, and a shop may add its own.

1 Pending	8 Denied	13 Chargeback
2 Processing	9 Canceled Reversal	14 Expired
3 Shipped	10 Failed	15 Processed
5 Complete	11 Refunded	16 Voided
7 Canceled	12 Reversed	

Three things are worth noticing. **There is no 4 and no 6** — the numbers are identifiers, not an order. **Several are about money moving backwards** (Refunded, Reversed, Chargeback, Canceled Reversal), which is Chapter 18's subject and a reminder that selling is the easy half. **And there is no zero.**

Measured: Status 0, and the orders nobody can see

From `code/m10_checkout.py -abandon`, which runs the checkout as far as `confirm` and then stops, exactly as a customer who changes their mind at the payment button does.

Orders at status 0 before	0
Orders at status 0 after	**1**
Order row written?	**yes**, total $854.40
Visible in the admin order list?	**no**

The order row is written at `confirm`, before any payment, and left at status 0. Status 0 is not one of the fourteen. The admin order list filters it out. Nothing will ever remove it.

This is not a bug; it is a deliberate design, and the reason for it is sound. The order number has to exist before the customer is sent to a payment provider, because the provider must be told what it is being paid for, and the answer that comes back has to find its way to something. The cost of that design is a table containing rows for orders that never happened.

Why it matters to you: a shop owner reporting "we have orders in the database that aren't in the orders list" has not found a fault. They have found their abandonment rate. Chapter 17 turns that into a number worth watching, and the ratio of status-0 rows to real orders is one of the most useful figures a small shop can compute from data it already has.

Two requests, one order — the problem confirm cannot solve alone

If the customer's connection drops after `confirm` is sent but before the reply arrives, their browser may retry. If the shop treats the second request as a new one, the customer is charged twice.

The property that prevents this is **idempotency**: performing an operation twice has the same effect as performing it once. OpenCart approaches it by keeping the order identifier in the session and updating that order rather than creating another, which is why `confirm` can be called repeatedly in a session without multiplying orders — and why the order identifier must be established *before* payment, not after.

It is not a complete answer, and Chapter 14 returns to it where it is hardest: when the second request comes not from the customer but from the payment provider, minutes later, over a connection that is not the customer's at all.

10.5 Where a Checkout Can Be Shortened, and Where It Cannot

Every step is a place to lose a customer, so the pressure to remove steps is constant. The measurements say which removals are free.

Four changes, ranked by what the evidence supports

1. **Remove the guest's shipping-address round trip.** It is measured at 21 ms and 32 queries, it redirects to a login page the guest cannot use, and the data it would set is already set. Pure waste, and the only change here with no trade-off.
2. **Merge the shipping and payment method steps.** 110 ms and 141 queries between them, and both are selections from short lists. The cost is that a shipping choice can change the total that the payment step displays, so they must then be recomputed together.
3. **Skip the address entirely for digital goods.** There is nothing to deliver. This is a real saving and the platform supports it — `hasShipping()` is already consulted.
4. **Remove the confirmation screen.** Tempting — 28% of the time. **Do not.** It is the last point at which the customer sees what they are agreeing to pay, and removing it moves every mistake from before the sale to after it, where it costs a refund, a complaint and Chapter 22's attention rather than a click.

Common Pitfalls

- **Testing a checkout only through the browser.** The fault at the start of this chapter was invisible there and obvious in the replies.
- **Assuming the slow step is the last step.** It is the most expensive single step, at 28%, which is not the same claim.
- **Counting orders without filtering status 0,** and reporting revenue that includes abandoned baskets.

- **Deleting status-0 rows to tidy up.** They are the audit trail of every customer who reached the payment button, and a payment provider's late reply may still be looking for one of them.
- **Pointing an order's product rows at the catalogue** instead of copying names and prices into the order. The catalogue changes; what a customer agreed to must not.
- **Removing the confirmation screen to raise conversion.** It raises conversion and refunds together.

Try It Yourself: Drive a checkout, then break it on purpose

```
# --- 1. Make the shop able to take an order at all -------------------
# Admin: System > Settings > Edit > Option. Tick "Payment Address
# required" and save. Write down what you expect that to change.

# --- 2. Place one order by hand, in the browser ----------------------
# Note how many pages you visited and roughly how long it took.

# --- 3. Now measure it -----------------------------------------------
cd code
python m10_checkout.py --repeat 5

# --- 4. Predict before you look --------------------------------------
# Which step has the most WRITES? Most people say "success".

# --- 5. Watch an order appear before it is paid for ------------------
python m10_checkout.py --abandon

# --- 6. Look for it in the admin panel -------------------------------
# Admin: Sales > Orders. It is not there. Now look in the table:
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT order_id, order_status_id, total FROM oc_order;"

# --- 7. Read the state machine off your own data ---------------------
python m10_checkout.py --states

# --- 8. Move an order by hand through three states -------------------
# Admin: Sales > Orders > View > History. Pending -> Processing ->
# Shipped. Then read oc_order_history and say what the status column
# alone would not have told you.

# --- 9. Clean up -----------------------------------------------------
python m10_checkout.py --clean
```

What to notice. Step 1 is the whole of Chapter 4's argument in one checkbox: a platform you did not write will have defects, and the skill that matters is finding them from the outside, not avoiding platforms.

Checkpoint — Test Yourself

1. A shop reports that customers abandon at the shipping step. Describe, in order, the three things you would do before accepting that the shipping charges are too high — and say what the first chapter measurement would have told you.
2. `confirm` is 28% of the checkout's time but 45% of its writes. Explain why those two shares differ, and what that implies for which step you would optimise if the shop became ten times busier.
3. Your client asks you to delete every order at status 0 because they "clutter the database". Give two distinct reasons to refuse, and one thing you would build instead.

Chapter Summary

- A checkout is a sequence because each answer constrains the next. An order copies what it needs out of the catalogue rather than pointing at it, so that later price changes cannot rewrite an agreement.
- Measured: a stock install *cannot complete a guest checkout*. The setting `config_checkout_payment_address` is read by ten controllers and written by none, so no shipping address is ever stored. Every page still returned 200.
- The way to find a fault of that shape is to drive the steps from a script and read the replies, then open the source for the one step that disagreed.
- Measured: 523 ms and 607 queries for one complete checkout — more than four category pages.
- Measured: no single step costs more than the rest together, but the three steps after the last keystroke cost 307 ms against 217 ms and nearly twice the writes.
- Measured: `confirm` performs 25 of the checkout's 55 writes, because it is the step that creates the order.
- Measured: abandoning at the payment button leaves an order row at status 0 — a status the shop does not define, that the admin list hides, and that nothing removes. It is the abandonment rate, not a fault.
- The order history, not the status column, is what a dispute is settled from.
- A step may be removed when the evidence says it does nothing. The confirmation screen is not that step.

Review Questions

1. **(MCQ)** In the measured checkout, which step performs the most database writes? (a) fill basket; (b) payment method; (c) confirm; (d) success.

2. **(MCQ)** An order sitting at status 0 means: (a) the order failed; (b) the order was refunded; (c) the order row was created but payment was never confirmed; (d) the status table is corrupt.

3. **(Short)** Explain why an order copies the product name and price instead of referring to the catalogue row, and give one thing that would go wrong if it did not.

4. **(Short)** Define idempotency, and explain why a checkout needs it in terms of a specific thing that would otherwise happen to a customer.

5. **(Short)** The stock shop answered every request with 200 while being unable to take a single order. Explain what this says about monitoring a shop by availability.

6. **(Applied)** You are given a shop with 1,200 rows in `oc_order`, of which 820 are at status 0. Say what you can and cannot conclude from that ratio, what two further figures you would want, and what you would recommend — with the reasoning for each step shown.

7. **(Applied)** Design a measurement that would tell you at which checkout step customers actually leave. State what you would record, where in the sequence you would record it, how you would avoid counting the same visitor twice, and how long you would need to run it before the answer meant anything for a shop taking thirty orders a day.

Chapter 11

Shipping, Tax and Zones

Learning Outcomes

By the end of this chapter you will be able to:

- **Define** a geo zone, and **explain** why both shipping and tax are expressed in terms of one.
- **Distinguish** a tax rate, a tax class and a tax rule, and **state** what each one decides.
- **Compute** an order total by hand, including tax on shipping, and **reconcile** it against what the shop recorded.
- **Choose** a shipping method for a given shop, and **justify** the choice against what it costs to operate.
- **Apply** Pakistani sales tax to an online sale, and **identify** what you would have to verify before relying on it.

Before You Start

Chapter 10: the checkout sequence, and that an order copies its totals rather than recomputing them. A working checkout — the setting repaired in Chapter 10 must be in place or no order can be completed.

Key Terms in This Chapter

geo zone • zone • tax rate • tax class • tax rule • tax base • destination principle • flat rate • weight-based shipping • handling fee • order total line • sort order • GST • inclusive and exclusive pricing

11.1 Everything Here Depends on Where the Customer Is

A shop quotes one price for a product and then has to answer two questions that the price does not settle: *what does it cost to get this to you*, and *what tax is due on this sale*. Both answers depend on where "you" are, and neither can be stored on the product.

So the platform needs a way to talk about places. That is a **geo zone**.

Zone, Geo Zone

A **zone** is an administrative subdivision of a country — a province, a state, a county. Pakistan's zones are Punjab, Sindh, Khyber Pakhtunkhwa, Balochistan, and the territories. The platform ships a list of them for every country and you do not maintain it.

A **geo zone** is a *named set* of (country, zone) pairs that you do maintain, created because you want to treat those places alike. "Punjab and Islamabad", "everywhere except Gilgit-Baltistan", "the whole of Pakistan" are each a geo zone if a rule of yours applies to exactly that set.

The important property: a geo zone has no meaning of its own. It is a label you attach rules to. The same shop may have one geo zone for shipping and a different one for tax,

overlapping in any way, because the places where delivery costs the same are not the places where tax is the same.

</> The two geo zones the demonstration shop ships with

System › Localisation › Geo Zones shows them, and they are a good illustration precisely because they are different shapes.

UK Shipping	One row: country 222, zone 0. Zone 0 means *every* zone — the whole country in a single entry.
UK VAT Zone	Many rows: country 222 and one zone each. The same country, enumerated region by region.

Why the difference? Because delivery is priced for the country as a whole, while the tax zone was built to allow regions to be excluded one at a time. **Reading a geo zone's shape tells you what its author expected to change.**

11.2 Tax: Three Things, Not One

Most people arrive expecting a tax setting. There are three, they sit in a hierarchy, and conflating them is the commonest source of a wrong total.

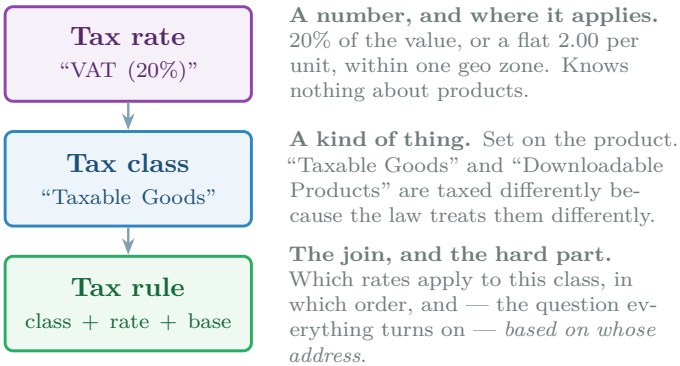


Figure 11.1: Tax in three layers. A rate is a number in a place; a class is a kind of product; a rule says which rates a class attracts, in what order, and which of the three addresses on the order decides.

The `based` column, which is the whole of cross-border tax

Every tax rule carries a **tax base**: `store`, `payment` or `shipping`. It names which address decides whether the rule applies.

- `store` — the shop's own address. Tax is the same for everyone. Simple, and correct only for a shop that sells domestically.
- `payment` — the billing address. Used where the law follows the buyer's financial residence.
- `shipping` — the delivery address. The **destination principle**: tax belongs where the goods are consumed. This is the usual answer for physical goods, and it is what the demonstration shop uses for its `Taxable Goods` rules.

One setting, three answers, and a shop that picks the wrong one is not slightly wrong — it is collecting the wrong tax from the wrong people and remitting it to the wrong authority. **System › Localisation › Taxes › Tax Rules** is where it is chosen, and it is the single most consequential dropdown in the admin panel.

11.3 An Order Total, Computed by Hand

The reliable way to understand a totals engine is to take one real order apart. This is the order placed by Chapter 10's harness: one MacBook at $500.00 and two iPhones at $101.00, delivered within the shop's own country.

Worked Example:

What the shop recorded. The rows of `oc_order_total`, in their own sort order:

Sub-Total	702.00	`sort_order` 1
Flat Shipping Rate	5.00	`sort_order` 3
Eco Tax (-2.00)	6.00	`sort_order` 5
VAT (20%)	141.40	`sort_order` 5
Total	**854.40**	`sort_order` 9

Step 1 — the sub-total. $500.00 + (2 \times 101.00) = 702.00$. Goods only; nothing else has been decided yet.

Step 2 — shipping. The flat rate is 5.00 regardless of what is in the basket. It is added as its own line, *before* tax, which matters for step 4.

Step 3 — the fixed-amount tax. "Eco Tax" is of type `F`: a flat 2.00 per unit, not a percentage. Three units were bought, so $3 \times 2.00 = 6.00$.

Step 4 — the percentage tax, and the part everyone gets wrong. 20% of *what*? Not of the sub-total:

$$0.20 \times 702.00 = 140.40 \neq 141.40$$

The difference is exactly 1.00, which is 20% of the 5.00 shipping charge:

$$0.20 \times (702.00 + 5.00) = 0.20 \times 707.00 = \mathbf{141.40}$$

Shipping is taxable, and the eco tax is not taxed again. Both of those are rule decisions, not arithmetic, and both would be different in another jurisdiction.

Step 5 — the total.

$$702.00 + 5.00 + 6.00 + 141.40 = \mathbf{854.40}$$

which is what the shop stored.

Two traps inside that one order

The `tax` column on an order line is *per unit*. `oc_order_product` recorded the iPhone line as quantity 2, total 202.00, tax **22.20**. That 22.20 is not the tax on the line; it is the tax on *one* iPhone — 20.20 of VAT plus 2.00 of eco tax. The line's tax is 44.40. A report that sums the `tax` column next to the `total` column adds a per-unit figure to a per-line figure and is wrong by a factor that changes with every basket.

The demonstration data is misleadingly named. "Eco Tax (-2.00)" reads as a two-unit *discount*. It is stored as rate +2.0000 and it **adds** 6.00 to this order. The name is the only thing negative about it. When you inherit a shop, the names in its tax table are the previous administrator's notes, not evidence.

11.4 Choosing How to Charge for Delivery

Five shipping methods ship with the platform. The choice is less about pricing than about what each one obliges you to maintain.

Method	Charges	What it obliges you to keep true
`flat`	One amount per order	Nothing. The cheapest to operate and the easiest to lose money on.
`item`	An amount per item	Nothing, but it punishes multi-item baskets, which is the opposite of what a shop wants.
`weight`	By total basket weight, in bands	**A weight on every product**, forever, including every product anyone adds later.
`free`	Nothing, above a threshold	A threshold chosen from your margin rather than from a competitor's.
`pickup`	Nothing; the customer collects	A real address and real opening hours.

How to choose, in one question

Ask: *what will this method be wrong about, and how often?*

A flat rate is wrong about every unusually heavy or distant order, and if those are 5% of your orders it is a rounding error you absorb. Weight-based shipping is right about every order and wrong whenever somebody adds a product and leaves the weight at zero — which is a defect that produces free delivery silently, and which Chapter 17 will not show you unless you ask it to.

The accurate method with a maintenance burden is worse than the approximate method with none, until the shop is big enough to carry the burden. That is the same argument Chapter 4 made about building versus extending, applied one level down.

11.5 Sales Tax on an Online Sale in Pakistan

Everything above is machinery. What a graduate working here will actually have to configure is Pakistani sales tax, and the machinery maps onto it directly.

Configuring a Pakistani shop

1. **Geo zone.** Create one named **Pakistan**, containing country Pakistan with zone 0 — every province. Sales tax on goods does not vary by province, so a single entry is correct and a list of provinces would be four more things to maintain.
2. **Tax rate.** One rate, type `P` (percentage), at the standard rate of **sales tax on goods** in force for the tax year. At the time of writing that standard rate is **18%**, but it has moved twice in recent years and moves with each Finance Act.
3. **Tax class.** **Taxable Goods**, applied to every physical product. Items on the Sixth Schedule are exempt and need a class of their own rather than a rate of zero, so that the exemption is visible in the data.
4. **Tax rule.** Base it on the **shipping** address. Goods are consumed where they are delivered.

5. **Display.** Decide whether displayed prices include tax. In Pakistan retail prices are customarily quoted inclusive, which means **System › Settings › Edit › Option** must be set to display prices with tax, or every product page will understate the price a customer pays.

What this section is not

It is a mapping from the platform's concepts to Pakistani tax concepts. It is **not** tax advice, and three things about it will go out of date:

- **The rate.** Verify the standard rate against the current Finance Act before configuring any shop. A handout printed in 2026 cannot be the authority for a rate set annually.
- **Services.** Sales tax on *services* is provincial — Sindh, Punjab, Khyber Pakhtunkhwa and Balochistan each levy their own at their own rate. A shop that sells only goods can ignore this; a shop that sells a service alongside goods cannot, and then the single geo zone above is no longer enough.
- **Marketplace obligations.** Where a shop sells through an online marketplace, the marketplace may carry withholding and reporting duties that a self-hosted shop does not. Chapter 22 returns to this with the E-Commerce Policy and FBR's registration requirements.

The professional habit this chapter is really teaching: configure the machinery so that a rate change is one field, and record where the number came from. A shop whose tax rate is hard-coded in three templates is a shop that will be wrong for a year after the next budget.

Common Pitfalls

- **Assuming tax applies to goods only.** It applied to shipping in the worked example, and 1.00 of the 141.40 was nobody's product.
- **Summing `oc_order_product.tax` as a line total.** It is per unit.
- **Trusting the names in an inherited tax table.** "Eco Tax (-2.00)" adds money.
- **Picking `store` as the tax base because it is first in the list.** It is correct only for a shop that never delivers anywhere else, and it is wrong silently.
- **Choosing weight-based shipping without an operational rule for entering weights.** The failure is free delivery, and it is invisible.
- **Changing a tax rate and expecting old orders to change.** They will not — an order copied its totals (Chapter 10), and that is the correct behaviour, not a bug.
- **Testing tax with the shop's own address.** Many rules only differ for a destination you have never tried.

Try It Yourself: Make your shop charge Pakistani tax, and prove the total

```
# --- 1. Look at what you have before changing it ---------------------
cd code
docker compose exec db mariadb -ushop -pshoppw opencart -e \
  "SELECT * FROM oc_geo_zone; SELECT * FROM oc_tax_rate;"

# --- 2. Place one order and take its total apart ---------------------
python m10_checkout.py --repeat 1
docker compose exec db mariadb -ushop -pshoppw opencart -e \
  "SELECT code,title,value FROM oc_order_total \
   WHERE order_id=(SELECT MAX(order_id) FROM oc_order) \
   ORDER BY sort_order;"
# Reproduce every line with a calculator before reading on.
```

```
# --- 3. Find the 1.00 ------------------------------------------------
# The VAT line is not 20% of the sub-total. Account for the difference
# exactly, and say which rule decision caused it.

# --- 4. Build a Pakistan geo zone ------------------------------------
# Admin: System > Localisation > Geo Zones > Add New.
# Pakistan, zone 0. One row.

# --- 5. Add the rate, the class and the rule -------------------------
# System > Localisation > Taxes > Tax Rates: percentage, current rate.
# System > Localisation > Taxes > Tax Classes: Taxable Goods.
# Base the rule on the SHIPPING address, and say why.

# --- 6. Disable the UK rules and order again -------------------------
python m10_checkout.py --repeat 1
# Predict the new total first. Then check the stored rows.

# --- 7. Break it deliberately ----------------------------------------
# Change the rule's base to "store" and order to the same address.
# Then order to a different country. Which one reveals the mistake?

# --- 8. Write the operating note -------------------------------------
# Five lines: where the rate came from, who changes it, when it is
# checked, and what else must change when it does.

# --- 9. Clean up -----------------------------------------------------
python m10_checkout.py --clean
```

What to notice. Step 7 is the lesson. A wrong tax base is invisible for every order delivered to the place you tested from, which is every order you will place while developing.

Checkpoint — Test Yourself

1. An order shows a sub-total of 702.00, shipping of 5.00 and a 20% VAT line of 141.40. Show where the 141.40 comes from, and state the two rule decisions the arithmetic reveals.
2. A shop in Lahore sells to customers across Pakistan and occasionally to the UAE. Its tax rule is based on `store`. Describe precisely what is wrong, which orders are affected, why nobody has noticed, and what you would change.
3. A colleague proposes weight-based shipping for a shop with 4,000 products, 600 of which have a weight of zero. Give the failure mode in terms of money, say why no error appears anywhere, and give one change that would make the failure visible.

Chapter Summary

- A geo zone is a named set of (country, zone) pairs. It has no meaning of its own; it is a label that shipping methods and tax rules attach to, and a shop's shipping zones need not match its tax zones.
- Tax is three things: a *rate* is a number in a place, a *class* is a kind of product, and a *rule* joins them and chooses which address decides.
- The `based` column — store, payment or shipping — is the single most consequential setting in the tax machinery. For physical goods the destination principle makes `shipping` the usual answer.
- Worked: 702.00 of goods + 5.00 shipping + 6.00 fixed eco tax + 141.40 VAT = 854.40, where the VAT is 20% of goods *plus shipping*, and the 1.00 difference is the whole lesson.

- `oc_order_product.tax` is per unit, not per line. Summing it beside `total` is wrong by the quantity.
- Shipping methods differ less in what they charge than in what they oblige you to keep true. Weight-based shipping requires a weight on every product forever, and fails as silent free delivery.
- For a Pakistani shop: one geo zone with zone 0, one percentage rate verified against the current Finance Act, a Taxable Goods class, and a rule based on the shipping address.
- Changing a rate changes future orders only. Orders copy their totals, and that is correct.

Review Questions

1. **(MCQ)** A tax rule's `based` column set to `shipping` means tax is decided by: (a) the shop's address; (b) the billing address; (c) the delivery address; (d) the shipping method chosen.

2. **(MCQ)** In the worked order, the VAT line exceeded 20% of the sub-total by exactly 1.00 because: (a) rounding; (b) shipping is taxable; (c) the eco tax was taxed; (d) there were three units.

3. **(Short)** Explain the difference between a tax rate and a tax class, with one example of two products that share a rate but not a class.

4. **(Short)** Why does a shop ever need more than one geo zone? Give a concrete case where the shipping zone and the tax zone must differ.

5. **(Short)** Explain why exempt products should be given their own tax class rather than a tax rate of zero.

6. **(Applied)** A shop quotes prices inclusive of tax and then changes the tax rate from 17% to 18%. Say exactly what must change in the configuration, what happens to existing orders, what happens to displayed prices, and what the shop must decide about who absorbs the 1%.

7. **(Applied)** Design the shipping configuration for a Bahawalpur shop selling 200 products weighing between 50 g and 12 kg, delivering nationwide, with free delivery intended above Rs 5,000. State your methods, your zones, your thresholds, what each one obliges somebody to maintain, and how you would detect the day it starts losing money.

Chapter 12

Discounts, Vouchers and Referrals

Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** the four price-reduction devices a storefront offers, and **choose** the right one for a stated objective.
- **Explain** why order totals are an ordered pipeline, and **predict** what moving a line in it does to the customer's bill.
- **Compute** the margin consequence of a discount, and **state** the extra volume needed to pay for it.
- **Describe** how a referral is tracked and paid, and **identify** what can be claimed twice.
- **Report** honestly on a feature a prescribed outline names and a platform does not provide.

Before You Start

Chapter 11: tax classes, rates and rules, and the worked order total. Chapter 10: the checkout sequence, and that an order copies its totals. A working checkout, so that discounts can be observed on real orders.

Key Terms in This Chapter

discount • quantity break • special price • coupon • coupon code • gift voucher • store credit • reward points • affiliate • referral • commission • tracking code • totals pipeline • sort order • gross margin

12.1 Four Devices, and What Each One Is For

"Give the customer money off" is one sentence and four different mechanisms, which differ in *who qualifies* and *when the shop finds out*.

Quantity break, Special, Coupon, Credit

A **quantity break** reduces the unit price above a threshold — buy ten, pay less each. It is attached to the product, applies to everyone, and the customer learns of it by filling the basket.

A **special** is a dated price that replaces the usual one for a period, for a customer group. Also attached to the product; also automatic; visible on the product page as a struck-through old price.

A **coupon** is a code the customer must type. It is attached to nothing — it is its own object, with its own dates, its own usage limits and its own restriction to particular products or categories. The code is the point: it lets the shop give a discount *to the people it gave the code to*.

Store credit is a balance on a customer's account that is spent like money. It is not a discount at all; it is a liability the shop already owes.

Which to reach for

- **To move stock** — a special. It needs no action from the customer and no code to leak.
- **To reward larger baskets** — a quantity break, which pays for itself in picking and delivery.
- **To measure a campaign** — a coupon, and *a different code per channel*. The code is the only part of a discount that tells you where the customer came from.
- **To settle a complaint** — store credit, which costs the shop its margin rather than its price list, and brings the customer back.

The mistake is to use a coupon for everything. A public code is a price cut with extra steps, and it teaches customers to search for a code before buying — which is a cost that arrives later and never goes away.

12.2 The Clause the Platform Does Not Implement

HEC's outline names "Discounts, *Vouchers*, and Referrals". A gift voucher — a code bought by one person as a stored-value instrument and spent by another — is a distinct thing from a coupon, and the shop in this course does not have one.

There are no gift vouchers in this platform

Checked directly rather than assumed:

- **No tables.** `SHOW TABLES LIKE '%voucher%'` returns nothing. The database has 159 tables and not one of them is a voucher.
- **No totals extension.** The nine shipped are `sub_total`, `shipping`, `coupon`, `tax`, `credit`, `reward`, `handling`, `low_order_fee` and `total`. No voucher.
- **One trace remains:** a demonstration image called `gift-voucher-birthday.jpg`. The feature existed in version 3 and was removed; the picture was not.

Why this is in the handout rather than quietly skipped. HEC's clause list is the contents page of a book published in 2010 (see the Course Specification), when stored-value vouchers were a standard cart feature. The outline has not been revised; the software has. Stating that plainly is more useful to you than either pretending the feature exists or pretending the clause does not.

What to do if a client asks for vouchers. Store credit (`oc_customer_transaction` and the `credit` total) already provides the spending half: a balance that draws down across orders. What is missing is the *buying* half — selling a voucher as a product, issuing a code, and letting a different person redeem it. That is a module, and Chapter 19 is where you would write it. It is the single most realistic piece of commissioned work in this course.

12.3 Totals Are a Pipeline, and Position Decides the Bill

Chapter 11 computed an order total by hand and found that the VAT line included the shipping charge. That was not arithmetic; it was *ordering*. Every total line is produced by a separate extension with a **sort order**, and each one works on the running total left by the lines before it.

Worked Example:

The same basket, with the coupon on either side of the tax. Two real orders, produced by `code/m12_discounts.py -sortorder`. The basket is Chapter 10's: one MacBook at $500.00 and two iPhones at $101.00. The coupon is the one the shop ships with, code `2222`, 10% off.

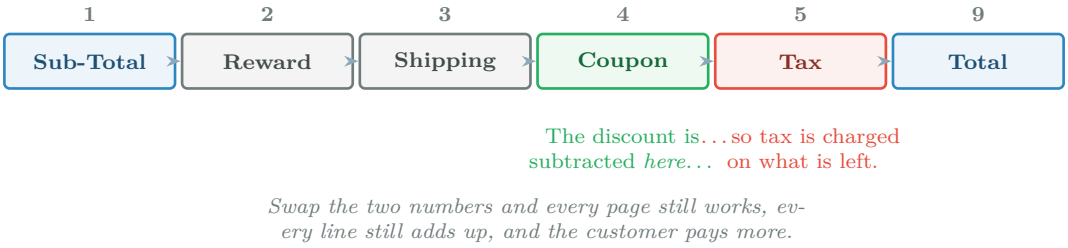


Figure 12.1: The totals pipeline as the shop ships it, with each extension's sort order above it. The numbers are editable fields in the admin panel, and nothing warns you what they mean.

As shipped — coupon at 4, tax at 5:

Sub-Total	702.00	
Flat Shipping Rate	5.00	
Coupon (2222)	−70.20	10% × 702.00
Eco Tax (-2.00)	6.00	3 × 2.00
VAT (20%)	127.36	20% × 636.80
Total	**770.16**	

The VAT base is $702.00 + 5.00 - 70.20 = 636.80$: the discount has already happened, so tax is charged on what the customer actually pays.

Coupon moved to 6, after tax:

Sub-Total	702.00	
Flat Shipping Rate	5.00	
Eco Tax (-2.00)	6.00	
VAT (20%)	141.40	20% × 707.00
Coupon (2222)	−70.20	
Total	**784.20**	

The difference is $14.04, and it is exactly 20% of the $70.20 discount — tax charged on money the customer was never asked for. On a $770 order that is 1.8%, produced by changing one integer in an admin field that offers no explanation of what it does.

Both orders are internally consistent. Both add up. Both display correctly. No validation objects, because there is nothing malformed about either — one of them is simply wrong about the world.

⚠ A sequencing rule the checkout does not tell you about

Applying a coupon *after* the customer has chosen a payment method invalidates the choice. The coupon changes the order total, the stored payment method no longer matches the recomputed list, and the final step answers:

```
{"error":"Payment method is incorrect!"}
```

— after which no order is created. The code was accepted, every earlier step reported success, and the sale is lost at the last click.

This was found by driving the sequence from a script, exactly as in Chapter 10, and it is the second fault in this course that is invisible from the browser. The remedy in your own shop is to put the coupon field *before* the method steps, or to re-select the method whenever a total changes.

12.4 What a Discount Actually Costs

A discount is not a marketing decision with a financial side effect. It is a financial decision, and the arithmetic is short enough to do before the meeting rather than after it.

Worked Example:

A product sells for **Rs 1,000** and costs the shop **Rs 700**.

Gross margin today: $1{,}000 - 700 =$ **Rs 300** per unit, 30%.

With a 10% coupon: the customer pays *Rs* 900; the cost is unchanged.

$$900 - 700 = \textbf{Rs 200}$$

The discount was 10% of the price and 33% of the profit.

$$\frac{300 - 200}{300} = 33.3\%$$

How much extra volume pays for it? To earn the same total profit:

$$\frac{300}{200} = 1.5$$

The shop needs 50% more orders to stand still. Not 10% more — 50%.

And that is the optimistic case, because it assumes every discounted sale is a sale that would not otherwise have happened. In practice some share of the customers who used the code would have bought anyway, and every one of those is pure loss. If half of them would have bought anyway, the required uplift roughly doubles again.

The general rule: at a gross margin of m and a discount of d (both as fractions of price), the volume multiple needed to break even is

$$\frac{m}{m - d}$$

which goes to infinity as d approaches m, and is *negative* — that is, unachievable at any volume — beyond it. A 30% discount on a 30% margin cannot be made up by selling more.

12.5 Referrals: Paying Someone Else to Send Customers

HEC's third term. The platform's mechanism is the **affiliate**: a customer account flagged as one, given a **tracking code**, and paid a commission on orders that arrive carrying it.

</> How a referral is tracked, in four steps

1. A customer applies at **Account › Affiliate** and is approved — or not, depending on **Affiliate Approval** in the settings.
2. They receive a tracking code and link their audience to `?tracking=code`.
3. A visitor arriving on that link has the code stored. The default commission in this shop is **5%**, set at **System › Settings › Edit › Option**.
4. When an order completes, a commission transaction is written against the affiliate's account, becoming store credit or a payable.

The question that decides whether this works: how long does the tracking code survive? If it lasts one request, the affiliate is paid only for customers who buy immediately.

If it lasts a month, the affiliate is paid for customers who were going to buy anyway. Neither extreme is right, and the shop — not the affiliate — must choose.

Where referral money leaks

- **Self-referral.** An affiliate buys through their own link. A 5% commission is a 5% personal discount, paid for by the shop and recorded as marketing spend.
- **Stacking with a coupon.** A 10% code plus a 5% commission is 15% off a margin that may be 30%, and half the profit is gone with two independently reasonable decisions.
- **Commission on refunded orders.** Unless the commission is reversed when the order is, the shop pays for a sale that was undone — Chapter 18's subject, and the commonest unreconciled figure in a small shop's accounts.
- **Commission on tax and shipping.** Paying 5% of $854.40 rather than 5% of the $702.00 of goods means paying the affiliate a share of money that belongs to the tax authority and the courier.

Common Pitfalls

- **Treating totals sort order as cosmetic.** It moved a bill by $14.04 and nothing warned.
- **Putting the coupon field after the payment step**, which loses the sale at the last click with an error nobody sees.
- **Assuming a demonstration coupon is usable.** The shipped codes are disabled and expired — which is correct of them, and the first thing to check when a code "does not work".
- **Approving a discount without the margin arithmetic.** 10% off a 30% margin needs 50% more orders.
- **Publishing one coupon code everywhere**, which makes the campaign unmeasurable and the discount permanent.
- **Paying commission on gross order value** rather than on goods.
- **Promising a client gift vouchers** because the outline mentions them. This platform has none; it is a module, not a setting.

Try It Yourself: Move one number, and change what the customer pays

```
# --- 1. Find out why the shipped coupon does not work ---------------
cd code
docker compose exec db mariadb -ushop -pshoppw opencart -e \
  "SELECT code,name,status,date_start,date_end FROM oc_coupon;"
# Two reasons, not one. Name both before fixing either.

# --- 2. Enable it ---------------------------------------------------
python m12_discounts.py --coupon on

# --- 3. Place one order with it and take the total apart ------------
python m12_discounts.py --order
# Reproduce every line with a calculator.

# --- 4. Predict, in writing -----------------------------------------
# If the coupon were applied AFTER tax instead of before, would the
# customer pay more, less, or the same? By how much? Commit first.

# --- 5. Run the experiment ------------------------------------------
python m12_discounts.py --sortorder

# --- 6. Account for the difference exactly --------------------------
```

```
# It is not an arbitrary number. Show where it comes from.

# --- 7. Build a measurable campaign ---------------------------------
# Admin: Marketing > Coupons. Make THREE codes for the same 10% offer,
# one per channel. Say how you would later tell which worked.

# --- 8. Do the margin arithmetic before you would ship it -----------
# Your product costs 700 and sells for 1000. Work out the volume
# uplift your 10% code needs. Then do it for 20%.

# --- 9. Put everything back -----------------------------------------
python m12_discounts.py --restore
```

What to notice. Step 4 against step 5. Most people predict "the same" — the discount and the tax are both there either way. The 1.8% difference is the whole chapter, and it lives in a numeric field with no label explaining it.

Checkpoint — Test Yourself

1. The same basket produced totals of 770.16 and 784.20 depending on one setting. Explain what the setting was, why the difference is exactly 14.04, and which total is correct — giving your reason in terms of what tax is owed on.
2. A client with a 25% gross margin wants to run a permanent 15% "loyal customer" code. Work out the volume uplift needed to break even, state two reasons the real requirement is higher than your figure, and propose an alternative that costs less.
3. HEC's outline requires vouchers and the platform has none. Describe what you would tell a client who asked for them, what the platform already provides, and what specifically you would have to build.

Chapter Summary

- Four devices, differing in who qualifies and when the shop learns of it: quantity breaks and specials are automatic and attached to products; a coupon is a code, which is what makes a campaign measurable; store credit is a liability, not a discount.
- This platform has **no gift vouchers** — no tables, no totals extension, only a leftover demonstration image. HEC's clause reflects a 2010 book, not current software. Say so rather than fudging it; the gap is a module, and Chapter 19 is where it would be written.
- Order totals are an ordered pipeline. Each line works on what the previous lines left, and every extension's position is an editable integer with no explanation attached.
- Worked: the same basket totalled 770.16 with the coupon before tax and 784.20 with it after — a difference of 14.04, exactly 20% of the 70.20 discount. Both are internally consistent; one is wrong about what tax is owed on.
- Applying a coupon after the payment method is chosen invalidates the choice and loses the order at the last click, with an error the browser does not show.
- Worked: a 10% discount on a 30% margin removes 33% of the profit and needs 50% more orders to break even — $m/(m-d)$ in general, and unachievable once d reaches m.
- A referral is tracked by a code and paid as commission. The leaks are self-referral, stacking with coupons, commission on refunded orders, and commission on tax and shipping.

Review Questions

1. (MCQ) Moving the coupon line after the tax line in the totals pipeline causes the customer to pay: (a) less; (b) the same; (c) more, by the tax on the discount; (d) more, by the discount.

2. **(MCQ)** A 10% discount on a product with a 30% gross margin removes what share of the profit? (a) 10%; (b) about a third; (c) half; (d) it depends on the volume.

3. **(Short)** Give two things a coupon can do that a special cannot, and one thing a special does better.

4. **(Short)** Explain why a shop should issue a different coupon code for each channel, even for the same offer.

5. **(Short)** Explain why paying affiliate commission on the order grand total rather than on the goods value is a mistake, using the worked order in Chapter 11 for your figures.

6. **(Applied)** A shop runs a 20% code and a 5% affiliate scheme on products with a 35% gross margin. Work out the margin on an order that uses both, say what the shop should do about it, and state how you would enforce your answer in the platform.

7. **(Applied)** Design the gift voucher the platform lacks. State the tables you would add, how a voucher is bought, how it is redeemed by a different person, what happens when it part-pays an order, where it sits in the totals pipeline and why, and the two ways it could be spent twice.

Part IV

Money and Identity

This is the part where mistakes cost money rather than marks. Taking payment, knowing who the customer is, and keeping both facts away from everybody else — three chapters of building, and one of trying to break what you built.

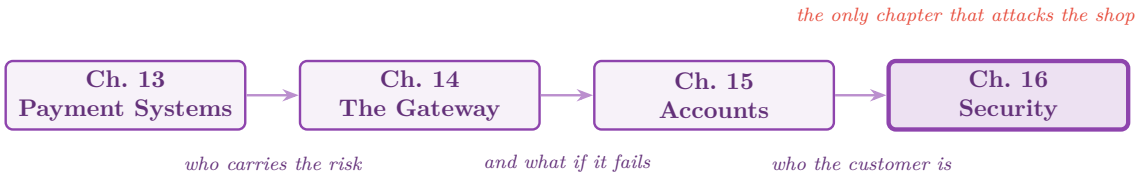


Figure P4.1: Reading path through Part IV. Chapter 16's laboratory runs against your own container and nothing else; the chapter says so in its first paragraph, and means it.

Chapter 13

Web Payment Systems

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the parties to an online payment and **state** what each one does and is paid for.
- **Distinguish** the payment instruments available to a Pakistani shop, and **compare** them on cost, settlement time and risk.
- **Explain** who carries the loss when a payment goes wrong, for each instrument.
- **Compute** the true cost of cash on delivery, including the returns it causes.
- **Select** a payment mix for a stated shop, and **defend** the choice against the alternatives.

Before You Start

Chapter 10: the order state machine, and that an order exists before it is paid for. Chapter 11: order totals, so that percentages have something to be percentages of.

Key Terms in This Chapter

payment instrument • card scheme • issuer • acquirer • payment gateway • payment service provider • merchant discount rate • settlement • chargeback • authorisation • capture • mobile wallet • cash on delivery • return-to-origin • Raast • PayPak

13.1 What Happens When Somebody Pays

A customer clicking **Pay** starts a conversation between five parties, four of which the shop never sees. Knowing who they are is not trivia: it is how you work out who to telephone when money is missing, and who will be out of pocket if it stays missing.

The five parties

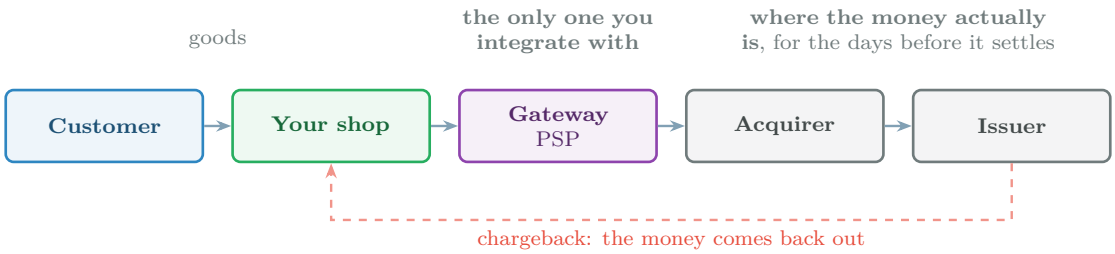
The **cardholder** or payer. Your customer.

The **issuer** — the bank that gave them the card or holds their wallet. It decides whether to approve the payment, and it is the party the customer complains to.

The **scheme** — Visa, Mastercard, PayPak — which operates the rules and the network between banks. It writes the rulebook that decides disputes.

The **acquirer** — the bank that holds the *shop's* account and accepts the payment on its behalf. It carries the risk that the shop fails to deliver, which is why it asks about your business before accepting you.

The **gateway** or **payment service provider** — the software company between the shop and the acquirer. It is the only one of the five you will integrate with, and the only one whose documentation you will read.



You pay all four. The merchant discount rate is a single percentage that is then divided between scheme, acquirer and gateway — which is why nobody will quote you its parts.

Figure 13.1: The parties to a card payment, and the direction a chargeback travels. The shop sits at one end of a chain it cannot see into, which is the central fact about accepting cards.

Authorisation is not payment

Two separate events, and confusing them is how shops ship goods they are never paid for.

Authorisation asks the issuer "would this payment succeed?" and reserves the amount. It is instant, it is reversible, and it is not money.

Capture claims the reserved amount. Only now is the shop owed anything.

Settlement is the acquirer actually paying the shop, typically days later and net of fees.

A gateway that reports "success" may mean any of the three. Read its documentation for which, because Chapter 14's entire difficulty is that your order status must mean the same thing as the gateway's.

13.2 The Instruments a Pakistani Shop Can Actually Offer

The platform ships no online payment method at all

Checked rather than assumed. The payment extensions in the box are `bank_transfer`, `cheque`, `cod` and `free_checkout` — **four methods, none of which takes money online**. Every one of them records an intention and leaves the shop to collect afterwards.

There is no card gateway, no wallet, nothing. Every real payment integration is a third-party extension, bought or written, and Chapter 14 is the chapter that deals with that. A plan that assumed "the platform handles payments" has already failed, and it is better to discover that here than in week fourteen.

Instrument	Typical cost	Settles	Who loses if it goes wrong	Main difficulty
Card (Visa, Mastercard)	2–3%	days	**the shop**, via chargeback	disputes you cannot win
PayPak card	lower	days	the shop	domestic acceptance only
Mobile wallet (JazzCash, Easypaisa)	1–2%	days	mostly the shop	reconciliation

Instrument	Typical cost	Settles	Who loses if it goes wrong	Main difficulty
Bank transfer / Raast	very low	near instant	effectively nobody	confirming which order was paid
Cash on delivery	1–3% plus returns	on remittance	**the shop**, heavily	refusal at the door

Rates are indicative and negotiated. Any figure here is a starting point for a conversation with a provider, not a price. What does not change is the *shape*: online card acceptance is the most expensive and carries the dispute risk; bank transfer is the cheapest and carries an operational burden instead; cash on delivery looks cheap and is not.

13.3 Cash on Delivery Is Not Free

Cash on delivery dominates Pakistani e-commerce for a reason that has nothing to do with technology: it asks the customer to trust nobody. They pay when the goods are in their hand. A shop that refuses COD is refusing most of its market.

It is also the most expensive instrument on the list, and the cost is invisible because it does not arrive as a percentage.

Worked Example:

A shop sells an item for **Rs 3,000** that costs it **Rs 2,100** — a 30% gross margin, Rs 900 per order.

The visible cost. The courier charges a COD service fee, say **2%** of the collected amount: $0.02 \times 3{,}000 = Rs\ 60$.

The invisible cost. Some proportion of COD orders are refused at the door — the customer is out, has changed their mind, or ordered on an impulse that has passed. Call it **20%**, which is not a pessimistic figure for unverified COD orders. A refused parcel is **return-to-origin**: the shop pays delivery *and* return, say **Rs 250** each way, and gets the item back with nothing to show.

Per 100 orders:

- 80 delivered: $80 \times (900 - 60 - 250) = 80 \times 590 = Rs\ 47{,}200$
- 20 refused: $20 \times (-250 - 250) = -Rs\ 10{,}000$
- **Net: Rs 37,200**, against Rs 72,000 of gross margin on the same 100 orders if they were prepaid and delivery were free.

The effective cost of COD here is 48% of gross margin — against 2–3% for a card. The refusal rate, not the service fee, is almost all of it: the 20 refused orders cost Rs 10,000 directly and also consumed the Rs 20,000 of margin they would have produced.

The lever that matters is therefore the refusal rate, not the fee. Dropping refusals from 20% to 10% is worth more than negotiating the courier fee to zero — which is why shops that take COD invest in confirming orders by telephone before dispatch, and why an order confirmation step that feels like friction is, here, the single most profitable thing in the pipeline.

What this says about the "frictionless checkout" advice

Most writing about e-commerce checkout comes from markets where payment is taken before dispatch. There, every removed step is pure gain, and the advice is sound.

Where the shop is paid on delivery, an unverified order is a liability rather than an asset, and a checkout optimised purely for completion produces more of them. **Advice imported without its assumptions is how a shop loses money by following best practice.** Chapter 8 made the same point about page weight; this is the version that costs real money.

13.4 The Local Rails

</> What a Pakistani shop is actually plugged into

- **State Bank of Pakistan** authorises and supervises the operators and providers. A gateway you integrate with should be an authorised one, and asking for that authorisation is a reasonable first question.
- **1LINK** operates the domestic switch connecting the banks, and **PayPak** is the domestic card scheme that runs over it. A PayPak card is cheaper to accept and works only in Pakistan.
- **Raast**, the State Bank's instant payment system, settles bank to bank in seconds at negligible cost. Its person-to-merchant use is the development most likely to change the arithmetic in this chapter, because it removes both the card fee and the COD refusal problem at once.
- **JazzCash** and **Easypaisa** are the mobile wallets, each with a merchant gateway, and between them they reach customers who have no card and no bank account — which in this market is the majority.

Two absences worth knowing. PayPal does not operate for merchants in Pakistan, and Stripe does not support Pakistan-registered businesses. A great deal of the integration tutorial material you will find online assumes one or the other, and none of it applies. Students regularly lose a week to this.

⚠ Dated facts, and how to treat them

Everything in this section is a fact about a particular year. Rates are negotiated, operators are authorised and de-authorised, and Raast's merchant rollout is still changing what is possible.

The durable part is the set of questions: who is authorised, what does it cost as a percentage and as a fixed fee, how long until the money is in the shop's account, who decides a dispute, and what proportion of customers can actually use it. Ask those five of any provider in any year and the answer will be current. Memorising this year's rates will not help you in 2029; knowing that settlement time and dispute liability are negotiable terms will.

Common Pitfalls

- **Treating an authorisation as a payment** and dispatching goods against it.
- **Quoting a client a card fee without the chargeback risk.** The 2–3% is the predictable part; the disputes are not.
- **Costing cash on delivery at the courier's service fee.** The refusal rate is most of the cost and appears in no invoice.
- **Offering only cards in a market where most customers have none.**
- **Following a payment integration tutorial written for PayPal or Stripe.** Neither is available to you here.
- **Assuming the platform can take payments out of the box.** Its four methods all collect later.

- **Letting the customer choose the currency without deciding who carries the exchange risk.**

Try It Yourself: Cost your own shop's payment mix

```
# --- 1. See what you actually have ----------------------------------
cd code
docker compose exec shop sh -c \
  'ls extension/opencart/admin/controller/payment/'
# Four files. Which of them takes money online? Write the answer down.

# --- 2. Enable bank transfer and read what it does ------------------
# Admin: Extensions > Extensions > Payments > Bank Transfer > Install.
# Place an order with it. What does the customer see, and what does
# the order status become?

# --- 3. Compare that with cash on delivery --------------------------
# Place one of each and compare the two orders' status histories.

# --- 4. Build the COD model for YOUR product ------------------------
# Take one product in your shop. Find its price. Assume a cost of 70%
# of price, Rs 250 each way for delivery, a 2% COD fee, and a 20%
# refusal rate. Compute net margin per 100 orders.

# --- 5. Find the break-even refusal rate ----------------------------
# At what refusal rate does COD earn the shop nothing at all?
# Solve it, do not guess.

# --- 6. Price the alternative ---------------------------------------
# Repeat step 4 for a 2.5% card fee, zero refusals, and the same
# delivery cost one way only. Which is better, and by how much?

# --- 7. Decide, and write the decision down -------------------------
# One paragraph: which instruments your shop will offer, in what
# order on the checkout page, and what you will measure to find out
# whether you were right.
```

What to notice. Step 5 surprises people. The break-even refusal rate is lower than most shops' actual refusal rate, which means a shop can run on COD for a year, grow its order count every month, and lose money on every single one.

Checkpoint — Test Yourself

1. A gateway reports "success" and your shop marks the order Processing and dispatches it. Describe two distinct ways the shop can end up having shipped goods it is never paid for, and say what you would check in the gateway's documentation to rule each one out.
2. Using the worked model, compute the net margin per 100 orders if the refusal rate falls from 20% to 10%. State the improvement as a percentage, and compare it with what the shop would gain by negotiating the courier's COD fee from 2% to 0%.
3. A client wants to accept international cards for customers abroad. List, in order, the four things you would establish before writing any code, and say which of them is most likely to stop the project.

Chapter Summary

- Five parties: customer, issuer, scheme, acquirer, gateway. You integrate with one and pay all of them through a single merchant discount rate whose parts nobody will itemise.

- Authorisation, capture and settlement are three different events. A gateway's "success" means one of them, and your order status must mean the same thing.
- The platform ships **no online payment method** — bank transfer, cheque, cash on delivery and free checkout all collect later. Every gateway is a third-party extension.
- Instruments differ less in price than in who carries the loss. Cards put the dispute risk on the shop; bank transfer puts an operational burden on it instead.
- Worked: cash on delivery at a 20% refusal rate costs 48% of gross margin, almost all of it from refusals rather than from the service fee. The refusal rate is the lever; the fee is not.
- Checkout advice written for prepaid markets does not transfer. Where payment is on delivery, an unverified order is a liability and confirming it is the most profitable step in the pipeline.
- Locally: SBP authorises, 1LINK switches, PayPak is the domestic scheme, Raast settles instantly, and the wallets reach customers with no card. PayPal and Stripe are not available to you.
- Rates and operators date; the five questions — authorised, cost, settlement time, who decides disputes, who can use it — do not.

Review Questions

1. **(MCQ)** Which party carries the loss when a card payment is successfully disputed? (a) the issuer; (b) the scheme; (c) the acquirer; (d) the shop.

2. **(MCQ)** How many of the payment extensions shipped with the platform take money online? (a) none; (b) one; (c) two; (d) all four.

3. **(Short)** Distinguish authorisation, capture and settlement, and say which one a shop should require before dispatching goods.

4. **(Short)** Explain why a 2% COD service fee understates the cost of cash on delivery, using one number from the worked model.

5. **(Short)** Give two reasons a Pakistani shop might prefer PayPak acceptance to Visa and Mastercard, and one reason it cannot rely on it alone.

6. **(Applied)** A shop takes 600 orders a month at an average value of Rs 4,000 and a 28% gross margin, all on cash on delivery with an 18% refusal rate and Rs 300 of delivery cost each way. Model the effect of introducing prepayment with a 2.5% fee that 35% of customers adopt, and state the monthly difference, showing every step.

7. **(Applied)** You are asked to decide whether to add a mobile wallet to a shop that currently takes only COD. Set out what you would need from the provider, what you would change in the shop, what you would measure for how long, and what result would make you remove it again.

Chapter 14

Taking Payment for Orders

Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the handshake between a shop and a gateway, and **name** the three outcomes of a payment request.
- **Measure** a gateway's response-time distribution, and **choose** a timeout from it rather than from its median.
- **Explain** why a timed-out payment has an unknown outcome, and **design** what the shop does next.
- **Implement** idempotency, and **demonstrate** what happens without it.
- **State** what PCI DSS requires of a shop that never touches a card number, and **identify** what would change that.

Before You Start

Chapter 13: the parties, and the difference between authorisation, capture and settlement.
Chapter 10: the order state machine, status 0, and why the order identifier exists before the payment.

Key Terms in This Chapter

gateway integration • hosted page • direct post • server-to-server • callback • webhook • timeout • latency distribution • percentile • long tail • idempotency key • replay • reconciliation • PCI DSS • SAQ-A

14.1 Three Ways to Take a Card, and Why One of Them Wins

A shop that wants a card number has three architectures available, and the choice determines how much of the payment industry's compliance regime applies to it.

Hosted page, Direct post, Server-to-server

A **hosted page** sends the customer to the provider's own site to type their card details, and receives them back afterwards with a result. The card number never reaches your server.

A **direct post** keeps the customer on your page, but the form submits straight to the provider. Your HTML is involved; your server is not.

Server-to-server means the card number arrives at your server and you forward it. You now hold card data, and almost every obligation in this chapter multiplies.

For a shop of the size this course is about, the answer is always the first or the second, and the reason is in the next box.

What PCI DSS actually asks of you

The Payment Card Industry Data Security Standard governs anyone who stores, processes or transmits card data. Its reporting burden is graded by how much of that you do, through a Self-Assessment Questionnaire.

- **SAQ-A** is for a merchant who has *entirely outsourced* card handling to a validated third party — a hosted page. It is the shortest questionnaire, and most of it asks whether you are keeping your own site from interfering with the provider's.
- **SAQ-A-EP** applies to direct post: your page constructs the form, so your page can be tampered with, and you are asked more.
- **SAQ-D** is the full standard, and is what server-to-server costs you. It runs to hundreds of requirements.

The design decision and the compliance decision are the same decision. Choosing a hosted page is not a shortcut around PCI DSS; it is the mechanism the standard offers for staying out of scope. A student who understands only this from the chapter has understood the important part.

What would change it: storing the card number "for the customer's convenience", logging the full request body, or accepting the number on your page "just to validate it". Each of these moves a shop from SAQ-A to SAQ-D with one line of code and no announcement.

14.2 The Course Ships Its Own Gateway

Measuring a real gateway needs a merchant account, which needs a registered business. A sandbox needs credentials that cannot go in a handout and will have expired before anyone reads it. So the stack in Appendix A includes a provider that does not exist.

What `code/gateway/provider.py` is, and is not

It is not a toy card processor. It implements nothing about cards. It is a gateway reduced to the **three properties that make integrating with one difficult**, each configurable so that you can make it behave worse on purpose:

- **It is slow, and irregularly so.** A median around 300 ms, with a minority of requests taking ten times that.
- **It sometimes does not answer at all.** About one request in fifty hangs, which is the case the rest of the chapter is about.
- **It tells you twice.** It honours an idempotency key if you send one, and charges twice if you do not.

Those are the properties a real integration is actually built against. Everything else a provider does — 3-D Secure, tokenisation, settlement reports — is detail you will read in their documentation. These three are what the *architecture* has to survive, and they are the same for every provider in every country.

14.3 Choosing a Timeout, Which Is Not a Performance Setting

Measured: 200 authorisations, waiting as long as it takes

From `code/m14_gateway.py -latency 200`, against the stack in Appendix A.

Median	321 ms	what a single test request shows you
Mean	700 ms	*more than twice the median*
90th percentile	3,180 ms	
95th percentile	3,774 ms	
99th percentile	4,815 ms	
Slowest answer	4,998 ms	**16× the median**
Never answered	4 of 200	2%

The mean is more than twice the median, which is the whole measurement. A distribution where the mean sits far above the median is one with a long tail, and an average response time of 700 ms describes almost none of the requests: most took a third of a second and a minority took five seconds. Reporting "average response time" here would be worse than reporting nothing, because it sounds like information.

Measured: What each candidate timeout would have cost

The same 200 requests, re-read against each timeout a developer might choose.

Timeout	Answered	Cut off	Share	What the shop is doing
0.5 s	171	25	12.8%	abandoning good payments
1.0 s	173	23	11.7%	abandoning good payments
2.0 s	173	23	11.7%	abandoning good payments
3.0 s	175	21	10.7%	abandoning good payments
5.0 s	**196**	**0**	**0.0%**	waiting
8.0 s	196	0	0.0%	waiting

The counter-intuitive result is the flat middle. Raising the timeout from 1 second to 3 seconds — tripling how long the customer waits — rescues two payments in two hundred. One percent. Almost nothing.

The reason is that the distribution is not a bell curve but *two groups*: requests that answer in under half a second, and requests that take between two and five seconds. There is hardly anything in between, so a timeout placed in between catches hardly anything. **Either you wait for the slow group or you do not, and every value between those two decisions buys you nothing while costing the customer time.**

A timeout chosen from the median rejects one payment in eight. And each rejection is not a failed payment — it is a payment whose outcome the shop does not know.

What to do about the third branch

A timed-out request has an unknown outcome. The shop must choose, in advance, what it believes.

1. **Never assume failure and never assume success.** Record it as *unknown*, which is a third state and needs a place to live — Chapter 10's status 0 is where it goes.

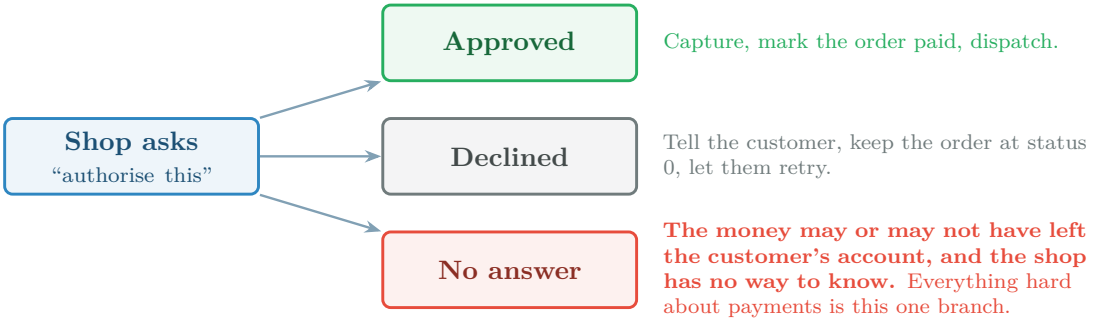


Figure 14.1: The three outcomes of a payment request. Most integration tutorials cover the first two.

2. **Ask the provider.** Every gateway offers a status query. Retry it on a schedule, not in the customer's request.
3. **Make the customer's retry safe.** If they press the button again, the same idempotency key must go with it, or you have charged them twice for being impatient.
4. **Reconcile daily.** The provider's settlement report against your orders. This is the only mechanism that catches everything the other three miss, and it is the one small shops skip.

14.4 Idempotency, Demonstrated

Measured: The same payment, sent twice

From `code/m14_gateway.py -idempotency`.

Without a key:

first	transaction `f59594f7d23c4446`	approved	125 ms
second	transaction `8171b6e85f9b4766`	approved	2,644 ms

Two transactions. The customer paid twice, and both payments are valid — there is nothing for the provider to detect, because two requests to charge the same amount are exactly what a customer buying two of something looks like.

With a key:

first	transaction `50f018abfd9243ec`	approved	149 ms
second	transaction `50f018abfd9243ec`	**replayed**	**5 ms**

The second request returned *the first request's answer*, in 5 ms, without charging anything. The provider saw four requests and charged three times.

Where the key must come from. Not from the request — a retry is a new request and would generate a new key, which defeats the entire mechanism. It must come from *the order*: generated once when the order row is created, stored on it, and sent unchanged with every attempt to pay for that order. This is the second reason Chapter 10's order row exists before the payment, and the better one.

Common Pitfalls

- **Choosing a timeout from the median response time.** It rejected one payment in eight here.
- **Reporting a gateway's "average" latency.** The mean was twice the median; the average describes no request that actually happened.
- **Treating a timeout as a failure** and letting the customer reorder.
- **Generating the idempotency key in the request** rather than storing it on the order.
- **Retrying inside the customer's request.** They are waiting; the retry belongs in a background job.
- **Logging the full request body** while integrating. If the body ever contains a card number, your log is now in PCI scope, and logs are backed up, shipped and searched by people who are not you.
- **Trusting a callback because it arrived.** Verify its signature; anyone can post to a public URL.
- **Skipping daily reconciliation** because the integration works. The integration working is what reconciliation is for.

Try It Yourself: Integrate against a gateway that misbehaves

```
# --- 1. Start the provider and look at it ----------------------------
cd code
docker compose up -d gateway
curl -s http://localhost:8092/health

# --- 2. Read it. It is one file --------------------------------------
# gateway/provider.py. Find the three properties it models, and the
# environment variable that controls each.

# --- 3. Measure before you decide ------------------------------------
python m14_gateway.py --latency 200

# --- 4. Predict your timeout, then check it --------------------------
# Write down the timeout you would have chosen from the median ALONE.
python m14_gateway.py --timeout

# --- 5. Demonstrate the double charge --------------------------------
python m14_gateway.py --idempotency

# --- 6. Make the provider worse, and re-decide -----------------------
# In docker-compose.yml set GATEWAY_TAIL_RATE to 0.35 and
# GATEWAY_HANG_RATE to 0.10, then:
docker compose up -d gateway
python m14_gateway.py --latency 100
# Does your timeout still hold? What changed, the median or the tail?

# --- 7. Put it back --------------------------------------------------
# Restore the defaults and restart the gateway.

# --- 8. Write the integration policy ---------------------------------
# One page, four headings: timeout and why; what an unknown outcome
# means for the order; where the idempotency key comes from; how
# reconciliation happens and who does it.
```

What to notice. Step 6 is the point. Most people's timeout is chosen once and never revisited, and a provider's tail is exactly the thing that changes without telling you — at the end of the month, on sale days, whenever their own upstream is busy.

Checkpoint — Test Yourself

1. A gateway's median response is 321 ms and its 95th percentile is 3,774 ms. A colleague proposes a 1-second timeout, reasoning that it is "three times the median, which is generous". Using the measured table, say what that choice actually costs and what you would propose instead, with your reason.
2. Explain why the measured timeout table barely improves between 1 second and 3 seconds, and what that shape tells you about the gateway that a single average could not.
3. Your shop charges a customer, the request times out, and the customer presses Pay again. Describe exactly what happens with and without an idempotency key, say where the key must be stored, and explain why generating it when the button is pressed would not work.

Chapter Summary

- A hosted page, a direct post and a server-to-server integration are the three architectures, and they map onto SAQ-A, SAQ-A-EP and SAQ-D. The design decision and the compliance decision are the same decision.
- A gateway is three hard properties: it is slow and irregular, it sometimes does not answer, and it tells you twice. The course ships a provider that does all three on purpose.
- Measured: median 321 ms, mean 700 ms, p95 3,774 ms, slowest 4,998 ms — sixteen times the median — and 2% that never answered at all. A mean twice the median is a tail, and an average is the wrong statistic for it.
- Measured: a 1-second timeout cuts off 11.7% of payments and a 3-second timeout cuts off 10.7%. The distribution is two groups with a gap between them, so a timeout in the gap buys nothing.
- A timed-out payment is not a failed payment. It is an unknown outcome, it needs a state of its own, and it is resolved by querying the provider and by daily reconciliation — not by guessing.
- Measured: without an idempotency key the same payment charged twice, with two valid transaction identifiers. With one, the second request returned the first answer in 5 ms and charged nothing.
- The key belongs to the order, not to the request. This is the better reason why the order row exists before the payment.

Review Questions

1. (MCQ) A shop using a hosted payment page completes which self-assessment questionnaire? (a) SAQ-A; (b) SAQ-A-EP; (c) SAQ-D; (d) none is required.

2. (MCQ) In the measured distribution, the mean was 700 ms and the median 321 ms. This indicates: (a) a measurement error; (b) a long tail of slow requests; (c) that half the requests failed; (d) that the gateway is overloaded.

3. (Short) Explain why a timed-out payment request has an unknown outcome rather than a failed one, and give the consequence of assuming either.

4. (Short) Give two things that would move a shop from SAQ-A to a heavier questionnaire, each of which a developer might do without realising.

5. (Short) Why must a callback's signature be verified, given that the callback comes from the provider?

6. (Applied) You are integrating a provider whose published median is 250 ms. Describe the measurement you would run before choosing a timeout, how many samples you would need and

why, which statistics you would report, and what you would do differently if the distribution turned out to be a smooth bell curve instead of two groups.

7. **(Applied)** Design the reconciliation process for a shop taking 200 orders a day. State what two records you compare, how you match them, the three kinds of mismatch you expect to find, what each one means, and who in the business acts on it.

Chapter 15

User Account Management

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why a password is stored as a deliberately slow hash, and **name** what the cost factor buys.
- **Measure** the cost factor against throughput, and **choose** one for a stated shop.
- **Demonstrate** user enumeration by timing, and **explain** why a stronger hash makes it worse.
- **Evaluate** a lockout policy, and **identify** what it enables as well as what it prevents.
- **Design** a password reset flow that does not become the weakest way in.

Before You Start

Chapter 9: sessions, cookies, and that the server decides what is true. Chapter 14: measuring a distribution rather than a single value. Appendix A — every exercise here runs against your own container.

Key Terms in This Chapter

authentication • password hash • bcrypt • cost factor • salt • rainbow table • credential stuffing • user enumeration • timing side channel • lockout • rate limiting • password reset • account takeover

15.1 Why a Password Is Stored Slowly on Purpose

Every other piece of engineering in this course tries to make something faster. Password storage is the one place where the software is deliberately slowed down, and understanding why is the whole of the chapter.

Hash, Salt, Cost Factor

A **password hash** is a one-way function of the password. The shop stores the output and never the password, so that a stolen database does not hand the attacker the passwords.

A **salt** is a random value mixed in, different for every user, so that two customers with the same password get different hashes — and so that an attacker cannot precompute a table of hashes once and look everybody up.

A **cost factor** says how much work the function does. Bcrypt's is a power of two: raising it by one **doubles** the time. That is a cost to your server once per login and a cost to the attacker once per *guess*, and the attacker is making billions of them.

What this shop actually does

- Passwords are stored with PHP's `password_hash(..., PASSWORD_DEFAULT)`, which here produces a `$2y$10$` hash — **bcrypt at cost 10**, PHP's default and unchanged since 2013.

- The column is `varchar(255)`, which is correct: it leaves room for whatever algorithm PHP's default becomes next.
- Version 3 of this platform used SHA-1 with a salt, which is far too fast to be a password hash at all. Version 4 moved to `password_hash`. **When you inherit a shop, check which one it is before anything else.**
- The minimum password length is **6 characters**, and none of the complexity settings — uppercase, lowercase, number, symbol — is set at all, so none is enforced.

Measured: The cost factor against the clock

From `code/m15_accounts.py -cost`, timed by PHP inside the shop container on the machine in Appendix A. "Logins per second" is one core doing nothing else.

Cost	Hash	Verify	Logins/sec	Work vs cost 8
8	13.6 ms	13.7 ms	73	1×
9	29.6 ms	26.8 ms	37	2×
10	**52.8 ms**	**60.3 ms**	**17**	4×
11	110.2 ms	103.0 ms	10	8×
12	217.5 ms	218.9 ms	5	16×
13	440.7 ms	448.2 ms	2	33×
14	846.8 ms	873.8 ms	1	64×

The doubling is exact, which is what a well-behaved cost factor looks like: 13.7, 26.8, 60.3, 103, 219, 448, 874. Nothing else in this course produces a table that clean, because nothing else in this course is a cost that somebody designed on purpose.

The shop ships at cost 10: 60 ms per login, 17 logins per second on one core. A shop with 17 simultaneous login attempts is a shop in trouble for other reasons, so cost 10 is not a throughput problem — but it is also twelve-year-old advice, and hardware has not stood still.

Cost 12 is the usual current recommendation: 219 ms, 5 logins per second. For a shop taking a few hundred orders a day that is free, and it makes every stolen hash sixteen times more expensive to attack than cost 8. **Raising it is one line, and the next section is the reason to think first.**

15.2 What the Login Form Tells an Attacker Who Has No Password

An attacker with a list of email addresses and no passwords still wants something from your login form: *which of these addresses have accounts here.* That list is worth money on its own, and it makes every later attack cheaper.

Measured: A registered email answers more slowly, and measurably so

From `code/m15_accounts.py -enumerate`: fifteen login attempts with a deliberately wrong password, against an address that exists and one that does not, timed from outside the container as an attacker would.

	Median	Fastest	Slowest
Registered email	**88.6 ms**	70.9 ms	142.7 ms
Unknown email	**23.0 ms**	15.2 ms	94.8 ms

A gap of 65.5 ms — 74% of the slower time — visible from outside, over the network, without a single correct password.

Where the 65 ms comes from is the uncomfortable part. It is almost exactly the cost of one bcrypt verify at cost 10, measured in the previous section as 60.3 ms. A wrong password on a real account must be checked against the stored hash; a wrong *email* matches no row, so there is nothing to check and the request returns early.

So the leak is produced by the security measure. Raise the cost factor to 12 for the excellent reasons above, and the gap grows from 65 ms to roughly 220 ms — four times louder, trivially measurable over any network. A change that makes stolen hashes sixteen times harder to crack makes your customer list four times easier to enumerate.

The fix is one line and it is not obvious: when the email matches no customer, verify the supplied password against a *dummy hash* anyway and discard the result. The request then costs the same whether the account exists or not. It feels like wasted work because it is wasted work — that is the point.

Timing is not the only way this leaks

Closing the timing channel is necessary and not sufficient. The same information escapes through:

- **Different messages.** "No such account" versus "Wrong password" states it outright. Use one message for both.
- **Registration.** "That email is already registered" is the same disclosure through a different form, and it is harder to remove, because the user genuinely needs to know. Send an email instead of answering on the page.
- **Password reset.** "We have sent you a link" must be shown whether or not the address exists.
- **The lockout itself.** See below — a locked account answers differently from one that was never there.

Enumeration is rarely the attack. It is the reconnaissance that makes the attack affordable, and that is why it is worth closing even though no single leak looks serious.

15.3 Lockout: What It Prevents, and What It Enables

Measured: Five wrong guesses, then what?

From `code/m15_accounts.py -lockout`. The shop's `config_login_attempts` is **5**.

Attempt	Time	Recorded	Result
1–5	70–81 ms	1, 2, 3, 4, 5	rejected, hash verified each time
6	**16 ms**	5	**locked**
7	**15 ms**	5	locked
correct password	**17 ms**	5	**still refused**

It works. After five failures the account stops answering, for an hour, keyed on the pair (email, IP address).

Two things the table shows that the setting does not.

First, the correct password is refused too. That is right — a lock that the attacker's next guess could open would be no lock — but it means **anyone who knows a customer's email address can lock them out of their own account at will**, with five wrong guesses, for an hour, repeatedly. Lockout converts a password-guessing attack into a denial-of-service one. It is the right trade for most shops and it is a trade, not a free win.

Second, the locked response is four times faster, because it never reaches the hash. The lockout is itself an enumeration channel: 16 ms says "this account exists and is locked", which is more than the 23 ms of an unknown address was giving away.

And the key is (email, IP). An attacker with a few hundred addresses has effectively no limit, while a school or an office behind one NAT shares a single counter — so the mechanism is weakest against the attacker it is for and strongest against the customers it is not.

Where the real risk is, and it is not your password form

Customers reuse passwords. An attacker who has a list of email and password pairs stolen from somewhere else tries them on your shop — **credential stuffing** — and some of them work, because the passwords are genuinely correct. Nothing in this chapter prevents that: the hash is fine, the lockout counts five failures and these are not failures.

What helps is **not** another password rule. It is: checking new passwords against lists of known-breached ones; offering a second factor; rate limiting by IP and by address across the whole site rather than per account; and noticing the pattern — many accounts, one address, high success rate — which is a Chapter 17 problem rather than a login problem.

Meanwhile this shop's minimum password length is six characters with no complexity requirement at all. That is the setting to change first, and it is in **System › Settings › Edit › Option**.

The reset flow is the other front door

An account is only as secure as the cheapest way to get into it, and that is usually the password reset.

1. **The token must be unguessable and single-use**, and must expire in minutes rather than days.
2. **It must not be in the URL of a page that loads third-party scripts**, which leak it in the `Referer` header.
3. **Resetting must invalidate existing sessions**, or an attacker who was already in stays in after the customer "fixes" it.
4. **The response must not reveal whether the address exists**, which is the enumeration problem again, in its easiest-to-fix form.
5. **The email itself is now the weakest link.** You have delegated your authentication to the customer's mail provider, and should know that you have done so.

Common Pitfalls

- **Choosing a cost factor without measuring it on the machine that will run it.** Cost 14 is one login per second.
- **Raising the cost factor without closing the timing channel**, which makes enumeration four times easier while you congratulate yourself.

- **Different error messages for "no account" and "wrong password".**
- **Believing lockout is free.** It hands anyone a denial-of-service tool against any customer whose email they know.
- **Rate limiting only per account**, which does nothing against an attacker trying one password across ten thousand accounts.
- **Leaving the minimum password length at six with no complexity rules**, which is this shop's shipped state.
- **A reset token that does not expire**, or that survives being used.
- **Testing any of this against a shop that is not yours.** See Appendix A.

Try It Yourself: Measure your own login, then make it quieter

```
# Everything here runs against YOUR container on localhost. Read the
# scope statement in Appendix A before you start.

# --- 1. Find out what your shop stores ------------------------------
cd code
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT email, LEFT(password,7) FROM oc_customer LIMIT 3;"
# The first seven characters name the algorithm and the cost.

# --- 2. Measure the cost factor -------------------------------------
python m15_accounts.py --cost

# --- 3. Choose one, and justify it in a sentence --------------------
# How many logins per second does your shop actually need? Work it
# out from orders per day, not from a feeling.

# --- 4. Predict, then measure the timing leak -----------------------
# Does a wrong password on a REAL account take longer than one on an
# address that does not exist? Predict yes or no, and by how much.
python m15_accounts.py --enumerate

# --- 5. Explain the number you got ----------------------------------
# Compare the gap with the verify time from step 2. They should
# match. Say why.

# --- 6. Watch the lockout, and find what it gives away --------------
python m15_accounts.py --lockout
# Two separate problems are visible in that table. Name both.

# --- 7. Tighten the policy ------------------------------------------
# Admin: System > Settings > Edit > Option. Raise the minimum
# password length and turn on the complexity requirements. Then
# register a new customer and confirm the rules are enforced.

# --- 8. Write the fix you cannot make in the admin panel ------------
# Three lines of pseudocode: how would you make a login to a
# non-existent account take the same time as one to a real account?

# --- 9. Clean up ----------------------------------------------------
python m15_accounts.py --clean
```

What to notice. Step 5. The timing leak is not a bug somebody left in — it is the direct, unavoidable consequence of doing the right thing with passwords. Security measures interact, and sometimes they interact badly.

Checkpoint — Test Yourself

1. Your shop runs bcrypt at cost 10 and takes 400 orders a day. Work out whether cost 12 is affordable, stating your assumptions about logins per order and peak concentration, and say what else you would have to change at the same time and why.
2. A wrong password on a registered address took 88.6 ms and on an unknown one 23.0 ms. Explain the cause, state what an attacker does with that, and give the one-line fix together with the reason it feels wrong to write.
3. A client asks you to remove the five-attempt lockout because customers complain about being locked out. Give the risk you are accepting, the risk you are removing, and a design that addresses both complaints better than either keeping or removing it.

Chapter Summary

- A password hash is slow on purpose. The salt defeats precomputation; the cost factor sets how expensive each guess is, and each step doubles it for you and for the attacker.
- This shop uses `password_hash` with bcrypt at cost 10, in a `varchar(255)` column. Version 3 used salted SHA-1; check which one any shop you inherit is using before anything else.
- Measured: cost 8 to 14 ran 13.7, 26.8, 60.3, 103, 219, 448 and 874 ms — an exact doubling. Cost 10 gives 17 logins per second on one core; cost 12, the usual current recommendation, gives 5.
- Measured: a wrong password on a registered address took 88.6 ms and on an unknown address 23.0 ms — a 65 ms gap, visible over the network, and almost exactly one bcrypt verify.
- The leak is caused by the defence. Raising the cost factor to 12 would make the gap roughly 220 ms. The fix is to verify against a dummy hash when no account matches, so that both paths cost the same.
- Enumeration also leaks through error messages, registration, password reset and the lockout response. It is reconnaissance, not the attack.
- Measured: five wrong guesses lock the account for an hour, keyed on (email, IP). The correct password is refused too — so lockout hands an attacker a denial-of-service tool — and the locked reply is four times faster, which is itself a disclosure.
- Credential stuffing defeats all of it, because the passwords are correct. The answers are breach-list checks, a second factor, site-wide rate limiting, and noticing the pattern.
- The password reset is the other front door: single-use, short-lived tokens; session invalidation; and the knowledge that you have delegated authentication to a mail provider.

Review Questions

1. (MCQ) Raising bcrypt's cost factor from 10 to 12 multiplies the work by: (a) 1.2; (b) 2; (c) 4; (d) 16.

2. (MCQ) The measured 65 ms gap between a registered and an unknown email is closest to: (a) network latency; (b) one bcrypt verify; (c) the database lookup; (d) template rendering.

3. (Short) Explain what a salt prevents, and why it does not need to be secret.

4. (Short) Give two ways a shop leaks which email addresses are registered, other than by timing, and say how to close each.

5. (Short) Explain why account lockout is a trade rather than an improvement, naming what it enables.

6. **(Applied)** A shop's database is stolen. All passwords are bcrypt at cost 10. Explain what the attacker can and cannot do, what the shop must tell its customers, and what it should change — and say why changing the cost factor now helps less than you might expect.

7. **(Applied)** Design rate limiting for a login form that must stop credential stuffing without locking out legitimate customers. State what you count, over what window, keyed on what, what happens at the limit, and how a school computer lab with forty students behind one address is not broken by your design.

Chapter 16

Securing the Shop

Learning Outcomes

By the end of this chapter you will be able to:

- **State** the legal boundary of security testing, and **confine** your own work inside it.
- **Measure** a shop's security posture as a set of yes/no questions, and **repeat** the measurement after a change.
- **Distinguish** faults the framework already prevents from faults the deployment introduces, with evidence for each.
- **Apply** a hardening configuration, and **quantify** what it closed and what it did not.
- **Judge** which remaining findings need a decision rather than a setting, and **argue** for one.

Before You Start

Chapter 5: the container, Apache, and where configuration lives. Chapter 7: the upload directory and what is served from it. Chapter 15: passwords, lockout, and that a defence can create a leak. **Appendix A's scope statement, which you must read before the lab.**

Key Terms in This Chapter

threat model • attack surface • security header • content security policy • clickjacking • MIME sniffing • CORS • information disclosure • hardening • defence in depth • SQL injection • cross-site scripting • CSRF token • PECA 2016

16.1 Where You May Point This Chapter

Read this before the laboratory. It is not a formality.

Every exercise in this chapter runs against **the container on your own machine**, reached at `localhost`, which you installed, which contains no real person's data, and which you can destroy and rebuild with one command.

Running the same checks against any system you do not own and have not been authorised in writing to test is a criminal offence in Pakistan under the **Prevention of Electronic Crimes Act 2016**, and under the equivalent law of almost every other country. Sections 3 and 4 of that Act cover unauthorised access to an information system and to data, with imprisonment and fine attached to each.

There is no educational exemption. "I was learning", "I did no harm", "the site was insecure" and "I told them afterwards" are not defences, and a university assignment is not authorisation. The authorisation that counts is a written scope agreement from the system's owner, naming what may be tested and when.

What this means in practice: the only address that appears in this chapter is `localhost:8090`. If you find yourself typing anything else, stop.

16.2 Measure First, and Measure the Right Thing

The temptation in a security chapter is to start with the famous attacks. It is worth resisting, because the famous attacks are the ones the framework already handles, and starting there teaches you to look in the wrong place.

Measured: The two attacks everyone expects, against your own search box

From `code/m16_security.py -injection`. Both probes are harmless: one is a quotation mark, the other is a script tag that would open an alert box in your own browser.

Probe	Bytes	Verbatim?	Escaped?
a normal search	38,406	yes	yes
one quotation mark	26,440	no	no
`ipod' OR '1'='1`	26,508	yes	yes
`<script>alert(1)</script>`	26,648	**no**	yes
`<img src=x onerror=...>`	26,626	**no**	yes

The SQL probe returns a *shorter* page than the plain search — 26,508 bytes against 38,406. It found nothing, because the whole string including the quotation marks was searched for literally. The query's meaning did not change, which is what a parameterised or properly escaped query means.

The script probes never appear as markup, only in their escaped form. Twig auto-escapes everything it prints unless explicitly told not to, so the tag becomes text.

So both classic attacks are already handled. That is the useful finding, and it redirects the rest of the chapter: whatever is wrong with this shop is not in its application code.

16.3 What Is Actually Wrong, and It Is All Deployment

Measured: Fifteen questions, asked of the shop as installed

From `code/m16_security.py`. Each is a yes/no question you could check by hand with `curl`. "Pass" always means the safe answer.
Part one: what the shop says about itself in every response.

Question	Before	After	Cost to fix
Server header hides the version	fail	pass	1 line
`X-Powered-By` absent	fail	pass	1 line
`X-Content-Type-Options: nosniff`	fail	pass	1 line
`X-Frame-Options` or frame-ancestors	fail	pass	1 line
Content-Security-Policy set	fail	pass	1 block
`Referrer-Policy` set	fail	pass	1 line
`Permissions-Policy` set	fail	pass	1 line
No wildcard CORS with credentials	fail	pass	2 lines

Part two: what the shop will hand over if you ask for it.

Question	Before	After	Cost to fix
Error log not readable over HTTP	**fail**	**pass**	1 block
Session cookie is HttpOnly	pass	pass	—
Session cookie has SameSite	pass	pass	—
`system/storage/` not browsable	pass	pass	—
`install/` removed	pass	pass	—
`config.php` not served as text	pass	pass	—
Admin not at the default path	fail	**fail**	a decision
Both parts together	**5 of 15**	**14 of 15**	

One configuration file closed nine findings. Not one of them required a change to the shop's code, and not one of them took more than a few lines.

The serious one is the error log. Before hardening, anybody at all who asked the shop for `system/storage/logs/error.log` was given it — verified by writing a marker string into the file and then fetching it back over HTTP. A PHP error log contains file paths, SQL fragments, and often the contents of whatever request failed.

⚠ Why the shipped protection does not work, in two parts

The platform knows about this. It ships a file called `.htaccess.txt` containing, among other things, exactly the rule that would have blocked the log. Every guide to securing this software says: rename it.

First, nobody renames it. The installer does not, the Docker image does not, and a fresh install therefore has no `.htaccess` at all.

Second, and worse: renaming it would not help here. Debian's `apache2.conf` sets `AllowOverride None` for the document root, so Apache never reads `.htaccess` files. You could rename it, confirm the file is in place, and have changed nothing — with no error, no warning, and no way to tell.

This is why the fix in `code/shop/security.conf` is a server configuration rather than a `.htaccess`. It is also why the advice you will find online for this platform is worse than useless: it is confidently wrong, it has been repeated for fifteen years, and following it produces a shop that is exactly as exposed as before while its owner believes otherwise.

Verify that a control works. Do not verify that you installed it.

16.4 Three Mistakes Made While Writing This Chapter

The hardening file did not work the first time, or the second. The failures are kept because each one is a way a real control silently does nothing.

</> What went wrong, and how the measurement found it

1. **`Header always unset` removed nothing.** The CORS headers were still there after hardening. `mod_headers` keeps two tables: `always` applies to every response including errors, and the default applies only to successful ones. A header PHP sets on a 200 lives in the second table. **Both forms are needed**, and using only the one whose name sounds more thorough removes nothing.

2. **`ServerTokens` was ignored after a reload.** It is read only when Apache *starts*. A graceful reload applies every other directive in the file and silently skips this one.
3. **The file was overridden by Debian's own.** Apache reads `conf-enabled` in filename order, and the last setting of a server-level directive wins. A file named `security-ecad.conf` sorts *before* `security.conf` — because `-` precedes `.` — so Debian's `ServerTokens OS` was applied after it. Renaming the file to `zz-security-ecad.conf` fixed it.

All three produced a valid configuration that Apache accepted without complaint, and in all three the header was simply absent. A security control that fails open and fails quietly is the only kind worth being afraid of, and the only defence is to measure the result rather than the action. Each of these took one run of the harness to find and would have survived any amount of re-reading the file.

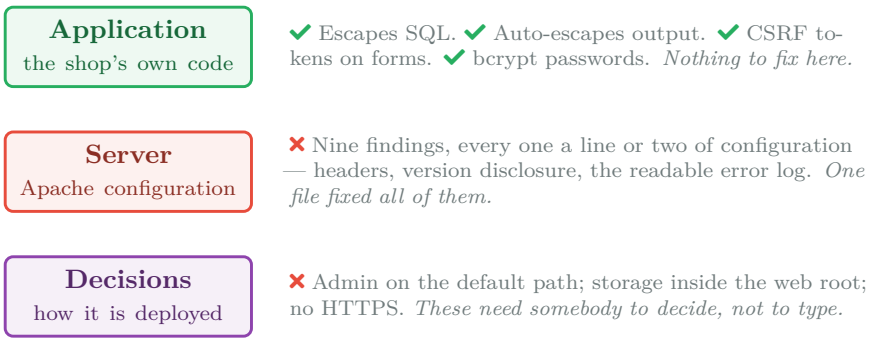


Figure 16.1: Where a stock shop's security problems actually live. The layer students expect to fix is the one that was already correct.

16.5 What a Header Cannot Fix

One finding survived the configuration file, and it is the honest one.

The three that need a decision

1. **The admin panel is at `/admin/`.** Renaming the directory and the matching path in `admin/config.php` is not security — anyone who looks will find it — but it removes the shop from every automated scan that tries one path and moves on. It costs a configuration change and a note for whoever maintains the shop next, which is why it is a decision rather than a setting.
2. **`system/storage/` is inside the web root.** Blocking it in the server config, as this chapter did, is the fix you can apply today. **Moving it outside the web root is the fix that is still correct after somebody edits the server config in two years.** Prefer the control that does not depend on remembering.
3. **There is no HTTPS.** On `localhost` it does not matter. On a real shop everything in this book — the session cookie, the password, the order — travels in clear text without it, and `Strict-Transport-Security` is deliberately left out of the hardening file because switching it on without a certificate makes the shop unreachable in your own browser for months. Chapter 20 turns it on where there is something to turn it on for.

The defaults this course has been using, and must not keep

The shop was installed with `admin`/`admin123`, a database user `shop`/`shoppw`, and a root password of `rootpw`. Those are in the handout, in the repository, and in this paragraph.

They are acceptable for exactly one thing: a container on your own machine, with no data in it, not reachable from your network. They are documented as teaching defaults rather than hidden, because a weak password nobody mentions is worse than a weak password everybody knows about.

Any shop that leaves this machine changes all three before it does, along with the minimum password length of six characters noted in Chapter 15. Chapter 20 makes that a checklist item with nothing optional about it.

Common Pitfalls

- **Testing anything but your own container.** See the first page of this chapter, and Appendix A.
- **Starting with SQL injection and cross-site scripting.** Both were already handled; the time was better spent elsewhere.
- **Renaming `.htaccess.txt` and believing you are done.** `AllowOverride None` means the file is never read.
- **Verifying that you applied a control rather than that it works.** Three separate controls in this chapter were applied correctly and did nothing.
- **Using `Header always unset` alone** to remove a header from a successful response.
- **Reloading instead of restarting** after changing a directive that is only read at startup.
- **Setting `Strict-Transport-Security` without a certificate**, which locks you out of your own shop.
- **Writing a Content-Security-Policy that breaks the theme**, then removing it entirely instead of narrowing it.
- **Treating a passing checklist as a secure shop.** Fourteen of fifteen is a measurement, not a guarantee.

Try It Yourself: Break into your own shop, then shut the door

```
# SCOPE: localhost only. Your container, your machine, no real data.
# Re-read the first page of this chapter before you begin.

# --- 1. Ask what it tells you, before changing anything -------------
cd code
curl -sI http://localhost:8090/ | head -20
# List every header that names a version or a product.

# --- 2. Read your own error log over HTTP ---------------------------
docker compose exec shop sh -c \
  'echo "SECRET-TEST-STRING" >> system/storage/logs/error.log'
curl -s http://localhost:8090/system/storage/logs/error.log
# If you see your string, so would anyone else.

# --- 3. Run the battery ---------------------------------------------
python m16_security.py --check

# --- 4. Try the two attacks you came here for -----------------------
python m16_security.py --injection
# Predict the result first. Most people predict at least one works.

# --- 5. Find out why the shipped protection does not ----------------
docker compose exec shop sh -c 'ls -la .htaccess*'
docker compose exec shop sh -c \
  'grep -n AllowOverride /etc/apache2/apache2.conf'
# Two separate reasons. Write both down.
```

```
# --- 6. Harden, and measure the difference --------------------------
python m16_security.py
# Read shop/security.conf first and predict the score.

# --- 7. Check the shop still works ----------------------------------
python waitfor.py
python m10_checkout.py --repeat 1
python m10_checkout.py --clean
# A hardening change that breaks checkout is not a hardening change.

# --- 8. Close the last finding --------------------------------------
# Rename the admin directory AND the path in admin/config.php. Then
# re-run the battery. Say why this is weaker than the other fixes.

# --- 9. Write the five lines you would hand to a client -------------
# What you changed, what is still open, what each one would cost,
# and which one you would do first.
```

What to notice. Step 4 against step 3. The attacks you have heard of were already stopped; the shop was giving away its error log. Security attention goes where the stories are, and the stories are about the wrong layer.

Checkpoint — Test Yourself

1. A colleague renames `.htaccess.txt` to `.htaccess` on this stack and reports the shop secured. Give the two independent reasons that is wrong, say what you would run to demonstrate it in one command, and state the general lesson about verifying controls.
2. The battery went from 5 of 15 to 14 of 15 with one configuration file. Explain why none of the nine fixes touched the shop's code, what that says about where to look when you inherit a shop, and which single remaining finding you would address first and why.
3. Three controls in this chapter were written correctly, accepted by Apache without complaint, and did nothing. Name the mechanism behind any two, and describe a working habit that would catch all three without knowing any of them in advance.

Chapter Summary

- Every exercise is confined to your own container on `localhost`. Testing anything else is a criminal offence under PECA 2016, with no educational exemption.
- Measured: SQL injection and cross-site scripting against the search box were both already handled — the injection string was searched for literally and returned fewer results, and the script tag appeared only escaped. The famous attacks were not the problem.
- Measured: the shop as installed passed 5 of 15 checks. After one configuration file it passed 14. Nothing in the shop's code changed.
- The serious finding was the PHP error log, readable over HTTP — confirmed by writing a marker into it and fetching it back.
- The shipped protection fails twice over: `.htaccess.txt` is never renamed, and `AllowOverride None` means it would not be read if it were. The advice repeated everywhere online for this platform does nothing on a modern deployment.
- Three controls were applied correctly and silently did nothing: `Header always unset` on a successful response, `ServerTokens` after a reload rather than a restart, and a file that sorted before Debian's own. Verify the result, never the action.

- What a header cannot fix needs a decision: move the admin path, move storage out of the web root, and serve over HTTPS. Prefer the control that does not depend on somebody remembering.
- The default credentials used throughout this course are documented rather than hidden, and are acceptable for a local container and nothing else.

Review Questions

1. **(MCQ)** The stock shop passed how many of the fifteen checks? (a) 0; (b) 5; (c) 11; (d) 14.
2. **(MCQ)** Renaming `.htaccess.txt` to `.htaccess` on this stack achieves: (a) full protection; (b) partial protection; (c) nothing, because Apache does not read it; (d) nothing, because the file is empty.
3. **(Short)** Explain what `X-Content-Type-Options: nosniff` prevents, and connect it to Chapter 7's upload findings.
4. **(Short)** Why is `Strict-Transport-Security` deliberately absent from this chapter's hardening file?
5. **(Short)** Give one reason moving `system/storage/` out of the web root is better than blocking it in the server configuration, even though both work today.
6. **(Applied)** You inherit a shop you did not build. Write the order in which you would investigate it, with the specific command or observation at each step, and say at which point you would stop and report rather than continue — and why that point exists.
7. **(Applied)** Design a Content-Security-Policy for a shop whose theme uses inline scripts, a payment provider's hosted field, and an analytics service. State each directive, what it allows, what it blocks, how you would roll it out without breaking the shop, and how you would know it was working.

Part V

Running and Extending the Shop

Launching a shop is a day's work; running one is the job. Part V is the half nobody demonstrates — the dashboard and which of its numbers deserve action, the back office where orders are actually fulfilled and refunded, the first thing the platform cannot do, and everything that happens after you deploy.

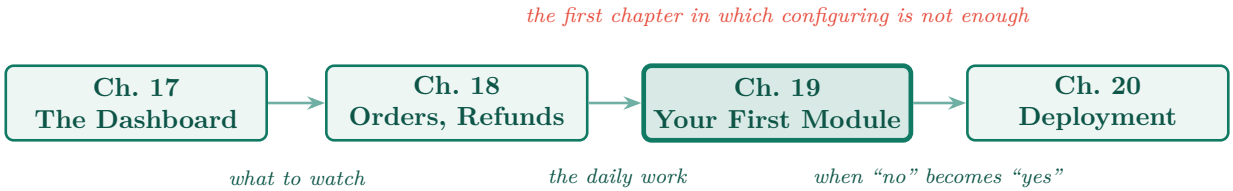


Figure P5.1: Reading path through Part V. Chapter 19 is where the course earns the words *Application Development* in its title; everything before it could have been done without writing code.

Chapter 17

Administration: The Dashboard, and What to Watch

Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** what each dashboard widget counts, and **state** the filter behind it.
- **Explain** why the dashboard's numbers disagree, and **reconcile** them against the underlying table.
- **Distinguish** a figure that describes the shop from a figure that would change a decision.
- **Derive** three measurements this course has produced that the dashboard does not show, and **write** the query for each.
- **Design** a panel for a stated shop, and **justify** every number on it.

Before You Start

Chapter 10: order statuses, and status 0. Chapter 6: reading the query a screen actually runs. Chapter 11 and Chapter 13: the costs that make some numbers worth watching and others not.

Key Terms in This Chapter

dashboard • widget • report • filter • vanity metric • actionable metric • reconciliation • abandonment rate • conversion • leading and lagging indicator • denominator

17.1 Eight Widgets, and What Each One Counts

The administrator's first screen carries eight panels. They are extensions like any other, each switched on at **Extensions › Extensions › Dashboard**, each with a width and a sort order.

Widget	Shows	Worth knowing
`order`	Orders today, yesterday, in total	Counts every order above status 0
`sale`	Money, over the same periods	Same filter, different query
`customer`	Registered customers	Not buyers — registrations
`online`	Visitors active now	**Empty unless you switch on tracking**
`activity`	What customers have been doing	**Also empty by default**
`chart`	Orders over time	**Counts only completed statuses**
`map`	Where orders came from	By billing country
`recent`	The last few orders	The most useful of the eight

Two of the eight are permanently empty, and that is a setting

`config_customer_online` and `config_customer_activity` are both **0** in a fresh install, so the `online` and `activity` panels have nothing to show and never will until somebody changes them.

Before switching them on, recall Chapter 9. Both work by writing a row for each visitor's activity — so turning them on adds a database write to page views that previously had none, on a shop whose session table already accumulated 645 rows without a single customer. The panels are not free; they are paid for in writes on your busiest pages.

Decide what you would do differently if you knew. If the answer is "nothing", leave them off. That question is the whole of this chapter.

17.2 The Numbers Disagree, and Nothing Says So

The four headline figures are produced by different queries. This is not a fault — each filter is right for its own purpose — but the screen presents them side by side with no indication that they are answering different questions.

Worked Example:

A known set of orders. Produced by `code/m17_dashboard.py`: five orders paid for through the checkout, two of which were then moved to **Complete**, plus one abandoned at the payment button.

What is actually in `oc_order`:

Status	Orders	Value	
0	1	854.40	*not a status at all*
1	3	2,563.20	Pending
5	2	1,708.80	Complete

What each part of the dashboard reports of exactly that data:

	Orders	Value	Filter
Orders widget	5	4,272.00	status > 0
Sales widget	5	4,272.00	status > 0
The chart	**2**	**1,708.80**	status in $(5, 3)$
The table itself	6	5,126.40	everything

Three different answers to "how many orders", all correct. Six in the table, five on the widgets, two on the chart — and a shopkeeper looking at the screen sees "5" above a chart whose bars add up to 2, with nothing explaining the difference.

Both gaps are information, once you know they are there.

Six minus five is the abandonment. One order in six reached the payment button and stopped. That is the single most useful number a small shop can compute, and it exists only as the difference between two figures the dashboard shows separately.

Five minus two is the work not yet done. Three orders are sitting at Pending: paid for, not shipped. On a Monday morning that is the day's job. On a Friday it is a problem.

And the chart's filter is a *setting*, not a constant: `config_complete_status` is `["5","3"]` here, so Shipped counts as complete. Change it and every historical bar changes with it.

How to read any dashboard you did not build

1. **For each number, ask what it counts and what it excludes.** The answer is in a query, and you now know how to find it.
2. **Reconcile against the table.** If the parts do not add up to the whole, the difference has a name — and the name is usually the interesting number.

3. **Find which filters are settings.** Those can change under you without any code changing.
4. **Check the denominator.** "Conversion" means nothing until you know whether the bottom of the fraction is visitors, sessions, baskets or people.

17.3 What the Dashboard Does Not Show

Every measurement in this course that would change a decision is absent from the administrator's first screen. That is not an accident: the dashboard shows what is cheap to count, and those are not the same things.

</> Three panels this course has already earned

1. **Abandonment rate.** From Chapter 10: orders at status 0 against orders above it, by day. `SELECT DATE(date_added) d, SUM(order_status_id=0) abandoned, SUM(order_status_id>0) placed FROM oc_order GROUP BY d ORDER BY d DESC;` A shop with a rising ratio has a checkout problem, a payment problem or a price problem, and knows to look before customers tell it.
2. **Products that cannot be delivered profitably.** From Chapter 11: enabled products with a weight of zero, which under weight-based shipping are quoted free delivery. `SELECT COUNT(*) FROM oc_product WHERE status=1 AND weight=0;` A number that should be zero and silently is not.
3. **Orders standing still.** From this chapter: how long orders have been at Pending. `SELECT order_status_id, COUNT(*) n, MIN(date_added) oldest FROM oc_order WHERE order_status_id>0 GROUP BY order_status_id;` The widgets say how many orders arrived. Nothing says how many are waiting, or for how long — which is the only one of the three a shopkeeper can act on this morning.

All three are one query each, and all three are Chapter 19's subject: a dashboard widget is the smallest useful module you can write.



Figure 17.1: A test for every number you are thinking of putting on a screen. All eight shipped widgets sit at the left-hand end.

Common Pitfalls

- **Comparing two dashboard numbers that have different filters.** Five orders and a chart showing two is not a bug report.
- **Reporting revenue from a table count** without excluding status 0.
- **Switching on the online and activity panels** without noticing the write they add to every page view.
- **Treating `config_complete_status` as a constant.** It is a setting, and changing it rewrites history.
- **Adding a number nobody would act on.** Every panel costs a query on the busiest screen in the admin panel.
- **Quoting a conversion rate without naming its denominator.**
- **Watching totals rather than ages.** "How many orders" is a report; "how long has this one been waiting" is a job.

Try It Yourself: Make the dashboard disagree with itself, then fix the gap

```
# --- 1. Build a known set of orders ---------------------------------
cd code
python m17_dashboard.py

# --- 2. Now look at the admin dashboard -----------------------------
# Admin: Dashboard. Write down the four headline numbers and what the
# chart shows. Compare with what step 1 printed.

# --- 3. Find the filter behind each number --------------------------
docker compose exec shop sh -c \
  'grep -n "order_status_id" admin/model/sale/order.php | head -3'
# Then do the same for the chart's model. Three different filters.

# --- 4. Account for both gaps ---------------------------------------
# 6 in the table, 5 on the widgets, 2 on the chart. Name each gap.

# --- 5. Change a setting and watch history move ---------------------
# Admin: System > Settings > Edit > Option. Add "Processing" to the
# complete order statuses. Reload the dashboard. Which numbers moved?

# --- 6. Write the three queries the dashboard does not have ---------
# Abandonment by day; enabled products with zero weight; orders by
# status with the age of the oldest. Run each one.

# --- 7. Decide what you would put on the screen ---------------------
# Six panels, no more. For each: the number, the query, and one
# sentence saying what you would DO if it moved.

# --- 8. Clean up ----------------------------------------------------
python m17_dashboard.py --clean
```

What to notice. Step 7 is harder than it looks. Most proposed panels fail the "what would you do" test, and the ones that survive are rarely the ones that were easiest to count.

Checkpoint — Test Yourself

1. The table holds 6 orders worth 5,126.40; the widgets report 5 orders worth 4,272.00; the chart shows 2. Explain each gap, say which of the three figures you would put in front of the shop's owner, and what you would label it.

2. A shop owner says "our conversion rate is 3%". List the questions you would ask before accepting that number, and explain why two shops both reporting 3% may not be comparable at all.
3. Propose a dashboard panel that would have caught Chapter 10's broken guest checkout — where every page returned 200 and no order could be placed. State the query, how often it runs, and the condition that raises the alarm.

Chapter Summary

- Eight widgets ship. Two of them — online and activity — are empty until tracking is switched on, and that tracking costs a database write on page views that previously had none.
- Worked: the same six orders were reported as 6, 5 and 2 by the table, the widgets and the chart. All three are correct; none is labelled with its filter.
- The gaps are the information. Table minus widgets is the abandonment rate; widgets minus chart is the work not yet done.
- The chart's filter is `config_complete_status`, a setting — so changing it rewrites every historical bar.
- To read any dashboard: ask what each number excludes, reconcile against the table, find which filters are settings, and check the denominator.
- A number earns its place on a screen only if somebody would do something differently on seeing it move. Most shipped widgets describe; few change a decision.
- Three panels this course has already earned, one query each: abandonment by day, enabled products with zero weight, and orders by status with the age of the oldest.
- Totals are a report; ages are a job. Prefer the number that names today's work.

Review Questions

1. (MCQ) The dashboard chart counts orders whose status is: (a) greater than zero; (b) in `config_complete_status`; (c) any status; (d) Pending only.

2. (MCQ) The difference between the order count in the table and the order count on the widgets is: (a) a bug; (b) refunded orders; (c) orders abandoned at the payment button; (d) orders from other stores.

3. (Short) Explain why switching on the "customers online" panel is not free, referring to a finding from Chapter 9.

4. (Short) Give the test that decides whether a number belongs on a dashboard, and apply it to "total registered customers".

5. (Short) Why does changing `config_complete_status` alter charts of months that have already happened?

6. (Applied) A shop's orders widget has been flat for three weeks while its sales widget has fallen 20%. Give three distinct explanations consistent with both, say what you would query to distinguish them, and state which explanation you would investigate first and why.

7. (Applied) Design the dashboard for a shop taking 40 orders a day on cash on delivery. Give six panels, the query behind each, how often it refreshes, and for every one a sentence beginning "if this moves, we will…". Justify anything you left off that the shipped dashboard includes.

Chapter 18

Managing Orders, Customers and Refunds

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** how the admin panel changes an order, and **say** why it does not do so directly.
- **Predict** which status changes move stock, from the rule rather than from the status names.
- **Distinguish** the two things the platform calls a refund, and **state** what each one actually does.
- **Identify** the work a refund requires that no screen performs.
- **Design** a back-office procedure that keeps money, stock and records in agreement.

Before You Start

Chapter 10: order statuses, the history table, and status 0. Chapter 17: that the dashboard's figures are filtered differently. Chapter 14: reconciliation, and that the shop's record and the provider's must be made to agree.

Key Terms in This Chapter

back office • order history • status transition • stock subtraction • return • return action • refund • store credit • customer transaction • API signature • allow-list • audit trail • reconciliation

18.1 The Admin Panel Does Not Edit Orders

The first surprise of the back office is architectural. When an administrator changes an order's status, the admin panel does not write to the database. It **signs a request and calls the storefront's own API over HTTP**, and the storefront makes the change.

Why that is the right design

Three things must happen when an order's status changes: stock must move, the customer may need an email, and a history row must be written. Those rules are not simple, and they must be identical whether the change comes from a human clicking a dropdown, from a payment provider's callback (Chapter 14), or from a script.

Writing them once, in the storefront, and having everything else call in is the only arrangement in which they cannot drift apart. The alternative — the admin panel having its own copy — is how a shop ends up with orders whose stock was never adjusted because they were edited from the wrong screen.

The consequence nobody mentions: the admin panel cannot change an order unless the API accepts the call. The back office is a client of the shop, and it can be locked out of it.

Measured: Out of the box, the API refuses every call

Produced by running `m18_orders.py -api` in `code/`. The request is signed exactly as the startup controller verifies it: an HMAC-SHA1 over the route, the call, the username, the host, the store, the language, the currency, an MD5 of the request body and a timestamp.

Rows in `oc_api_ip`	**0**
A correctly signed call	`HTTP 403 -- You do not have permission`
After adding one address	accepted

The API user ships enabled, and its address allow-list ships empty. An empty allow-list permits nothing — the check is "is the caller's address in this list", and the answer for an empty list is always no. So every signed call is refused, including the admin panel's own.

This is the third fault of this shape in the course, after Chapter 10's missing setting and Chapter 16's unread `.htaccess`: a feature that is present, configured, enabled, and inert. In each case the system answered, politely, and did nothing.

18.2 Stock Moves on Transitions, Not on Statuses

The rule that governs stock is not written on any screen, and it is not what the status names suggest.

The rule, from `catalog/model/checkout/order.php`

The statuses are partitioned into a set — the union of `config_processing_status` and `config_complete_status` — and everything else.

Moving *into* that set subtracts stock. Moving *out of* it adds stock back. Moving within it does nothing.

In this shop the set is $\{1, 2, 3, 5, 12\}$ — Pending, Processing, Shipped, Complete and *Reversed*. Everything else, including 0, Canceled, Denied, Failed and *Refunded*, is outside it.

Measured: One order, seven status changes, and the stock after each

From `code/m18_orders.py -stock`: one order for one unit of product 43 and two of product 40, driven through the API.

Status	Where it sits	Stock 43	Stock 40
(before the order)	—	904	920
1 Pending	in the set	903	918
3 Shipped	still in the set	903	918
7 Canceled	**outside**	**904**	**920**
1 Pending	back inside	903	918
11 Refunded	**outside**	**904**	**920**
1 Pending	back inside	903	918
12 Reversed	**in the set**	**903**	**918**

Shipped changed nothing, although it is the status at which the goods physically leave. The stock left when the order was paid for.

Refunded and Reversed behave oppositely. Both names mean money went back to the customer. Status 11 returns the goods to stock; status 12 does not, because somebody put 12 in `config_processing_status`. Nothing on any screen says so, and the two statuses sit next to each other in the dropdown.

This is a setting, so it is your problem. Whoever configures a shop decides which statuses are "processing", and in doing so decides the stock behaviour of every status change the shop will ever make. It is one of the highest-consequence settings in the admin panel and it looks like a list of checkboxes.

18.3 Two Different Things Are Called a Refund

⚠ A return marked "Refunded" does nothing at all

The platform has a returns system: **Sales › Returns**, with its own statuses (Pending, Awaiting Products, Complete), five reasons, and four actions — **Waiting for return**, **Credit Issued**, **Replacement Sent** and **Refunded**.

Reading `admin/model/sale/returns.php`, recording a return performs exactly two statements:

- `UPDATE oc_return SET return_status_id = ...`
- `INSERT INTO oc_return_history ...`

And nothing else. It does not change the order's status. It does not restore stock. It does not create a customer transaction or any store credit. It does not move a single rupee.

A return marked "Refunded" is a note. It records that somebody decided to refund. The money must be sent back by hand through the payment provider, the stock adjusted by hand, and the order's own status changed by hand — and if none of that is done, the order still reads **Complete** to everyone who looks at it, including the dashboard and every report.

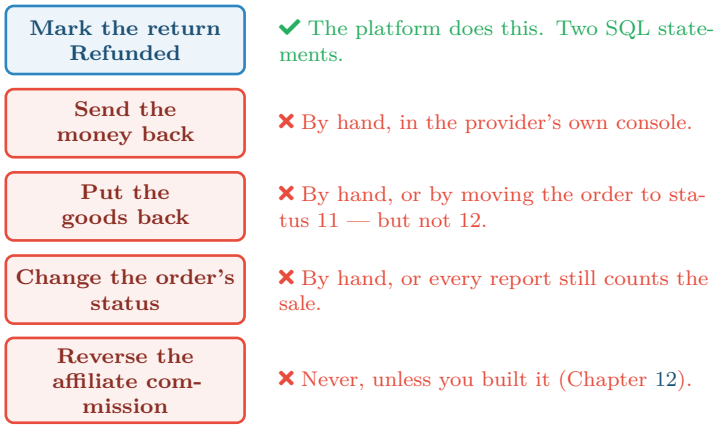


Figure 18.1: What a refund actually requires, and the one part of it the software performs. Four of the five are somebody's responsibility, and in most small shops nobody has been told which.

The procedure a shop actually needs

Because the software performs one fifth of a refund, the other four fifths have to be a written procedure, in this order:

1. **Send the money back first**, through the provider, and record the provider's reference.
2. **Move the order to status 11**, not 12, which returns the goods to stock — and only if the goods actually came back.
3. **Record the return** with the provider's reference in the comment, so that the shop's record and the provider's can be matched later (Chapter 14).
4. **Reverse any commission**, which nothing will do for you.
5. **Check the dashboard agrees** the next morning. The gap between what the shop thinks it sold and what the provider says it collected is the only thing that catches a step somebody forgot.

Steps 2 and 3 are in that order on purpose. A return recorded before the order is moved leaves a window in which the shop's records say "refunded" and its stock and totals say "sold". The window is usually a few minutes, and month-end is when somebody finds the one that lasted a year.

18.4 Customers, and What You Are Keeping

The customer side of the back office is shorter but carries more risk, because the records are about people.

What Customers › Customers lets you do, and the question attached to each

- **Edit a customer's details.** Who is allowed to, and is the change recorded? An order copies its addresses (Chapter 10), so editing the customer does not rewrite history — which is correct, and worth confirming rather than assuming.
- **Reset a password.** The shop cannot show you the old one, and anyone who offers to has stored it wrongly (Chapter 15).
- **See everything they bought.** Convenient for support, and a complete purchase history of a named person — which Chapter 22 treats as the regulated asset it is.
- **Add store credit** (`oc_customer_transaction`). This is real money the shop owes, and it is the one place in the back office where an administrator can create a liability with no approval step.
- **Delete a customer.** Their orders remain, because the orders are the shop's accounting records and are not the customer's to delete. Be able to explain that distinction before you are asked it.

Common Pitfalls

- **Assuming the admin panel writes to the database.** It calls the storefront, and can be refused.
- **Leaving `oc_api_ip` empty** and then reporting that order editing is broken without knowing why.
- **Reading the status names to predict stock.** Refunded returns stock and Reversed does not.
- **Putting a money-returned status into `config_processing_status`**, which is how Reversed got there.
- **Believing a return marked Refunded refunded anything.**
- **Recording the return before moving the order**, leaving a window where the two disagree.
- **Refunding without reversing commission**, which Chapter 12 named as the commonest unreconciled figure in a small shop.

- **Editing an order to "correct" it** instead of changing its status, which loses the history a dispute would be settled from.

Try It Yourself: Refund an order properly, then find what the shop still believes

```
# --- 1. Find out why order editing does not work --------------------
cd code
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT * FROM oc_api; SELECT * FROM oc_api_ip;"
# One table has a row and the other does not. Say what that means.

# --- 2. Prove it from outside ---------------------------------------
python m18_orders.py --api

# --- 3. Predict the stock table, then watch it ----------------------
# Which of these returns goods to stock: Shipped, Canceled, Refunded,
# Reversed? Commit to an answer for each BEFORE running:
python m18_orders.py --stock

# --- 4. Explain the two that surprised you --------------------------
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT value FROM oc_setting \
      WHERE \`key\` IN ('config_processing_status', \
                        'config_complete_status');"

# --- 5. Record a return in the admin panel --------------------------
# Admin: Sales > Returns > Add New. Action: Refunded. Save.
# Now check: did the order's status change? Did stock move? Did any
# money move? Check all three in the database, not on the screen.
python m18_orders.py --refund

# --- 6. Do the refund properly --------------------------------------
# Follow the five-step procedure in this chapter. Write down which
# steps the software performed and which you performed.

# --- 7. Find the disagreement ---------------------------------------
# Admin: Reports > Sales > Orders. Does the total match what the shop
# actually kept? Account for every difference.

# --- 8. Write the procedure -----------------------------------------
# One page a shop assistant could follow, with the order of steps
# and a stated reason for that order.

# --- 9. Clean up ----------------------------------------------------
python m18_orders.py --clean
```

What to notice. Step 5 is the one students argue about. The screen says **Refunded**, the save succeeds, a confirmation appears — and the database shows two rows written in a table nothing else reads.

Checkpoint — Test Yourself

1. An order is moved from Pending to Shipped, then to Reversed. State what happened to stock at each step and why, and say what you would change in the shop's configuration if you wanted Reversed to behave like Refunded.
2. A shop records 40 returns a month, all marked **Refunded**, and its stock figures drift steadily away from reality. Explain the mechanism precisely, say what else is also drifting, and give the smallest change that would stop it.

3. The admin panel reports that it cannot change an order. Describe how you would diagnose this in four steps, naming what you would look at each time, and explain why the fault is easy to misread as a permissions problem with the administrator's account.

Chapter Summary

- The admin panel changes an order by signing a request and calling the storefront's API, so that stock, notification and history follow one code path whatever triggered the change.
- Measured: the API user ships enabled with an *empty* address allow-list, which permits nothing — so every signed call is refused, including the admin panel's own. The third inert-feature fault in this course.
- Stock is decided by the transition between sets of statuses, not by the status. Moving into the processing-or-complete set subtracts; moving out of it adds back; moving within it does nothing.
- Measured: Shipped moved no stock. Canceled and Refunded returned it. **Reversed did not**, because 12 sits in `config_processing_status` — a setting, and therefore your responsibility.
- A return marked **Refunded** performs two SQL statements and nothing else: no order status change, no stock, no credit, no money. It is a note.
- A refund is five jobs and the software does one. The other four need a written procedure, in a stated order, because the order of steps decides how long the records disagree.
- Store credit is a liability an administrator can create with no approval step. Deleting a customer does not delete their orders, and should not.

Review Questions

1. **(MCQ)** Moving an order to status 12 (Reversed) causes stock to: (a) be returned; (b) be subtracted again; (c) stay as it is; (d) depend on the payment method.

2. **(MCQ)** Recording a return with the action **Refunded**: (a) refunds the customer; (b) restores stock; (c) changes the order's status; (d) writes two rows and does nothing else.

3. **(Short)** Explain why the admin panel calls the storefront's API instead of writing to the database, and give one problem that design prevents.

4. **(Short)** Why does an empty `oc_api_ip` refuse every call rather than allowing every call? Give the general lesson about allow-lists.

5. **(Short)** An order's addresses are copied at checkout. Explain what this means for editing a customer's address afterwards, and why it is the correct behaviour.

6. **(Applied)** A shop's stock counts have drifted by several hundred units over six months. Give four distinct mechanisms from this chapter that could produce that, say how you would test for each in the database, and state which you would expect to find first.

7. **(Applied)** Write the refund procedure for a shop taking 200 orders a day with a 6% return rate, two staff in the back office and a payment provider whose console is separate from the shop. State each step, who performs it, what is recorded, the order and the reason for the order, and the daily check that would catch a missed step.

Chapter 19

Extending the Platform: Your First Module

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what an extension is made of, and **state** why each part exists.
- **Write** a storefront module and a shipping method that add behaviour without editing anything the platform ships.
- **Install** an extension the way the marketplace installer does, in three steps you can read.
- **Measure** what a module costs a page, and **account** for every query it adds.
- **Judge** when to extend, when to configure, and when to refuse.

Before You Start

Chapter 8: themes, templates and overrides. Chapter 11: the shipping quote interface and tax classes. Chapter 6: counting the queries a page runs. Appendix B for enough PHP and Twig to read the files.

Key Terms in This Chapter

extension • module • extension point • namespace • autoloading • manifest • controller • model • language file • module instance • layout position • upgrade safety • permission check

19.1 Why Not Just Edit the File

Every feature in this chapter could be added faster by opening the shop's own code and changing it. The reason not to is one sentence long, and it arrives about eighteen months later: **the next upgrade overwrites your file, and nothing tells you which ones you changed.**

The three ways to change what a shop does, in order of preference

1. **Configure it.** If a setting exists, use it. Chapter 11's tax rules and Chapter 17's widgets are settings, and a setting survives everything.
2. **Extend it.** Add a directory under `extension/` that the platform loads. Nothing it ships is touched, the upgrade leaves your directory alone, and `git status` tells you exactly what you added.
3. **Edit it.** Only when the first two cannot reach, and then with the change recorded somewhere an upgrade will make you re-read.

Most commissioned work on this platform is the second. That is what the department's title adds to HEC's bare "E-Commerce", and it is the difference between configuring a shop and developing one.

19.2 What an Extension Is Made Of

Extension, Module, Instance

An **extension** is a directory under `extension/` whose internal shape mirrors the shop's own — an `admin/` half, a `catalog/` half, and a manifest. It may provide several things at once: this chapter's provides a module and a shipping method.

A **module** is a block of storefront output that a layout can place. It has two controllers: one that renders it for a customer, one that lets the shopkeeper configure it.

An **instance** is one placement of a module, with its own settings. The same module may appear three times on a page with three different configurations, which is why its render method is handed its settings rather than reading them from the global configuration.

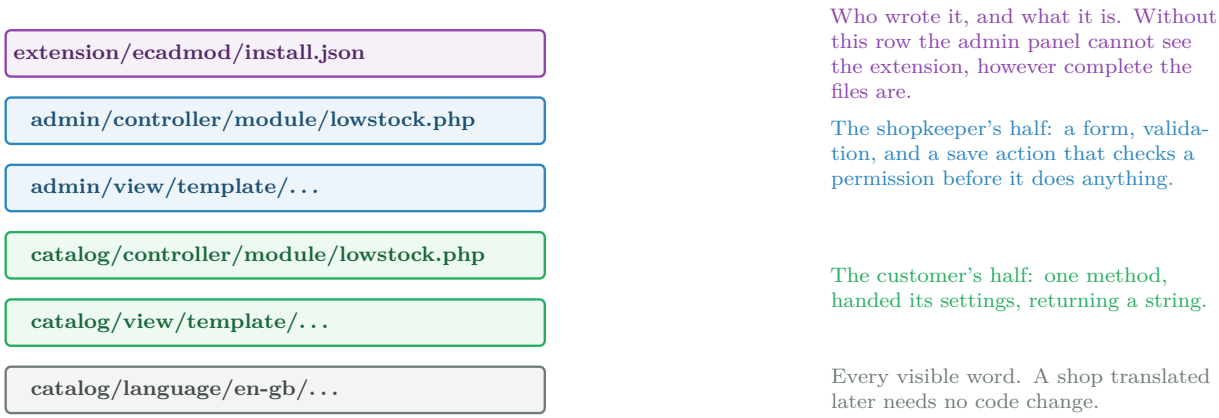


Figure 19.1: The twelve files of this chapter's extension. The namespace of every class is its path, which is how the loader finds it and the first thing to get wrong.

The namespace is the path, and the error message is unhelpful

The class

`Opencart\Catalog\Controller\Extension\Ecadmod\Module`

lives in `extension/ecadmod/catalog/controller/module/`. Get a single segment wrong — the wrong case, a missing `Extension`, the folder named differently from the namespace — and the loader reports that the class does not exist.

Which is true, and tells you nothing about why. **When an extension does not load, check the namespace against the path before anything else**; it is the cause more often than every other possibility combined.

19.3 The Storefront Half, in Full

The whole customer-facing side of the module is one method. It is worth reading in full, because almost nothing in it is specific to this module.

```
namespace Opencart\Catalog\Controller\Extension\Ecadmod\Module;

class LowStock extends \Opencart\System\Engine\Controller {
    public function index(array $setting): string {
        $this->load->language('extension/ecadmod/module/lowstock');

        $data['heading_title'] = $this->language->get('heading_title');
        $threshold = (int)($setting['threshold'] ?? 5);
```

```
        $limit     = (int)($setting['limit'] ?? 4);

        $query = $this->db->query("SELECT p.product_id,
            p.quantity, pd.name FROM ...
            WHERE p.status = '1' AND p.quantity > '0'
              AND p.quantity <= '" . (int)$threshold . "'
            ORDER BY p.quantity ASC LIMIT " . (int)$limit);
```

and then, in the same method, the rows become view data and the view becomes a string:

```
        $data['products'] = [];

        foreach ($query->rows as $row) {
            $data['products'][] = [
                'name'     => $row['name'],
                'quantity' => $row['quantity'],
                'href'     => $this->url->link('product/product',
                    'language=' . $this->config->get('config_language')
                    . '&product_id=' . $row['product_id'])
            ];
        }

        if (!$data['products']) { return ''; }

        return $this->load->view(
            'extension/ecadmod/module/lowstock', $data);
    }
}
```

Four decisions in thirty lines

1. **It takes `$setting` as an argument** rather than reading `$this->config`. That is what allows the module to be placed twice with different thresholds.
2. **It returns a string and never prints.** The layout decides where the string goes; a module that echoes has taken a decision that is not its own.
3. **It returns `''` when there is nothing to say.** An empty panel with a heading is worse than no panel.
4. **Every integer is cast.** `(int)$threshold` inside a query string is the difference between a parameter and an injection — and Chapter 16 showed that the platform is careful about this everywhere, which means your extension is now the weakest part.

19.4 Installing It, in Three Steps You Can Read

The marketplace installer is a button. What it does is three things, and doing them by hand once is the fastest way to understand why an extension that looks installed sometimes is not.

What `code/m19_modules.py -install` performs

1. **Copy the files** to `extension/ecadmod/` and make them readable by the web server. Twelve files.
2. **Insert the manifest row** into `oc_extension_install` — name, author, version, link. **Without this the admin panel cannot see the extension at all**, however complete the files are. This is the step people miss, and the symptom is an extension that is plainly on disk and absent from every list.
3. **Insert one row per thing provided** into `oc_extension`: `(ecadmod, module, lowstock)` and `(ecadmod, shipping, zonerate)`. This is what makes each appear in its own list rather than all of them in one.

Uninstalling is the same three in reverse, and the script does that too — because an extension you cannot remove cleanly is one you cannot test.

19.5 What a Module Costs

The objection to building features as modules is that they must be slow. It is worth measuring rather than assuming, in both directions.

Measured: The same home page with 0, 1, 3 and 5 instances

From `code/m19_modules.py -measure` against the stack in Appendix A. Each instance is a separate placement of the same module on the home layout.

Instances	Page	Queries	ms each	queries each
0	76 ms	117	—	—
1	82 ms	121	5.8	**4.0**
3	90 ms	129	4.7	**4.0**
5	96 ms	137	4.0	**4.0**

The cost is linear and small: about 4 ms and exactly four queries per instance. Five instances added 20 ms to a 76 ms page. A module is not expensive.

But the module contains *one* query. The other three are the framework's, and reading them from the slow log names each one:

`SELECT * FROM oc_module WHERE module_id=16`	this instance's settings
`SELECT * FROM oc_translation WHERE route=...`	its translations
`SELECT p.product_id, p.quantity ...`	**the one I wrote**
`SELECT * FROM oc_theme WHERE route=...`	its template override

Three quarters of what a module costs is the extension mechanism itself, spent finding your settings, your words and your template. That is the price of the arrangement that lets you place the module twice, translate it without touching code, and override its template from a theme.

It is also the explanation for Chapter 6's 144 queries on a category page, which was never explained at the time: a page is mostly modules, and every one of them pays this toll.

When to extend, and when to refuse

Extend when the behaviour is yours, the platform offers a point for it, and you can state what the shop would do without it.

Configure instead when a setting already reaches. Rewriting a setting as a module is the commonest waste in commissioned work.

Refuse when the request is for something the platform deliberately does not do, and the reason it does not is a good one. "Store the card number so customers do not have to retype it" is a module somebody will ask you to write, and Chapter 14 is the reason to say no — in writing, with the consequence named.

Common Pitfalls

- **Editing a shipped file** because it is faster today.
- **A namespace that does not match the path.** The error says the class does not exist, which is unhelpfully true.
- **Forgetting the `oc_extension_install` row**, which leaves an extension on disk and invisible.
- **Reading `$this->config` in a module's render method** instead of its `$setting` argument, so it cannot be placed twice.
- **Omitting the permission check in a save action.** The form is only shown to an administrator; the action is a URL.
- **Interpolating an uncast value into SQL.** The platform is careful everywhere else, so your extension becomes the weakest part.
- **Returning a tax-inclusive cost from a shipping quote**, which the totals pipeline then taxes again (Chapter 11).
- **Hard-coding visible text** in a controller or template rather than a language file.

Try It Yourself: Write, install, measure, and then break it

```
# --- 1. Read the twelve files before running anything ---------------
cd code
find modules/ecadmod -type f | sort
# Match each class's namespace to its path. Say the rule out loud.

# --- 2. Install it the way the marketplace would --------------------
python m19_modules.py --install

# --- 3. Place it, and look ------------------------------------------
python m19_modules.py --place 1
# Open the storefront. Nothing shows -- the demo products all have
# high stock. Fix that by raising the threshold, and say why an
# empty module returning '' is better than an empty panel.

# --- 4. Predict the cost, then measure it ---------------------------
# How many queries does ONE instance add? Commit to a number.
python m19_modules.py --measure

# --- 5. Account for all four ----------------------------------------
docker compose exec db mariadb -uroot -prootpw -e \
  "SELECT LEFT(CONVERT(sql_text USING utf8),80) FROM mysql.slow_log;"
# One is yours. Name the other three and say what each is for.

# --- 6. Break the namespace on purpose ------------------------------
# Rename the class to Lowstock (small s) and reload. Read the error.
# This is the error you will see most often; meet it deliberately now.

# --- 7. Configure your shipping method ------------------------------
# Admin: Extensions > Extensions > Shipping > Zone Rate > Edit.
# Rates one per line as zone_id:cost. Place an order and check the
# quote, then check oc_order_total for how the tax was applied.

# --- 8. Add the free-delivery threshold and test both sides ---------
# One order below it, one above. Verify from oc_order_total, not
# from the screen.

# --- 9. Remove it completely ----------------------------------------
python m19_modules.py --remove
# Then confirm the shop is exactly as it was: no files, no rows,
```

```
# no settings. An extension you cannot remove is one you cannot test.
```

What to notice. Step 5 is the chapter. Your module is one query; it costs four. Knowing which three are the mechanism's is what lets you judge whether a page with twenty modules is badly written or merely busy.

Checkpoint — Test Yourself

1. Five instances of a module added 20 ms and 20 queries to a 76 ms page. A colleague concludes modules are expensive and proposes editing the theme instead. Give your answer, with the per-instance breakdown, and say what you would measure before agreeing to anything.
2. A module's render method reads its threshold from `$this->config->get('module_lowstock_threshold')` instead of from its `$setting` argument. The shop works. Describe exactly when it stops working and what the shopkeeper will report.
3. You are asked to add a field to the checkout that the platform does not offer. Set out how you would decide between configuring, extending and editing, what you would need to know first, and what you would write down whichever you chose.

Chapter Summary

- Configure first, extend second, edit last. An extension is a directory the upgrade leaves alone; an edit is a file the upgrade overwrites without telling you.
- An extension mirrors the shop's shape: an admin half, a catalog half, a manifest, and language files for every visible word.
- The namespace of every class is its path. A mismatch reports that the class does not exist, and is the commonest cause by a wide margin.
- A module's render method takes its settings as an argument, returns a string, returns `''` when it has nothing to say, and casts every value it puts in a query.
- Installing is three steps: the files, a manifest row in `oc_extension_install`, and one row per provided thing in `oc_extension`. Miss the second and the extension is on disk and invisible.
- Measured: 4 ms and exactly 4 queries per module instance; five instances added 20 ms to a 76 ms page. Modules are cheap.
- Measured: of those four queries, *one* is the module's. The other three find its settings, its translations and its template — the price of being placeable, translatable and overridable, and the reason a category page runs 144 queries.
- A shipping method's whole interface is one method that is handed an address and returns a quote without tax, or nothing at all.
- Refusing is a professional act. "Store the card number" is a module somebody will ask for; decline it in writing, with the consequence named.

Review Questions

1. **(MCQ)** One instance of the measured module added how many database queries? (a) 1; (b) 2; (c) 4; (d) it varies with the catalogue.

2. **(MCQ)** An extension whose files are all present but which appears in no admin list is most likely missing: (a) a language file; (b) its `oc_extension_install` row; (c) file permissions; (d) a template.

3. **(Short)** Explain why a module's render method is handed its settings instead of reading the global configuration.

4. **(Short)** Give the rule relating a class's namespace to its file path, and the symptom of breaking it.

5. **(Short)** Why must a shipping quote return a cost without tax? Refer to Chapter 11's totals pipeline.

6. **(Applied)** You are asked for a module showing "customers who bought this also bought". Specify it: what it queries, where its settings live, how many queries it will add per page, how you would keep that number from growing with the catalogue, and what you would measure before and after.

7. **(Applied)** A client wants a field on the product page that their staff fill in and customers see. Compare doing this with a product attribute (configuration), a module (extension) and a template edit. Give the work, the upgrade risk and the maintenance burden of each, and recommend one with your reasons.

Chapter 20

Deployment, Maintenance and Performance

Learning Outcomes

By the end of this chapter you will be able to:

- **Measure** a shop's throughput and latency together, and **locate** its saturation point.
- **Explain** why throughput can fall as concurrency rises, and **say** what that means for a sale day.
- **Quantify** what the compiler cache is worth, and **order** optimisations by measured effect.
- **Distinguish** the cost of a backup from the cost of a restore, and **state** which one matters.
- **Assemble** a deployment checklist from this course's findings, with nothing on it that was not measured.

Before You Start

Chapter 5: images, containers, volumes and the Compose file. Chapter 16: the hardening file and what it closed. Chapter 14: reading a distribution rather than an average.

Key Terms in This Chapter

throughput • latency • concurrency • saturation point • queueing • opcode cache • reverse proxy • backup • restore • recovery time • rollback • end of life • pinning • HSTS

20.1 Two Numbers, and Why One of Them Lies

"How fast is the shop" has two answers that move in opposite directions, and quoting either alone is how capacity is mis-planned.

Throughput, Latency, Saturation

Throughput is requests served per second. It is a property of the server, and it is the number in the sales brochure.

Latency is how long one request took. It is what the customer experiences, and it is the number in the complaint.

The **saturation point** is the concurrency at which throughput stops rising. Below it, extra customers are served from spare capacity. Above it, extra customers join a queue — so latency rises steeply while throughput stays flat or *falls*.

Measured: The same page, requested by more people at once

From `code/m20_deploy.py -load` against the stack in Appendix A. Sixty requests at each level, spread across that many concurrent clients.

Clients	req/sec	Median	p95	Worst	Failed
1	11.0	81 ms	147 ms	158 ms	0
5	**42.1**	95 ms	181 ms	258 ms	0

20	**38.7**	**455 ms**	667 ms	814 ms	0

Throughput peaked at five clients and then fell. Twenty concurrent customers were served at 38.7 requests per second — *fewer* than five were. More customers, less work done.

And the median response grew 5.6 times, from 81 ms to 455 ms. Every one of those twenty customers waited half a second for a page that one customer alone gets in a twelfth of that.

Nothing failed. No error, no timeout, no log entry. The shop was past its saturation point and the only observable was that it had become slow — which is the same observable as a badly written query, a slow network or a customer's own machine. Load does not announce itself.

The saturation point is the number to know before a sale day. It is five here, on a laptop, serving an uncached home page. What matters is not the figure but that there *is* one, that it is measurable in ten minutes, and that past it every additional visitor makes the shop worse for everybody already in it.

20.2 The Cheapest Optimisation Is Already a Setting

Before any caching layer, any reverse proxy and any rewriting of queries, there is one line of configuration worth measuring.

Measured: The same load, with the compiler cache off and on

From `code/m20_deploy.py -opcache`. PHP compiles source to bytecode; the opcode cache keeps the result, so the files are compiled once rather than on every request.

Clients	req/sec off	req/sec on	Gain	Median off → on
1	8.1	10.1	1.3×	109 → 82 ms
5	**16.8**	**34.0**	**2.0×**	245 → 131 ms
20	21.7	30.3	1.4×	715 → 497 ms

Twice the throughput at five clients, for one line of configuration. No code was changed, no caching layer added, no query rewritten.

The gain is largest in the middle, and that is not a coincidence. With one client there is spare processor to waste, so the compilation is almost free. At twenty, the queue dominates and the saving is diluted. At five — the saturation point — the processor is the constraint, and removing work from it is worth exactly what it costs.

Check this before optimising anything else. A shop running without it is being benchmarked as a compiler rather than as a shop, and every conclusion drawn from such a measurement is about the wrong program. Chapter 5's Dockerfile installs it for this reason, and said so at the time.

The order to work in, from this course's own measurements

1. **Confirm the compiler cache is on.** 2×, one line.
2. **Minify the stylesheet** (Chapter 8). 240 KB off every page, one file, no visible change.
3. **Index what the pages actually sort by** (Chapter 6) — and re-measure, because at 50,000 products the indexes made it *slower*.

4. **Cache whole pages** for visitors who are not logged in. The largest win available and the one that introduces staleness, so it comes after the three that do not.
5. **Add servers.** Last, because everything above is cheaper, and because Chapter 9 showed that file-based sessions stop working the moment there are two.

20.3 Backups, and the Number Nobody Measures

Measured: What a backup costs, and what a restore costs

From `code/m20_deploy.py -backup`.

Database dump	**1.0 s**	572 KB
Database restore	**6.4 s**	
Shop files	0.8 s	52 MB in 4,024 files

The restore takes six times as long as the backup, and it is the only one of the two numbers that matters — because it is the one you will be timing while the shop is down and somebody is standing behind you.

Those figures are for a demonstration catalogue. At Chapter 6's 50,000 products the dump was a different order of magnitude, and the restore grew with it. **Measure your own, on your own data, and write the number down**: it is your recovery time, and it is what you will be asked for.

A backup nobody has restored is a file, not a backup. The only way to know a backup works is to restore it somewhere and look at the shop. Do it on a schedule, not after an incident.

Three things in the platform's own deployment advice that are wrong

Recorded because they are the first things a student will find, and all three are dated rather than debatable.

- **Its own `Dockerfile` does not build**, because the one in the release contains the line `docker-php-ext-install zip && && docker-php-ext-enable zip` — a shell syntax error. It has therefore never worked for anybody.
- **Its own compose file pins `mysql:5.7`**, which reached end of life in October 2023 and takes no further security fixes. This course's stack uses MariaDB 11 and Chapter 5 said why.
- **Its security guidance says to rename `.htaccess.txt`**, which Chapter 16 measured as doing nothing at all on a modern Apache.

Upstream artefacts are evidence, not authority. Read them to learn what the authors intended, then check whether they work.

20.4 Going Live

The checklist, with a measurement behind every line

Nothing here is advice this handout has not demonstrated.

1. **Change all three default passwords** — admin, database user, database root — and raise the minimum password length from six (Chapter 15, Chapter 16).
2. **Apply the hardening configuration** and re-run the battery. It moved 5 of 15 to 14 of 15 (Chapter 16).

3. **Obtain a certificate, serve over HTTPS, and only then switch on HSTS.** Before the certificate it locks you out of your own shop.
4. **Move `system/storage/` out of the web root**, rather than relying on a configuration file somebody will edit (Chapter 16).
5. **Rename the admin directory** and the matching path in `admin/config.php`.
6. **Confirm the compiler cache is on** — 2×, measured above.
7. **Fix `config_checkout_payment_address`**, without which no guest can complete a checkout (Chapter 10).
8. **Populate `oc_api_ip`**, without which the admin panel cannot change an order (Chapter 18).
9. **Take a backup, restore it somewhere else, and time the restore.** Write the number down.
10. **Know your saturation point** before the first sale day.

Four of those ten are faults in the shipped software that every page of the shop will hide from you. That is the argument of the whole course in one list.

Common Pitfalls

- **Quoting throughput without latency.** At twenty clients this shop served 38.7 requests a second and every customer waited half a second.
- **Assuming more concurrency means more work done.** Past saturation it means less.
- **Benchmarking with the compiler cache off** and drawing conclusions about anything else.
- **Adding a caching layer before checking the free settings.**
- **Measuring the backup and never the restore.**
- **Never restoring a backup at all.**
- **Switching on HSTS before there is a certificate.**
- **Copying upstream's own Dockerfile or compose file.** One does not build and the other pins software that is out of support.
- **Deploying with the course's teaching passwords.**

Try It Yourself: Find where your shop falls over, then make it fall over later

```
# --- 1. Predict your saturation point --------------------------------
# How many concurrent customers before throughput stops rising?
# Write a number down before measuring.
cd code
python m20_deploy.py --load

# --- 2. Read both columns together -----------------------------------
# At the level where req/sec peaks, what is the median? And at
# twice that concurrency? Write one sentence a shop owner would
# understand.

# --- 3. Measure the free optimisation --------------------------------
python m20_deploy.py --opcache
# Explain why the gain is biggest in the MIDDLE row.

# --- 4. Apply Chapter 8's finding and re-measure ---------------------
# Minify bootstrap.css, then run --load again. Did req/sec move?
# It should not, by much. Say why not, and what DID improve.
```

```
# --- 5. Time a restore ------------------------------------------------
python m20_deploy.py --backup

# --- 6. Now do it for real --------------------------------------------
# Seed 50,000 products (seed/catalogue.py), then time the dump and
# the restore again. Record both. That is your recovery time.
# Restore the demo catalogue afterwards.

# --- 7. Prove the backup works ----------------------------------------
# docker compose down -v, up -d, restore your dump, and open the
# shop. If the products are not there, you did not have a backup.

# --- 8. Write the go-live checklist -----------------------------------
# Ten lines, each with the measurement or chapter that justifies it.
# Anything you cannot justify, take off the list.
```

What to notice. Step 7 is the only step that proves anything. Everyone takes backups; the question that separates a professional from an amateur is when they last restored one.

Checkpoint — Test Yourself

1. At 5 concurrent clients the shop served 42.1 requests per second; at 20 it served 38.7. Explain the mechanism, say what a shop owner should do with that information before a sale day, and give one change that would move the turning point.
2. The compiler cache doubled throughput at 5 clients but gave only 1.3× at 1 and 1.4× at 20. Explain both ends of that pattern, and say what it implies about when to benchmark.
3. Your client's backup runs nightly and has never been restored. Write the case for a restore test in terms your client will act on, state exactly what you would do and how long it would take, and say what you would measure and record.

Chapter Summary

- Throughput and latency are different questions and move in opposite directions. Quoting either alone mis-plans capacity.
- Measured: throughput peaked at 5 concurrent clients (42.1 req/sec) and *fell* to 38.7 at 20, while the median response grew 5.6× from 81 ms to 455 ms. Nothing failed and nothing was logged.
- The saturation point is measurable in ten minutes and is the number to know before a sale day.
- Measured: the opcode cache doubled throughput at the saturation point for one line of configuration, and the gain was largest in the middle — spare capacity at one client, queueing at twenty.
- Optimise in measured order: compiler cache, then assets, then indexes (re-measuring, because at 50,000 products they hurt), then page caching, then servers.
- Measured: a 1.0 s backup took 6.4 s to restore. The restore is the number that matters, and a backup nobody has restored is a file.
- Upstream's own Dockerfile does not build, its compose file pins software out of support, and its security advice does nothing on a modern Apache. Artefacts are evidence, not authority.
- Four of the ten go-live checklist items are faults in the shipped software that every page of the shop hides from you.

Review Questions

1. **(MCQ)** Going from 5 to 20 concurrent clients changed throughput from 42.1 to: (a) 168 req/sec; (b) 84 req/sec; (c) 38.7 req/sec; (d) it failed.
2. **(MCQ)** The opcode cache gave its largest gain at: (a) 1 client; (b) 5 clients; (c) 20 clients; (d) the same at all three.
3. **(Short)** Define the saturation point and explain why throughput can fall beyond it.
4. **(Short)** Why is the restore time, rather than the backup time, the figure a business should be given?
5. **(Short)** Explain why benchmarking a PHP application with the opcode cache disabled produces conclusions about the wrong program.
6. **(Applied)** A shop expects 300 concurrent visitors for a two-hour sale. Its measured saturation point is 5 and its median response at 20 is 455 ms. Set out what you would do, in order, with an expected effect for each step and a statement of which steps you could verify beforehand and which you could not.
7. **(Applied)** Design the backup and recovery procedure for a shop taking 200 orders a day. State what is backed up, how often, where it is kept, how long you may lose, how long a restore takes, who performs it, how the procedure is tested, and how you would discover that a night's backup had silently failed.

Part VI

Reach, Law and Practice

A shop that cannot be found has no customers, and a shop that ignores the law has them only briefly. Part VI closes the course with being found, being lawful, and defending what you built.

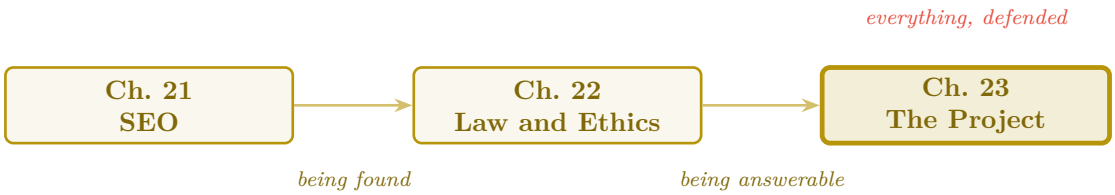


Figure P6.1: Reading path through Part VI. Chapter 22 is the only chapter whose content is set by statute rather than by the platform, and the statute it uses is Pakistan's.

Chapter 21

SEO and Digital Marketing

Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** the parts of search optimisation you control and can check from the parts you cannot.
- **Measure** a shop's crawlability: URLs, titles, descriptions, canonical links, sitemap and first byte.
- **Explain** why readable URLs are two halves, and **apply** both.
- **Diagnose** a configuration change that makes a shop worse while appearing to improve it.
- **Prioritise** marketing work by what is checkable rather than by what is advised.

Before You Start

Chapter 16: `AllowOverride None`, and why `.htaccess` is never read on this stack. Chapter 8: page weight and time to first byte. Chapter 6: the catalogue tables behind the pages.

Key Terms in This Chapter

crawler • indexing • readable URL • URL rewriting • canonical link • meta description • title tag • duplicate content • sitemap • robots.txt • faceted navigation • time to first byte • structured data

21.1 What Is Checkable, and What Is Not

Most writing about search optimisation concerns things nobody outside the search engine can verify: how a ranking is computed, how a change affected it, what the algorithm rewards this month. This chapter does not go there, because a course that cannot check its claims has no business making them.

The question this chapter asks instead

Not "how do I rank higher", but: **can a crawler read these pages, tell them apart, and find all of them?**

Every part of that is checkable from outside, in about a minute, with the shop in front of you. And a shop that fails it cannot rank at all — so it is the part worth doing first regardless of what you believe about the rest.

21.2 Readable URLs Are Two Halves

Readable URL, URL rewriting

A **readable URL** identifies a page by words rather than by parameters. This course's demonstration category is reached at

```
/en-gb/catalog/desktops
```

rather than at

```
index.php?route=product/category&path=20
```

It needs two independent things.

The **application half** generates them: the shop looks each link up in a table of keywords and writes the readable form into its HTML.

The **server half** resolves them: the web server turns the readable path back into the parameters the application understands, *before* the application sees the request.

Either half alone is useless. The second alone leaves the links unchanged. The first alone is *worse than nothing.*

Measured: Each half on its own, and both together

From `code/m21_seo.py -fix`. The shop ships with **78 prepared keywords** in `oc_seo_url` and `config_seo_url` set to **0**.

Configuration	Readable?	HTTP	A link the page emits
setting off, rewrite off	no	200	`index.php?route=...`
setting ON, rewrite off	yes	**404**	`/en-gb/catalog/desktops`
setting ON, rewrite ON	yes	200	`/en-gb/catalog/desktops`

The middle row is the finding. Switching on the setting that every guide tells you to switch on rewrites every link in the shop into a readable one — and **every one of them returns 404**. The home page still works, because its own address is unchanged; everything reachable from it is gone.

And the cause is Chapter 16's. The rewrite ships in `.htaccess.txt`, and `AllowOverride None` means Apache never reads `.htaccess` at all. One misconfiguration, found while looking for an exposed error log, silently disables an unrelated feature two chapters later.

The application half is faultless. Fetching `index.php?_route_=htc-touch-hd` by hand returns 200 — the lookup, the keyword table and the routing all work. Only the server was never told.

A change that makes a shop worse while appearing to improve it

Consider what the shopkeeper sees after ticking that box. The admin panel reports success. The home page loads. The URLs in the address bar are now pretty, which is visibly what was asked for. Nothing in any log is unusual, because a 404 is a perfectly ordinary response.

What has actually happened is that **every product and category page in the shop has stopped existing**, and the first person to find out will be a customer or a crawler.

The habit this argues for is one step long: after any configuration change, follow a link. Not the page you changed — a link from it. The whole of this failure is visible in one click and invisible in any amount of looking at the setting.

21.3 Can a Crawler Tell the Pages Apart?

Measured: Titles and descriptions across the catalogue

From `code/m21_seo.py -meta`, over all 19 product pages.

Product pages checked	19	
Distinct titles	**19**	no duplicates — good
Duplicate titles	0	
Pages with no meta description	**19**	**all of them**
Distinct descriptions	0	

Titles are fine. Each is the product's name, and because product names differ, so do the titles. That is the one thing the platform gets right here, and it gets it right by accident rather than by design — a shop selling "Blue T-Shirt" in four sizes as four products would have four identical titles.

Not one page has a meta description. The field exists in `oc_product_description`; the demonstration data leaves it empty, and the platform writes nothing in its place. A search engine will then compose a description from whatever text it finds on the page, which for a product page is usually the navigation.

And no page carries a canonical link — not the home page, not a category, not a product. So the same product reached through two categories is two URLs with identical content, and nothing tells a crawler which one is the real one.

Measured: What a crawler is told, and how fast

From `code/m21_seo.py -crawl`.

`robots.txt`	HTTP 200	16 `Disallow` rules
… names a sitemap?	**no**	
`/sitemap.xml`	**HTTP 404**	
A sitemap extension	**HTTP 404**	none ships

	Canonical?	First byte (median)
Home	no	114 ms
Category	no	133 ms
Product	no	110 ms

The `robots.txt` is good work. Its sixteen rules block sorted, paged and filtered variants of listing pages — `?sort=`, `&page=`, `?filter_name=` — which is exactly the faceted navigation that otherwise generates thousands of near-identical URLs from twenty products. Somebody thought about this.

There is no sitemap of any kind, and `robots.txt` does not name one. A crawler therefore finds pages only by following links, so anything more than a few hops from the home page is, in practice, not there. For a catalogue of nineteen products this does not matter; at Chapter 6's fifty thousand it is most of the shop.

First byte is 110–133 ms, which is the server's own thinking time and is respectable. Chapter 8 showed that the other 600 KB is what the customer actually waits for — so this number, the one most often quoted, is the smallest part of the experience.

21.4 The Order to Work In

Six things, each of which you can verify yourself

1. **Make the pages reachable.** Both halves of readable URLs, or neither. Then follow a link.
2. **Publish a sitemap** and name it in `robots.txt`. The platform ships none, so this is a module (Chapter 19) — a query over products and categories emitting XML.
3. **Give every page a canonical link.** One template override (Chapter 8), and it ends the duplicate-content question for the whole shop.
4. **Write meta descriptions**, starting with the twenty products that sell. It is data entry, it is nobody's favourite afternoon, and it is the only one of these six a search engine shows to a human.
5. **Keep `robots.txt` blocking the facets.** It already does; the risk is somebody replacing it.
6. **Then look at speed** — and from Chapter 8's measurements, at page weight rather than at first byte.

Everything above is checkable in your own shop, today, with no access to anybody's analytics. Advice that cannot be checked that way goes after all six.

On the marketing half of this chapter's title

The 2025 curriculum renames this course *Digital Marketing & E-Commerce*, and a handout traced to the 2017 outline should say what it is not covering. Campaign strategy, content marketing, social advertising and attribution modelling are a different subject with different methods, and three weeks of this course cannot do them justice.

What this course contributes to them is narrower and more durable: **a shop that can be crawled, a coupon code per channel (Chapter 12) so that a campaign is measurable at all, and the habit of asking what the denominator is (Chapter 17) before believing a rate.** Without those three, a marketing budget is being spent on a shop that cannot report what it bought.

Common Pitfalls

- **Switching on readable URLs without the rewrite.** Every link in the shop becomes a 404 and nothing reports it.
- **Changing a setting and checking the page you changed** instead of following a link from it.
- **Assuming `.htaccess` does anything on this stack.**
- **Leaving meta descriptions empty** and letting the engine write them from your navigation.
- **Shipping without canonical links**, so that one product at four URLs competes with itself.
- **Removing the `robots.txt` facet rules** to "let everything be indexed".
- **Optimising first byte** when Chapter 8 measured 600 KB of assets after it.
- **Believing any claim about ranking that you cannot test.**

Try It Yourself: Make your shop findable, and break it first

```
# --- 1. Look at a link before changing anything ---------------------
cd code
```

```
curl -s "http://localhost:8090/index.php?route=product/category&path=20" \
  | grep -o 'href="http://localhost:8090/[^"]*"' | head -3

# --- 2. Turn on readable URLs the way every guide says --------------
# Admin: System > Settings > Edit > Server > Use SEO URLs = Yes.

# --- 3. Now FOLLOW A LINK -------------------------------------------
# Click any category. Write down what happened before reading on.

# --- 4. Measure all three configurations ----------------------------
python m21_seo.py --fix

# --- 5. Explain the middle row --------------------------------------
# Name the chapter that already measured the cause.

# --- 6. Audit what a crawler can tell apart -------------------------
python m21_seo.py --meta
python m21_seo.py --crawl

# --- 7. Fix the cheapest finding ------------------------------------
# Add a canonical link to the product template in your child theme
# (Chapter 8). One line. Re-run --crawl and confirm it appears.

# --- 8. Write the sitemap module ------------------------------------
# Chapter 19's structure: one controller, one query over enabled
# products and categories, XML out. Then name it in robots.txt.

# --- 9. Write ten meta descriptions ---------------------------------
# For ten real products. Then say honestly how long 50,000 would
# take, and what you would do instead.

# --- 10. Restore ----------------------------------------------------
python m21_seo.py --restore
```

What to notice. Step 3. Almost everybody ticks the box, sees a pretty URL in the address bar, and concludes it worked. The entire shop is broken at that moment, and one click shows it.

Checkpoint — Test Yourself

1. A client turns on readable URLs and reports that "the site looks the same but customers say pages are missing". Explain precisely what has happened, name the two halves and which one is absent, give the command that demonstrates it, and say why nothing appeared in any log.
2. All 19 product pages have distinct titles and none has a meta description. Say which of those two facts is luck and which is design, what happens on the pages with no description, and what you would do first for a catalogue of 50,000.
3. This shop has no sitemap and no canonical links. Explain what each absence costs, say which matters more for a 19-product shop and which for a 50,000-product shop, and justify the difference.

Chapter Summary

- Ask what is checkable: can a crawler read the pages, tell them apart, and find all of them? A shop failing that cannot rank whatever else is true.
- Readable URLs are two halves — the application generating them and the server resolving them. Either alone is useless and the first alone is harmful.

- Measured: with the setting on and the rewrite off, every link the shop emits returns **404**, while the home page and `index.php?_route_=...` work perfectly. The cause is Chapter 16's `AllowOverride None`.
- After any configuration change, follow a link. The whole of that failure is visible in one click and invisible in any amount of inspecting the setting.
- Measured: 19 distinct titles and 0 duplicates — by luck, because product names differ — and **19 pages with no meta description at all**, and no canonical link on any page.
- Measured: `robots.txt` blocks the faceted-navigation parameters, which is good work; there is no sitemap of any kind and nothing names one. First byte is 110–133 ms, which Chapter 8 showed is the smallest part of what the customer waits for.
- Work in order: reachable, sitemap, canonical, descriptions, keep the facet rules, then speed — and page weight before first byte.
- This course's contribution to marketing is a shop that can be crawled, a code per channel so campaigns are measurable, and always asking what the denominator is.

Review Questions

1. **(MCQ)** With `config_seo_url` on and no server rewrite, a link the category page emits returns: (a) 200; (b) 301; (c) 404; (d) 500.
2. **(MCQ)** How many of the 19 product pages carried a meta description? (a) all 19; (b) about half; (c) one; (d) none.
3. **(Short)** Explain what a canonical link is for, with an example of two URLs in this shop that need one.
4. **(Short)** Why does `robots.txt` block `?sort=` and `?filter_name=`, and what would happen if it did not?
5. **(Short)** Give the one-step habit this chapter argues for after any configuration change, and the failure it would have caught.
6. **(Applied)** You inherit a shop with 8,000 products, no sitemap, no canonical links and meta descriptions on 200 of them. Set out your plan in order, with the effort and the expected benefit of each step, and say which steps you would automate and how.
7. **(Applied)** Specify the sitemap module this platform lacks: what it queries, how it handles a 50,000-product catalogue within the sitemap size limits, how often it is regenerated, how it is named in `robots.txt`, and how you would verify it works without waiting for a search engine.

Chapter 22

Social, Legal and Ethical Issues

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the Pakistani instruments that bind an online shop, and **state** what each one governs.
- **Explain** what makes an online contract enforceable, and **identify** where consumer law actually comes from here.
- **Recognise** a dark pattern, including ones this course produced by accident.
- **Assess** what personal data a shop holds, and **justify** keeping or discarding each kind.
- **Argue** a professional position on a request you are asked to implement.

Before You Start

Chapter 9: what the shop records about visitors who never buy. Chapter 12: the totals pipeline and what moving a line does. Chapter 16: PECA 2016, and what the shop exposed. Chapter 18: what a refund does and does not do.

Key Terms in This Chapter

electronic contract • Electronic Transactions Ordinance 2002 • PECA 2016 • E-Commerce Policy 2019 • consumer protection • provincial jurisdiction • personal data • data minimisation • retention • dark pattern • accessibility • WCAG • professional responsibility

22.1 What Makes an Online Order a Contract

A shop takes money from strangers it never meets, on the strength of a click. The question of why that click binds anybody is older than the software and is answered by one instrument.

Electronic Transactions Ordinance 2002

Pakistan's **Electronic Transactions Ordinance 2002** gives legal recognition to electronic documents, records and signatures: a contract is not void merely because it was made electronically, and an electronic record satisfies a legal requirement for writing.

Without it, every order in this course would be unenforceable. With it, the click is a contract — which cuts both ways, because it is also what makes your shop's published terms binding on *you*.

Practical consequence: the order record is the contract. That is why Chapter 10 copies the price, the address and the totals into the order rather than pointing at the catalogue, and why Chapter 18 treats the order history as the thing a dispute is settled from. Those were engineering decisions in those chapters; here they are the reason they were the right ones.

22.2 The Instruments, and What Each Governs

Instrument	Governs	Why it reaches you
Electronic Transactions Ordinance 2002	Electronic records, signatures, contracts	Your orders are enforceable
PECA 2016	Offences involving information systems	Unauthorised access is criminal — Chapter 16
E-Commerce Policy of Pakistan 2019	Policy direction: consumer protection, payments, taxation, data	The framework the rest is built under
Provincial consumer protection Acts	Goods, services, redress	**Not federal** — see below
Sales tax law, administered by FBR	Registration, collection, returns	Chapter 11's rate and rules

Two things about Pakistani law that surprise students

First: consumer protection is provincial, not federal. Punjab, Sindh, Khyber Pakhtunkhwa and Balochistan each have their own consumer protection legislation, with their own authorities and their own procedures. A shop in Bahawalpur selling to Karachi may be answerable under a different Act from the one it was set up under, and "the consumer protection law" is not a thing that exists.

Second: Pakistan has no comprehensive data protection Act in force. A Personal Data Protection Bill has been in draft for years. What this means in practice is **not** that there are no obligations — contract, the E-Commerce Policy, sectoral rules from the State Bank, and the plain law of confidence all reach — but that *the law is not what will stop you doing something stupid with a customer list.* Your own judgement is, and that is an uncomfortable position to be in rather than a comfortable one.

Both of these can change, and the second is likely to. Check the current position before advising anybody. What this chapter teaches is how to ask, not what the answer was in 2026.

22.3 What Your Shop Already Knows About People

Before any question of obligation, a question of fact: what is in there? This course has already produced the answer, in pieces.

An inventory, from this course's own measurements

- **645 session rows on a shop with no customers** (Chapter 9) — addresses, timestamps and a currency, for visitors who were mostly robots, kept for 24 hours by a clock rather than discarded on departure.
- **An address, a telephone number and an email** on every order, copied and kept permanently, for guests who never created an account (Chapter 10).
- **A complete purchase history per named person**, reachable in two clicks from the admin panel (Chapter 18).
- **An IP address on every customer row**, written at registration and never updated or removed.

- **A PHP error log, readable by anyone over HTTP** before Chapter 16's hardening — and a PHP error log can contain the contents of the request that failed, which on a checkout is somebody's address.
- **Uploaded files in a web-served directory** (Chapter 7), protected by an unguessable name.

Not one of those was a decision. Every one is a default. The shop collects them because the software does, and it keeps them because nothing deletes them.

Three questions worth asking of each

1. **Why do we have this?** If the answer is "it came with the software", that is not an answer, and the thing to find out is what breaks if it goes.
2. **How long do we need it?** An order is an accounting record and has a statutory life. A session row from a crawler has none. They are not the same object and should not have the same fate.
3. **What happens if it leaks?** Rank by that, not by volume. The session table is large and dull; the order table is small and is somebody's home address next to what they bought.

Data minimisation is not a legal doctrine you are importing. It is the engineering observation that you cannot lose what you did not keep, and it is free at design time and expensive afterwards.

22.4 Dark Patterns, Including the Ones We Built

Dark pattern

A **dark pattern** is an interface that leads a user to do something they did not intend and would not have chosen, by exploiting how they read rather than by lying to them. Each element is usually true. The arrangement is the deception.

Three from this handout, two of them accidental

1. Charging tax on a discount the customer never received. Chapter 12 measured it: moving the coupon line after the tax line changed the customer's bill from 770.16 to 784.20. Every line was correct and the total added up. **Nobody designed this as a deception — it is one integer in an admin field** — and a shop that did it deliberately would be indistinguishable from a shop that did it by accident. That is precisely why the arithmetic in Chapter 11 and Chapter 12 is somebody's professional responsibility and not a detail.

2. Removing the confirmation screen to raise conversion. Chapter 10 measured it as 28% of the checkout's time, and listed it as the one step not to remove. Removing it does raise completed orders. It raises them by taking the last moment at which a customer sees what they are agreeing to pay.

3. Optimising a cash-on-delivery checkout for completion. Chapter 13 showed that an unverified COD order is a liability, and that confirming orders by telephone is the most profitable step in the pipeline. Advice imported from prepaid markets — remove every step, reduce friction — produces more unwanted orders, and the customer who refuses at the door was helped into an order they did not want.

A test you can apply before writing the code

Would you be comfortable explaining this arrangement to the customer it affects, in one sentence, out loud?

- "We put the discount before the tax so you pay tax only on what you actually pay." Comfortable.
- "We put the discount after the tax so you pay us an extra 1.8%." Not comfortable. Same software, one integer apart.
- "We show you the total before you pay." Comfortable.
- "We removed that screen because fewer people stopped." Not.

The test is not a substitute for law and does not need to be. It is faster than law, it applies before the code is written, and it has never once in this course disagreed with the right answer.

22.5 Accessibility, Which Is Not Optional and Is Usually Skipped

Four things that cost nothing at build time

Chapter 8 rebuilt a theme without once mentioning the customers who cannot see it. That omission is normal and is a mistake.

1. **Every image has alt text**, and a product photograph's alt text is the product's name — which the shop already has.
2. **Every form field has a label** tied to it, not a placeholder pretending to be one. A placeholder disappears when typing starts, which is exactly when a confused customer needs it.
3. **Colour is never the only signal.** "Out of stock" in red is invisible to a large minority of men.
4. **The checkout works from the keyboard alone.** Try it. It takes four minutes and it is the only one of these four that usually fails.

All four are cheaper than page weight was in Chapter 8, and unlike page weight they cannot be retrofitted by minifying anything. A shop that cannot be used by somebody with a screen reader has excluded a customer who had money and wanted to spend it, which is a commercial fact before it is an ethical one.

22.6 Being Asked to Do Something You Should Not

Four requests you will receive, and an answer to each

- **"Store the card number so customers need not retype it."** No — and the reason is not squeamishness: it moves the shop from PCI DSS SAQ-A to SAQ-D (Chapter 14), which is hundreds of requirements and an audit. *Offer instead:* the provider's own tokenisation, which gives the same convenience and keeps the number where it is already somebody's job to protect.
- **"Email our customer list to this marketing company."** Ask what the customers were told when they gave it, and whether that covers this. If nobody knows, the answer is no. *Offer instead:* send the campaign yourself, which keeps the list where the promise was made.

- **"Delete these orders, they're embarrassing."** No. They are accounting records and Chapter 18 showed they are also what a dispute is settled from. *Offer instead:* a status, a note, and a report that excludes them — which is what was actually wanted.
- **"Check whether our competitor's site has the same vulnerability."** No, and say why plainly: that is unauthorised access to an information system, it is an offence under **PECA 2016**, and no commercial reason changes that. *Offer instead:* check your own, which is what Chapter 16 did.

In every case the answer has the same shape: say no once and clearly, give the reason in terms of consequence rather than of principle, and offer the nearest thing you can do. A refusal with no alternative gets overruled by somebody who will not refuse.

Common Pitfalls

- **Assuming there is one national consumer protection law.** There is not; it is provincial.
- **Assuming no data protection Act means no obligations.** It means the law will not stop you, which is worse.
- **Treating a legal rate or rule as permanent.** Check it against the current instrument before advising anybody.
- **Keeping data because the software collected it.** That is not a reason, and it is not a defence.
- **Ranking data risk by volume** rather than by what a leak would do.
- **Building a dark pattern by accident**, which is what moving one integer did in Chapter 12.
- **Leaving accessibility until after launch**, when it cannot be retrofitted cheaply.
- **Refusing a request without offering an alternative.**
- **Testing any system you do not own** (Chapter 16).

Try It Yourself: Audit your own shop as though it were somebody else's

```
# --- 1. Inventory what you hold about people --------------------------
cd code
docker compose exec db mariadb -ushop -pshoppw opencart -e \
  "SELECT COUNT(*) FROM oc_customer; \
   SELECT COUNT(*) FROM oc_session; \
   SELECT COUNT(*) FROM oc_order; \
   SELECT COUNT(*) FROM oc_customer_ip;"

# --- 2. Find the most sensitive single row ----------------------------
# Not the biggest table. The one whose leak would do most harm.
# Name it and say why in one sentence.

# --- 3. Decide a retention period for each ----------------------------
# Three columns: what, how long, why. "Forever" needs a reason too.

# --- 4. Build the dark pattern, then look at it -----------------------
python m12_discounts.py --sortorder
# Read the two totals. Write the sentence you would have to say to
# the customer to explain the second one.

# --- 5. Put it back ---------------------------------------------------
python m12_discounts.py --restore

# --- 6. Try your checkout with the keyboard only ----------------------
```

```
# No mouse, no trackpad. Tab and Enter. Get to a completed order.
# Write down where you got stuck.

# --- 7. Check your images ---------------------------------------
curl -s "http://localhost:8090/index.php?route=product/category&path=20" \
  | grep -o '<img[^>]*>' | grep -c 'alt='
curl -s "http://localhost:8090/index.php?route=product/category&path=20" \
  | grep -c '<img'
# Two numbers. They should be equal.

# --- 8. Write your terms ----------------------------------------
# One page a customer could read: who you are, what you sell, the
# price including tax, delivery, how to cancel, how to complain, and
# what you do with their data. Then check your shop does what it says.
```

What to notice. Step 8 is the exercise. Almost every shop's published terms describe a shop that does not exist, because nobody checked the sentences against the software — and this course has spent twenty-one chapters learning how to check.

Checkpoint — Test Yourself

1. A shop moves its coupon line after its tax line and collects 1.8% more on every discounted order. Nothing was concealed and every figure is correct. Argue whether this is a dark pattern, state what would change your answer, and say what you would do if you found it in a shop you had inherited.
2. Your shop holds 645 session rows and 6 orders. Rank the two tables by risk, justify the ranking, give a retention period for each with your reasoning, and say which one you would be more reluctant to delete and why.
3. A client asks you to store card numbers "because our competitor does". Write the reply you would send: the refusal, the reason in terms of consequence, the alternative, and what you would do if the client insisted.

Chapter Summary

- The Electronic Transactions Ordinance 2002 is what makes an online order a contract — and makes your published terms binding on you. The order record is the contract, which is why it copies its data.
- PECA 2016 makes unauthorised access an offence; the E-Commerce Policy 2019 sets the framework; FBR administers the tax.
- **Consumer protection in Pakistan is provincial**, with separate Acts and authorities, so "the consumer protection law" does not exist as a single thing.
- **Pakistan has no comprehensive data protection Act in force.** That means the law will not stop you, not that nothing binds you — an uncomfortable position rather than a comfortable one.
- This course's own measurements are the data inventory: 645 session rows from robots, addresses and telephone numbers on guest orders, a purchase history per named person, an IP on every customer, and — before hardening — an error log anyone could read.
- Ask of each: why do we have it, how long do we need it, and what happens if it leaks. Rank by harm, not volume. You cannot lose what you did not keep.
- A dark pattern is an arrangement, not a lie, and Chapter 12 built one by moving a single integer. Test by whether you could explain the arrangement out loud to the customer it affects.

- Four accessibility measures cost nothing at build time and cannot be retrofitted cheaply. The keyboard-only checkout is the one that usually fails.
- Refuse clearly, give the consequence rather than the principle, and always offer the nearest thing you can do.

Review Questions

1. **(MCQ)** Consumer protection legislation in Pakistan is: (a) federal only; (b) provincial; (c) governed by PECA 2016; (d) not yet enacted anywhere.
2. **(MCQ)** What makes an electronic order legally enforceable in Pakistan? (a) PECA 2016; (b) the E-Commerce Policy 2019; (c) the Electronic Transactions Ordinance 2002; (d) the shop's terms.
3. **(Short)** Explain why an order copies its prices and addresses rather than referring to them, in legal rather than technical terms.
4. **(Short)** Give the one-sentence test for a dark pattern, and apply it to a countdown timer that resets when the page is reloaded.
5. **(Short)** Why is "we have no data protection Act" a reason for more care rather than less?
6. **(Applied)** Write the data retention schedule for a shop taking 50 orders a day: every table holding anything about a person, how long it is kept, the reason, what deletes it, and who checks. Mark anything you would need advice on rather than deciding yourself.
7. **(Applied)** Your employer asks you to implement a checkout that pre-ticks an optional insurance product. Set out your analysis: what the law here does and does not say, what the one-sentence test gives, what you would propose instead, what you would do if overruled, and at what point you would decline to build it at all.

Chapter 23

The Semester Project

Learning Outcomes

By the end of this chapter you will be able to:

- **Assemble** the shop you have built since Chapter 5 into a deliverable with a stated scope.
- **Deploy** it from nothing, on a machine that is not yours, in one command.
- **Measure** your own shop and **report** the figures honestly, including the ones that did not improve.
- **Present** and **defend** the decisions you made, naming what you would do differently.
- **Judge** another team's shop against the same criteria.

Before You Start

Everything. Specifically: a shop that stands up (Chapter 5), takes an order (Chapter 10), charges correctly (Chapter 11), is hardened (Chapter 16), carries at least one module you wrote (Chapter 19), and can be restored from backup (Chapter 20).

Key Terms in This Chapter

scope • deliverable • reproducible build • baseline • evidence • defence • peer review • handover • known limitation

23.1 What Is Being Assessed

HEC's own teaching methodology for this course names *Project* and *Presentations*, and this chapter discharges both. The project is not a new piece of work: **it is the shop you have been adding to since week three**, finished, deployed, measured and defended.

The one sentence the whole project is marked against

Can somebody else bring your shop up from nothing, place an order in it, and check your numbers?

Everything below follows from that. A shop that only runs on your laptop is not deployed. A claim you cannot reproduce is not a result. A decision you cannot explain is not a decision.

23.2 The Deliverable

Five things, and nothing else

1. **A repository that builds.** `docker compose up -d --build` on a clean machine produces a working shop with your catalogue in it. If a human step is needed, it is one line in the README and it is tested.
2. **The shop itself**: a real catalogue of at least thirty products with options, categories, prices, weights and images; working shipping and tax for a stated place; a completable checkout; at least one payment method; and the Chapter 16 hardening applied.

3. **At least one module you wrote** (Chapter 19), with its admin half, its language file, and an install step — and a sentence saying what the shop could not do without it.
4. **A measurement report**, two to four pages: what you measured, with what, on what machine, what you found, and what you changed as a result. Figures you did not improve are part of the report.
5. **A ten-minute presentation and a defence.**

Not wanted: a redesigned theme as the main contribution, a slide deck longer than the shop, or a feature list. The marks are in whether it works, whether it is measured, and whether you can defend it.

Assessed on	Weight	What earns it
It builds and runs elsewhere	20%	Somebody else runs one command
The shop works end to end	20%	An order can be placed and fulfilled
Your own extension	15%	It installs, removes cleanly, and is needed
Hardening and correctness	15%	The Chapter 16 battery, and the go-live list from Chapter 20
Measurement report	20%	Reproducible figures, honestly reported
Presentation and defence	10%	You can answer "why"

23.3 The Measurement Report

This is where the course differs from a web-development project, and it is worth being explicit about what a good one looks like.

Four measurements, chosen because you have already made them once

1. **A catalogue measurement** (Chapter 6): a page's time and query count against catalogue size, with and without an index you chose. *Report the case where the index did not help, if there is one.*
2. **A page weight measurement** (Chapter 8): bytes and requests, before and after one change, with the change named.
3. **A security measurement** (Chapter 16): the battery before and after your hardening, with the findings you could not close listed and costed.
4. **One of your own choosing**, with a stated question it answers.

Every figure must name the file that produces it, exactly as this handout does. A number with no command behind it is an assertion.

What a good report does that a weak one does not

It states the machine. Appendix A of this handout exists because every timing in it is a fact about one laptop. Yours is a fact about yours.

It reports the measurement that disagreed with you. This handout printed three wrong versions of Chapter 6's experiment, the store identifier that made 50,000 products invisible, and three security controls that were applied correctly and did nothing. Every one of those is worth more than a result that went as expected, because it shows the method working.

It separates what it measured from what it believes. "The index made the page slower at 50,000 products on this machine" is a measurement. "Our shop is fast" is not.

It says what it could not measure. Chapter 8's harness could not see fonts requested by CSS, and said so. Knowing the limits of your instrument is the difference between a report and a brochure.

23.4 The Defence

Ten minutes of presentation and then questions. The questions are not a test of memory.

</> What you will be asked, and what a good answer sounds like

- **"Why this platform?"** Not "it was in the course". Chapter 4 is the argument; make it yours, with the alternative you rejected and why.
- **"Show me an order going through."** Live, on the running shop. Know what status it lands in and what moved in the database when it did (Chapter 10, Chapter 18).
- **"What does this number mean?"** Pointing at any figure in your report. Be able to say what it counts, what it excludes, and what would change it.
- **"What is still wrong with it?"** The best question and the one most teams are unprepared for. A team that names three real limitations is further along than one that claims none.
- **"What would you do with another month?"** Ordered by measured benefit, not by what would be interesting to build.
- **"Who did what?"** Answered the same way by every member.

Reviewing another team's shop

Each team reviews one other, using the same instruments, and submits one page.

1. **Clone and build it.** Time how long until you see a storefront. That number is the review's first finding.
2. **Place an order.** Note every step that confused you.
3. **Run the security battery** against *their* container on *your* machine. This is within scope precisely because you have built it yourself and it is running on your own computer — Chapter 16's rule is about systems you do not control, and a classmate's repository running on your laptop is one you do.
4. **Check one of their numbers.** Run the file they name, and report what you got.
5. **Write one page**: what works, what you could not get working, one thing you will steal, and one thing you would change.

Reproducing somebody else's number is the single most valuable hour of the semester, in both directions. It is also the only part of this course that cannot be done alone.

23.5 A Realistic Schedule

Week	Milestone	What is actually due
10	Proposal	One page: what you sell, to whom, where you deliver, which payment methods, and the one feature you will build
12	Catalogue complete	30+ real products with weights and images, and your first measurement
14	Your module works	Installs and removes cleanly on a clean container
15	Hardened and deployed	The go-live list from Chapter 20, and a restore you have timed
16	Present and defend	And review another team

Common Pitfalls

- **Leaving deployment to the last week.** It is the deliverable with the most ways to fail and the least room to recover.
- **A shop that runs only on the machine it was built on.**
- **Measuring once, at the end.** A baseline you did not take in week 10 cannot be taken in week 16.
- **Reporting only the figures that improved.**
- **Quoting a number without the command that produces it.**
- **Spending the project on the theme.** It is the most visible work and the least of the marks.
- **Twelve demo products with no weights**, which breaks shipping in a way Chapter 11 told you about.
- **A module that cannot be removed**, which means it was never tested from a clean state.
- **Arriving at the defence unable to say what is still wrong with it.**
- **Running anybody's security battery against anything but your own or your classmate's container on your own machine** (Chapter 16).

Try It Yourself: Ship it

```
# --- 1. Prove it builds from nothing ---------------------------------
cd code
docker compose down -v
docker compose up -d --build
python waitfor.py
# Time this. It is the first number in your report.

# --- 2. Place an order on the fresh shop -----------------------------
python m10_checkout.py --repeat 1
python m10_checkout.py --states
# If this fails on a clean build, your README is wrong.

# --- 3. Install your own extension -----------------------------------
python m19_modules.py --install
python m19_modules.py --measure

# --- 4. Harden, and record the battery before and after --------------
python m16_security.py
```

```
# --- 5. Take the four measurements ---------------------------------
python m06_catalogue.py --quick
python m08_pageweight.py --compare
python m20_deploy.py --load
# Plus your own. Save every output; it is your evidence.

# --- 6. Back it up, destroy it, restore it -------------------------
python m20_deploy.py --backup
# Then do it for real: down -v, up, restore, and check the catalogue
# is there. Record the restore time.

# --- 7. Write the README somebody else will follow -----------------
# Hand it to a classmate and watch them. Every question they ask is
# a line missing from it. Do not help them; write it down.

# --- 8. Write the report -------------------------------------------
# Four measurements, the machine, what changed, what did not, and
# what you could not measure.

# --- 9. Prepare three limitations ----------------------------------
# Real ones, with what each would cost to fix. You will be asked.
```

What to notice. Step 7 is the one that decides the first 20% of the marks, and watching somebody fail to follow your README is the only way to write one. Every course does this badly; this is your chance not to.

Checkpoint — Test Yourself

1. Your shop runs perfectly on your laptop and a classmate cannot get it started. List the five most likely causes in order, say how you would find out which it is without access to their machine, and name the one thing you should have done in week 12 that would have prevented all five.
2. You measured your category page at 180 ms before an index and 195 ms after. Say what you would put in the report, what you would investigate first, and why this result is worth more marks than one that went the way you expected.
3. At the defence you are asked "what is still wrong with it?". Give three answers for the shop as this handout left it, each with the chapter that found it and the cost of fixing it.

Chapter Summary

- The project is the shop you have built since Chapter 5, finished and defended — not a new piece of work.
- One sentence decides it: can somebody else bring it up from nothing, place an order, and check your numbers?
- Five deliverables: a repository that builds, a shop that works end to end, an extension you wrote, a measurement report, and a defence.
- Every figure names the file that produces it, every timing names the machine, and the measurements that disagreed with you are worth more than the ones that did not.
- A good report separates what it measured from what it believes, and says what its instrument could not see.
- The hardest defence question is "what is still wrong with it?". A team naming three real limitations is ahead of one claiming none.
- Reproducing another team's number is the most valuable hour of the semester, and the only part of the course that cannot be done alone.

- Deploy early. A baseline not taken in week 10 cannot be taken in week 16.

Review Questions

1. **(MCQ)** The largest single share of the project mark is for: (a) the theme; (b) the number of features; (c) that it builds and runs on another machine; (d) the length of the presentation.
2. **(MCQ)** A measurement in your report that did not improve should be: (a) omitted; (b) re-run until it does; (c) reported with what you think caused it; (d) replaced with an estimate.
3. **(Short)** Explain why a baseline must be taken early, with an example from this course.
4. **(Short)** Give two things a measurement report must state about the machine, and say why each matters.
5. **(Short)** Why is being able to name your shop's limitations worth marks, rather than costing them?
6. **(Applied)** Write the week-10 proposal for a shop selling a product of your choosing in Pakistan: what you sell, to whom, where you deliver, which payment methods and why, the tax configuration, and the one feature you will build with the reason the shop needs it.
7. **(Applied)** You have one month after the semester to take the shop to real customers. Give your ordered plan, each step justified by a measurement or a chapter from this course, and state the three things you would refuse to launch without.

Appendices

Chapter A

The Stack: Setting It Up and Keeping It

Everything in this book from Chapter 5 onward assumes a shop you can reach in a browser. This appendix gets you one, in a single sitting, and lists what to do when it refuses — which on a university laptop it sometimes will.

A.1 What You Install

One thing: a container runtime. Not a web server, not PHP, not a database. Those three run inside containers and never touch your own machine, which is why the shop is identical on every laptop in the class and why uninstalling it is one command.

- **Windows or macOS:** Docker Desktop, from `docker.com`. On Windows it needs WSL 2, which its installer offers to set up.
- **Linux:** the `docker.io` and `docker-compose-v2` packages from your distribution, then add yourself to the `docker` group and log out and in again.

Check it with:

```
docker --version
docker compose version
docker run --rm hello-world
```

The third is the one that matters. The first two only prove the program is installed; the third proves the *engine* is running, which is a different claim and the one that usually fails.

A.2 Starting the Shop

```
cd code
cp .env.example .env
docker compose up -d --build
python waitfor.py
```

The first run takes a few minutes and downloads about 1.5 GB; later runs take seconds. Chapter 5 measures both and explains why the difference matters.

What	Where	Sign in with
The storefront	`http://localhost:8090/`	nobody; you are a customer
The admin panel	`http://localhost:8090/admin/`	`admin` / `admin123`
The database console	`http://localhost:8091/`	server `db`, user `shop`, password `shoppw`, database `opencart`

Those credentials are deliberately bad

`admin` / `admin123` is exactly the kind of credential Chapter 16 goes looking for and finds. It is the default so that you can measure the shop before and after hardening it; Chapter 16 asks you to change it, and until then every student in the class can sign in to every other student's shop if they can reach it.

Which they cannot, because nothing here is published to the internet. If you ever expose this stack beyond your own machine, change the credentials in `.env` *first*.

A.3 The Four Commands Worth Memorising

Command	What it does
`docker compose up -d`	Start everything. Safe to run when it is already running
`docker compose down`	Stop everything. **Your shop is kept**
`docker compose logs -f shop`	Watch the shop's log. This is where PHP errors go, and they go nowhere else
`docker compose down -v`	Stop everything **and delete your shop**. The reset button, and the one mistake with no undo

A.4 When It Will Not Start

Five failures account for nearly all of them, and four are not really about Docker.

1. **"Cannot connect to the Docker daemon"** — the engine is not running. On Windows and macOS, start Docker Desktop and wait for its icon to stop animating; the command line is ready several seconds before the whale stops moving.

2. **"Ports are not available"** or "address already in use" — something else on your machine already holds port 8090 or 8091. This is common; the development machine for this book had both 8080 and 8081 taken, which is why the defaults are 8090 and 8091 in the first place. Edit `.env`, change `SHOP_PORT`, and bring it up again. Change only the port in `.env` — `OC_HTTP_SERVER` reads the same variable, so links keep working.

3. **The shop answers but every page is an error** — the install did not finish. Read `docker compose logs shop` and look for "SUCCESS! OpenCart successfully installed". If it is absent, the database was not ready; `docker compose down -v` and start again.

4. **WSL 2 is not installed, or virtualisation is disabled** — Windows only. Docker Desktop's installer handles the first; the second is a BIOS setting (often called "Intel VT-x", "AMD-V" or "SVM Mode") that some laptops ship disabled.

5. **It worked yesterday and today it does not** — almost always disk space. `docker system df` will tell you; `docker system prune` reclaims what is not in use. It does not touch named volumes, so your shop survives — but read what it proposes before agreeing.

Read the log before changing anything

The single most useful habit in this appendix. A shop that is broken is broken for a reason, and the reason is almost always written down in `docker compose logs shop`.

Restarting, rebuilding and resetting are the three things students try first, and all three destroy the evidence. Look at the log, then act.

A.5 Backing Up, and Why You Will Wish You Had

Your shop lives in two named volumes. `down -v` deletes them, and so does an afternoon of clearing disk space without reading the prompt.

```
# Dump the database into the repository, where git can see it.
docker compose exec -T db mariadb-dump -ushop -pshoppw opencart \
  > backups/shop-$(date +%Y%m%d).sql

# Put it back.
```

```
docker compose exec -T db mariadb -ushop -pshoppw opencart \
  < backups/shop-20261001.sql
```

Take one after every chapter's laboratory. It costs two seconds, and Chapter 20 measures what the alternative costs.

Where to keep your own work

Anything you write that must survive belongs in `code/`, in the repository — not inside a running container, where it is deleted at the next rebuild, and not only in the admin panel, where it is deleted by `down -v`.

This matters most in Chapter 19, where you write a module. Write it in `code/modules/` and mount it; do not edit it in place inside the container. Students who learn this in Week 14 learn it the expensive way.

A.6 The Machine Every Measurement Was Taken On

Every figure in a **Measured** box in this book was produced on this machine. Yours will differ; the shapes and ratios should not.

Item	Value
Host	Windows 11 Pro, 4 CPU cores, 7.7 GiB available to the container runtime
Container runtime	Docker 28.1.1, Linux containers
Web and application	Apache 2.4.68, PHP 8.2.34 with OPcache enabled
Database	MariaDB 11.8.9
Application	OpenCart 4.1.0.4, installed from the pinned release in `code/shop/Dockerfile`
Everything on	one machine, so no figure here includes real network latency

The scope of Chapter 16's security work

Chapter 16 asks you to attack a shop: SQL injection, cross-site scripting, missing security headers, default credentials, an exposed administrative path. Every one of those exercises is scoped, in the chapter and here, to **the container running on your own machine, reached at `localhost`, which you installed and which contains no real person's data**.

That is the whole of it. Running the same checks against any system you do not own and have not been authorised in writing to test is a criminal offence in Pakistan under the Prevention of Electronic Crimes Act 2016, and a sackable one nearly everywhere else. The skill is valuable precisely because it is dangerous, and the boundary is not a formality.

Chapter 22 returns to this with the statute in front of it.

Chapter B

Enough PHP, SQL and Twig to Read the Shop

The sibling courses give this appendix to mathematics. This one spends it on language, because that is what this course actually assumes and does not teach.

You are not asked to be a strong PHP programmer. Until Chapter 19 you will read far more code than you write, and this appendix is calibrated to that: enough to follow what the platform is doing, and enough to change it when Chapter 19 asks you to.

B.1 PHP, in Two Pages

B.1.1 The shape of a file

PHP is a language for producing a response to an HTTP request. A file runs top to bottom, every time the page is requested, and whatever it prints becomes the page.

```php
<?php
declare(strict_types=1);

$name  = 'Keyboard';       // string, single quotes: no interpolation
$price = 2499.00;          // float
$qty   = 3;                // int
$total = $price * $qty;

// Double quotes interpolate; single quotes do not. This is the single
// most common source of confusion in a first week of PHP.
echo "Total for $name: $total";   // Total for Keyboard: 7497
echo 'Total for $name';           // Total for $name
```

Three things that surprise everyone

Variables start with `$`, always — including inside strings and in function parameters.

`==` is not `===`. The two-character form converts types before comparing, so `'0' == false` is true and `'abc' == 0` was true in older versions. **Use `===` unless you have a reason not to**, and in Chapter 16 you will see what the other one costs.

`declare(strict_types=1)` is not the default. Without it PHP quietly converts `"5 apples"` into `5` when a function wants an integer. OpenCart's own code does not use it; your modules should.

B.1.2 Arrays, which are really two things

```php
<?php
// A list.
$sizes = ['S', 'M', 'L'];
echo $sizes[0];                    // S
echo count($sizes);                // 3

// A map -- the same type, with keys. Almost every piece of data in
// OpenCart arrives as one of these.
$product = [
    'product_id' => 42,
    'name'       => 'Keyboard',
    'price'      => 2499.00,
];
echo $product['name'];             // Keyboard
```

```php
// Walking one. Note the "as $key => $value" form.
foreach ($product as $field => $value) {
    echo "$field = $value\n";
}

// The ?? operator: use this value, or that one if it is absent.
// Without it, a missing key is a warning and a null.
$image = $product['image'] ?? 'placeholder.png';
```

B.1.3 Functions, classes and the arrow

```php
<?php
function formatPrice(float $amount, string $symbol = 'Rs. '): string
{
    return $symbol . number_format($amount, 2);
}

class ShippingQuote
{
    private float $rate;

    public function __construct(float $rate)
    {
        $this->rate = $rate;          // $this-> reaches a property
    }

    public function forWeight(float $kg): float
    {
        return $this->rate * max(1.0, $kg);
    }
}

$q = new ShippingQuote(120.0);
echo formatPrice($q->forWeight(2.5));    // Rs. 300.00
```

The three arrows, which are different

PHP uses three symbols that beginners merge into one.

- `$obj->method()` — call a method on an object, or read a property. An *object*.
- `$array['key']` — read a key from an array. An *array*. Square brackets, quoted key.
- `Class::CONSTANT` — reach something on the class itself rather than on an instance.

OpenCart hands you arrays far more often than objects, so `['key']` is the one you will type most. `$this->request->get['product_id']` is all three ideas in one expression: a property of a property, then an array key.

B.1.4 Where request data arrives, and why it is dangerous

```php
<?php
// Submitted by a form with method="get"  -> $_GET
// Submitted by a form with method="post" -> $_POST
$qty = $_POST['quantity'] ?? 1;

// EVERY value in those arrays is a STRING, and every one of them was
// typed by somebody who may not be a customer. Chapter 16 is about
// what happens when this is forgotten; Chapter 15 is about sessions,
// which is how the shop remembers who you are between requests.
$qty = (int)$qty;                    // now it is certainly an integer
```

```
if ($qty < 1 || $qty > 999) {
    $qty = 1;                          // and certainly in range
}
```

B.2 SQL, as This Course Uses It

You met SQL in *Database Systems*. This section is a reminder of the five statements this course needs and the two ideas it leans on.

```
-- Read. The only one you will type often.
SELECT p.product_id, pd.name, p.price
  FROM oc_product p
  JOIN oc_product_description pd ON pd.product_id = p.product_id
 WHERE pd.language_id = 1
   AND p.status = 1
 ORDER BY p.price DESC
 LIMIT 20;

-- Count. Chapter 6 shows why a category page runs one of these as
-- well as the query above, every single time.
SELECT COUNT(*) FROM oc_product WHERE status = 1;

-- Change one row. The WHERE is not optional; without it you have
-- just repriced the entire catalogue.
UPDATE oc_product SET price = 2999.00 WHERE product_id = 42;

-- Remove. Same warning, more so.
DELETE FROM oc_product_to_category WHERE product_id = 42;

-- Make a query fast, or fail to -- see Chapter 6.
CREATE INDEX idx_sort_order ON oc_product (sort_order);
```

The two ideas everything else rests on

A join combines rows from two tables on a matching column. OpenCart splits a product across eighteen tables (Chapter 6), so almost every useful question needs at least one join.

An index is a sorted structure the database can search instead of reading every row. It makes reads faster and writes slower, and it is useless if the query hides the column inside a function — `ORDER BY LCASE(name)` cannot use an index on `name`, which is the central finding of Chapter 6.

EXPLAIN, which answers "why is this slow?"

Put `EXPLAIN` in front of any `SELECT` and the database tells you what it intends to do rather than doing it. Three words in the output carry nearly all the information:

- **`type: ALL`** — a full table scan. Every row read. At fifty thousand products this is the one to fix.
- **`Using filesort`** — the result had to be sorted after it was fetched, because no index gave it in the right order.
- **`Using temporary`** — a temporary table was built to hold intermediate results, usually because of `DISTINCT` or `GROUP BY`.

Chapter 6 runs `EXPLAIN` against the real category query and finds all three at once.

B.3 Twig, Which Is Just the Page

OpenCart's templates are Twig. A template is HTML with three kinds of hole in it, and that is very nearly the whole language.

```
{# A comment. It does not reach the browser. #}

{# 1. Print something. #}
<h1>{{ heading_title }}</h1>
<span class="price">{{ product.price }}</span>

{# 2. Do something -- a loop or a condition. #}
{% if products %}
  <ul>
    {% for product in products %}
      <li>
        <a href="{{ product.href }}">{{ product.name }}</a>
        {% if product.special %}
          <s>{{ product.price }}</s> {{ product.special }}
        {% else %}
          {{ product.price }}
        {% endif %}
      </li>
    {% endfor %}
  </ul>
{% else %}
  <p>{{ text_empty }}</p>
{% endif %}

{# 3. Filter -- transform a value on the way out. #}
<p>{{ product.description|striptags|slice(0, 120) }}</p>
<p>{{ product.name|escape }}</p>
```

|escape and what it prevents

Twig escapes output by default in most configurations, and that default is the single thing standing between a product description written by somebody else and a cross-site scripting hole in your shop.

Twig also offers `|raw`, which turns the escaping off. There are legitimate uses, and every one of them is a decision to trust the source of that value completely. Chapter 16 looks for places where somebody has used it without meaning to.

B.4 YAML, in Ten Lines

The Compose file is YAML, and YAML has exactly two rules worth knowing.

```
# 1. Indentation is structure. Two spaces per level, NEVER tabs --
#    a tab is a syntax error and the message will not say so clearly.
services:
  db:
    image: mariadb:11
    environment:
      MARIADB_DATABASE: opencart

# 2. A leading dash is a list item.
    ports:
      - "8090:80"
      - "3306:3306"
```

That is enough to read `code/docker-compose.yml` and to change the one thing Chapter 5 asks you to change, which is a port number.

B.5 Where To Look Next

For	Go to
PHP	`php.net/manual` — the official manual, and unusually good. Each function page has user-contributed notes; treat those as hints, not authority
SQL	The MariaDB knowledge base, and your *Database Systems* notes for the theory this appendix skips
Twig	`twig.symfony.com/doc` — the "for template designers" page is all you need
OpenCart	`docs.opencart.com`, and the source in your own container, which is more current than the documentation

Read the source; it is right there

The shop's entire source is inside your container, and you can read it:

```
docker compose exec shop \
  cat /var/www/html/catalog/model/catalog/product.php
```

This is not a last resort. Chapter 6 finds the category page's real query this way, and it is the only way to answer "what does this page actually do?" with certainty rather than with a plausible guess. The documentation describes a version; the container holds yours.

Chapter C

Answers to the Checkpoints

Three answers per chapter, in order. Attempt the checkpoint from memory before reading these, and argue with them where you disagree — several are judgement calls rather than facts, and they say so.

Chapter 1 — What E-Commerce Is, and What It Changed

1. **Electronic business, not electronic commerce.** Viewing results and downloading transcripts are business processes conducted over a network, but no value changes hands through the system — the transaction happens at a bank counter. The moment fee payment moves online it becomes e-commerce, and the university acquires every problem to the right of the dashed line in Figure 1.1: a payment that may have succeeded while the network dropped, a receipt that must exist exactly once, a refund path.

2. Many are acceptable. One good answer: **refusing to sell the last item when it is visibly damaged.** In software this requires a stock record that distinguishes "held" from "sellable", somebody whose job is to update it, a rule for what the storefront does when the two disagree, and a decision about what the customer sees — out of stock, or a quiet substitution? The shopkeeper does all of that in half a second without recording that a decision was made, which is exactly why it is forgotten when the shop goes online.

3. **Two uncounted costs:** the acquisition cost of finding customers who currently arrive free (Chapter 2 puts this at Rs. 1,200 an order for a typical small shop, which can exceed the commission), and the labour of running the four functions themselves — payment integration, delivery arrangements, customer service, returns. The commission was buying those, not just the listing. **What they gain that is not money:** the customer relationship. They learn who bought, can contact them again, and cannot be delisted. For a business with repeat custom that is frequently worth more than the 15%.

Chapter 2 — Business Models and Revenue Models

1. **Business model:** a marketplace connecting restaurants with diners, with fulfilment performed in-house — three and a half of the four functions. **Revenue models: three at once** — *commission* from restaurants, a *service or delivery fee* from customers, and *advertising* in the form of promoted placement. **Most profitable per unit: the advertising.** It has almost no marginal cost, where commission is earned against riders and support that must be paid for, and the delivery fee rarely covers the rider. Promoted placement is close to pure margin, which is why these platforms push it so hard.

2. Viable when LTV $>$ CAC, that is when $400 \times n > 1{,}500$, so when the average customer orders $n > 3.75$ — **at least four times** — before they stop. *What to measure:* the repeat rate and the time over which it happens. Four orders over six months is a business; four orders over six years is not, because the cash was spent today and returns over a period during which the venture has to survive.

3. B2C platforms store **one price per product** — a column on the product row. B2B pricing is a *relation* between customer and product, often with quantity breaks and expiry dates. That is not a setting you have failed to find; it is a different cardinality, and making it work means either a different platform or an extension that intercepts every price lookup in the system

(Chapter 19 shows how much work that is). The assumption that breaks is "a product has *a* price".

Chapter 3 — Auctions, Portals and Communities

1. **Bid Rs. 15,000** — your true valuation. You win, and you pay the second-highest bid, **Rs. 13,500**, gaining Rs. 1,500. Bidding Rs. 18,000 does not improve this outcome at all: you still pay Rs. 13,500, because the price is set by the others. What it does is expose you to winning when somebody bids Rs. 16,500, at which point you pay Rs. 16,500 for a thing worth Rs. 15,000 to you. The overbid can only ever buy you an auction you did not want.

2. It **never closes**. The auction stays open indefinitely; the winning bidder is never notified, the seller is never paid, and the listing accumulates further bids whenever somebody wanders past. The seller sees an item that appears to be permanently "ending soon" and a buyer who has given up. Worse, the failure is *silent* and inversely proportional to interest: popular auctions close correctly and unpopular ones do not, so the bug is invisible in testing, where you load every page you create.

3. Any two of: **be a seller yourself** until real ones arrive (cost: you take on stock and inventory risk, and you compete with the sellers you are recruiting); **narrow the market** to one city or one category (cost: you cap your growth and may pick the wrong niche); **give sellers a tool worth having without buyers** (cost: you are now building and supporting a second product); **list supply gathered elsewhere** (cost: legally risky and reputationally expensive if discovered).

Chapter 4 — Planning an E-Commerce Framework

1. (a) **Functional**, and it *does not* discriminate — all four options reset passwords. (b) **Functional**, and it **discriminates sharply**: per-customer pricing eliminates most hosted platforms and the marketplace route outright, and is the single most common reason to choose self-hosted or bespoke. (c) **Non-functional**, and it discriminates *in the opposite direction* to what students expect — surviving a twentyfold spike is something a large hosted platform does effortlessly and your single virtual server does not.

2. **Hosting** (Rs. 324,000 in the worked example), **setup and theming labour** (Rs. 175,000 plus Rs. 100,000 of custom modules), and above all **maintenance labour** (Rs. 360,000). Their stated saving of Rs. 792,000 is therefore closer to Rs. 339,000 — still a saving, but less than half what they claimed, and it vanishes entirely if maintenance takes eight hours a month instead of four.

3. Many are acceptable. A clear one: **a home baker selling six kinds of cake to one city, taking orders through Instagram.** Self-hosting would mean paying for a server and maintaining it to serve perhaps thirty orders a week, with nobody to patch it — which is not a cheaper shop but an unpatched one. **Choose marketplace-only or a cheap hosted platform**: the business's scarce resource is the baker's time, the volume is low, nothing about the pricing is unusual, and every hour spent on a server is an hour not spent baking.

Chapter 5 — The Stack, and Your Shop in One Command

1. **Working:** the operating system and the web server — a 500 is generated and served by Apache, so the request reached it and a response came back. **Failed:** the application layer; PHP raised an error and died before producing a page. The database is *unknown* — it may be the cause (the application could not connect) or entirely healthy. **Look first** at the PHP error log, with `docker compose logs shop`; the message is never on the screen, because `display_errors` is off, and Chapter 16 is about why you want it that way.

2. `depends_on` orders container *starts*, and a container that has started is not a database that will accept a login — MariaDB opens its TCP port several seconds before it finishes initialising. The shop therefore starts, runs the installer, and the installer is refused by a database that is listening but not ready. **The failure is intermittent and worse on a fast machine**, because the faster the shop container starts, the more certainly it arrives early. The `service_healthy` condition waits for the database's own healthcheck to pass, which tests a login rather than a port.

3. **Lost:** both named volumes — the entire database (products, categories, orders, customers, settings) and the shop's installed files, including any module you had written in place. **Not lost:** anything in the repository — the Compose file, the Dockerfile, `code/`, and any module kept there rather than edited inside the container. **What would have made it minor:** a database dump taken on a schedule and kept outside the volume, plus the discipline of writing modules in `code/modules/` and mounting them, so that the only thing destroyed is state you can regenerate. Chapter 20 makes both of these routine.

Chapter 6 — Products, Categories and Their Tables

1. **Move to `oc_product_description`:** the product name and the meta description. Both are *words*, and words differ by language. **Stay in `oc_product`:** price, weight and stock quantity — a kilogram is a kilogram in Urdu, and a shop that let the price depend on the reader's language would eventually charge two different prices for one product. The test is the first of the four questions: *does this fact depend on who is reading?*

2. **Buys:** a single-query answer to "every category beneath this one", so a category page costs one query instead of a loop whose length depends on how deep the tree happens to be. **Costs:** the same fact is now stored twice, so the two can disagree, and every write that changes the shape of the tree must rewrite the derived rows. **The operation that makes it visible:** *moving a category.* Changing one `parent_id` invalidates the path rows of that category and of everything beneath it, so the admin panel must rewrite them all — which is why moving a category with many descendants is noticeably slower than renaming one.

3. Two of several good answers. **(a) The database was never the bottleneck**: if the database is 20% of the page, halving it improves the page by 10%, which can be smaller than the run-to-run variance. **(b) The expensive thing is the *number* of queries, not the cost of any one** — the measurement in this chapter found 146 queries for one page, and an index helps exactly one of them. **To tell them apart:** turn on the query log with `long_query_time=0`, request the page once, and read `COUNT(*)` and `SUM(query_time)` from `mysql.slow_log`. A large total with few queries is (a); a small total spread over a hundred queries is (b). That single measurement separates them, and it is the one in `m06_catalogue.py`.

Chapter 7 — Product Variations, Options and Uploads

1. **The photograph is an option** — specifically a `file` option. It is supplied by the customer, it is different every time, and the shop holds no stock of it. **The size is a variant**, because a small mug and a large mug are two different objects on two different shelves, each with its own cost, its own barcode and its own count. **What the shop holds in stock:** blank mugs in two sizes. Nothing else — which is exactly why the photograph cannot be a variant: there is no such thing as stock of "the mug with Ahmed's photograph on it" until Ahmed orders one.

2. **Option value rows:** $4 + 3 + 2 = \mathbf{9}$, plus three rows in `oc_product_option`. **As variants:** $4 \times 3 \times 2 = 24$ **combinations, so 25 `oc_product` rows** — twenty-four plus the master — and 25 description rows, 25 store rows and 25 category rows beside them. Nine against a hundred, for the same catalogue.

3. **Passes:** the filename sanitiser (nothing illegal in "logo.svg"), the length check, the extension allow-list (`svg` is on the stock list), and the MIME allow-list — it claimed `image/png`, which is allowed, and nothing verifies that claim. **Fails:** nothing. All five checks pass. **For it to harm a visitor**, the file must be *served to them from your origin and rendered*. Two things would have to be true: the attacker must learn the stored URL, which carries a 32-character random token and so is not guessable — but it *is* shown to whoever uploaded it and may be forwarded to an administrator who opens it — and the server must serve it with `Content-Type: image/svg+xml` rather than as a download, which Chapter 5's stock install does. The realistic victim is therefore the administrator reviewing the order, and the realistic fix is to drop `svg` from the list and serve uploads with `Content-Disposition: attachment` (Chapter 16).

Chapter 8 — Themes, Layout and the Customer's Experience

1. **Best case for compressing the photographs:** the images are 713 − 597 − 54 = about **61 KB**. Halving them saves 30 KB, which is **4%** of the page — and that is the *best* case, because you cannot compress them to nothing. **Propose instead:** minify `bootstrap.css`, which is 265 KB of the 597 and has no minified copy in the tree. That single change saves roughly 240 KB, about **34%** of the page — eight times the whole image budget — and changes nothing a customer can see. Then remove the unused parts of Bootstrap and FontAwesome, and only then look at the images.

2. **Layout change** if a "trust badge" module already exists: **Design › Layouts › Product**, add the module to **Content Bottom**, set its sort order. No code, visible to the next person, survives upgrades. **Template override** if it must sit *immediately under the button*, inside the markup the theme generates: there is no position there, so you override `product/product.twig` in your child theme and add it. The general rule: a position that exists is a layout change; markup that must differ is a template change.

3. **Two reasons it is lower.** (a) **It follows only assets named in the HTML.** Fonts are requested by the CSS once the browser parses it, so none of the FontAwesome `.woff2` files — 248 KB in that directory — are counted. (b) **It does not execute JavaScript**, so anything a script loads afterwards (a tracking pixel, a lazy-loaded image, a carousel's later slides) is invisible to it. **Which it could fix:** the first, by parsing the CSS for `url()` references and following them — a dozen lines. The second it cannot fix, because fixing it means running a browser, which is a different kind of tool. Knowing which limitation is laziness and which is fundamental is the useful judgement here.

Chapter 9 — The Shopping Basket, and Where State Lives

1. **What would change:** the two queries per request that the `db` engine spends reading and writing the session record. On this host that was worth about 20% of the page — 89 ms to 71 ms — when those two queries were removed altogether by the `file` engine, and Redis would remove them from the database in the same way. **What would not change: the basket.** It is rows in `oc_cart`, and the `Cart rows` column was 1 under both engines. **The number that settles it:** the session record is **18 bytes** and holds `{"currency":"USD"}`. Whatever you move it to, you are moving eighteen bytes. The right place to look for basket performance is the six cart queries an empty basket costs, not the store the session sits in.

2. **Where they came from:** a session row is written on every request, before the visitor has asked for anything, so every page view by every crawler, uptime check and measurement harness created one. The 645 rows here are the by-product of writing this handout. **Why they are not removed on departure:** nothing tells a server that a visitor has left — HTTP has no "goodbye". Rows can therefore only expire on a clock, and `config_session_expire` is 86,400 seconds. **What to measure first:** the proportion of sessions that ever acquire a basket

or a login. If almost none do, the expiry is not the problem and robots are; shortening the expiry would then also shorten the life of the few baskets that matter, which is the opposite of what was wanted. Measure before tuning the number that is easiest to reach.

3. **What each removes:** `down` removes the containers — the running processes and their writable layers. The named volumes `dbdata` and `shopdata` are separate objects and are left alone, so the database files survive and the basket with them; `up -d` attaches the new containers to the same volumes and the shop resumes mid-visit. `down -v` additionally deletes those volumes, which is where every byte of durable state lives, so the entrypoint finds an empty database and reinstalls from scratch. **One other thing it destroys:** every product, category, order, customer and setting you have made since Chapter 5 — in this measurement the shop came back in 17 seconds looking perfectly healthy, with all of it gone. Uploaded images go with the `shopdata` volume at the same time.

Chapter 10 — The Checkout and the Order State Machine

1. **Three things, in order.** (a) **Drive the checkout end to end from a script and read every reply**, not the rendered pages. (b) **Find the first step whose answer disagrees with the step before it** — here, an address accepted with `success` and then reported missing. (c) **Read the source for that one step**, which the previous two have now identified out of several thousand files. **What the measurement would have told you:** that the shipping quote answered `"Shipping address required!"` — a fault, not a price objection. The shipping charges were never the question, and a shop that had lowered them would have changed nothing and concluded that customers were simply difficult.

2. **Why the shares differ.** Time is spread across all eight steps because every one of them loads the framework, starts a session, rebuilds the basket and renders something — fixed costs that have nothing to do with what the step is for. Writes are not spread: seven steps write almost nothing, because collecting an address changes a session, while `confirm` commits the whole order — `oc_order`, its products, its totals and its addresses — in one burst. **What it implies at ten times the traffic:** optimise the *fixed* cost, not `confirm`. Reads scale by caching and by the work of Chapter 6; writes do not cache, but 55 writes per order against 607 queries means the order itself is the small part. The 271 queries spent before the customer has committed to anything are the ones to attack.

3. **Two reasons to refuse.** (a) **They are evidence.** Each one is a customer who reached the payment button and stopped; the ratio of status-0 rows to real orders *is* the abandonment rate, and deleting them destroys the only record of it the shop has. (b) **A payment provider may still be looking for one.** A gateway's reply can arrive minutes or hours later, and it finds the order by an identifier that must still exist (Chapter 14); deleting the row turns a late but successful payment into money received against nothing. **What to build instead:** a report that counts them by day and by last-reached step, and an archiving rule that moves rows older than the gateway's longest callback window — so that nothing is destroyed and the live table stays small.

Chapter 11 — Shipping, Tax and Zones

1. **Where the 141.40 comes from.** It is not 20% of the sub-total: $0.20 \times 702.00 = 140.40$, which is 1.00 short. The missing 1.00 is 20% of the 5.00 shipping charge, so the base is $702.00 + 5.00 = 707.00$ and $0.20 \times 707.00 = 141.40$. **The two rule decisions this reveals.** (a) **Shipping is taxable** in this configuration — a jurisdictional choice, not arithmetic, and one that differs between countries. (b) **The fixed eco tax is not itself taxed**: it contributed 6.00 to the total but nothing to the VAT base, so the two tax lines are applied to the same

base in parallel rather than one on top of the other. Had they compounded, the VAT would have been $0.20 \times 713.00 = 142.60$.

2. **What is wrong.** A base of `store` means every order is taxed as though it were delivered to the shop's own address in Lahore. **Which orders are affected:** only the UAE ones — they are charged Pakistani sales tax on an export that should not carry it, and the shop is either over-collecting from the customer or absorbing the amount itself, while the correct treatment is recorded nowhere. Every domestic order is, by coincidence, correct. **Why nobody has noticed:** the configuration is right for the overwhelming majority of orders and for every order anyone has ever tested, because testing is done from Pakistan. A fault that is correct in the common case and wrong in the rare one does not announce itself. **What to change:** set the rule's base to `shipping`, and confine the rate's geo zone to Pakistan, so that a UAE delivery matches no rule and attracts no Pakistani tax. Then place an order to each, because a change made on reasoning alone has not been verified.

3. **The failure mode in money.** Weight-based shipping charges from a band table keyed on total basket weight. A product with weight zero contributes nothing, so a basket of only zero-weight products falls in the lowest band — usually free or near-free delivery — and the shop pays the courier out of its own margin. With 600 of 4000 products affected, roughly one basket in seven will be understated, and the worst cases are the heaviest products whose weight was never entered. **Why no error appears:** zero is a valid weight. The band lookup succeeds, a quote is returned, the order completes, and every page returns 200 — the same shape of silent failure as Chapter 10's missing setting. **One change that makes it visible:** refuse to quote for a basket containing a zero-weight product, or, less disruptively, add a dashboard panel (Chapter 17) counting enabled products with a weight of zero. The first prevents the loss; the second reports it. Either is better than an accurate method nobody maintains.

Chapter 12 — Discounts, Vouchers and Referrals

1. **The setting** was the coupon total extension's *sort order*: 4 as shipped, which places it before the tax line at 5, versus 6, which places it after. **Why 14.04:** the discount is 70.20, and the VAT rate is 20%. With the coupon first, the VAT base is $702.00 + 5.00 - 70.20 = 636.80$ and the VAT is 127.36; with it last, the base is $702.00 + 5.00 = 707.00$ and the VAT is 141.40. The difference $141.40 - 127.36 = 14.04$ is exactly 0.20×70.20 — tax on the discount. **Which is correct: 770.16**, the shipped order. Tax is owed on *consideration* — what the customer actually pays — and the customer was never asked for the 70.20, so no tax is due on it. Charging 784.20 collects 14.04 of tax on money that changed hands with nobody.

2. **The break-even uplift.** With margin $m = 0.25$ and discount $d = 0.15$, the multiple is $m/(m-d) = 0.25/0.10 = \mathbf{2.5}$ — the shop needs **150% more orders**, two and a half times the volume, to make the same profit. **Two reasons the real requirement is higher.** (a) **Cannibalisation**: a permanent code for *loyal* customers goes, by definition, to people who already buy. Every one of them who would have bought anyway is pure loss, and no new volume is created to offset it. (b) **Variable costs scale with volume**: 150% more orders means 150% more picking, packing, delivery and support, so the gross-margin arithmetic above understates the cost of getting there — and a permanent code also resets the reference price, so the 15% is not recoverable later. **A cheaper alternative:** store credit issued *after* an order — say 5% back, usable on the next purchase. It costs less (not every credit is redeemed), it cannot be found by someone searching for a code, it rewards repeat buying rather than discounting a single sale, and it brings the customer back rather than merely letting them leave cheaper.

3. **What to tell the client.** That the platform has no gift vouchers at all — no tables, no totals extension, only a leftover demonstration image from the version that did — and that the outline naming them reflects a 2010 book rather than current software. Say it before quoting,

not after. **What the platform already provides:** the *spending* half. Store credit (`oc_customer_transaction` and the `credit` total) is a balance on an account that draws down across orders, handles partial payment, and already sits in the totals pipeline. **What must be built:** the *buying* half — selling a voucher as a purchasable product; generating a code on order completion; delivering it to a named recipient; and redeeming it into *a different customer's* credit, with the balance, expiry and single-use guarantees that implies. That is a module (Chapter 19), and the hard parts are the two ways it can be spent twice: two simultaneous redemptions of the same code, and a redemption on an order that is later refunded.

Chapter 13 — Web Payment Systems

1. **Two ways to ship goods you are never paid for.** (a) **Authorisation mistaken for capture.** The gateway reserved the funds and answered "success", but nothing was ever claimed. The reservation lapses after a few days and the money returns to the customer, who has the goods. (b) **Capture followed by a chargeback.** The money arrived and was later taken back by the issuer — on a disputed or fraudulent card, this is the shop's loss, not the acquirer's. **What to check in the documentation:** for the first, whether the endpoint you call performs authorisation only or authorisation-and-capture, and if the former, which call captures and when it must happen. For the second, the provider's chargeback terms: how long the window is, what evidence wins a dispute, and whether a delivery signature is among it. Both questions have documented answers, and neither is visible in the integration's happy path.

2. **Net margin at a 10% refusal rate.** Per 100 orders: 90 delivered at $900 - 60 - 250 =$ Rs 590 each, so $90 \times 590 =$ Rs 53,100; 10 refused at −Rs 500 each, so −Rs 5,000. **Net Rs 48,100**, against Rs 37,200 at a 20% refusal rate — an improvement of Rs 10,900, or **29%**. **Against removing the courier fee entirely:** the fee is Rs 60 on each of 80 delivered orders, $80 \times 60 =$ Rs 4,800, which raises the net to Rs 42,000 — an improvement of **13%**. So halving the refusal rate is worth more than twice as much as winning a free COD service, and the courier would never agree to the second while the first is entirely within the shop's own control. **The lesson:** negotiating hard over the thing you cannot change while ignoring the thing you can is exactly backwards, and the arithmetic says so in one line.

3. **Four things, in order.** (1) **Can you legally receive the money?** Foreign-currency receipts by a Pakistani merchant are subject to State Bank rules, and the answer determines whether the project is possible at all. (2) **Will an acquirer accept this business?** Cross-border card acceptance is underwritten against the risk that the shop does not deliver, and a new shop with no trading history is often declined. (3) **What does it cost, including the dispute rate?** International card-not-present carries both a higher discount rate and a materially higher chargeback rate than domestic. (4) **What will you ship, and how will you prove you shipped it?** Winning a dispute requires delivery evidence, and international delivery often produces none. **Most likely to stop the project: the second.** The technical integration is a week; acquirer onboarding for cross-border acceptance by a small Pakistani merchant is the step that fails — and it fails after everything has been built, if you leave it until last. That is why it belongs second on this list and not fourth.

Chapter 14 — Taking Payment for Orders

1. **What a 1-second timeout actually costs.** From the measured table it cuts off **23 of 196 answered requests, 11.7%** — not a few stragglers but one payment in eight and a half, each with an *unknown* outcome rather than a failed one, so the customer may already have been charged. The colleague's reasoning is sound for a bell-shaped distribution, and this one is not bell-shaped: the median is 321 ms and the 95th percentile is 3,774 ms, so "three times the median" lands in the empty gap between the two groups. **What to propose: 5 seconds**,

which cut off nothing at all, and the measurement to justify it. The customer waits longer in the rare slow case, and the shop stops discarding good payments. If five seconds is judged too long to make anyone wait, the answer is not a shorter timeout but an asynchronous design — accept, show a pending page, and resolve from the provider's callback.

2. **Why the middle is flat.** Because the latency distribution is *bimodal*: one group answers in under about half a second and a second group takes between two and five seconds, with very little in between. A timeout placed in the gap therefore catches almost nothing extra — going from 1 s to 3 s rescued two requests in two hundred. **What the shape tells you that an average could not:** that there are two distinct code paths inside the provider rather than one variable one — a fast path, and something slower behind it such as a risk check or an internal retry — and that the only meaningful choices are "before the slow path" and "after it". An average of 700 ms invites tuning around 700 ms, which is precisely the region where no choice makes any difference at all.

3. **Without a key.** The second request is a new, valid authorisation. The provider has no way to tell it from a customer legitimately buying twice, so it charges again and returns a different transaction identifier — in the measurement `f59594f7d23c4446` and `8171b6e85f9b4766`, both approved. The customer has paid twice and the shop has one order. **With a key.** The provider recognises it, returns the first answer unchanged — same transaction identifier, in 5 ms — and charges nothing. **Where it is stored: on the order row**, generated once when the order is created. **Why generating it when the button is pressed would not work:** the retry is a new request and would generate a new key, which is a key the provider has never seen, so it charges again. The key must identify *the payment attempt for this order*, not the HTTP request — and the order is the only thing that survives between the two.

Chapter 15 — User Account Management

1. **Is cost 12 affordable?** Assumptions, stated: 400 orders a day; perhaps three logins per order once failed attempts, repeat visits and account pages are counted, so about 1,200 logins a day; and traffic is not flat, so assume the busiest hour carries 20% of the day's logins — 240 in an hour, about **0.07 logins per second** on average and perhaps one per second in a burst. Cost 12 delivers **5 logins per second** on one core. The headroom is more than fiftyfold, so **yes, comfortably** — and cost 13, at 2 per second, would still be adequate. **What else must change at the same time:** the timing leak. Cost 12 raises the enumeration gap from about 65 ms to about 220 ms, four times louder, so the dummy-hash fix must ship *with* the cost change rather than after it. Existing hashes also remain at cost 10 until each customer next logs in, so the code must rehash on successful login or the change never reaches anybody.

2. **The cause.** A wrong password on a registered address must be checked against the stored hash, which costs one bcrypt verify — measured at 60.3 ms at cost 10. An unknown address matches no row, so there is nothing to verify and the request returns early. The 65.5 ms gap is that verify. **What an attacker does with it:** runs a list of email addresses through the login form with any password at all, and keeps the slow ones. That is a confirmed customer list — saleable in itself, and it makes credential stuffing and phishing against this shop's customers far cheaper, because every address is known to be live. **The one-line fix:** when no customer matches, verify the supplied password against a fixed dummy hash and discard the result. **Why it feels wrong to write:** it is deliberately wasted work in the hot path of a login form, and every instinct a programmer has says to delete it. A reviewer who does not know why it is there will delete it, which is exactly why the reason belongs in a comment beside it.

3. **The risk you are accepting** by removing lockout: unlimited online password guessing against any account. At 60 ms per attempt a single attacker can try tens of thousands of passwords an hour against one account, which breaks any password a customer actually chooses. **The risk**

you are removing: anyone who knows a customer's email address can lock that customer out at will — five wrong guesses, an hour at a time, repeatedly — so the complaint is real, and the customers complaining may be victims rather than the forgetful. **A design that addresses both better than either:** replace the hard count with *progressive delay* keyed on the address being guessed — no delay for the first three attempts, then a delay that doubles with each further failure and resets on success. A customer who mistypes twice notices nothing; an attacker is throttled to a few guesses an hour, and no account is ever locked, so there is nothing to abuse. Add site-wide rate limiting by source address to catch the attacker who spreads attempts across many accounts — which neither lockout nor per-account delay sees at all — and offer a second factor to the customers who want one.

Chapter 16 — Securing the Shop

1. **Two independent reasons.** (a) **Apache never reads the file.** Debian's `apache2.conf` sets `AllowOverride None` for the document root, so `.htaccess` files are ignored entirely — the rename is correct and has no effect. (b) **Even read, it would not be the whole fix.** The shipped file blocks `system/storage` by rewrite and sets one header; it says nothing about the content security policy, the referrer policy, the permissions policy, the version banners or the wildcard CORS headers, which were eight of the nine findings. **One command to demonstrate it:** write a marker into the error log and fetch it — `docker compose exec shop sh -c 'echo MARKER » system/storage/logs/error.log'` then `curl -s http://localhost:8090/system/storage/logs/error.log`. If the marker comes back, the control is not working whatever the file is called. **The general lesson:** a control that fails open and fails silently is indistinguishable from a control that works, until somebody measures the outcome. Verify the result, never the action — and the gap between "I installed it" and "it does something" is where almost every real breach lives.

2. **Why none of them touched the shop's code.** Because none of them was a programming fault. Every one was a property of how the software was *served*: which headers Apache attaches, what it will serve from which directory, and what it volunteers about itself. The application had already escaped its SQL, auto-escaped its output, tokenised its forms and hashed its passwords properly. **What that says about inheriting a shop:** look at the deployment first. The framework's defences are written once by people who do this full time and are the same in every installation; the deployment is different in every installation and is usually nobody's job. **Which remaining finding first: move `system/storage/` out of the web root.** The admin path is the one still failing the battery, but renaming it only defeats automated scanners, while the storage directory holds the error log, the sessions and every upload — and today it is protected only by a block in a configuration file that the next administrator may edit, replace or lose during an upgrade. Moving it removes the risk instead of covering it, and is the control that survives being forgotten.

3. **Two mechanisms, any two of the three.** (a) **`Header always unset` on a successful response.** `mod_headers` keeps two tables — `always` for every response including errors, and the default for successful ones only. A header PHP sets on a 200 lives in the second, so the directive whose name sounds more thorough touches nothing. (b) **`ServerTokens` after a reload.** It is consumed when Apache starts and used to build the banner once; a graceful reload applies every neighbouring directive and skips this one. (c) **Filename ordering in `conf-enabled`.** Apache reads the directory in filename order and the last setting of a server-level directive wins, so `security-ecad.conf` lost to `security.conf` because `-` sorts before `.`. **The habit that catches all three without knowing any of them:** *measure the observable outcome before and after every change, from outside the system.* Not "is the directive present", not "did Apache accept the config" — all three of these produced a configuration Apache accepted without a single warning — but "does the response now contain the header, and does

the URL now refuse to serve the file". One script, run twice, found all three in the time it would have taken to re-read the file once. That is the entire argument for the measured strand in this course, in its most concrete form.

Chapter 17 — Administration: The Dashboard, and What to Watch

1. **Each gap. 6 to 5:** the table counts every row, including the one at status 0 — an order created at `confirm` whose customer never paid (Chapter 10). That difference is the *abandonment*. **5 to 2:** the widgets count every status above 0, while the chart counts only those in `config_complete_status`, here Complete and Shipped. The three missing orders are paid for and not yet despatched — the *work outstanding*. **Which figure to put in front of the owner: 5, labelled "orders to fulfil and bank"**, because it is the number the shop will actually be paid for. But put it beside the other two rather than instead of them, each with its filter named: "5 paid, 3 still to despatch, 1 abandoned today". One number with a label beats three without, and the labels are the whole content.

2. **Questions to ask.** (a) **What is the denominator?** Sessions, unique visitors, people, or baskets created? Each gives a different answer from the same shop, and Chapter 9 showed that sessions are created by robots that never see a page. (b) **What counts as a conversion?** An order row, an order above status 0, or an order that was actually paid and not refunded? (c) **Over what period, and does it include the campaign spike?** (d) **Are bots excluded, and how? Why two shops at 3% may not be comparable:** one may divide paid orders by human visitors and the other all order rows by all sessions. With 645 session rows and no human customers at all — this shop's measured state — the second definition can report a rate that is pure arithmetic about crawlers. Comparing the numbers without comparing the definitions compares nothing.

3. **A panel that would have caught it. The query:** count orders created in the last hour, split by whether they are above status 0:

   ```
   SELECT SUM(order_status_id=0) stalled,
   SUM(order_status_id>0) completed
   FROM oc_order
   WHERE date_added > NOW() - INTERVAL 1 HOUR;
   ```

   How often: hourly during trading hours. **The alarm condition:** *completed* is zero while the shop has had traffic. That is the exact signature of Chapter 10's fault — customers arriving, baskets filling, no order ever reaching status 1 — and it is detectable without knowing anything about the cause. Note what makes it work: it alarms on the *absence* of a thing. Every monitoring system in the course so far watched for errors, and this fault produced none; the only observable was that something which should happen had stopped happening.

Chapter 18 — Managing Orders, Customers and Refunds

1. **Pending to Shipped.** Nothing happened to stock. Both statuses are inside the processing-or-complete set, and the rule fires only on entering or leaving it — the goods had already been deducted when the order reached Pending. **Shipped to Reversed.** Also nothing, which is the surprise: 12 is in `config_processing_status` in this shop, so the order never left the set. The customer's money has gone back and the goods are still counted as sold. **What to change:** remove 12 from `config_processing_status` at **System › Settings › Edit › Option**. Then moving to Reversed leaves the set and returns the stock, exactly as Refunded does. Note what this implies: the stock behaviour of every status in the shop is decided by two checkbox lists, and nothing in the interface connects them to the word "stock".

2. **The mechanism.** Marking a return **Refunded** writes two rows — one updating `oc_return`, one appending to `oc_return_history` — and touches nothing else. The goods physically come back to the shelf, but `oc_product.quantity` was never incremented, because the *order* was never moved out of the processing set. So after 40 returns a month the system believes it holds 40 fewer units than it does, every month, cumulatively. **What else is drifting:** the revenue figures, because the orders still read as sold and the dashboard counts them (Chapter 17); any affiliate commission paid on them, which nothing reverses (Chapter 12); and the shop's record against the payment provider's, which is what Chapter 14's reconciliation exists to catch. **The smallest change that stops it:** make moving the *order* to status 11 a required step of recording a return. One line in the procedure, and it fixes stock and revenue together, because both follow from the same transition.

3. **Four steps.** (1) **Reproduce it outside the browser** with a signed API call, which separates "the admin panel is broken" from "the API refuses". (2) **Read the response**: a 403 with a permission message, not a 500 and not a validation error. (3) **Check `oc_api`** — a user exists and is enabled, so it is not a missing credential. (4) **Check `oc_api_ip`** — empty, and an empty allow-list permits nothing. **Why it reads as a user permissions problem:** the error message is the generic "You do not have permission to access this page", which is the same text an administrator sees when their own user group lacks a privilege. So the natural first move is to inspect the administrator's permissions at **System › Users › User Groups**, find them correct, grant more anyway, and still fail — while the refusal is happening in a different application, to a different identity, over a connection the administrator never sees. The lesson is the one from Chapter 16: an error message tells you what the system decided, not where it decided it.

Chapter 19 — Extending the Platform: Your First Module

1. **The answer.** Five instances cost 20 ms on a 76 ms page — about 4 ms and exactly 4 queries each — and of those four queries *one* is the module's own. The other three are the framework finding the instance's settings, its translations and its template override. So the measured cost of "being a module" is 3 queries, and the cost of the feature is 1. **Editing the theme instead** would remove those three and keep the one, saving roughly 3 ms per instance — at the price of a change the next upgrade overwrites, that `git status` will not distinguish from the vendor's code, and that cannot be placed twice or disabled without another edit. **What to measure before agreeing to anything:** whether the page is actually too slow, and against what target. A 96 ms page with five modules is not a performance problem — Chapter 8 measured 600 KB of assets after it and Chapter 20 measured the saturation point at five concurrent clients. Optimising 3 ms while 600 KB goes unexamined is the error this whole course is written against.

2. **When it stops working.** The moment the module is placed a *second* time with a different threshold. Both instances will then read the same global value, so both show the same products, and whichever instance was configured last silently wins — or, if no such global setting exists at all, both read null and fall back to the default, so neither reflects what was typed. **What the shopkeeper will report:** "I set the second panel to 20 and it still shows the same four items as the first one." They will not report a bug in the module; they will report that the admin panel does not save. That is the characteristic shape of this fault — the symptom points at the form, and the cause is in the render method, which is the one place nobody looks because it is visibly working.

3. **How to decide.** First ask whether a *setting* reaches it: a custom field on the customer or the address, or a product attribute, covers more requests of this kind than people expect, and a setting survives every upgrade. If not, ask whether the platform offers an *extension point* at the place you need — for checkout there are events on the controllers, and an event handler

is an extension. Only if neither reaches do you consider editing, and then the question is whether the field is worth owning a merge conflict for ever. **What I would need to know first:** who fills the field in and who reads it; whether it must appear on the *order* as well as in the session, because an order copies its data at checkout (Chapter 10) and a field that is not copied disappears from history; and whether it is required, because a required field changes the validation path and therefore the completion rate. **What I would write down whichever I chose:** the decision, the two options rejected and why, and — if it was an edit — the file, the version it was made against, and what must be re-checked at the next upgrade. The note is the deliverable; the code is the easy part.

Chapter 20 — Deployment, Maintenance and Performance

1. **The mechanism.** At five concurrent clients the server's processors are fully occupied and nothing is waiting. Past that point the extra requests do not find spare capacity — they join a queue. A queued request still consumes memory, a connection and a worker slot, and the context switching between twenty workers costs real time, so the server does *less* useful work while every request waits longer. That is why throughput fell from 42.1 to 38.7 while the median rose from 95 ms to 455 ms. **What the owner should do before a sale day:** know the number, and decide what happens past it — queue politely with a wait page, or shed load — rather than discovering it at the busiest hour of the year. **One change that moves the turning point:** cache whole pages for visitors who are not logged in. It removes PHP and the database from the common path entirely, which is where every millisecond in the measurement was spent; the saturation point then becomes a property of the web server rather than of the application, and that is a different order of magnitude.

2. **Both ends. At one client** the processor is mostly idle: a request arrives, is compiled and served, and then nothing happens until the next one. Removing compilation work from a processor that has spare capacity buys little, so 1.3×. **At twenty** the bottleneck has moved from the processor to the *queue*: every request waits behind nineteen others, and shortening each one by a third shortens the queue proportionally but does not change the fact that there is one, so the benefit is diluted to 1.4×. **In the middle**, at the saturation point, the processor is the binding constraint and nothing else is — so work removed from it converts almost one-for-one into throughput, hence 2.0×. **What it implies about when to benchmark:** measure at or near the saturation point, because that is where an optimisation's effect is both largest and most honestly represented. Benchmarking a single client flatters nothing and shows nothing; benchmarking far past saturation measures the queue and attributes it to your change.

3. **The case, in terms a client will act on.** Not "your backups might be corrupt" — that is a hypothetical and clients discount hypotheticals. Instead: "We do not know how long the shop would be closed. I can find out this afternoon." Recovery time is a business number, it belongs in the same sentence as a day's takings, and nobody has ever been given it. **What I would do:** bring up a second empty stack, restore last night's dump into it, open the shop, and check that the catalogue, the orders and the settings are all there. **How long:** under an hour for this shop, including writing it down; the restore itself was 6.4 seconds on the demonstration catalogue and will be minutes on a real one. **What I would measure and record:** the restore time, the age of the newest data in the dump — which is how much the shop would lose, and is a different number from how long it is down — and anything that was *not* in the backup, which in this stack is the uploaded images in the `shopdata` volume. That last item is the one such a test usually finds, and finding it is the entire point.

Chapter 21 — SEO and Digital Marketing

1. **What has happened.** The client switched on `config_seo_url`, which is the *application* half of readable URLs: the shop now looks every link up in `oc_seo_url` and writes `/en-gb/catalog/desktops` into its HTML. The *server* half — the rewrite that turns that path back into `index.php?_route_=en-gb/catalog/desktops` before PHP sees it — is absent, because it ships in `.htaccess` and Chapter 16 measured `AllowOverride None`, which means Apache never reads `.htaccess` on this stack. **Which half is absent: the server half. The command that demonstrates it:** `curl -s -o /dev/null -w "%{http_code}" http://the-shop/en-gb/catalog/desktops` returns 404, while the same page via `index.php?_route_=...` returns 200 — which proves the application is right and the server was never told. **Why nothing appeared in any log:** a 404 is an ordinary response. It is not an error, not an exception, and not a failure of anything; the server was asked for a path it has no rule for and said so, correctly. The shop is working exactly as configured, and the configuration is wrong.

2. **Which is luck and which is design.** The distinct titles are **luck**: the title is the product's name, and the demonstration products happen to have different names. A shop selling the same garment in four sizes as four products would have four identical titles and the same code would produce them. The missing descriptions are **design** — or rather, the absence of design: the field exists in `oc_product_description`, the demonstration data leaves it empty, and the platform writes nothing in its place. **What happens on those pages:** the search engine composes its own summary from whatever text it finds, which on a product page is usually the navigation menu — so the snippet a customer reads in the results is a list of categories rather than a description of the product. **For 50,000 products:** do not write them by hand. Generate a reasonable default from data you already have — name, manufacturer, category and price — as a module (Chapter 19), which gets every page off the floor; then write proper descriptions by hand for the few hundred products that actually sell, because those are the only ones where the difference between adequate and good is worth an afternoon.

3. **What each absence costs. No sitemap** means a crawler finds pages only by following links from the home page, so anything several hops away is effectively invisible — and depth grows with catalogue size, since a 50,000-product shop paginates its categories into hundreds of pages that `robots.txt` itself blocks. **No canonical links** means the same product reached through two categories, or with a sort parameter, is several URLs with identical content competing with one another, and the engine picks one arbitrarily. **For 19 products the canonical matters more:** every page is one click from the home page so nothing is unfindable, but nineteen products across several categories already generate duplicate paths. **For 50,000 the sitemap matters more:** duplication is still a problem, but it is a problem about pages that are at least known to exist, and the greater loss by far is the tens of thousands that are never reached at all. **The difference:** canonical links are about which of the pages a crawler found should count; a sitemap is about whether it found them. Being counted wrongly is a smaller harm than not being counted.

Chapter 22 — Social, Legal and Ethical Issues

1. **Is it a dark pattern? Yes** — and the argument has to be made carefully, because every individual element is honest. The sub-total is right, the discount is right, the tax is arithmetically right on the base it is applied to, and the total adds up. What makes it a dark pattern is the *arrangement*: the customer is shown a discount of 70.20 and charged tax on money that discount means they never paid, and no part of the screen shows them that this is what happened. That is the definition — leading someone to something they would not have chosen, by how it is presented rather than by a false statement. **What would change my answer:** if the tax authority's own rule required tax on the undiscounted amount, the arrangement would

be compelled rather than chosen, and the shop would be a collector rather than a beneficiary. The test is who keeps the 1.8%. **What I would do in an inherited shop:** assume it was an accident, because Chapter 12 showed it is one integer in an unlabelled field and far more likely to be a mistake than a scheme. Fix it, then say what was fixed, how long it had been that way, and roughly what it came to — because if the shop has been over-collecting, somebody else's money is involved and that is now a question about refunds rather than about configuration.

2. **The ranking: the 6 orders are higher risk than the 645 sessions**, by a wide margin, and the volume points the other way. **Justification:** a session row holds an address, a timestamp and a currency for an unidentified visitor who was probably a crawler (Chapter 9); leaked, it is noise. An order row holds a named person's home address, telephone number, email address and a list of what they bought, delivered to where they live; leaked, six of them would do more harm than all 645 sessions together. Risk is harm per record times records, and the first term dominates by orders of magnitude. **Retention. Sessions: 24 hours**, which is already the configured expiry — they exist to make the next click work and have no purpose after the visit, so the only argument for keeping them is that nothing deletes them, which is not an argument. **Orders: years**, because they are accounting records with a statutory life and are the contract (Chapter 22's opening). The *personal* parts of an old order may be reducible — a telephone number is not needed to satisfy a tax inspector — but that is a decision to take advice on, not one to take alone. **Which I would be more reluctant to delete: the orders**, and the reason is the opposite of the risk ranking. They are the most dangerous thing the shop holds *and* the thing it is least free to discard, which is exactly why they deserve the most protection rather than the most deletion. The uncomfortable pairing of those two facts is the whole of data governance in one example.

3. **The reply.** "No, and I want to be clear about why rather than just refusing. Storing card numbers moves us from the shortest PCI DSS questionnaire to the longest — from a handful of questions about keeping our site out of the way, to several hundred requirements and an external audit, on us, annually. That is a real recurring cost and a real liability, and it arrives the moment the first number is stored, not when something goes wrong. What our competitor is doing is almost certainly not what it looks like. The convenience you are describing is *tokenisation*: the provider stores the card and gives us a meaningless reference, so the customer does not retype anything and we never hold a number. Every provider we would use offers it, it is usually a configuration change rather than development, and it keeps the card where it is already somebody else's full-time job to protect. I can have that working this week." **If the client insisted:** say once more, in writing, what the consequence is and who carries it — the merchant, not the developer and not the provider. Then decline to implement it. This is not a matter of taste: it is a documented obligation the client would be taking on unknowingly, and building it after explaining that would make me a party to it rather than a contractor on it.

Chapter 23 — The Semester Project

1. **Five likely causes, in order.** (1) **A port already in use** on their machine — 8090 or 8091 is the commonest, and this handout hit it in Chapter 5. (2) **A step that lives only in your shell history**: a setting you changed once in the admin panel, a SQL statement you ran by hand, a file you copied. It works for you because you did it and forgot. (3) **Data that is not in the repository** — product images, a database dump — because the volume held it and the volume is not the repository. (4) **A line ending or path assumption**: an entrypoint script saved with CRLF will not run in the container, and the error is unhelpful. (5) **A different Docker version or no Docker at all. How to find out without their machine:** ask for three things — the exact command they ran, the complete output, and `docker compose ps`. Those three separate "it never started" from "it started and the shop is not installed" from

"it is running and they are at the wrong address", which is the whole diagnostic tree. **What should have been done in week 12:** build it from scratch on a machine that is not yours — `docker compose down -v` at minimum, a classmate's laptop ideally — because every one of the five causes is invisible on the machine where the shop was built and obvious on any other.

2. **What goes in the report:** both numbers, the index definition, the catalogue size, the machine, and the command that produces them — and then the honest sentence: the index made the page 8% slower, and here is what we think is happening. **What to investigate first:** whether the query is using the index at all. `EXPLAIN` before and after is one command and splits the result in two: if the optimiser ignored it, the question is why (a function on the column, as Chapter 6 found with `LCASE`, or a selectivity it did not like); if it used it and was slower, the index is being read in a different order from the table and the scattered reads cost more than the sort it saved — which is exactly what Chapter 6 measured at 50,000 products, twice. **Why it is worth more marks:** 15 ms is within noise for a single run, so the first thing it tests is whether you measured properly — repeats, a median, a warm cache. And if it survives that, it is a real result that contradicts the textbook rule, which is the kind of thing this course exists to teach you to notice. A result that went as expected demonstrates that you can follow instructions; this one demonstrates that you can measure.

3. **Three things still wrong with the shop as this handout left it.** (1) **The admin panel is still at `/admin/`** (Chapter 16, the one finding the hardening file could not close). *Cost:* an hour — rename the directory and the matching path in `admin/config.php`, then re-run the battery and tell whoever maintains it next. (2) **`system/storage/` is still inside the web root** (Chapter 16), blocked by a configuration file that any future administrator may replace during an upgrade. *Cost:* half a day — move it out, change the two constants in `config.php` and `admin/config.php`, and re-test uploads, sessions and the error log. (3) **There is no sitemap and no canonical link on any page** (Chapter 21). *Cost:* two or three days for a module following Chapter 19's structure, plus a one-line template override for the canonical, which is an afternoon and should be done first. **A fourth, if pressed:** the shop serves plain HTTP, so everything in Chapter 15 and Chapter 14 travels in clear — which is free to fix with a certificate and is the first thing to do before real customers, and is on Chapter 20's go-live list for that reason.

Chapter D

Glossary

Terms are defined as this course uses them. Where a word is used differently elsewhere in the industry, that is said. The chapter in brackets is where the term is introduced and where the evidence for any claim about it lives.

abandonment. A basket or checkout the customer began and did not finish. In this platform it is visible as the gap between rows in `oc_order` and orders above status 0. (Chapter 10, Chapter 17)

acquirer. The bank holding the shop's account, which accepts card payments on its behalf and carries the risk that the shop fails to deliver. (Chapter 13)

affiliate. A customer paid commission on orders arriving through their tracking code. Default commission here is 5%. (Chapter 12)

authorisation. A request asking the issuer whether a payment would succeed, reserving the amount. Reversible, instant, and not money. (Chapter 13)

bcrypt. A deliberately slow password hashing function with a tunable cost factor. PHP's `PASSWORD_DEFAULT` produces bcrypt at cost 10 here. (Chapter 15)

B2B, B2C, C2C. Business to business, to consumer, and consumer to consumer. A statement about who the parties are, not about the software. (Chapter 2)

capture. Claiming an amount previously authorised. Only now is the shop owed anything. (Chapter 13)

cash on delivery (COD). Payment collected by the courier on handover. Dominant in this market, and the most expensive instrument once refusals are counted. (Chapter 13)

chargeback. A card payment reversed by the issuer after a dispute. The loss falls on the shop. (Chapter 13)

container. A running instance of an image, with its own filesystem and process space, sharing the host's kernel. Not a virtual machine. (Chapter 5)

content security policy (CSP). A response header listing where a page may load scripts, styles and images from. Enforced by the browser. (Chapter 16)

cookie. A named value the server asks the browser to store and return on later requests. The only standard way to relate two requests. Lives on the customer's machine, where they can read and change it. (Chapter 9)

cost factor. Bcrypt's work parameter. Each step doubles the time for you and for an attacker. (Chapter 15)

coupon. A discount the customer must type a code to obtain. The code is what makes a campaign measurable. (Chapter 12)

credential stuffing. Trying email and password pairs stolen elsewhere. Defeats password rules, because the passwords are correct. (Chapter 15)

CSRF token. A one-time value in a form, checked on submission, so that another site cannot submit the form on a visitor's behalf. This shop's login carries one. (Chapter 15, Chapter 16)

destination principle. Tax belongs where the goods are consumed — hence a tax rule based on the shipping address. (Chapter 11)

Docker Compose. A file describing several containers, their networks and their volumes, brought up together by one command. (Chapter 5)

gateway (payment). The software company between the shop and the acquirer. The only one of the five parties you integrate with. (Chapter 13, Chapter 14)

geo zone. A named set of (country, zone) pairs that shipping methods and tax rules attach to. It has no meaning of its own. (Chapter 11)

hosted page. A payment architecture where the customer types their card on the provider's site. Keeps the shop in PCI DSS SAQ-A. (Chapter 14)

idempotency key. A value sent with a request so that repeating it has the effect of sending it once. Must belong to the order, not the request. (Chapter 14)

image (container). A read-only layered filesystem holding an application and its dependencies. Built once, identical everywhere. (Chapter 5)

issuer. The bank that gave the customer their card or holds their wallet. Decides whether to approve, and hears their complaints. (Chapter 13)

long tail. A latency distribution where a minority of requests take far longer than the median, so the mean sits well above it. A timeout must be chosen against the tail. (Chapter 14)

marketplace. A venue where third parties sell, with the operator taking a fee and holding neither stock nor the customer relationship. (Chapter 2)

merchant discount rate. The single percentage a shop pays on a card payment, divided afterwards between scheme, acquirer and gateway. (Chapter 13)

module. A unit of functionality added to the platform without editing it: a shipping method, a payment method, a dashboard panel. (Chapter 19)

option (product). A choice offered on a product — size, colour — stored as rows that grow by addition. Contrast *variant*. (Chapter 7)

order. A row recording a commitment to buy, with the products, totals and addresses *copied* as they were at the time. It does not point at the catalogue. (Chapter 10)

order history. The list of an order's status changes with dates. The status says where an order is; the history says how it got there, and is what a dispute is settled from. (Chapter 10, Chapter 18)

order status. The state an order is in. The set is the shop's own, not a standard. Status 0 is not in the set. (Chapter 10)

PayPak. Pakistan's domestic card scheme, running over the 1LINK switch. Cheaper to accept and domestic only. (Chapter 13)

PCI DSS. The card industry's data security standard. Its reporting burden is graded by how much card data you touch; a hosted page keeps you on the shortest questionnaire. (Chapter 14)

PECA 2016. Pakistan's Prevention of Electronic Crimes Act. Unauthorised access to an information system is an offence under it, with no educational exemption. (Chapter 16, Chapter 22)

percentile. The value below which a given share of measurements fall. The 95th percentile of a response time tells you what a slow customer experiences; the mean does not. (Chapter 14)

Raast. The State Bank's instant payment system, settling bank to bank in seconds at negligible cost. (Chapter 13)

return-to-origin. A delivery refused at the door and sent back. The shop pays both legs and keeps the goods. (Chapter 13)

salt. A random per-user value mixed into a password hash, so that identical passwords hash differently and precomputed tables are useless. Need not be secret. (Chapter 15)

session. Server-side state filed under a key, with the browser carrying only the key. Distinct from the cookie that points at it. (Chapter 9)

session engine. Where session records are kept — database, files or Redis. It does not decide where the basket is. (Chapter 9)

settlement. The acquirer actually paying the shop, days later and net of fees. (Chapter 13)

sort order (totals). The position of a total line in the pipeline. It decides what each line is computed from, and therefore what the customer pays. (Chapter 12)

special. A dated replacement price for a product and customer group, applied automatically. (Chapter 12)

tax base. Which address a tax rule is decided by: store, payment or shipping. The most consequential dropdown in the admin panel. (Chapter 11)

tax class. A kind of product for tax purposes, such as taxable goods or downloadable products. (Chapter 11)

tax rate. A number and the geo zone it applies in. Percentage or fixed amount. (Chapter 11)

tax rule. The join between a class and a rate, with a priority and a base. (Chapter 11)

template override. A copy of one theme file in a child theme, replacing the original for that one page. The override directory is the list of your changes. (Chapter 8)

theme. A directory of Twig templates and assets. Never edit the default one. (Chapter 8)

timing side channel. Information leaked by how long an operation takes. Here, a registered email answered 65 ms slower than an unknown one. (Chapter 15)

Twig. The template language this platform renders pages with. It auto-escapes output, which is why the script probes in Chapter 16 did nothing.

user enumeration. Discovering which accounts exist, by timing, messages or behaviour. Reconnaissance rather than an attack, and worth closing for that reason. (Chapter 15)

variant. A product generated per combination of choices, growing by multiplication. Contrast *option*. (Chapter 7)

volume (container). Storage managed by the container runtime and outliving any container. `down` keeps it; `down -v` deletes it. (Chapter 5, Chapter 9)

voucher (gift). A stored-value instrument bought by one person and spent by another. HEC's outline names it; **this platform does not have one**. (Chapter 12)

Chapter E

Where to Read Further

HEC's outline for this course lists four reference books. Three of them are cited at editions that are now between nine and nineteen years old, and one teaches a runtime that lost security support in 2018. This appendix records what HEC prescribed, what this course used instead, and — in each case — why, so that a reviewer can see that the substitution was made deliberately rather than by neglect.

E.1 What HEC Prescribed

Reference, as HEC lists it	Edition	Status
Laudon & Traver, *E-Commerce: Business, Technology, Society*	13th, 2017	Still the standard text; much newer editions exist
Peacock, *PHP 5 E-commerce Development*	2010	**Targets a runtime unsupported since 2018**
Rayport & Jaworski, *Introduction to E-Commerce*	2007	Predates the mobile web
Schneider, *Electronic Commerce*	—	Current editions exist

On the Peacock book, which is the interesting one

It is listed as reference 2, and **the "Course Content" paragraph of HEC's outline is that book's table of contents**, nearly verbatim and in order. That finding is recorded in the Course Specification at the front of this handout, and it explains a great deal: why the clause list reads like a build sequence, why "Checkout" appears twice, and why gift vouchers are named (Chapter 12 found the platform has none — they were a feature of the 2010 cart the book builds).

Why this course does not follow it. PHP 5 reached end of life on 31 December 2018 and has received no security fix since. Teaching a 2010 cart on a dead runtime would mean teaching, as current practice, patterns that are now vulnerabilities — unprepared queries, `mysql_*` functions, MD5 password hashing. Chapter 15 measured what this platform does instead.

It is still worth half an hour of your time. Read its contents page beside HEC's clause list, and you will understand how a curriculum outline is made, which is a more useful lesson than anything in the chapters.

E.2 What This Course Used Instead

Business and models — Chapter 1 to Chapter 4

- **Laudon & Traver**, *E-Commerce: Business, Technology, Society*, **current edition**. The same book HEC names, read at an edition that knows about mobile commerce and marketplaces. Its business-model taxonomy is what Chapter 2 is organised around.
- **Pakistan Bureau of Statistics** and the **State Bank of Pakistan**'s payment system reviews, for market figures. Prefer a primary statistic you can cite to a number from a blog post.

The platform — Chapter 5 to Chapter 18

- **OpenCart's own documentation** at `docs.opencart.com`, and, more usefully, **its source**. This handout read the source for every claim it makes about behaviour, because the documentation is thin and in several places describes an older version.
- **The MariaDB Knowledge Base** for `EXPLAIN`, the slow query log and index behaviour — Chapter 6's method comes from it.
- **The Docker documentation** on Compose, volumes and healthchecks. Chapter 5 and Chapter 9 rest on the volume semantics it defines.
- **The PHP manual** on `password_hash` and sessions. It is unusually good and is the primary source for Chapter 15.

Security — Chapter 16

- **OWASP Top Ten** — the list everyone cites. Read it to know the vocabulary, then note that Chapter 16 measured the two most famous entries as already handled by the framework.
- **OWASP Application Security Verification Standard (ASVS)** — far more useful than the Top Ten for actual work, because it is a checklist of requirements at three levels rather than a ranking of fashionable attacks.
- **OWASP Secure Headers Project**, which is where this course's `security.conf` comes from.
- **PCI DSS and the SAQ-A questionnaire**, from the PCI Security Standards Council. Chapter 14 summarises it; read the actual questionnaire once, because it is shorter than its reputation.
- **Mozilla Developer Network** on Content Security Policy, which is the clearest writing on the subject anywhere.

Law and policy — Chapter 22

Primary sources only. Secondary summaries of Pakistani e-commerce law are frequently out of date and occasionally wrong about which tier of government is responsible.

- **Electronic Transactions Ordinance 2002** — what makes an online order enforceable.
- **Prevention of Electronic Crimes Act 2016** — read sections 3 and 4 before Chapter 16's laboratory, not after.
- **E-Commerce Policy of Pakistan (Ministry of Commerce, 2019)** — the policy framework, and short enough to read in one sitting.
- **The provincial consumer protection Acts** — Punjab (2005), Sindh (2014), Khyber Pakhtunkhwa, Balochistan. Read the one your shop is in.
- **Federal Board of Revenue** for the current sales tax position, and the **Finance Act** of the year in question for the rate. Chapter 11 deliberately refuses to be the authority for a number set annually.

How to read any of this

1. **Prefer the primary source.** The manual, the standard, the Act. Every secondary account of them in this handout can be checked against one.
2. **Prefer the source you can run.** `EXPLAIN` beats an article about indexes; the shop's own code beats its documentation. This is the method of the whole course.
3. **Date everything.** A rate, a version, an end-of-life date and a statutory limit all expire. A claim with no date is a claim you cannot check.

4. **Distrust anything about ranking, conversion or performance that comes without a method.** Chapter 21 declined to make claims it could not check, and the same standard applies to what you read.

E.3 A Note on Using Tutorials

Most of what you will find searching for this platform was written for version 1.5 or 3.x and does not apply. Two findings from this course make the point sharply:

- **"Rename `.htaccess.txt`"** is in every guide, and Chapter 16 measured it as doing nothing at all on a modern Apache where `AllowOverride` is `None`. The advice has been repeated for fifteen years and has quietly stopped being true.
- **"Turn on SEO URLs"** is in every guide, and Chapter 21 measured that doing so — without the server-side rewrite the same guides assume is present — points *every link in the shop* at a 404.

Neither piece of advice was ever wrong. Both stopped being right without anybody updating them. That is the normal fate of written advice about software, and it is why this course measures rather than cites. Read tutorials for the idea; verify the behaviour in your own container.

Chapter F

The Boundary With Other Courses

This course sits downstream of four others and alongside a fifth. It assumes a good deal and repeats a little, and both the assumptions and the repetitions are deliberate. This appendix records what belongs where, so that a lecturer can see what to assume and a student can see why a familiar topic reappears.

F.1 The Edition History, Which Is Unusual

Before the boundaries, the provenance — because this course's position in the degree is less settled than any of its siblings'.

Edition	Title	Where	Content published?
2017	E-Commerce	BSSE elective, p. 38	**Yes — p. 94, the only full specification in any edition**
2023	—	**Absent entirely**	One incidental mention, p. 23
2025	Digital Marketing & E-Commerce	Interdisciplinary / allied list, item 11	No — title and credit hours only

So the single published outline for this course is a **2017 specification for an elective in a different degree**, the current editions do not specify it at all, and the department teaches it as **INFT-4041, BSIT, 5th semester, 3 credit hours**. Everything in this handout is traced to that 2017 outline, with each addition justified in the Course Specification at the front.

A prerequisite under another name

HEC's 2017 entry gives the prerequisite as **Web Engineering**. The department has no course of that name; its equivalent is **CS-207 / INFT-4005 Web Technologies** at 4th semester.

The name differs and the dependency is satisfied. Recorded here rather than silently matched, because a reviewer comparing the two documents will notice, and the answer should be written down before they ask.

F.2 Web Technologies — the Prerequisite

The rule

Web Technologies owns the request. This course owns what is at the other end of it.

A student arriving here should be able to read an HTTP request and response, write HTML and CSS, and know what a form submission is. What this course adds is *an application that answers*, with a database behind it, money passing through it, and consequences attached to getting it wrong.

Topic	Owned by	Here
HTTP methods, status codes, headers	Web Technologies	Assumed; used in Chapter 16 and Chapter 21
HTML, CSS, responsive layout	Web Technologies	Assumed; Chapter 8 is about *overriding* templates, not writing CSS
Cookies, as a browser feature	Web Technologies	Chapter 9 asks where the state they point at actually lives
Forms and validation	Web Technologies	Chapter 10 is about the *sequence* of forms and what each commits
A server-side language	Web Technologies	Appendix B is a refresher, not a first course

F.3 Database Systems

The rule

Database Systems owns the model. This course owns the consequences of somebody else's model.

You will not design a schema here. You will read one you did not write, find out what it costs, and discover that the answer changes with its size.

- **Assumed:** SQL selects and joins, primary and foreign keys, normalisation, what an index is.
- **Added here:** reading the query a screen *actually* ran (Chapter 6); measuring an index rather than assuming it (Chapter 6 found two that made the page slower at 50,000 products); and a schema whose shape is a historical record of somebody's decisions — 18 product tables, options that grow by addition and variants that grow by multiplication (Chapter 7).
- **The useful repetition:** every Database Systems student has been told that indexes make queries faster. Chapter 6 is where that becomes a measurement with a crossover point in it.

F.4 Information Security

The rule

Information Security owns the principles. Chapter 16 owns one shop, measured twice.

That chapter is not a survey of attacks. It is a battery of fifteen checks run against a real installation before and after one configuration file, and its central finding — that the famous attacks were already handled and every real problem was in the deployment — is one a principles course cannot deliver.

- **Assumed:** the CIA triad, authentication versus authorisation, hashing versus encryption, what injection and cross-site scripting *are*.

- **Added here:** measuring them (Chapter 16); the cost factor as a trade-off with a number attached (Chapter 15); a timing side channel caused by the defence itself; and **the legal boundary**, which is stated in Appendix A, in Chapter 16 and again in Chapter 22.
- **Not covered here:** cryptographic construction, network security, forensics. Those stay with Information Security.

F.5 Software Engineering

- **Assumed:** requirements, version control, testing, the idea that a system has a lifecycle.
- **Added here:** the build-versus-buy-versus-extend decision made against a real platform (Chapter 4); extension as the discipline that keeps a system upgradable (Chapter 19); and deployment measured rather than described (Chapter 20).
- **The overlap worth naming:** Chapter 4's decision is a software engineering decision, taught here because it is the one that determines everything after it in this course. A student who has made it in Software Engineering should make it again here with a specific platform in front of them, which is a different exercise.

F.6 Virtual Systems and Services — the Same Semester

⚠ This overlap needs stating, because the two courses are taught side by side

INFT-4008 Virtual Systems and Services is also a 5th-semester BSIT course, and it also uses containers. A student taking both will meet Docker twice in one semester.

The split. Virtual Systems and Services owns *virtualisation as a subject*: hypervisors, container internals, orchestration, cloud service models, how a container differs from a virtual machine at the kernel level.

This course uses containers as a tool and says so. Chapter 5 spends one chapter on what a container is — enough to use Compose confidently and no more — because its purpose is a reproducible shop, not an education in virtualisation. Chapter 20 returns to deployment, again from the application's side: saturation, backup, restore.

For the lecturer: if the two courses are taught in the same semester, Chapter 5 can be shortened by agreement, and the time spent on Chapter 6, which always needs more. If they are not, Chapter 5 stands alone as written.

F.7 What This Course Does Not Teach

Stated so that nobody is surprised at the end of it.

- **Front-end frameworks.** No React, no Vue. Chapter 8 overrides templates in the platform's own engine.
- **Writing a cart from scratch.** Chapter 4 argues explicitly against it, and the argument is the lesson.
- **Digital marketing as a discipline.** Chapter 21 covers what is checkable and says plainly what it is leaving out, which matters because the 2025 edition renames the course.
- **Cloud deployment on a named provider.** Chapter 20 measures a deployment; which cloud it sits on is a different course's subject and dates faster than this handout.
- **Payment integration with a live provider.** Chapter 14 ships its own, because a merchant account cannot be a prerequisite for a laboratory.

Chapter G

The Sixteen-Week Schedule

This appendix is the **source of record** for the teaching schedule. The one-page `ECAD_Fall2026_Lecture_Schedule.pdf` distributed to students is generated from it, and the slide pipeline parses the table below. Change this first; regenerate the one-page version afterwards.

Twenty-three chapters are taught across sixteen weeks, so seven weeks carry two chapters. The pairings are not arbitrary. In every case the two chapters are either one topic the printed book splits for length — the catalogue and its variants, the two halves of the back office — or a pair in which the second is unusable without the first: a basket with nothing to check out, a gateway with no payment method chosen.

One pairing is deliberate in the other direction. **Chapter 16, Security, has a week to itself**, although it would pair naturally with Chapter 15. Its laboratory attacks the student's own shop and then hardens it, and that does not fit in half a week alongside anything else.

G.1 The Schedule

Wk	Lecture topic	Chapters	Lab	Assessment
1	What e-commerce changed; business and revenue models	1, 2	Read four real shops; the arithmetic of one order	—
2	Auctions, portals and communities; planning a framework	3, 4	Three shapes compared; your requirements and your decision	—
3	The stack, and your shop in one command	5	Compose up; verify, break and reset the shop	—
4	Products, categories and the data model beneath them	6	50,000 products, and the index that rescues the category page	—
5	Product variations, options and user uploads	7	Option combinations, and what an upload field accepts	**Quiz 1 (Ch 1–5)**
6	Themes, layout and the user experience	8	Page weight before and after, measured in bytes and requests	**Assignment 1 set**
7	The shopping basket, and where state lives	9	Session, cookie and database baskets across a restart	—
8	The checkout and the order state machine	10	Midterm, then queries counted per checkout step	**Midterm (Ch 1–9)**

Wk	Lecture topic	Chapters	Lab	Assessment
9	Shipping, tax and zones; discounts and vouchers	11, 12	Geo zones and GST; a voucher applied in the wrong order	**Assignment 1 due**
10	Web payment systems; taking payment for orders	13, 14	A sandbox gateway, and what happens when it times out	**Project proposal**
11	User account management	15	Password cost against logins per second	—
12	Securing the shop	16	The OWASP checks against your own container, before and after hardening	**Quiz 2 (Ch 10–15)**
13	Administration: the dashboard, orders, customers, refunds	17, 18	A refund through the order state machine	**Assignment 2 set**
14	Extending the platform: your first module	19	A shipping method and a dashboard widget, written from nothing	—
15	Deployment, maintenance and performance; SEO	20, 21	Load test with cache on and off; crawlability measured	**Assignment 2 due**
16	Social, legal and ethical issues; project defence	22, 23	Project presentations	**Project due; presentations**

G.2 How the Assessment Adds Up

HEC publishes the assessment instruments for this course — sessional examination, home assignments, quizzes, project, presentations, final examination — without weights. The weights below are the department's, and are stated here so that a student can see the whole shape of the semester on one page.

Instrument	Weight	What it examines
Two quizzes	10%	Weeks 5 and 12. Short, closed-book, on the chapters named in the schedule
Two assignments	15%	Set in weeks 6 and 13. Both are done on your own shop and submitted as a change to it plus a short write-up
Midterm examination	20%	Week 8, Chapters 1–9
Semester project	25%	The shop itself, built across the whole semester (Chapter 23), submitted in week 16
Presentation and defence	5%	Week 16. Ten minutes, plus questions you are expected to be able to answer about your own decisions
Final examination	25%	All chapters, with emphasis after the midterm

⚠ The project is not a week-16 activity

Twenty-five per cent of the marks are for a shop that is built in the laboratory of every chapter from Chapter 5 onward. A student who treats the laboratories as optional homework and intends to build the project in the last fortnight will be building, in two weeks and without help, something the rest of the class built in fourteen weeks with a lecturer in the room.

Chapter 23 states what is required and how it is marked. Read it in Week 3, not in Week 15.